CoE paper figure book

every figure, in the order it was made, with its Python source

Built Sun Oct 25 2026

Order: by "originally made" time (CT), oldest first; ties broken by git commit, then file time to the second, then paper number. "Copy time" means only when the file was copied onto the box (Fri Oct 23 2026, 8:20–8:31 AM CT); the true original is likely earlier. "This morning" = paper as of 8:30 AM CT Oct 25; "now" = rewritten paper. Every image is set at `\textwidth`; very tall images get a taller page instead of being shrunk.

Contents

1. Passing-universe domains **3**

2. First stochastic-field fit (stage A) **6**

3. Parameter degeneracies **9**

4. Same source at R = 4, 5, 6 Gpc **11**

5. Low multipoles: LCDM, Planck, CoE one-sky **14**

6. Damped-harmonic fits to eps_l **16**

7. Final CMB model vs Planck TT **18**

8. Density and t vs a (radiation + matter) **24**

9. CatWISE-locked density laws **27**

10. Cosmic-web ingredients **30**

11. Fitted P_delta terms and window W(chi) **33**

12. Observables: Migkas A/A_all with bars and rotation overlay **36**

13. Sky-orientation sequence (four rows) **45**

14. Full Migkas A/A_all with model and residual bars (key plot) 51

15. Geometry set-up 62

16. Realistic cosmic web slab 70

17. Migkas 71 bins with CoE direct fits 73

18. Volume vs time (radiation + matter) 76

19. Target profiles matched numerically 78

20. Axisymmetric harmonics on a ring 81

21. Target peak matched numerically 84

22. Density modulation m(r) 90

23. A/B running-mean residuals and sky maps 92

24. Spot-scores table 100

25. Predicted MAX/MIN bands on Migkas Fig 8 102

26. Web smoothed ladder (R = 0.5-14 Gpc) 124

27. Harmonic decomposition 2x2 132

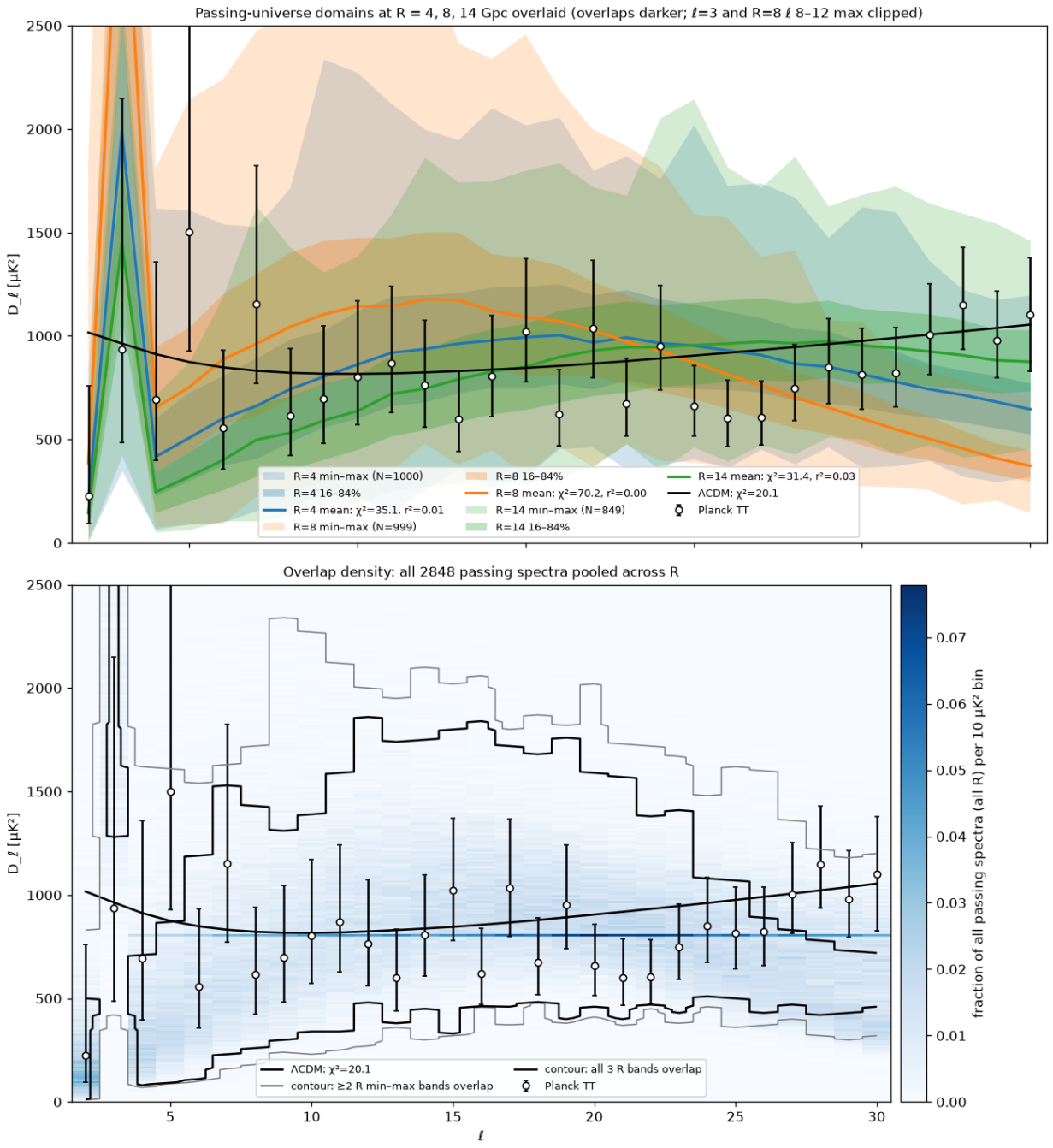
1. Passing-universe domains

Originally made: Wed Oct 21 2026, 4:19 PM CT (*git commit ae85632 (coe-cosmology-gh), adding mc_domain_overlay_lin.png*)
Paper figure number: this morning Fig. 18; now Fig. 18
Files: `f05.png`
Code status: original (git)

Notes

Domains at R = 4, 8, 14 Gpc overlaid, with spreads over Monte-Carlo realisations of the realistic web. Derived from the redweb2/realweb Monte-Carlo runs; no earlier figure in this book is an input. Script saves mc_domain_overlay_lin.png, which is md5-identical to f05.png.

Paper caption (caveats as stated): Passing-universe domains at $R = 4$, 8, 14 Gpc overlaid (top; bands are spreads over realisations; $\ell = 3$ and the $R = 8$ ℓ 8–12 maxima clipped) and the density of sky-plane domains (bottom) against Planck.



f05.png

Computing chain (run order, full listings)

/workspace/redweb2/realweb/mc/mc_overlay.py

```
import numpy as np
exec(open('/workspace/redweb2/realweb/analyze.py').read().split('O=')[0])
t=np.loadtxt('/workspace/joint/lcdm.txt');Lt=np.interp(l,t[:,0],t[:,1])
z=np.load('mc_mean_passing.npz');fig,ax=plt.subplots(2,1,figsize=(11,12),sharex=True)
X=ax[0];allD=[]
```

```
for R,c in zip([4.0,8.0,14.0],['tab:blue','tab:orange','tab:green']):
  D=z[f'D{R}'];allD.append(D);m=D.mean(0);p16,p84=np.percentile(D,[16,84],0)
  X.fill_between(l,D.min(0)[2:31],D.max(0)[2:31],color=c,alpha=.2,lw=0,label=f'R={R:g} min–max
  ↪ (N={len(D)})')
  X.fill_between(l,p16[2:31],p84[2:31],color=c,alpha=.3,lw=0,label=f'R={R:g} 16–84%')
  X.plot(l,m[2:31],color=c,lw=2,label=f'R={R:g} mean: χ²={chi(m):.1f}, r²={r2(m):.2f}')
for A in ax:
  A.plot(l,Lt,'k-',lw=1.6,label='ΛCDM: χ²=20.1')
  A.errorbar(Pl[:,0],Pl[:,1],yerr=[Pl[:,2],Pl[:,3]],fmt='o',color='k',mfc='w',ms=5,capsize=2,label='Planc
  ↪ k
  ↪ TT')
  A.set_ylim(0,2500);A.set_ylabel('D_□ [μK²]')
X.set_title('Passing-universe domains at R = 4, 8, 14 Gpc overlaid (overlaps darker; □=3 and R=8 □ 8–12
↪ max clipped)',fontsize=10)
X.legend(loc='lower center',ncol=3,fontsize=8)
A=np.vstack(allD);yb=np.linspace(0,2500,251);yc=(yb[1:]+yb[:-1])/2
F=np.array([[np.mean((A[:,L]<=y)&False) for y in yc] for L in l])  # placeholder
F=np.array([np.histogram(A[:,L],bins=yb)[0] for L in l],float)
# fraction of all spectra whose min-max... use per-l density of all passing spectra
F/=len(A)
# count of R min-max bands covering
C=np.array([[sum((D[:,L].min()<=y<=D[:,L].max()) for D in allD) for y in yc] for L in l])
im=ax[1].pcolormesh(np.r_[l-.5,30.5],yb,F.T,cmap='Blues',shading='flat',vmin=0)
ax[1].contour(l,yc,C.T,levels=[1.5,2.5],colors=['gray','k'],linewidths=[1,1.5])
plt.colorbar(im,ax=ax[1],pad=.01,label='fraction of all passing spectra (all R) per 10 μK² bin')
ax[1].plot([],[],color='gray',label='contour: ≥2 R min–max bands
↪ overlap');ax[1].plot([],[],color='k',lw=1.5,label='contour: all 3 R bands overlap')
ax[1].set_title(f'Overlap density: all {len(A)} passing spectra pooled across R',fontsize=10)
ax[1].legend(loc='lower center',ncol=2,fontsize=8);ax[1].set_xlabel('□')
plt.tight_layout();plt.savefig('mc_domain_overlay_lin.png',dpi=120)
```

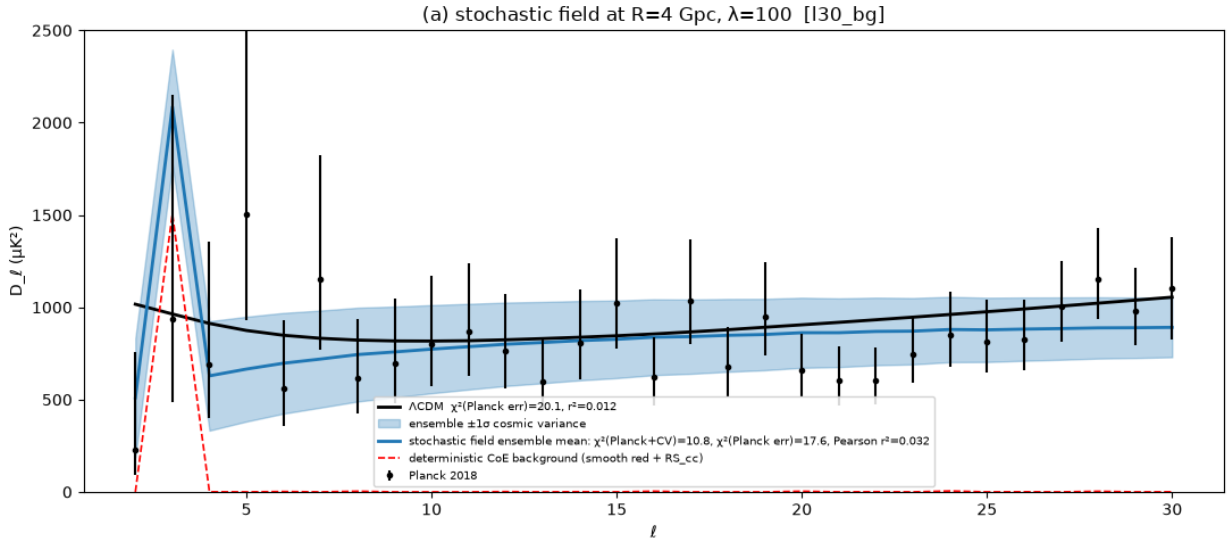
2. First stochastic-field fit (stage A)

Originally made: Thu Oct 22 2026, 12:42 AM CT (*git commit c18eae2, adding stochfield/a_cl_l30_bg.png*)
Paper figure number: this morning Fig. 12; now Fig. 21
Files: `f15.png`
Code status: original (git)

Notes

Gaussian random field riding the CoE flow, R = 4 Gpc, fitted to Planck low multipoles. Lives in the stochfield/ work; builds on the CoE flow geometry. Script (from GitHub kiwipumpkin/coe-cosmology, commit c18eae2, work/2026-10-21_22/stochfield/figs.py) saves a_cl_{tag}.png; run with tag l30_bg it writes a_cl_l30_bg.png, which is md5-identical to f15.png. fwd.py is its imported helper.

Paper caption (caveats as stated): First stochastic-field fit (stage A: Gaussian random field riding the CoE flow, $R = 4\,\mathrm{Gpc}$, ℓ 2–30, CoE background included, $\lambda = 100$). Blue: ensemble mean with $\pm 1\sigma$ cosmic-variance band ($\chi^2_{\mathrm{Planck+CV}} = 10.8$, $\chi^2_{\mathrm{Planck}} = 17.6$, Pearson $r^2 = 0.032$); red dashed: deterministic CoE background; black: ΛCDM ($\chi^2 = 20.1$, $r^2 = 0.012$); points: Planck 2018.



f15.png

Computing chain (run order, full listings)

/workspace/coepdf/book/gitsrc/figs.py

```
import sys,json,numpy as np,healpy as hp,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from fwd import Fwd
tag=sys.argv[1];LAMI=int(sys.argv[2]) if len(sys.argv)>2 else None
R=json.load(open(f'fit_{tag}.json'));Pf=np.array(R['planck']);l=Pf[:,0];Lt=np.array(R['lcdmD']);Dfix=np.array(R['Dfix']);LC=R['Lcurve']
```

```
# chosen: L-curve corner = max curvature in (log chi2, log reg); fallback lam=0 entry
x=np.log10([max(e['chi2_cv'],1e-9) for e in LC[1:]]);y=np.log10([max(e['reg'],1e-12) for e in LC[1:]])
if LAMI is None:
    dx,dy=np.gradient(x),np.gradient(y);ddx,ddy=np.gradient(dx),np.gradient(dy);k=(dx*ddy-dy*ddx)/np.maximum((dx**2+dy**2)**1.5,1e-12);LAMI=1+int(np.argmax(np.abs(k[1:-1])))+1
S=LC[LAMI];p=S['p'];F=Fwd(lmax=int(R['LMAX']));t=F.terms(p);Df=t['scalar']+t['mlens'];Dm=Df+Dfix;cv=np.sqrt(2/(2*l+1))*Df
lab=lambda c2,c2p,r2:f'χ²(Planck+CV)={c2:.1f}, χ²(Planck err)={c2p:.1f}, Pearson r²={r2:.3f}'
fig,ax=plt.subplots(figsize=(11,5));ax.plot(l,Lt,'k-',lw=2,label=f'ΛCDM  χ²(Planck err)={R["lcdm"]["chi2_planck"]:.1f}, r²={R["lcdm"]["r2"]:.3f}')
ax.fill_between(l,Dm-cv,Dm+cv,color='C0',alpha=.3,label='ensemble ±1σ cosmic variance');ax.plot(l,Dm,'C0-',lw=2,label='stochastic field ensemble mean: '+lab(S['chi2_cv'],S['chi2_planck'],S['r2']))
if R['BG']:ax.plot(l,Dfix,'r--',lw=1.2,label='deterministic CoE background (smooth red + RS_cc)')
ax.errorbar(l,Pf[:,1],yerr=[Pf[:,2],Pf[:,3]],fmt='o',color='k',ms=3,label='Planck 2018')
ax.set_xlabel('□');ax.set_ylabel('D_□ (μK²)');ax.set_ylim(0,max(2500,1.1*Pf[:,1].max()) if R['LMAX']>30 else 2500);ax.set_title(f'(a) stochastic field at R=4 Gpc, λ={S["lam"]:.3g} [{tag}]');ax.legend(loc='lower center',fontsize=7,framealpha=.85)
fig.tight_layout();fig.savefig(f'a_cl_{tag}.png',dpi=100);plt.close()
# (b) P(k) and W
fig,(a1,a2)=plt.subplots(1,2,figsize=(12,4.5));kk=np.geomspace(1e-2,100,400)
for j,e in enumerate(LC[1::2]):a1.loglog(kk,F.Pd(e['p'],kk),color=plt.cm.viridis(j/6),lw=1,label=f'λ={e["lam"]:.2g}')
a1.loglog(kk,F.Pd(p,kk),'k',lw=2.5,label=f'chosen λ={S["lam"]:.2g}');a1.set_xlabel('k (1/Gpc)');a1.set_ylabel('P_δ(k) (Gpc³)');a1.legend(loc='lower center',fontsize=7,ncol=2)
a1.set_title(f'(b) P_δ: n={p["n"]:.2f}, k_c={p["kc"]:.3g}/Gpc, σ_δ(0.078)={S["reg_sig_delta"]:.3g}')
for j,e in enumerate(LC[1::2]):a2.plot(F.ch,F.W(e['p'])[0],color=plt.cm.viridis(j/6),lw=1)
a2.plot(F.ch,F.W(p)[0],'k',lw=2.5);a2b=a2.twinx();a2b.semilogy(F.ch,abs(F.KM),'C3:',label='|moving-lens kernel K_M|');a2b.legend(loc='lower center',fontsize=7)
a2.set_xlabel('distance from observer χ (Gpc)');a2.set_ylabel('W(χ)');a2.set_title(f'radial window: centre {p["chiw"]:.2f}, width {p["sw"]:.3g} Gpc (R=4)')
fig.tight_layout();fig.savefig(f'b_pk_w_{tag}.png',dpi=100);plt.close()
# (c) L-curve
fig,ax=plt.subplots(figsize=(6,5));X=[e['chi2_cv'] for e in LC[1:]];Y=[e['reg'] for e in LC[1:]];ax.loglog(X,Y,'o-')
for e in LC[1:]:ax.annotate(f'{e["lam"]:.2g}',(e['chi2_cv'],e['reg']),fontsize=7)
ax.loglog([S['chi2_cv']],[S['reg']],'r*',ms=14,label='chosen');ax.set_xlabel('χ² (Planck+CV)');ax.set_ylabel('regularizer');ax.set_title(f'(c) L-curve [{tag}]');ax.legend(loc='lower center')
fig.tight_layout();fig.savefig(f'c_lcurve_{tag}.png',dpi=100);plt.close()
json.dump(dict(chosen=S['lam'],p=p,chi2_cv=S['chi2_cv'],chi2_planck=S['chi2_planck'],r2=S['r2'],reg=S['reg'],sig_delta=S['reg_sig_delta']),open(f'chosen_{tag}.json','w'),indent=1)
print('chosen',S['lam'],p,S['chi2_cv'],S['chi2_planck'],S['r2'])
```

/workspace/coepdf/book/gitsrc/fwd.py

```
# Semi-analytic forward model: ensemble C_l of dT/T from a Gaussian random potential field (envelope W(chi)) riding the CoE flow.
# Kernels copied from los_flow_R/flowR.py (grow variant, full CoE radial velocity in moving lens), evaluated at x=1 (CoE-radius of observer).
import numpy as np,healpy as hp
from scipy.special import spherical_jn
from scipy.interpolate import RegularGridInterpolator as RGI
c=2.998e5;Gc=4.30e-12;rho=42152249999.99999*1e9;T0=2.7255e6
T=np.load('/workspace/redweb2/realweb/los_flow_R/tab.npz');L1=np.load('/workspace/redweb2/realweb/los_flow_R/ltb.npz')
lTs=np.log(T['Ts']);Stf=RGI((lTs,T['xs']),T['St']);ix1=np.argmin(abs(T['xs']-1));a_=T['S'][:,ix1];eta=T['eta']
```

```
A=lambda t:np.interp(np.log(t),np.log(L1['T']),L1['Agrow'])
def t_of_chi(ch,R):return np.exp(np.interp(-ch/R*eta[0],-eta,lTs))
def kernels(R,dch=0.004):
    ch=np.arange(0,R,dch)+dch/2;te=t_of_chi(np.r_[ch-dch/2,R],R);tt=t_of_chi(ch,R);dt=-np.diff(te)
    ai=np.interp(np.log(tt),lTs,a_);Am=0.5*(A(te[:-1])+A(te[1:]))
    KI=(2/c**2)*(A(te[:-1])-A(te[1:]))/dch                          # per (km/s)^2 per Gpc
    Rd=Stf(np.c_[np.log(tt),np.ones_like(tt)])
    KM=(2/c**2)*(-(Am/ai*dt)*42*Rd)/dch                              # multiplies u.grad Phi0
    ↪ [(km/s)^2/Gpc]
    SW=A(t_of_chi(R-1e-9,R))/(3*c**2)
    return ch,KI,KM,SW
class Fwd:
    def __init__(s,R=4.0,lmax=100,nk=360,kmin=1e-2,kmax=120,dch=0.004):
        s.R=R;s.l=np.arange(2,lmax+1);s.lx=np.arange(0,lmax+2);s.k=np.geomspace(kmin,kmax,nk);s.lnk=np.lo⌋
        ↪ g(s.k)
        s.ch,s.KI,s.KM,s.SW=kernels(R,dch);s.dch=dch
        s.J=np.empty((len(s.lx),nk,len(s.ch)),np.float32);s.JR=np.empty((len(s.lx),nk))
        for i,L in
        ↪ enumerate(s.lx):s.J[i]=spherical_jn(L,s.k[:,None]*s.ch[None]);s.JR[i]=spherical_jn(L,s.k*R)
    def W(s,p,der=False):  # radial envelope on potential: Gaussian in distance, centre chiw, width sw
    ↪ (Gpc)
        w=np.exp(-0.5*((s.ch-p['chiw'])/p['sw'])**2)
        if der:return -w*(s.ch-p['chiw'])/p['sw']**2
        return w,np.exp(-0.5*((s.R-p['chiw'])/p['sw'])**2)
    def Pd(s,p,k=None):  # density power spectrum P_delta(k) [Gpc^3]: A (k/1)^n exp(-(k/kc)^2)
        k=s.k if k is None else k;return 10**p['lgA']*k**p['n']*np.exp(-(k/p['kc'])**2)
    def Pphi(s,p):return (4*np.pi*Gc*rho)**2*s.Pd(p)/s.k**4
    def terms(s,p):
        w,wR=s.W(p);wd=s.W(p,True);Is=(np.einsum('lkc,c->lk',s.J,(s.KI*w).astype(np.float32))*s.dch+s.SW*⌋
        ↪ wR*s.JR)[2:-1]
        Am=np.einsum('lkc,c->lk',s.J,(s.KM*w).astype(np.float32))*s.dch;Bm=np.einsum('lkc,c->lk',s.J,(s.K⌋
        ↪ M*wd).astype(np.float32))*s.dch
        P=s.Pphi(p);f=2/np.pi*s.k**3*P   # integrand per dlnk
        Cs=np.trapezoid(f*Is**2,s.lnk,axis=1);Cm=np.zeros(len(s.l));k=s.k
        # moving lens  int K_M u.grad(W phi) = W u.grad phi + phi W'(n.u): exact m-averaged ensemble C_l
        ↪ via cos(theta) Gaunt couplings (u = z)
        if not hasattr(s,'cf'):
            a=lambda l,m:np.sqrt(((l+1)**2-m**2)/((2*l+1)*(2*l+3)));b=lambda
            ↪ l,m:np.sqrt(np.maximum(l**2-m**2,0)/((2*l-1)*(2*l+1)))
            cf=[]
            for l in s.l:
                m=np.arange(-l,l+1);mi=np.abs(m)<=l-1
                cf.append([np.sum(a(l,m)**2+b(l,m)**2),np.sum(b(l+1,m)**2),np.sum(a(l-1,m[mi])**2),np.sum⌋
                ↪ (a(l,m)*b(l+1,m)),np.sum(b(l,m[mi])*a(l-1,m[mi]))])
            s.cf=np.array(cf)/(2*s.l[:,None]+1)
        L=s.l;A0=Am[L];Bp=Bm[L+1];Bn=Bm[L-1];c=s.cf
        acc=k**2*A0**2*c[:,:1]+Bp**2*c[:,1:2]+Bn**2*c[:,2:3]+2*k*A0*Bp*c[:,3:4]-2*k*A0*Bn*c[:,4:5]
        Cm=np.trapezoid(f*acc,s.lnk,axis=1)
        dl=s.l*(s.l+1)/(2*np.pi)*T0**2;return dict(scalar=Cs*dl,mlens=Cm*dl)
    def Dl(s,p):t=s.terms(p);return t['scalar']+t['mlens']
    def sig_delta(s,p,Rs=0.078):  # rms density contrast smoothed with Gaussian of Rs Gpc
        kk=np.geomspace(1e-3,1e3,4000);return
        ↪ float(np.sqrt(np.trapezoid(kk**3*s.Pd(p,kk)*np.exp(-(kk*Rs)**2)/(2*np.pi**2),np.log(kk))))
```

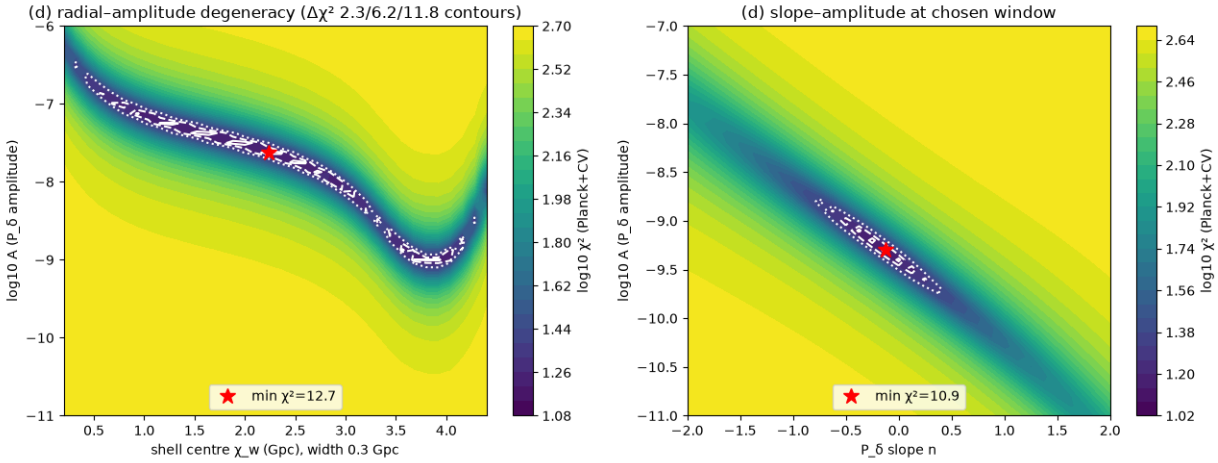
3. Parameter degeneracies

Originally made: Thu Oct 22 2026, 12:42 AM CT (*git commit c18eae2 (same commit as #2; ordered after it by paper number), adding stochfield/d_degeneracy_l30_bg.png*)
Paper figure number: this morning Fig. 22; now Fig. 23
Files: `f14.png`
Code status: original (git)

Notes

Delta-chi2 contours of shell position vs amplitude for the stochastic-field fit. Depends on the stage-A fit (#2). Script writes d_degeneracy_l30_bg.png (md5 match to f14.png).

Paper caption (caveats as stated): Parameter degeneracies. Left: shell position vs amplitude ($\Delta\chi^2 = 2.3, 6.2, 11.8$, i.e. 1, 2, 3σ for two parameters). A farther shell needs a different amplitude because K_M varies with χ. Right: tilt vs amplitude at the chosen window.



`f14.png`

Computing chain (run order, full listings)

`/workspace/redweb2/stochfield/deg.py`

```
import json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt,sys
sys.argv=['fit.py','30','bg'];src=open('fit.py').read().split('rng=np.random')[0];exec(src)
C=json.load(open('chosen_l30_bg.json'));p0=C['p']
def c2(**kw):q=dict(p0,**kw);Df=F.Dl(q);return chi2(Df+Dfix,Df)[0]
# slice 1: thin shell (width 0.3 Gpc): shell centre vs amplitude
cw=np.linspace(0.2,4.4,36);lA=np.linspace(-11,-6,41);S1=np.array([[c2(chiw=c,sw=0.3,lgA=a) for c in cw]
↪  for a in lA])
# slice 2: slope vs amplitude at chosen window
ns=np.linspace(-2,2,33);lA2=np.linspace(-11,-7,41);S2=np.array([[c2(n=n,lgA=a) for n in ns] for a in lA2])
np.savez('deg_l30_bg.npz',cw=cw,lA=lA,S1=S1,ns=ns,lA2=lA2,S2=S2)
fig,(a,b)=plt.subplots(1,2,figsize=(13,5));m1=S1.min();m2=S2.min()
for ax,X,Y,S,m,xl in ((a,cw,lA,S1,m1,'shell centre χ_w (Gpc), width 0.3 Gpc'),(b,ns,lA2,S2,m2,'P_δ slope
↪  n')):
    cs=ax.contourf(X,Y,np.log10(S),30,cmap='viridis');plt.colorbar(cs,ax=ax,label='log10 χ² (Planck+CV)')
```

```
    ax.contour(X,Y,S-m,[2.3,6.18,11.8],colors=['w','w','w'],linestyles=['-','--',':']);i,j=np.unravel_index(S.argmin(),S.shape);ax.plot(X[j],Y[i],'r*',ms=12,label=f'min χ²={m:.1f}')
    ax.set_xlabel(xl);ax.set_ylabel('log10 A (P_δ amplitude)');ax.legend(loc='lower center')
a.set_title('(d) radial–amplitude degeneracy (Δχ² 2.3/6.2/11.8 contours)');b.set_title('(d) slope–amplitude at chosen window')
fig.tight_layout();fig.savefig('d_degeneracy_l30_bg.png',dpi=100)
print('slice1 min',m1,'at',cw[np.argmin(S1.min(0))],'; per-centre best lgA:',[(round(c,2),round(lA[np.argmin(S1[:,j])],2),round(S1[:,j].min(),1)) for j,c in enumerate(cw)][::5])
```

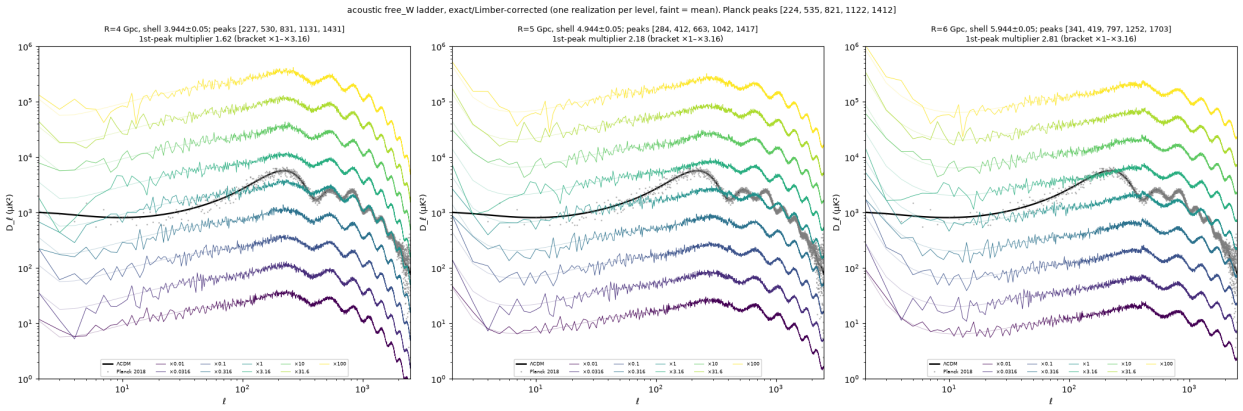
4. Same source at R = 4, 5, 6 Gpc

Originally made: Thu Oct 22 2026, 12:42 AM CT (*git commit c18eae2, adding stochfield/acoustic_ladder_456.png (pixel-identical to f03.png)*)
Paper figure number: this morning Fig. 21; now Fig. 22
Files: `f03.png`
Code status: original (git)

Notes

Peak positions vs shell radius for the same source parameters. Script (commit c18eae2, work/2026-10-21_22/stochfield/acoustic_ladder456.py) ends with savefig('acoustic_ladder_456.png'); that committed PNG is pixel-for-pixel identical to f03.png (f03 is a re-saved copy). acoustic_model.py is its imported module.

Paper caption (caveats as stated): The same source parameters with $R = 4$, 5, 6 Gpc (shell at 3.944, 4.944, 5.944 Gpc): peaks [227, 530, 831, 1131, 1431], [284, 412, 663, 1042, 1417], [341, 419, 797, 1252, 1703]. Moving the shell without refitting r_s changes $\chi_{\rm eff}$ in Eq. (20).



f03.png

Computing chain (run order, full listings)

/workspace/coepdf/book/gitsrc/acoustic_ladder456.py

```
# Ladder R=4,5,6 Gpc with per-R exact/Limber correction (13 nodes l 51-400, log-ratio interpolated).  Field only.
import os,json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from scipy.special import spherical_jn
import acoustic_model as M
from acoustic_model import AFwd,Gc,rho,T0,LMAX
import acoustic_fit as A
from acoustic_ladder import lim,pk,MULT,p0,PlP,pl1
CK='acoustic_ladder_456.json';S=json.load(open(CK)) if os.path.exists(CK) else {};ll=np.arange(LMAX+1)
NODES=(51,60,75,90,110,130,150,175,200,240,280,330,400)
def exact(F,p,L):
    ch,KM,dch=F.ch,F.KM,F.dch;W=np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2);Wd=-W*(ch-p['chiw'])/p['sw']**2; m=W>1e-4;ch,KM,W,Wd=ch[m],KM[m],W[m],Wd[m]
    k=np.linspace(L/ch.max()*0.7,min((L+60)/ch.min()*2.5,4000),2500);x=k[:,None]*ch[None]
```

```
    Aa=spherical_jn(L,x)@(KM*W)*dch;Bp=spherical_jn(L+1,x)@(KM*Wd)*dch;Bm=spherical_jn(L-1,x)@(KM*Wd)*dch
    acc=k**2*Aa**2/3+(Bp**2+Bm**2)/6+k*Aa*(Bp-Bm)/3;Pp=(4*np.pi*Gc*rho)**2*F.Pd(p,k)/k**4
    return 2/np.pi*np.trapezoid(k**2*Pp*acc,k)*L*(L+1)/(2*np.pi)*T0**2
for R in (4.0,5.0,6.0):
    key=f'R{R:g}'
    if key in S and 'D1' in S[key]:continue
    F=AFwd(R=R,lmax=M.LX,kmax=200,nk=500);p=dict(p0,chiw=R-0.056,sw=0.05);t=F.terms(p)
    Dl=lim(F,p,np.arange(M.LX+1,LMAX+1));D1=np.r_[0,0,t['scalar']+t['mlens'],Dl]
    rr=S.get(key,{}).get('ratio',[])
    for L in NODES[len(rr):]:
        rr.append(float(exact(F,p,L)/lim(F,p,np.array([L]))[0]));S[key]=dict(ratio=rr);json.dump(S,open(C
        ↪ K,'w'))   # checkpoint per
        ↪ node
    c=np.ones(LMAX+1);sel=(ll>M.LX)&(ll<=400);c[sel]=np.exp(np.interp(ll[sel],NODES,np.log(rr)));D1=D1*c
    h=float(D1[150:350].max());lev=MULT*h;i=[j for j in range(8) if lev[j]<=pl1<=lev[j+1]]
    S[key]=dict(ratio=rr,D1=D1.tolist(),peaks=pk(D1),first_peak_height_x1=h,first_peak_l=int(150+np.argma
    ↪ x(D1[150:350])),bracket=[(float(MULT[j]),float(MULT[j+1])) for j in
    ↪ i],mult_to_match=pl1/h,jump_l50=float(D1[51]/D1[50]))
    json.dump(S,open(CK,'w'));print(key,S[key]['peaks'],'h %.4g'%h,S[key]['bracket'],'mult
    ↪ %.3g'%(pl1/h),'D51/D50 %.3f'%S[key]['jump_l50'],np.round(rr,3).tolist(),flush=True)
fig,axs=plt.subplots(1,3,figsize=(21,7));cs=plt.cm.viridis(np.linspace(0,1,9));l=ll[2:]
for ax,R in zip(axs,(4,5,6)):
    s=S[f'R{R:g}'];D1=np.array(s['D1'])
    ax.plot(l,A.Lt[2:],'k-',lw=1.8,label='ΛCDM');ax.plot(A.Pf[:,0],A.Pf[:,1],'.',color='grey',ms=1.5,alph
    ↪ a=.6,label='Planck
    ↪ 2018')
    for j,m in enumerate(MULT):
        Dm=m*D1[2:];Dr=Dm*np.random.default_rng(42+j).chisquare(2*l+1)/(2*l+1);ax.plot(l,Dm,'-',color=cs[
        ↪ j],lw=.6,alpha=.3);ax.plot(l,Dr,'-',color=cs[j],lw=.6,label=f'×{m:.3g}')
    ax.set_xscale('log');ax.set_yscale('log');ax.set_ylim(1,1e6);ax.set_xlim(2,2500);ax.set_xlabel('□');a
    ↪ x.set_ylabel('D_□
    ↪ (μK²)')
    ax.set_title(f'R={R} Gpc, shell {R-0.056:.3f}±0.05; peaks {s["peaks"]}\n1st-peak multiplier
    ↪ {s["mult_to_match"]:.2f} (bracket
    ↪ ×{s["bracket"][0][0]:.3g}–×{s["bracket"][0][1]:.3g})',fontsize=9);ax.legend(loc='lower
    ↪ center',ncol=6,fontsize=6,framealpha=.85)
fig.suptitle(f'acoustic free_W ladder, exact/Limber-corrected (one realization per level, faint = mean).
↪ Planck peaks {PlP}',fontsize=10);fig.tight_layout();fig.savefig('acoustic_ladder_456.png',dpi=100)
```

/workspace/coepdf/book/gitsrc/acoustic_model.py

```
# Acoustic-feature stochastic field: exact projection for l<=LX, Limber for l>LX (moving-lens term
↪ dominant; SW/ISW scalar kept only in exact part).
import json,numpy as np
from fwd import Fwd,T0,Gc,rho
LX=50;LMAX=2500
class AFwd(Fwd):
    def Pd(s,p,k=None):
        k=s.k if k is None else k
        return 10**p['lgA']*k**p['n']*(1+p['Aw']*np.cos(k*p['rs']+p['phi']))*np.exp(-(k/p['kD'])**2)
EX=AFwd(lmax=LX,kmax=200,nk=500)
lL=np.arange(LX+1,LMAX+1)
def limber(p,l=lL,n=600):
    a=max(0.01,p['chiw']-6*p['sw']);b=min(EX.R,p['chiw']+6*p['sw'])
    if b<=a+1e-6:return np.zeros(len(l))
    ch=np.linspace(a,b,n);KM=np.interp(ch,EX.ch,EX.KM)
    W=np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2);Wd=-W*(ch-p['chiw'])/p['sw']**2
    k=(l[:,None]+0.5)/ch[None];Pp=(4*np.pi*Gc*rho)**2*EX.Pd(p,k)/k**4
```

```
    I=(KM*W)**2*k**2/3*Pp/ch**2   # Limber: only transverse k; <(k.u)^2> over sky = k^2/3; W' term is part of the radial (suppressed) modes
    return np.trapezoid(I,ch,axis=1)*l*(l+1)/(2*np.pi)*T0**2
def Dfield(p):
    t=EX.terms(p);return np.r_[0,0,t['scalar']+t['mlens'],limber(p)]   # index = l, 0..2500
Dfix=np.zeros(LMAX+1);Dfix[2:31]=json.load(open('/workspace/redweb2/stochfield/fit_l30_bg.json'))['Dfix']
def Dmodel(p):Df=Dfield(p);return Df+Dfix,Df
from scipy.special import spherical_jn
def exact_nodes(p,nodes=(51,60,75,90,110,130,150,175,200,240,280,330,400)):
    """large-l-limit exact projection (validated vs Fwd exact to <2% at l 20-50) at nodes; returns ratio exact/limber"""
    ch,KM,dch=EX.ch,EX.KM,EX.dch;W=np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2);Wd=-W*(ch-p['chiw'])/p['sw']**2;m=W>1e-4;ch,KM,W,Wd=ch[m],KM[m],W[m],Wd[m]
    r=[]
    for L in nodes:
        k=np.linspace(L/ch.max()*0.7,min((L+60)/ch.min()*2.5,4000),2500);x=k[:,None]*ch[None]
        A=spherical_jn(L,x)@(KM*W)*dch;Bp=spherical_jn(L+1,x)@(KM*Wd)*dch;Bm=spherical_jn(L-1,x)@(KM*Wd)*dch
        acc=k**2*A**2/3+(Bp**2+Bm**2)/6+k*A*(Bp-Bm)/3;Pp=(4*np.pi*Gc*rho)**2*EX.Pd(p,k)/k**4
        C=2/np.pi*np.trapezoid(k**2*Pp*acc,k)*L*(L+1)/(2*np.pi)*T0**2;r.append(C/limber(p,np.array([L]))[0])
    return np.array(nodes),np.array(r)
def corr_curve(p):
    n,r=exact_nodes(p);c=np.ones(LMAX+1);ll=np.arange(LMAX+1);sel=(ll>LX)&(ll<=400);c[sel]=np.exp(np.interp(ll[sel],n,np.log(r)));return c,n,r
```

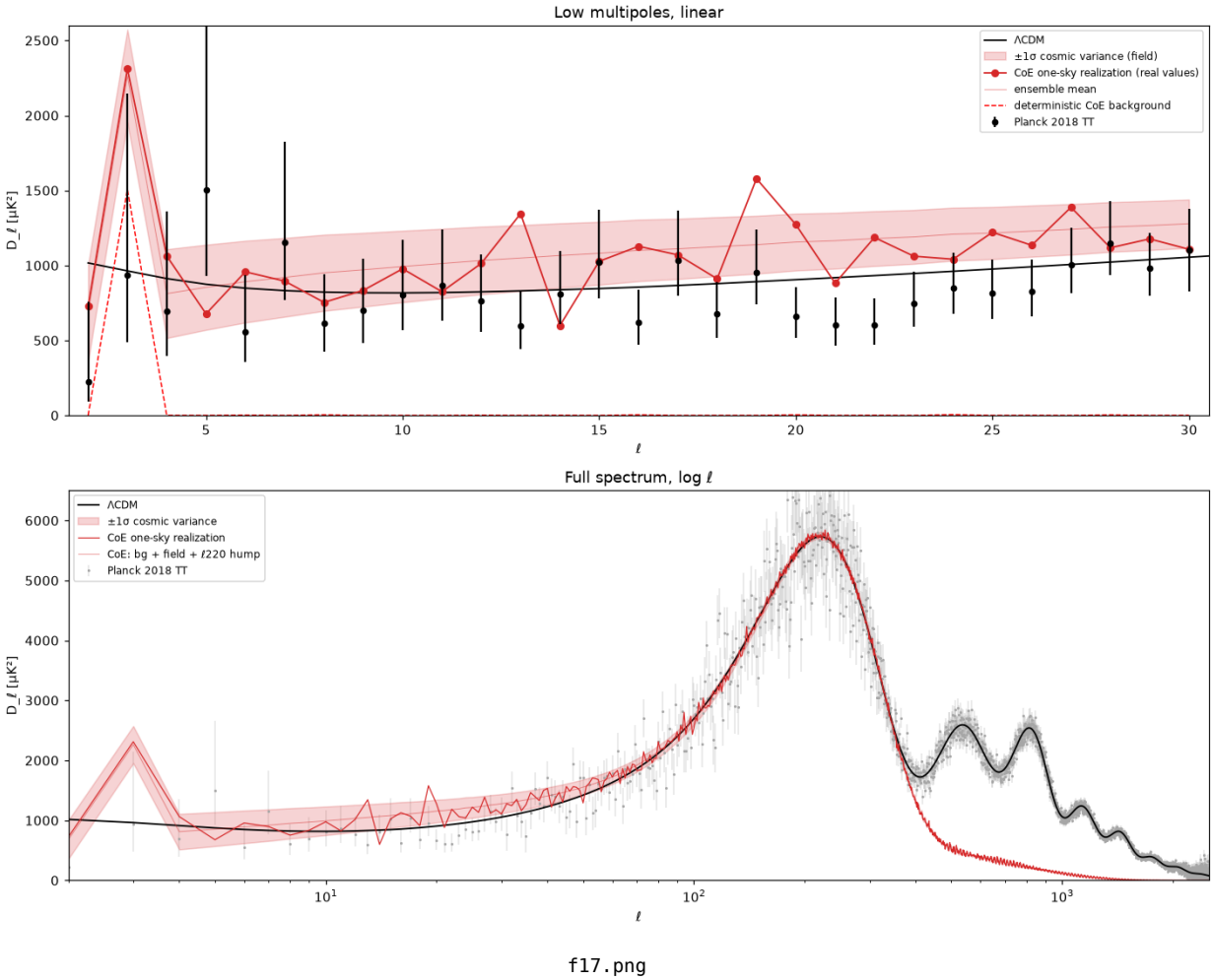
5. Low multipoles: LCDM, Planck, CoE one-sky

Originally made: Thu Oct 22 2026, 12:42 AM CT (*git commit c18eae2, adding stochfield/combo10.png (pixel-identical to f17.png)*)
Paper figure number: this morning Fig. 20; now Fig. 27
Files: `f17.png`
Code status: original (git)

Notes

CoE one-sky realisation in its cosmic-variance band. Script (commit c18eae2, work/2026-10-21_22/stochfield/combo10.py) ends with savefig('combo10.png'); that committed PNG is pixel-for-pixel identical to f17.png.

Paper caption (caveats as stated): Low multipoles (linear axes): ΛCDM (black), Planck (grey), CoE one-sky realisation (red) in its $\pm 1\sigma$ cosmic-variance band (Eq. 8) with faint ensemble mean. Bottom: full spectrum.



`f17.png`

Computing chain (run order, full listings)

/workspace/coepdf/book/gitsrc/combo10.py

```
import json,glob,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
d=json.load(open('fit_l30_bg.json'));Df=np.load('Df_2500.npy');L=np.arange(2,2501)
bg=np.zeros(len(L));B=np.array(d['Dfix']);bg[:len(B)]=B
def h(l,a,c,s1,s2):return a*np.where(l<c,np.exp(-.5*((l-c)/s1)**2),np.exp(-.5*((l-c)/s2)**2))
tot=bg+Df+h(L,4973.3,219.6,83.9,92.4)
th=np.loadtxt(glob.glob('/workspace/planck/*minimum*')[0]);P=np.loadtxt('/workspace/planck/COM_PowerSpect_CMB-TT-full_R3.01.txt')
MC=np.load('../realweb/mc/mc_mean_passing.npz')['D4.0'];mc=MC.mean(0);q16,q84=np.percentile(MC,[16,84],0);lm=np.arange(MC.shape[1])
rg=np.random.default_rng(1);nu=2*L+1;real=tot-Df+Df*rg.chisquare(nu)/nu
fig,ax=plt.subplots(2,1,figsize=(13,10))
a=ax[0];Pl=P[P[:,0]<=30];a.errorbar(Pl[:,0],Pl[:,1],yerr=[Pl[:,2],Pl[:,3]],fmt='ko',ms=4,label='Planck 2018 TT',zorder=5)
a.plot(th[:,0],th[:,1],'k-',lw=1.2,label='ΛCDM');s=L<=30;cv=np.sqrt(2/(2*L+1))*Df
a.fill_between(L[s],(tot-cv)[s],(tot+cv)[s],color='C3',alpha=.2,label='±1σ cosmic variance (field)')
a.plot(L[s],real[s],'C3-o',ms=5,lw=1.2,label='CoE one-sky realization (real values)');a.plot(L[s],tot[s],'C3-',lw=.6,alpha=.5,label='ensemble mean')
a.plot(L[s],bg[s],'r--',lw=1,label='deterministic CoE background')
a.set(xlim=(1.5,30.5),ylim=(0,2600),xlabel='□',ylabel='D_□ [μK²]',title='Low multipoles, linear');a.legend(loc='upper right',fontsize=8)
a=ax[1];a.errorbar(P[:,0],P[:,1],yerr=[P[:,2],P[:,3]],fmt='.',color='0.65',ms=2,lw=.3,label='Planck 2018 TT',zorder=1)
a.plot(th[:,0],th[:,1],'k-',lw=1.2,label='ΛCDM');a.fill_between(L,tot-cv,tot+cv,color='C3',alpha=.2,label='±1σ cosmic variance');a.plot(L,real,'-',color='C3',lw=.8,label='CoE one-sky realization');a.plot(L,tot,'C3-',lw=.6,alpha=.5,label='CoE: bg + field + □220 hump')
a.set(xscale='log',xlim=(2,2500),ylim=(0,6500),xlabel='□',ylabel='D_□ [μK²]',title='Full spectrum, log □');a.legend(loc='upper left',fontsize=8)
fig.tight_layout();fig.savefig('combo10.png',dpi=110)
```

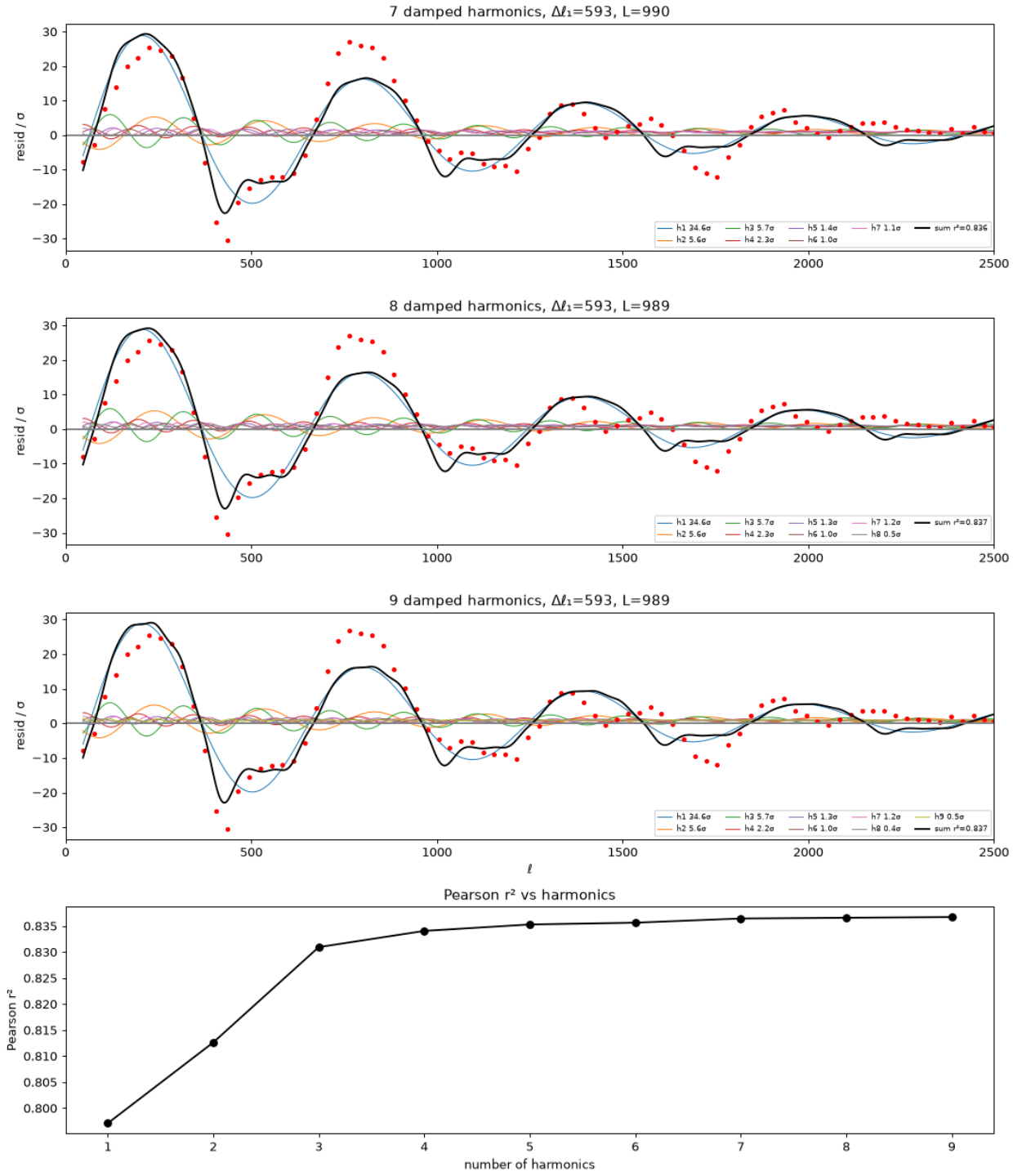
6. Damped-harmonic fits to eps_l

Originally made: Thu Oct 22 2026, 12:42 AM CT (*git commit c18eae2, adding stochfield/resid_damped_7to9.png (pixel-identical to f18.png)*)
Paper figure number: this morning Fig. 23; now Fig. 24
Files: `f18.png`
Code status: not found

Notes

Fits with 7, 8, 9 harmonics to eps_l. Original name resid_damped_7to9.png found in git; no .py in any commit or under /workspace contains resid_damped, so its script was never saved (likely run inline).

Paper caption (caveats as stated): Damped-harmonic fits to ϵ_ℓ with 7, 8, 9 harmonics ($\Delta\ell = 593$, $L = 990$) and Pearson r^2 versus number of harmonics (bottom).



f18.png

Computing chain (run order, full listings)

Generating script not found. Searched all `.py` files under /workspace (excluding virtualenvs) for the image filename and its known aliases, and the git repos.

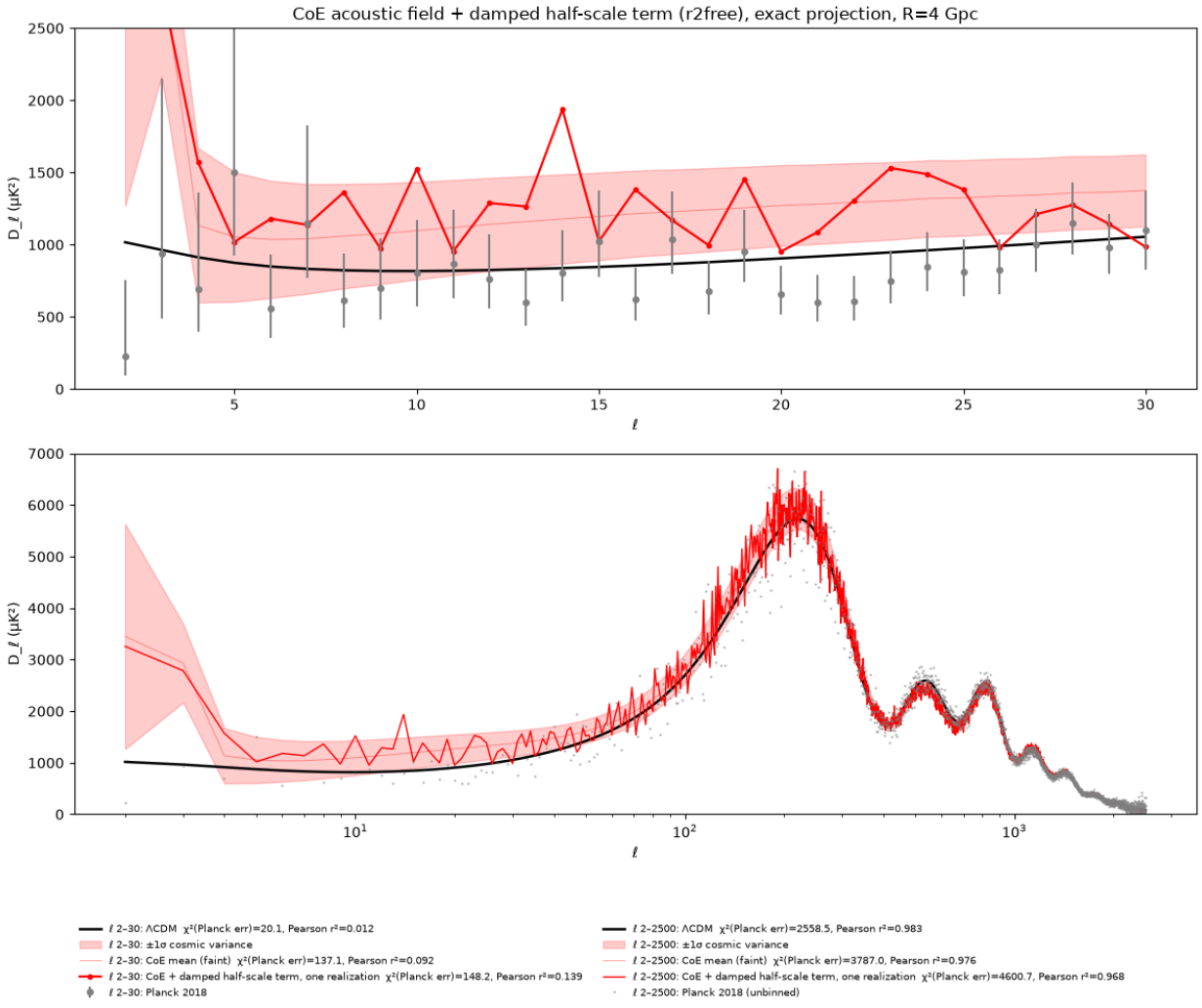
7. Final CMB model vs Planck TT

Originally made: Thu Oct 22 2026, 11:14 AM CT (*git commit 2b5677e, adding acoustic_half_cl_fixed.png*)
Paper figure number: this morning Fig. 24; now Fig. 25
Files: `f01.png`
Code status: original (git)

Notes

CoE acoustic field plus damped half-scale term vs Planck TT, exact projection to l = 2500. Depends on the stochastic-field / acoustic fits (#2, #3). Helper modules half_fit, acoustic_fit and acoustic_model are included below.

Paper caption (caveats as stated): Final model: acoustic field + damped half-scale term (r_2 free), exact/Limber-corrected projection, $R \approx 4\,\mathrm{Gpc}$, plus D_ℓ^{fix}. Top: low ℓ (linear). Bottom: full spectrum. ΛCDM black, Planck grey, CoE one-sky realisation red in its $\pm 1\sigma$ band. $\chi^2_{2-2500} = 3{,}787$, Pearson $r^2 = 0.976$.



f01.png

Computing chain (run order, full listings)

/workspace/redweb2/stochfield/half_figs_fixed.py

```
import json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from scipy.signal import find_peaks,savgol_filter
import half_fit as H;import acoustic_fit as A;import acoustic_model as M
ll=np.arange(M.LMAX+1);Lt=A.Lt;Pf=A.Pf;Pb=A.Pb;J=H.J
def peaks(D,lo=100):
    s=savgol_filter(D[lo:],61,3);pk,_=find_peaks(s,prominence=20);pk=(pk+lo)[:5];return pk.tolist(),[round(float(s[p-lo]),0) for p in pk]
def real(Df,seed=42):return Df*np.random.default_rng(seed).chisquare(2*np.maximum(ll,1)+1)/(2*np.maximum(ll,1)+1)
out={};models={}
Df=M.Dfield(J['free_W']['best']['p'])*H.CORR0;models['old']=(Df+M.Dfix,Df)
Cp=json.load(open('half_ckpt_r2free.json'));H.CORR[:]=Cp['corr'];Dp=M.Dfield(Cp['best']['p'])*H.CORR;models['prev']=(Dp+M.Dfix,Dp)
modes=[m for m in ('half','r2free') if 'best' in json.load(open(f'fixed_ckpt_{m}.json'))]
for m in modes:
    C=json.load(open(f'fixed_ckpt_{m}.json'));H.CORR[:]=C['corr'];Df=M.Dfield(C['best']['p'])*H.CORR;models[m]=(Df+M.Dfix,Df);out[m]=C['best']
bm=min(modes,key=lambda m:out[m]['l2_2500']['chi2_planck'])
sPl=savgol_filter(Pf[:,1][98:],61,3);pk,_=find_peaks(sPl,prominence=20);pk=pk[:5];out['peaks_planck']=((pk+100).tolist(),[round(float(sPl[p]),0) for p in pk])
for k,(D,Df) in models.items():out.setdefault(k,{})['peaks']=peaks(D)
out['peaks_lcdm']=peaks(Lt);out['lcdm']=J['lcdm']
D,Df=models[bm];Dr=M.Dfix+real(Df);st={k:A.stats(Dr,Df,*r) for k,r in dict(l2_30=(2,30),l2_2500=(2,2500)).items()};out[bm]['realization_seed42']=st
fig,ax=plt.subplots(2,1,figsize=(12,10))
for a,(l0,l1),key in zip(ax,((2,30),(2,M.LMAX)),('l2_30','l2_2500')):
    m=(ll>=l0)&(ll<=l1);a.plot(ll[m],Lt[m],'k-',lw=1.8,label=f'ΛCDM  χ²(Planck err)={J["lcdm"][key]["chi2_planck"]:.1f}, Pearson r²={J["lcdm"][key]["r2"]:.3f}')
    cv=np.sqrt(2/(2*np.maximum(ll,1)+1))*Df;a.fill_between(ll[m],(D-cv)[m],(D+cv)[m],color='r',alpha=.2,label='±1σ cosmic variance')
    a.plot(ll[m],D[m],'r-',lw=.7,alpha=.35,label=f'CoE mean (faint)  χ²(Planck err)={out[bm][key]["chi2_planck"]:.1f}, Pearson r²={out[bm][key]["r2"]:.3f}')
    a.plot(ll[m],Dr[m],'r-',lw=1.0 if l1>30 else 1.6,marker='' if l1>30 else 'o',ms=3,label=f'CoE + damped half-scale term, one realization  χ²(Planck err)={st[key]["chi2_planck"]:.1f}, Pearson r²={st[key]["r2"]:.3f}')
    mm=(Pf[:,0]>=l0)&(Pf[:,0]<=l1)
    if l1==30:a.errorbar(Pf[mm,0],Pf[mm,1],yerr=[Pf[mm,2],Pf[mm,3]],fmt='o',color='grey',ms=4,label='Planck 2018');a.set_ylim(0,2500)
    else:a.plot(Pf[mm,0],Pf[mm,1],'.',color='grey',ms=1.5,alpha=.5,label='Planck 2018 (unbinned)');a.set_xscale('log');a.set_ylim(0,7000)
    a.set_xlabel('□');a.set_ylabel('D_□ (μK²)')
ax[0].set_title(f'CoE acoustic field + damped half-scale term ({bm}), exact projection, R=4 Gpc');h0,l0=a[0].get_legend_handles_labels();h1,l1=ax[1].get_legend_handles_labels();fig.legend(h0+h1,['□ 2–30: '+x for x in l0]+['□ 2–2500: '+x for x in l1],loc='lower center',ncol=2,fontsize=7.5,frameon=False);fig.tight_layout(rect=(0,0.13,1,1));fig.savefig('acoustic_half_cl_fixed.png',dpi=100);plt.close()
fig,a=plt.subplots(figsize=(12,4.8))
for k,c,lab in (('old','tab:orange','old CoE (free_W)'),('prev','tab:purple','previous half-scale (□=50 step)'),(bm,'r','new CoE (+half-scale term)')):
```

```
    a.plot(Pb[:,0],(Pb[:,1]-A.binned(models[k][0]))/Pb[:,2],'o-',color=c,ms=3,label=f'Planck − {lab}')
a.plot(Pb[:,0],(Pb[:,1]-A.binned(Lt))/Pb[:,2],'ko-',ms=3,label='Planck − ΛCDM');a.axhline(0,color='grey');a.set_xlabel('□ (binned, linear)');a.set_ylabel('residual / σ_Planck');fig.legend(loc='lower center',ncol=3,fontsize=8,frameon=False);fig.tight_layout(rect=(0,0.09,1,1));fig.savefig('acoustic_half_resid_fixed.png',dpi=100);plt.close()
# leftover period: Lomb-Scargle on new residual
from scipy.signal import lombscargle
r=(Pb[:,1]-A.binned(models[bm][0]))/Pb[:,2];per=np.linspace(100,1500,1400);pw=lombscargle(Pb[:,0],r-r.mean(),2*np.pi/per);out['leftover_period']=float(per[pw.argmax()]);out['resid_rms']={k:float(np.sqrt(np.mean(((Pb[:,1]-A.binned(v[0]))/Pb[:,2])**2))) for k,v in models.items()}
out['resid_rms']['lcdm']=float(np.sqrt(np.mean(((Pb[:,1]-A.binned(Lt))/Pb[:,2])**2)));out['best_mode']=bm
np.save('resid_half_fixed.npy',np.c_[Pb[:,0],r]);json.dump(out,open('acoustic_half_fixed.json','w'),indent=1)
print(json.dumps({k:(v if k not in modes else {kk:v[kk] for kk in ('p','l2_2500','l2_30','peaks')}) for k,v in out.items() if k!='lcdm'},indent=0,default=str))
```

/workspace/redweb2/stochfield/half_fit.py

```
# Damped half-scale term: P = P0 k^n [1 + A cos(k rs+phi) + B e^{-k/kB} cos(k r2 + phi2)] e^{-(k/kD)^2};  r2 = rs/2 ('half') or free ('r2free')
# Checkpointed: each Nelder-Mead start writes half_ckpt_<mode>.json; rerun resumes.
import sys,os,json,numpy as np
from scipy.optimize import minimize
from multiprocessing import Pool
import acoustic_model as M
import acoustic_fit as A
def Pd(s,p,k=None):
    k=s.k if k is None else k;r2=p.get('r2',p['rs']/2)
    return 10**p['lgA']*k**p['n']*(1+p['Aw']*np.cos(k*p['rs']+p['phi'])+p.get('B',0)*np.exp(-k/p.get('kB',1e9))*np.cos(k*r2+p.get('phi2',0)))*np.exp(-(k/p['kD'])**2)
M.AFwd.Pd=Pd
J=json.load(open('acoustic_fit.json'));OLD=J['free_W']['best'];CORR0=np.array(J['free_W']['corr'])
NM=['lgA','n','Aw','rs','phi','lkD','chiw','lsw','B','phi2','lkB','r2']
def P(x,mode):
    p=dict(lgA=x[0],n=x[1],Aw=x[2],rs=x[3],phi=x[4]%(2*np.pi),kD=10**x[5],chiw=x[6],sw=10**x[7],B=x[8],ph
    i2=x[9]%(2*np.pi),kB=10**x[10])
    if mode=='r2free':p['r2']=x[11]
    return p
def ok(x,mode):
    b=-25<x[0]<5 and -3<x[1]<6 and 0<=x[2]<=1 and 0.015<x[3]<0.25 and 0.5<x[5]<3.5 and 0.3<x[6]<=4.0 and  np.log10(0.05)<x[7]<0.4 and 0<=x[8]<=3 and 0.5<x[10]<4.5
    return b and (mode!='r2free' or 0.01<x[11]<0.2)
CORR=CORR0.copy()
def obj(x,mode):
    if not ok(x,mode):return 1e15
    p=P(x,mode);Df=M.Dfield(p)*CORR;return A.chi2_fit(Df+M.Dfix)
def run(a):
    x0,mode,fev=a;r=minimize(obj,np.array(x0,float),args=(mode,),method='Nelder-Mead',options=dict(maxfev=fev,xatol=1e-5,fatol=1e-3,adaptive=True));return  x0,r.x.tolist(),float(r.fun)
def full(x,mode):
    p=P(x,mode);Df=M.Dfield(p)*CORR;D=Df+M.Dfix
    return dict(x=list(x),p=p,chi2_fit=A.chi2_fit(D),**{k:A.stats(D,Df,*r) for k,r in  dict(l2_30=(2,30),l2_2500=(2,2500),l30_2500=(30,2500)).items()})
if __name__=='__main__':
    mode=sys.argv[1];stage=sys.argv[2];ck=f'half_ckpt_{mode}.json'
    C=json.load(open(ck)) if os.path.exists(ck) else dict(starts=[],stage2=[],corr=None)
```

```
    if C['corr']:CORR[:]=C['corr']
    xo=OLD['x']
    if stage=='1':
        # seed B from residual amplitude: |Planck-CoE| in muK2 / D_CoE at first swings, phi2 grid incl. residual phase 5.55
        R=np.load('resid_free_W.npy');sig=np.interp(R[:,0],A.Pb[:,0],A.Pb[:,2]);Dold=M.Dfield(OLD['p'])*C ORR0+M.Dfix
        m=R[:,0]<700;B0=float(np.median(np.abs(R[m,1])*sig[m]/np.interp(R[m,0],np.arange(len(Dold)),Dold) ))*2
        print('B0 seed',B0,flush=True)
        seeds=[]
        for ph2 in (5.55,5.55-np.pi,0.0,np.pi/2,np.pi,3*np.pi/2):
            for B in (B0,0.5*B0):
                s=xo+[B,ph2,np.log10(250)]+([xo[3]/2] if mode=='r2free' else [])
                if mode=='r2free':s[11]*= (1.0 if B==B0 else 1.05)
                seeds.append(s)
        done=[tuple(np.round(s[0],6)) for s in C['starts']];todo=[s for s in seeds if tuple(np.round(s,6)) not in done]
        with Pool(8) as pl:
            for x0,x,f in pl.imap_unordered(run,[(s,mode,1500) for s in todo]):
                C['starts'].append([x0,x,f]);json.dump(C,open(ck,'w'));print('start',round(f,1),np.round( x,4).tolist(),flush=True)
    if stage=='2':   # refresh Limber->exact correction at current best, polish top 4
        C['starts'].sort(key=lambda t:t[2]);best=C['starts'][0][1]
        c,nn,rr=M.corr_curve(P(best,mode));CORR[:]=c;C['corr']=c.tolist();print('corr',np.round(rr,3).tol ist(),flush=True)
        with Pool(4) as pl:
            for x0,x,f in pl.imap_unordered(run,[(s[1],mode,2500) for s in C['starts'][:4]]):
                C['stage2'].append([x0,x,f]);json.dump(C,open(ck,'w'));print('polish',round(f,1),flush=Tr ue)
    if stage=='3':
        b=min(C['stage2'] or C['starts'],key=lambda t:t[2])[1];C['best']=full(b,mode);json.dump(C,open(ck,'w'),indent=1);print(C['best'])
```

/workspace/redweb2/stochfield/acoustic_fit.py

```
import sys,json,numpy as np
from scipy.optimize import minimize
from multiprocessing import Pool
from acoustic_model import *
Pf=np.loadtxt('/workspace/planck/COM_PowerSpect_CMB-TT-full_R3.01.txt');Pf=Pf[(Pf[:,0]>=2)&(Pf[:,0]<=LMAX )]
Pb=np.loadtxt('/workspace/planck/COM_PowerSpect_CMB-TT-binned_R3.01.txt');lo=30+30*np.arange(len(Pb));hi= np.minimum(lo+29,LMAX)
th=np.loadtxt('/workspace/planck/COM_PowerSpect_CMB-base-plikHM-TTTEEE-lowl-lowE-lensing-minimum-theory_R 3.01.txt');Lt=np.zeros(LMAX+1);Lt[2:]=np.interp(np.arange(2,LMAX+1),th[:,0],th[:,1])
low=Pf[Pf[:,0]<=29]
def binned(D):return np.array([D[a:b+1].mean() for a,b in zip(lo,hi)])
def chi2_fit(D,Df=None,cv=False):   # low l unbinned (2-29) + binned l>=30 ; Planck errors (side-dependent), optional cosmic variance of field
    l=low[:,0].astype(int);d=D[l];e=np.where(d>low[:,1],low[:,3],low[:,2])**2
    if cv:e=e+2/(2*l+1)*Df[l]**2
    c=np.sum((d-low[:,1])**2/e);db=binned(D);eb=Pb[:,2]**2
    if cv:eb=eb+2/((2*Pb[:,0]+1)*30)*binned(Df)**2
    return float(c+np.sum((db-Pb[:,1])**2/eb))
def stats(D,Df,a,b):   # unbinned full file, l in [a,b]
    m=(Pf[:,0]>=a)&(Pf[:,0]<=b);l=Pf[m,0].astype(int);d=D[l];e=np.where(d>Pf[m,1],Pf[m,3],Pf[m,2])**2
```

```
    c=float(np.sum((d-Pf[m,1])**2/e));ccv=float(np.sum((d-Pf[m,1])**2/(e+2/(2*l+1)*Df[l]**2)));return dict(chi2_planck=c,chi2_cv=ccv,r2=float(np.corrcoef(d,Pf[m,1])[0,1]**2),npts=int(m.sum()))
NM=['lgA','n','Aw','rs','phi','lkD','chiw','lsw']
FIXW=dict(chiw=4.0,sw=0.1)
def P(x,fix):
    p=dict(lgA=x[0],n=x[1],Aw=x[2],rs=x[3],phi=x[4]%(2*np.pi),kD=10**x[5])
    if fix:p.update(FIXW)
    else:p.update(chiw=x[6],sw=10**x[7])
    return p
def ok(x,fix):
    b=-25<x[0]<5 and -3<x[1]<6 and 0<=x[2]<=1 and 0.015<x[3]<0.25 and 0.5<x[5]<3.5
    return b and (fix or (0.3<x[6]<=4.0 and np.log10(0.05)<x[7]<0.4))
def reg(p):return p['Aw']**2+(0.1/p['sw'])**2+EX.sig_delta(p)**2
CORR=np.ones(LMAX+1)
def obj(x,fix,lam):
    if not ok(x,fix):return 1e15
    p=P(x,fix);Df=Dfield(p)*CORR;return chi2_fit(Df+Dfix)+lam*reg(p)
def run(a):
    x0,fix,lam,it=a;x0=np.array(x0,float)
    if lam==0 and it==0:   # level the amplitude first
        p=P(x0,fix);Df=Dfield(p);x0[0]+=np.log10(np.median(Pb[:,1])/max(np.median(binned(Df)),1e-300))
    r=minimize(obj,x0,args=(fix,lam),method='Nelder-Mead',options=dict(maxiter=2500,maxfev=4000,xatol=1e-5,fatol=1e-3,adaptive=True));return r.x.tolist(),float(r.fun)
if __name__=='__main__':
    out={'params':NM,'fixedW':FIXW,'lcdm':{k:stats(Lt,Lt,*r) for k,r in dict(l2_30=(2,30),l2_2500=(2,2500),l30_2500=(30,2500)).items()},'lcdm_fitmeasure':chi2_fit(Lt)}
    out['lcdm']['fit_measure_chi2']=out['lcdm_fitmeasure']
    print('LCDM',out['lcdm'],flush=True)
    res={}
    for fix in (False,True):
        nm='fixed_shell' if fix else 'free_W';seeds=[]
        for rs in (0.042,0.063,0.084,0.105):
            for ph in (0,np.pi/2,np.pi,3*np.pi/2):
                for W in ([None] if fix else [(4.0,np.log10(0.1)),(3.5,np.log10(0.3))]):
                    x=[-9,0.5,0.6,rs,ph,np.log10(400)]+([] if fix else list(W));seeds.append(x)
        CORR[:]=1
        with Pool(8) as pl:R=pl.map(run,[(s,fix,0,0) for s in seeds])
        R.sort(key=lambda t:t[1]);print(nm,'top starts',[(round(f,1),np.round(x,3).tolist()) for x,f in R[:4]],flush=True)
        # Limber -> exact-node correction iteration (2 passes)
        best=R[0][0];hist=[]
        for it in range(2):
            c,nn,rr=corr_curve(P(best,fix));CORR[:]=c
            with Pool(4) as pl:RR=pl.map(run,[(x,fix,0,1) for x,_ in R[:4]])
            best,f=min(RR,key=lambda t:t[1]);hist.append(dict(nodes=nn.tolist(),ratio=rr.tolist(),chi2=f));print(nm,'corr pass',it,f,np.round(rr,3).tolist(),flush=True)
        c,nn,rr=corr_curve(P(best,fix));CORR[:]=c
        Lc=[]
        for lam in (0,10,100,1000):
            with Pool(2) as pl:RR=pl.map(run,[(best,fix,lam,1),(R[1][0],fix,lam,1)])
            x,f=min(RR,key=lambda t:t[1]);p=P(x,fix);Df=Dfield(p)*CORR;D=Df+Dfix
            Lc.append(dict(lam=lam,x=x,p=p,obj=f,chi2_fit=chi2_fit(D),chi2_fit_cv=chi2_fit(D,Df,True),reg=reg(p),sig_delta=EX.sig_delta(p),
                **{k:stats(D,Df,*r) for k,r in dict(l2_30=(2,30),l2_2500=(2,2500),l30_2500=(30,2500)).items()}))
            print(nm,'lam',lam,round(Lc[-1]['chi2_fit'],1),{k:round(v,4) for k,v in p.items()},flush=True)
        res[nm]=dict(starts=[(x,f) for x,f in R],corr_hist=hist,corr=CORR.tolist(),Lcurve=Lc)
```

```
        json.dump(dict(out,**res),open('acoustic_fit.json','w'),indent=1)
```

/workspace/redweb2/stochfield/acoustic_model.py

```
# Acoustic-feature stochastic field: exact projection for l<=LX, Limber for l>LX (moving-lens term dominant; SW/ISW scalar kept only in exact part).
import json,numpy as np
from fwd import Fwd,T0,Gc,rho
LX=50;LMAX=2500
class AFwd(Fwd):
    def Pd(s,p,k=None):
        k=s.k if k is None else k
        return 10**p['lgA']*k**p['n']*(1+p['Aw']*np.cos(k*p['rs']+p['phi']))*np.exp(-(k/p['kD'])**2)
EX=AFwd(lmax=LX,kmax=200,nk=500)
lL=np.arange(LX+1,LMAX+1)
def limber(p,l=lL,n=600):
    a=max(0.01,p['chiw']-6*p['sw']);b=min(EX.R,p['chiw']+6*p['sw'])
    if b<=a+1e-6:return np.zeros(len(l))
    ch=np.linspace(a,b,n);KM=np.interp(ch,EX.ch,EX.KM)
    W=np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2);Wd=-W*(ch-p['chiw'])/p['sw']**2
    k=(l[:,None]+0.5)/ch[None];Pp=(4*np.pi*Gc*rho)**2*EX.Pd(p,k)/k**4
    I=(KM*W)**2*k**2/3*Pp/ch**2   # Limber: only transverse k; <(k.u)^2> over sky = k^2/3; W' term is part of the radial (suppressed) modes
    return np.trapezoid(I,ch,axis=1)*l*(l+1)/(2*np.pi)*T0**2
def Dfield(p):
    t=EX.terms(p);return np.r_[0,0,t['scalar']+t['mlens'],limber(p)]   # index = l, 0..2500
Dfix=np.zeros(LMAX+1);Dfix[2:31]=json.load(open('/workspace/redweb2/stochfield/fit_l30_bg.json'))['Dfix']
def Dmodel(p):Df=Dfield(p);return Df+Dfix,Df
from scipy.special import spherical_jn
def exact_nodes(p,nodes=(51,60,75,90,110,130,150,175,200,240,280,330,400)):
    """large-l-limit exact projection (validated vs Fwd exact to <2% at l 20-50) at nodes; returns ratio exact/limber"""
    ch,KM,dch=EX.ch,EX.KM,EX.dch;W=np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2);Wd=-W*(ch-p['chiw'])/p['sw']**2;m=W>1e-4;ch,KM,W,Wd=ch[m],KM[m],W[m],Wd[m]
    r=[]
    for L in nodes:
        k=np.linspace(L/ch.max()*0.7,min((L+60)/ch.min()*2.5,4000),2500);x=k[:,None]*ch[None]
        A=spherical_jn(L,x)@(KM*W)*dch;Bp=spherical_jn(L+1,x)@(KM*Wd)*dch;Bm=spherical_jn(L-1,x)@(KM*Wd)*dch
        acc=k**2*A**2/3+(Bp**2+Bm**2)/6+k*A*(Bp-Bm)/3;Pp=(4*np.pi*Gc*rho)**2*EX.Pd(p,k)/k**4
        C=2/np.pi*np.trapezoid(k**2*Pp*acc,k)*L*(L+1)/(2*np.pi)*T0**2;r.append(C/limber(p,np.array([L]))[0])
    return np.array(nodes),np.array(r)
def corr_curve(p):
    n,r=exact_nodes(p);c=np.ones(LMAX+1);ll=np.arange(LMAX+1);sel=(ll>LX)&(ll<=400);c[sel]=np.exp(np.interp(ll[sel],n,np.log(r)));return
    c,n,r
```

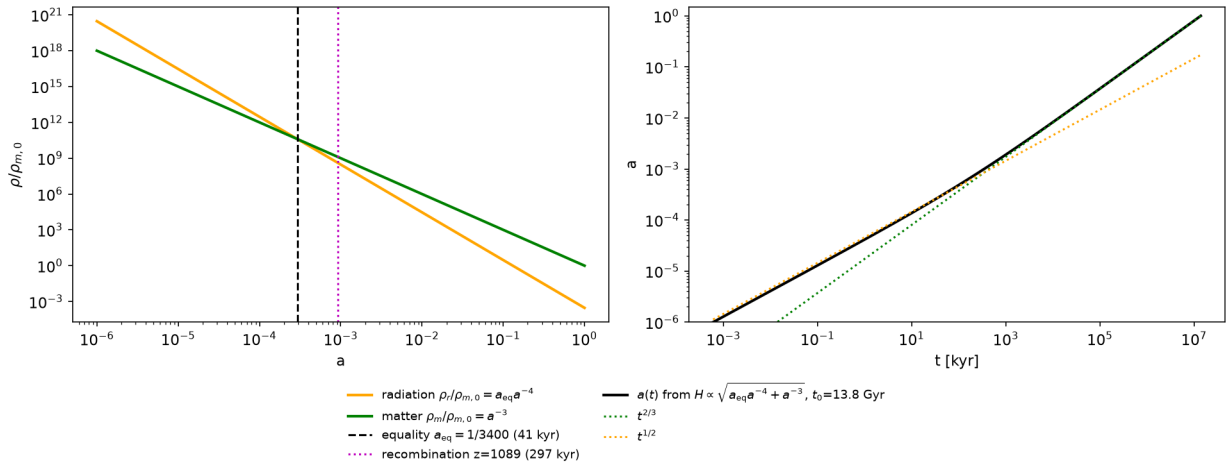
8. Density and t vs a (radiation + matter)

Originally made: Fri Oct 23 2026, 12:37 AM CT (*git commit 54f717f, adding math_pdf/fig/d4_radmat.png*)
Paper figure number: this morning Fig. 3; now Fig. 3
Files: `d4_radmat.png`
Code status: original (git)

Notes

Eq. 1 density and Eq. 2 t(a) for a_eq = 1/3400, t0 = 13.8 Gyr. Analytic; no figure inputs. The four d*.png figures (#5–#8) come from one commit; ordered by paper number.

Paper caption (caveats as stated): Eq. (1) (left, units of today's matter density) and Eq. (2) (right) for $a_{\rm eq} = 1/3400$, $t_0 = 13.8\,$Gyr. Dotted: the limits $t^{1/2}$ and $t^{2/3}$.



`d4_radmat.png`

Computing chain (run order, full listings)

`/workspace/coepdf/dens_plots.py`

```
import numpy as np,pandas as pd,json,sys,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from scipy.integrate import quad
plt.rcParams.update({'font.size':11})
def legbelow(fig,axs,n=3):
    h,l=[],[]
    for a in axs:
        hh,ll=a.get_legend_handles_labels();h+=hh;l+=ll
    d=dict(zip(l,h));fig.legend(d.values(),d.keys(),loc='lower center',ncol=n,frameon=False,fontsize=9)
N='/workspace/cosmology-course/notes/'
# 1 Migkas density laws
df=pd.read_csv(N+'catwise-density-levelset.csv');e=pd.read_csv(N+'catwise-eta-density.csv')
J=json.load(open(N+'catwise-density-levelset-fit.json'))['density_params']
lab={'exp':r'exp: $e^{-r/L}$, L=923','pl':r'power law: $(1+r/L)^{-n}$, L=1049, n=1.20','cored':r'cored:
 $1/(1+(r/L)^2)$, L=322',
 'gauss':r'Gaussian: $e^{-(r/L)^2/2}$, L=232','soft':r'soft core+tail: $(1+(r/L)^{\alpha})^{-\beta}$,
  L=5110, $\alpha$=0.30, $\beta$=1',
```

```
 'eta':r'stretched exp: $e^{-(r/L)^{\eta}}$, L=15246, $\eta$=0.25'}
r=df.r_Gpc.values;fig,ax=plt.subplots(1,2,figsize=(13,5.6))
for c in df.columns[1:]:
    k=c.split('_')[3]
    for a in ax:a.plot(r,df[c],lw=2,label=lab[k])
ce=[c for c in e.columns if 'eta0.05' in c][0]
for a in ax:a.plot(e.r_Gpc,e[ce],'k--',lw=1.5,label=r'$\eta$-edge: $\eta$=0.050, L=0.849 Gpc')
D,d=58.2,14.3005
for a in ax:
    a.axvline(D,color='grey',lw=1);a.axvspan(D-d,D+d,color='grey',alpha=.12,label=r'CatWISE lock window
    ↪  $D\pm d$')
    a.set_xlabel('r from CoE [Gpc]');a.set_ylabel(r'$\rho/\rho_{\rm crit}$')
ax[0].set_xlim(0,max(r));ax[1].set_xlim(D-1.3*d,D+1.3*d);ax[1].set_ylim(0.28,0.32)
ax[0].set_title('Density laws, all normalised to $\\rho(D)=0.3\\rho_{\\rm
↪  crit}$');ax[1].set_title(r'Zoom: every law has $[\rho(D-d)-\rho(D+d)]/\rho(D)=0.031$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.2,1,1));fig.savefig('fig/d1_migkas_laws.png',dpi=150)
# 2 web
sys.path.insert(0,'/tmp/coe/work/2026-10-20_21/redweb2/realweb');import cosmo as C
fig,ax=plt.subplots(1,3,figsize=(14,4.8))
M=np.geomspace(1e14,2e15,41);n,s=C.ps_dndlnM(M);ax[0].loglog(M,n,'b',lw=2,label='Press–Schechter
↪  $dn/d\\ln M$ (halos)')
ax[0].set_xlabel(r'M [M$_\odot$]');ax[0].set_ylabel(r'Mpc$^{-3}$')
Rv=np.geomspace(10,50,41);ax[1].loglog(Rv,C.svdw_dndlnR(Rv),'r',lw=2,label='Sheth–van de Weygaert
↪  $dn/d\\ln R$ (voids)');ax[1].set_xlabel('void radius R [Mpc]')
ax[2].stem([0],[-1],linefmt='r-',markerfmt='ro',basefmt=' ',label=r'void as painted: point deficit $-M_d$
↪  at centre, $M_d=0.85\bar\rho_m(4\pi/3)R_v^3$')
ax[2].stem([1.25],[1],linefmt='b-',markerfmt='bo',basefmt=' ',label=r'compensating wall: 8 random points,
↪  $M_d/8$ each, at $1.25R_v$')
ax[2].axhline(0,color='k',lw=.5);ax[2].set_xlim(-0.2,1.6);ax[2].set_ylim(-1.3,1.3);ax[2].set_xlabel(r'$r/
↪  R_v$');ax[2].set_ylabel(r'mass /
↪  $M_d$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.16,1,1));fig.savefig('fig/d2_web.png',dpi=150)
# 2b modulation shp
CC=[-0.041,-0.035,0.1353,0.0];Rb=1.5*(1+1/2.937)
shp=lambda x:np.exp(sum(c*(np.minimum(np.abs(x),Rb)-1)**(k+1) for k,c in enumerate(CC)))
# 3 stochastic
p=json.load(open('/tmp/coe/work/2026-10-21_22/stochfield/acoustic_half_fixed.json'))['r2free']['p']
k=np.linspace(0.5,600,3000)  # 1/Gpc
base=10**p['lgA']*k**p['n']*np.exp(-(k/p['kD'])**2)
ac=1+p['Aw']*np.cos(k*p['rs']+p['phi']);dm=p['B']*np.exp(-k/p['kB'])*np.cos(k*p['r2']+p['phi2'])
fig,ax=plt.subplots(1,2,figsize=(13,5))
km=k/1000
ax[0].plot(km,base,'grey',lw=1.5,label=r'$P_0k^ne^{-(k/k_D)^2}$');ax[0].plot(km,base*ac,'b',lw=1.2,label=
↪  '+ acoustic
↪  term')
ax[0].plot(km,base*(ac+dm),'r',lw=2,label=r'full $P_\delta$ (+ damped $r_2$ term)')
ax[0].set_xlabel(r'k [Mpc$^{-1}$]');ax[0].set_ylabel(r'$P_\delta$ [code units,
↪  Gpc$^3$]');ax[0].set_yscale('log');ax[0].set_title('Fitted source spectrum ($r_2$ free)')
ch=np.linspace(2.0,5.2,400);ax[1].plot(ch,np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2),'r',lw=2,label=r'$W(\c
↪  hi)=e^{-(\chi-R_c)^2/2w^2}$, $R_c$=3.572, $w$=0.336
↪  Gpc')
ax[1].set_xlabel(r'$\chi$ [Gpc]');ax[1].set_ylabel('W');ax[1].set_title('Radial window')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d3_stoch.png',dpi=150)
# 4 radiation+matter
ae=1/3400;f=lambda a:a/np.sqrt(ae+a);T=quad(f,0,1)[0];sc=13.8e9/T
a=np.geomspace(1e-6,1,300);t=np.array([quad(f,0,x)[0]*sc for x in a])
fig,ax=plt.subplots(1,2,figsize=(13,5))
ax[0].loglog(a,ae/a**4,'orange',lw=2,label=r'radiation $\rho_r/\rho_{m,0}=a_{\rm
↪  eq}a^{-4}$');ax[0].loglog(a,a**-3.,'g',lw=2,label=r'matter $\rho_m/\rho_{m,0}=a^{-3}$')
```

```
ax[0].axvline(ae,color='k',ls='--',label=r'equality $a_{\rm eq}=1/3400$ (41 kyr)');ax[0].axvline(1/1090,color='m',ls=':',label='recombination z=1089 (297 kyr)')
ax[0].set_xlabel('a');ax[0].set_ylabel(r'$\rho/\rho_{m,0}$')
ax[1].loglog(t/1e3,a,'k',lw=2,label=r'$a(t)$ from $H\propto\sqrt{a_{\rm eq}a^{-4}+a^{-3}}$, $t_0$=13.8 Gyr')
ax[1].loglog(t/1e3,a[-1]*(t/t[-1])**(2/3),'g:',label=r'$t^{2/3}$');ax[1].loglog(t/1e3,ae*(t/40.8e3)**0.5, 'orange',ls=':',label=r'$t^{1/2}$')
ax[1].set_ylim(1e-6,1.5);ax[1].set_xlabel('t [kyr]');ax[1].set_ylabel('a')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.18,1,1));fig.savefig('fig/d4_radmat.png',dpi=150)
# 5 modulation
fig,ax=plt.subplots(figsize=(13,3.8));xx=np.linspace(0,3,400);ax.plot(xx*42,shp(xx),'b',lw=2,label=r'$m(r)=\exp\sum_k c_k(\min(r/42,R_b)-1)^{k+1}$, $c$=[−0.041, −0.035, 0.1353, 0], $R_b$=2.011')
ax.set_xlabel('distance from CoE axis point [Gpc]');ax.set_ylabel('m (before normalising)');legbelow(fig,[ax],1);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d5_mod.png',dpi=150)
print(p)
```

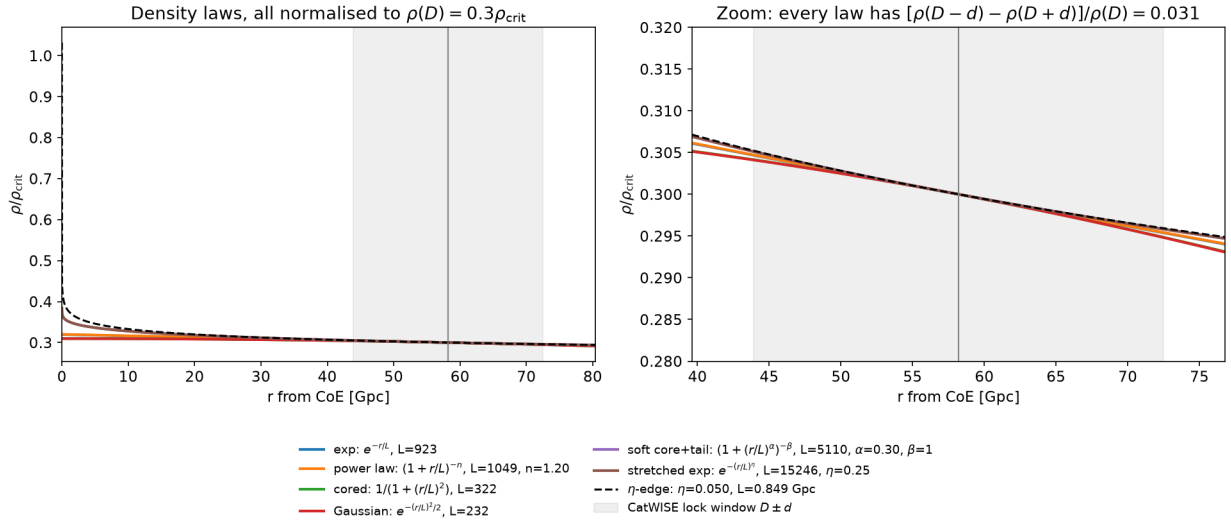
9. CatWISE-locked density laws

Originally made: Fri Oct 23 2026, 12:37 AM CT (*git commit 54f717f*)
Paper figure number: this morning Fig. 5; now Fig. 5
Files: `d1_migkas_laws.png`
Code status: original (git)

Notes

rho/rho_crit vs distance from the CoE with zoom on D +/- d, from the CatWISE-locked level-set laws. Input: CatWISE 3% dipole lock.

Paper caption (caveats as stated): CatWISE-locked laws $\rho/\rho_{\rm crit}$ versus distance from the CoE (left) and zoom on $D \pm d$ (right), plotted from `catwise-density-levelset.csv` and `catwise-eta-density.csv`. $D = 58.2\,$Gpc (line), $d = 14.3005\,$Gpc (band).



`d1_migkas_laws.png`

Computing chain (run order, full listings)

`/workspace/coepdf/dens_plots.py`

```
import numpy as np,pandas as pd,json,sys,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from scipy.integrate import quad
plt.rcParams.update({'font.size':11})
def legbelow(fig,axs,n=3):
    h,l=[],[]
    for a in axs:
        hh,ll=a.get_legend_handles_labels();h+=hh;l+=ll
    d=dict(zip(l,h));fig.legend(d.values(),d.keys(),loc='lower center',ncol=n,frameon=False,fontsize=9)
N='/workspace/cosmology-course/notes/'
# 1 Migkas density laws
df=pd.read_csv(N+'catwise-density-levelset.csv');e=pd.read_csv(N+'catwise-eta-density.csv')
J=json.load(open(N+'catwise-density-levelset-fit.json'))['density_params']
lab={'exp':r'exp: $e^{-r/L}$, L=923','pl':r'power law: $(1+r/L)^{-n}$, L=1049, n=1.20','cored':r'cored:
↪  $1/(1+(r/L)^2)$, L=322',
```

```
 'gauss':r'Gaussian: $e^{-(r/L)^2/2}$, L=232','soft':r'soft core+tail: $(1+(r/L)^{\alpha})^{-\beta}$, L=5110, $\alpha$=0.30, $\beta$=1',
 'eta':r'stretched exp: $e^{-(r/L)^{\eta}}$, L=15246, $\eta$=0.25'}
r=df.r_Gpc.values;fig,ax=plt.subplots(1,2,figsize=(13,5.6))
for c in df.columns[1:]:
    k=c.split('_')[3]
    for a in ax:a.plot(r,df[c],lw=2,label=lab[k])
ce=[c for c in e.columns if 'eta0.05' in c][0]
for a in ax:a.plot(e.r_Gpc,e[ce],'k--',lw=1.5,label=r'$\eta$-edge: $\eta$=0.050, L=0.849 Gpc')
D,d=58.2,14.3005
for a in ax:
    a.axvline(D,color='grey',lw=1);a.axvspan(D-d,D+d,color='grey',alpha=.12,label=r'CatWISE lock window $D\pm d$')
    a.set_xlabel('r from CoE [Gpc]');a.set_ylabel(r'$\rho/\rho_{\rm crit}$')
ax[0].set_xlim(0,max(r));ax[1].set_xlim(D-1.3*d,D+1.3*d);ax[1].set_ylim(0.28,0.32)
ax[0].set_title('Density laws, all normalised to $\\rho(D)=0.3\\rho_{\\rm crit}$');ax[1].set_title(r'Zoom: every law has $[\rho(D-d)-\rho(D+d)]/\rho(D)=0.031$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.2,1,1));fig.savefig('fig/d1_migkas_laws.png',dpi=150)
# 2 web
sys.path.insert(0,'/tmp/coe/work/2026-10-20_21/redweb2/realweb');import cosmo as C
fig,ax=plt.subplots(1,3,figsize=(14,4.8))
M=np.geomspace(1e14,2e15,41);n,s=C.ps_dndlnM(M);ax[0].loglog(M,n,'b',lw=2,label='Press–Schechter $dn/d\\ln M$ (halos)')
ax[0].set_xlabel(r'M [M$_\odot$]');ax[0].set_ylabel(r'Mpc$^{-3}$')
Rv=np.geomspace(10,50,41);ax[1].loglog(Rv,C.svdw_dndlnR(Rv),'r',lw=2,label='Sheth–van de Weygaert $dn/d\\ln R$ (voids)');ax[1].set_xlabel('void radius R [Mpc]')
ax[2].stem([0],[-1],linefmt='r-',markerfmt='ro',basefmt=' ',label=r'void as painted: point deficit $-M_d$ at centre, $M_d=0.85\bar\rho_m(4\pi/3)R_v^3$')
ax[2].stem([1.25],[1],linefmt='b-',markerfmt='bo',basefmt=' ',label=r'compensating wall: 8 random points, $M_d/8$ each, at $1.25R_v$')
ax[2].axhline(0,color='k',lw=.5);ax[2].set_xlim(-0.2,1.6);ax[2].set_ylim(-1.3,1.3);ax[2].set_xlabel(r'$r/R_v$');ax[2].set_ylabel(r'mass / $M_d$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.16,1,1));fig.savefig('fig/d2_web.png',dpi=150)
# 2b modulation shp
CC=[-0.041,-0.035,0.1353,0.0];Rb=1.5*(1+1/2.937)
shp=lambda x:np.exp(sum(c*(np.minimum(np.abs(x),Rb)-1)**(k+1) for k,c in enumerate(CC)))
# 3 stochastic
p=json.load(open('/tmp/coe/work/2026-10-21_22/stochfield/acoustic_half_fixed.json'))['r2free']['p']
k=np.linspace(0.5,600,3000)  # 1/Gpc
base=10**p['lgA']*k**p['n']*np.exp(-(k/p['kD'])**2)
ac=1+p['Aw']*np.cos(k*p['rs']+p['phi']);dm=p['B']*np.exp(-k/p['kB'])*np.cos(k*p['r2']+p['phi2'])
fig,ax=plt.subplots(1,2,figsize=(13,5))
km=k/1000
ax[0].plot(km,base,'grey',lw=1.5,label=r'$P_0k^ne^{-(k/k_D)^2}$');ax[0].plot(km,base*ac,'b',lw=1.2,label='+ acoustic term')
ax[0].plot(km,base*(ac+dm),'r',lw=2,label=r'full $P_\delta$ (+ damped $r_2$ term)')
ax[0].set_xlabel(r'k [Mpc$^{-1}$]');ax[0].set_ylabel(r'$P_\delta$ [code units, Gpc$^3$]');ax[0].set_yscale('log');ax[0].set_title('Fitted source spectrum ($r_2$ free)')
ch=np.linspace(2.0,5.2,400);ax[1].plot(ch,np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2),'r',lw=2,label=r'$W(\chi-R_c)^2/2w^2}$, $R_c$=3.572, $w$=0.336 Gpc')
ax[1].set_xlabel(r'$\chi$ [Gpc]');ax[1].set_ylabel('W');ax[1].set_title('Radial window')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d3_stoch.png',dpi=150)
# 4 radiation+matter
ae=1/3400;f=lambda a:a/np.sqrt(ae+a);T=quad(f,0,1)[0];sc=13.8e9/T
a=np.geomspace(1e-6,1,300);t=np.array([quad(f,0,x)[0]*sc for x in a])
fig,ax=plt.subplots(1,2,figsize=(13,5))
```

```
ax[0].loglog(a,ae/a**4,'orange',lw=2,label=r'radiation $\rho_r/\rho_{m,0}=a_{\rm eq}a^{-4}$');ax[0].loglog(a,a**-3.,'g',lw=2,label=r'matter $\rho_m/\rho_{m,0}=a^{-3}$')
ax[0].axvline(ae,color='k',ls='--',label=r'equality $a_{\rm eq}=1/3400$ (41 kyr)');ax[0].axvline(1/1090,color='m',ls=':',label='recombination z=1089 (297 kyr)')
ax[0].set_xlabel('a');ax[0].set_ylabel(r'$\rho/\rho_{m,0}$')
ax[1].loglog(t/1e3,a,'k',lw=2,label=r'$a(t)$ from $H\propto\sqrt{a_{\rm eq}a^{-4}+a^{-3}}$, $t_0$=13.8 Gyr')
ax[1].loglog(t/1e3,a[-1]*(t/t[-1])**(2/3),'g:',label=r'$t^{2/3}$');ax[1].loglog(t/1e3,ae*(t/40.8e3)**0.5, 'orange',ls=':',label=r'$t^{1/2}$')
ax[1].set_ylim(1e-6,1.5);ax[1].set_xlabel('t [kyr]');ax[1].set_ylabel('a')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.18,1,1));fig.savefig('fig/d4_radmat.png',dpi=150)
# 5 modulation
fig,ax=plt.subplots(figsize=(13,3.8));xx=np.linspace(0,3,400);ax.plot(xx*42,shp(xx),'b',lw=2,label=r'$m(r)=\exp\sum_k c_k(\min(r/42,R_b)-1)^{k+1}$, $c$=[−0.041, −0.035, 0.1353, 0], $R_b$=2.011')
ax.set_xlabel('distance from CoE axis point [Gpc]');ax.set_ylabel('m (before normalising)');legbelow(fig,[ax],1);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d5_mod.png',dpi=150)
print(p)
```

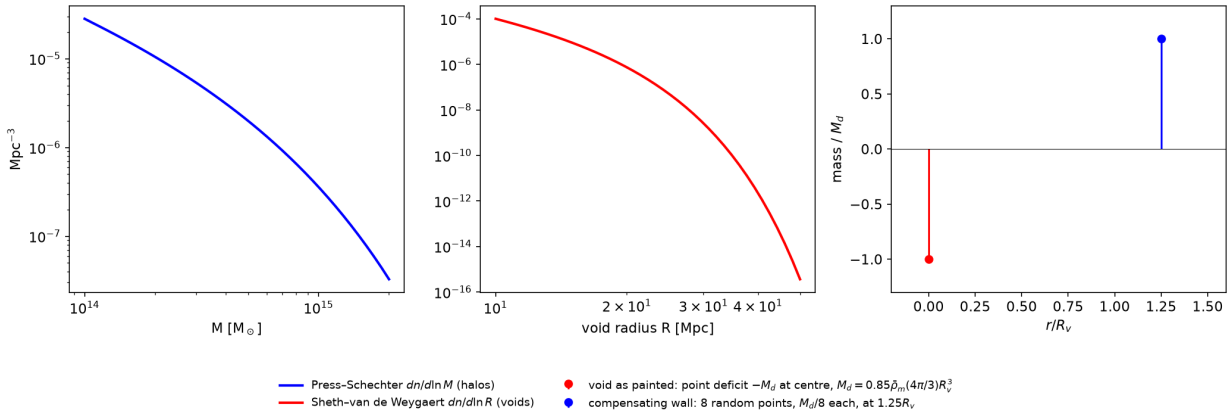
10. Cosmic-web ingredients

Originally made: Fri Oct 23 2026, 12:37 AM CT (*git commit 54f717f*)
Paper figure number: this morning Fig. 13; now Fig. 7
Files: `d2_web.png`
Code status: original (git)

Notes

Halo mass function, void size function and compensated void mass placement, recomputed from cosmo.py. Input to the realistic web (#14) and later web figures.

Paper caption (caveats as stated): Web ingredients recomputed from `cosmo.py`: halo mass function (left), void size function (middle), compensated void mass placement (right).



d2_web.png

Computing chain (run order, full listings)

`/workspace/coepdf/dens_plots.py`

```
import numpy as np,pandas as pd,json,sys,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from scipy.integrate import quad
plt.rcParams.update({'font.size':11})
def legbelow(fig,axs,n=3):
    h,l=[],[]
    for a in axs:
        hh,ll=a.get_legend_handles_labels();h+=hh;l+=ll
    d=dict(zip(l,h));fig.legend(d.values(),d.keys(),loc='lower center',ncol=n,frameon=False,fontsize=9)
N='/workspace/cosmology-course/notes/'
# 1 Migkas density laws
df=pd.read_csv(N+'catwise-density-levelset.csv');e=pd.read_csv(N+'catwise-eta-density.csv')
J=json.load(open(N+'catwise-density-levelset-fit.json'))['density_params']
lab={'exp':r'exp: $e^{-r/L}$, L=923','pl':r'power law: $(1+r/L)^{-n}$, L=1049, n=1.20','cored':r'cored:
↪  $1/(1+(r/L)^2)$, L=322',
 'gauss':r'Gaussian: $e^{-(r/L)^2/2}$, L=232','soft':r'soft core+tail: $(1+(r/L)^{\alpha})^{-\beta}$,
 ↪  L=5110, $\alpha$=0.30, $\beta$=1',
 'eta':r'stretched exp: $e^{-(r/L)^{\eta}}$, L=15246, $\eta$=0.25'}
r=df.r_Gpc.values;fig,ax=plt.subplots(1,2,figsize=(13,5.6))
```

```
for c in df.columns[1:]:
    k=c.split('_')[3]
    for a in ax:a.plot(r,df[c],lw=2,label=lab[k])
ce=[c for c in e.columns if 'eta0.05' in c][0]
for a in ax:a.plot(e.r_Gpc,e[ce],'k--',lw=1.5,label=r'$\eta$-edge: $\eta$=0.050, L=0.849 Gpc')
D,d=58.2,14.3005
for a in ax:
    a.axvline(D,color='grey',lw=1);a.axvspan(D-d,D+d,color='grey',alpha=.12,label=r'CatWISE lock window
    ↪  $D\pm d$')
    a.set_xlabel('r from CoE [Gpc]');a.set_ylabel(r'$\rho/\rho_{\rm crit}$')
ax[0].set_xlim(0,max(r));ax[1].set_xlim(D-1.3*d,D+1.3*d);ax[1].set_ylim(0.28,0.32)
ax[0].set_title('Density laws, all normalised to $\\rho(D)=0.3\\rho_{\\rm
↪  crit}$');ax[1].set_title(r'Zoom: every law has $[\rho(D-d)-\rho(D+d)]/\rho(D)=0.031$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.2,1,1));fig.savefig('fig/d1_migkas_laws.png',dpi=150)
# 2 web
sys.path.insert(0,'/tmp/coe/work/2026-10-20_21/redweb2/realweb');import cosmo as C
fig,ax=plt.subplots(1,3,figsize=(14,4.8))
M=np.geomspace(1e14,2e15,41);n,s=C.ps_dndlnM(M);ax[0].loglog(M,n,'b',lw=2,label='Press–Schechter
↪  $dn/d\\ln M$ (halos)')
ax[0].set_xlabel(r'M [M$_\odot$]');ax[0].set_ylabel(r'Mpc$^{-3}$')
Rv=np.geomspace(10,50,41);ax[1].loglog(Rv,C.svdw_dndlnR(Rv),'r',lw=2,label='Sheth–van de Weygaert
↪  $dn/d\\ln R$ (voids)');ax[1].set_xlabel('void radius R [Mpc]')
ax[2].stem([0],[-1],linefmt='r-',markerfmt='ro',basefmt=' ',label=r'void as painted: point deficit $-M_d$
↪  at centre, $M_d=0.85\bar\rho_m(4\pi/3)R_v^3$')
ax[2].stem([1.25],[1],linefmt='b-',markerfmt='bo',basefmt=' ',label=r'compensating wall: 8 random points,
↪  $M_d/8$ each, at $1.25R_v$')
ax[2].axhline(0,color='k',lw=.5);ax[2].set_xlim(-0.2,1.6);ax[2].set_ylim(-1.3,1.3);ax[2].set_xlabel(r'$r/
↪  R_v$');ax[2].set_ylabel(r'mass /
↪  $M_d$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.16,1,1));fig.savefig('fig/d2_web.png',dpi=150)
# 2b modulation shp
CC=[-0.041,-0.035,0.1353,0.0];Rb=1.5*(1+1/2.937)
shp=lambda x:np.exp(sum(c*(np.minimum(np.abs(x),Rb)-1)**(k+1) for k,c in enumerate(CC)))
# 3 stochastic
p=json.load(open('/tmp/coe/work/2026-10-21_22/stochfield/acoustic_half_fixed.json'))['r2free']['p']
k=np.linspace(0.5,600,3000)  # 1/Gpc
base=10**p['lgA']*k**p['n']*np.exp(-(k/p['kD'])**2)
ac=1+p['Aw']*np.cos(k*p['rs']+p['phi']);dm=p['B']*np.exp(-k/p['kB'])*np.cos(k*p['r2']+p['phi2'])
fig,ax=plt.subplots(1,2,figsize=(13,5))
km=k/1000
ax[0].plot(km,base,'grey',lw=1.5,label=r'$P_0k^ne^{-(k/k_D)^2}$');ax[0].plot(km,base*ac,'b',lw=1.2,label=
↪  '+ acoustic
↪  term')
ax[0].plot(km,base*(ac+dm),'r',lw=2,label=r'full $P_\delta$ (+ damped $r_2$ term)')
ax[0].set_xlabel(r'k [Mpc$^{-1}$]');ax[0].set_ylabel(r'$P_\delta$ [code units,
↪  Gpc$^3$]');ax[0].set_yscale('log');ax[0].set_title('Fitted source spectrum ($r_2$ free)')
ch=np.linspace(2.0,5.2,400);ax[1].plot(ch,np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2),'r',lw=2,label=r'$W(\c
↪  hi)=e^{-(\chi-R_c)^2/2w^2}$, $R_c$=3.572, $w$=0.336
↪  Gpc')
ax[1].set_xlabel(r'$\chi$ [Gpc]');ax[1].set_ylabel('W');ax[1].set_title('Radial window')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d3_stoch.png',dpi=150)
# 4 radiation+matter
ae=1/3400;f=lambda a:a/np.sqrt(ae+a);T=quad(f,0,1)[0];sc=13.8e9/T
a=np.geomspace(1e-6,1,300);t=np.array([quad(f,0,x)[0]*sc for x in a])
fig,ax=plt.subplots(1,2,figsize=(13,5))
ax[0].loglog(a,ae/a**4,'orange',lw=2,label=r'radiation $\rho_r/\rho_{m,0}=a_{\rm
↪  eq}a^{-4}$');ax[0].loglog(a,a**-3.,'g',lw=2,label=r'matter $\rho_m/\rho_{m,0}=a^{-3}$')
ax[0].axvline(ae,color='k',ls='--',label=r'equality $a_{\rm eq}=1/3400$ (41
↪  kyr)');ax[0].axvline(1/1090,color='m',ls=':',label='recombination z=1089 (297 kyr)')
```

```
ax[0].set_xlabel('a');ax[0].set_ylabel(r'$\rho/\rho_{m,0}$')
ax[1].loglog(t/1e3,a,'k',lw=2,label=r'$a(t)$ from $H\propto\sqrt{a_{\rm eq}a^{-4}+a^{-3}}$, $t_0$=13.8 Gyr')
ax[1].loglog(t/1e3,a[-1]*(t/t[-1])**(2/3),'g:',label=r'$t^{2/3}$');ax[1].loglog(t/1e3,ae*(t/40.8e3)**0.5, 'orange',ls=':',label=r'$t^{1/2}$')
ax[1].set_ylim(1e-6,1.5);ax[1].set_xlabel('t [kyr]');ax[1].set_ylabel('a')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.18,1,1));fig.savefig('fig/d4_radmat.png',dpi=150)
# 5 modulation
fig,ax=plt.subplots(figsize=(13,3.8));xx=np.linspace(0,3,400);ax.plot(xx*42,shp(xx),'b',lw=2,label=r'$m(r)=\exp\sum_k c_k(\min(r/42,R_b)-1)^{k+1}$, $c$=[−0.041, −0.035, 0.1353, 0], $R_b$=2.011')
ax.set_xlabel('distance from CoE axis point [Gpc]');ax.set_ylabel('m (before normalising)');legbelow(fig,[ax],1);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d5_mod.png',dpi=150)
print(p)
```

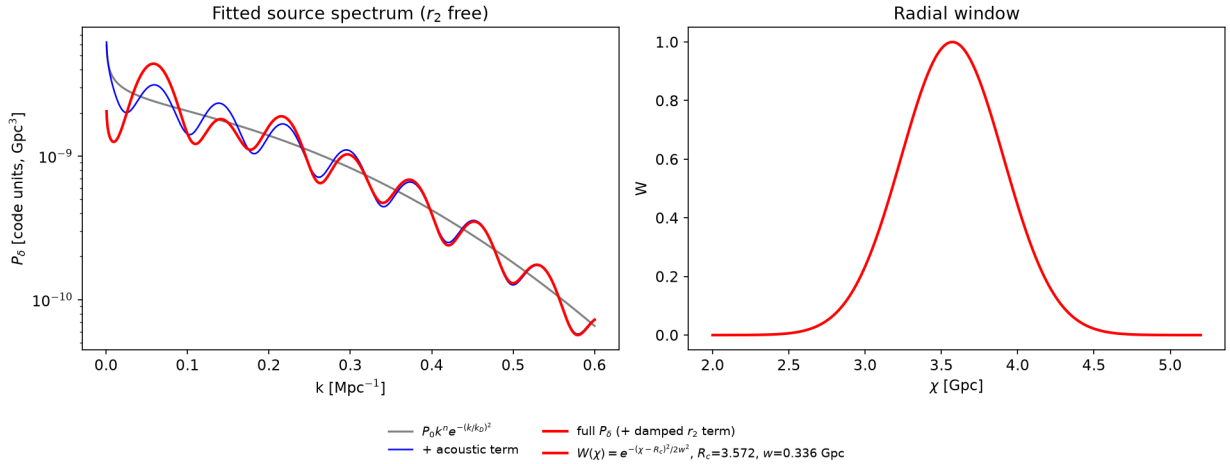
11. Fitted P_delta terms and window W(chi)

Originally made: Fri Oct 23 2026, 12:37 AM CT (*git commit 54f717f*)
Paper figure number: this morning Fig. 19; now Fig. 26
Files: `d3_stoch.png`
Code status: original (git)

Notes

Fitted P_delta split into its terms and the radial window. Depends on the stochastic-field fit (#2).

Paper caption (caveats as stated): Fitted P_δ split into its terms (left) and the radial window $W(\chi)$ (right).



`d3_stoch.png`

Computing chain (run order, full listings)

`/workspace/coepdf/dens_plots.py`

```
import numpy as np,pandas as pd,json,sys,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from scipy.integrate import quad
plt.rcParams.update({'font.size':11})
def legbelow(fig,axs,n=3):
    h,l=[],[]
    for a in axs:
        hh,ll=a.get_legend_handles_labels();h+=hh;l+=ll
    d=dict(zip(l,h));fig.legend(d.values(),d.keys(),loc='lower center',ncol=n,frameon=False,fontsize=9)
N='/workspace/cosmology-course/notes/'
# 1 Migkas density laws
df=pd.read_csv(N+'catwise-density-levelset.csv');e=pd.read_csv(N+'catwise-eta-density.csv')
J=json.load(open(N+'catwise-density-levelset-fit.json'))['density_params']
lab={'exp':r'exp: $e^{-r/L}$, L=923','pl':r'power law: $(1+r/L)^{-n}$, L=1049, n=1.20','cored':r'cored:
↪  $1/(1+(r/L)^2)$, L=322',
 'gauss':r'Gaussian: $e^{-(r/L)^2/2}$, L=232','soft':r'soft core+tail: $(1+(r/L)^{\alpha})^{-\beta}$,
 ↪  L=5110, $\alpha$=0.30, $\beta$=1',
 'eta':r'stretched exp: $e^{-(r/L)^{\eta}}$, L=15246, $\eta$=0.25'}
r=df.r_Gpc.values;fig,ax=plt.subplots(1,2,figsize=(13,5.6))
```

```
for c in df.columns[1:]:
    k=c.split('_')[3]
    for a in ax:a.plot(r,df[c],lw=2,label=lab[k])
ce=[c for c in e.columns if 'eta0.05' in c][0]
for a in ax:a.plot(e.r_Gpc,e[ce],'k--',lw=1.5,label=r'$\eta$-edge: $\eta$=0.050, L=0.849 Gpc')
D,d=58.2,14.3005
for a in ax:
    a.axvline(D,color='grey',lw=1);a.axvspan(D-d,D+d,color='grey',alpha=.12,label=r'CatWISE lock window
    ↪  $D\pm d$')
    a.set_xlabel('r from CoE [Gpc]');a.set_ylabel(r'$\rho/\rho_{\rm crit}$')
ax[0].set_xlim(0,max(r));ax[1].set_xlim(D-1.3*d,D+1.3*d);ax[1].set_ylim(0.28,0.32)
ax[0].set_title('Density laws, all normalised to $\\rho(D)=0.3\\rho_{\\rm
↪  crit}$');ax[1].set_title(r'Zoom: every law has $[\rho(D-d)-\rho(D+d)]/\rho(D)=0.031$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.2,1,1));fig.savefig('fig/d1_migkas_laws.png',dpi=150)
# 2 web
sys.path.insert(0,'/tmp/coe/work/2026-10-20_21/redweb2/realweb');import cosmo as C
fig,ax=plt.subplots(1,3,figsize=(14,4.8))
M=np.geomspace(1e14,2e15,41);n,s=C.ps_dndlnM(M);ax[0].loglog(M,n,'b',lw=2,label='Press–Schechter
↪  $dn/d\\ln M$ (halos)')
ax[0].set_xlabel(r'M [M$_\odot$]');ax[0].set_ylabel(r'Mpc$^{-3}$')
Rv=np.geomspace(10,50,41);ax[1].loglog(Rv,C.svdw_dndlnR(Rv),'r',lw=2,label='Sheth–van de Weygaert
↪  $dn/d\\ln R$ (voids)');ax[1].set_xlabel('void radius R [Mpc]')
ax[2].stem([0],[-1],linefmt='r-',markerfmt='ro',basefmt=' ',label=r'void as painted: point deficit $-M_d$
↪  at centre, $M_d=0.85\bar\rho_m(4\pi/3)R_v^3$')
ax[2].stem([1.25],[1],linefmt='b-',markerfmt='bo',basefmt=' ',label=r'compensating wall: 8 random points,
↪  $M_d/8$ each, at $1.25R_v$')
ax[2].axhline(0,color='k',lw=.5);ax[2].set_xlim(-0.2,1.6);ax[2].set_ylim(-1.3,1.3);ax[2].set_xlabel(r'$r/
↪  R_v$');ax[2].set_ylabel(r'mass /
↪  $M_d$')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.16,1,1));fig.savefig('fig/d2_web.png',dpi=150)
# 2b modulation shp
CC=[-0.041,-0.035,0.1353,0.0];Rb=1.5*(1+1/2.937)
shp=lambda x:np.exp(sum(c*(np.minimum(np.abs(x),Rb)-1)**(k+1) for k,c in enumerate(CC)))
# 3 stochastic
p=json.load(open('/tmp/coe/work/2026-10-21_22/stochfield/acoustic_half_fixed.json'))['r2free']['p']
k=np.linspace(0.5,600,3000)  # 1/Gpc
base=10**p['lgA']*k**p['n']*np.exp(-(k/p['kD'])**2)
ac=1+p['Aw']*np.cos(k*p['rs']+p['phi']);dm=p['B']*np.exp(-k/p['kB'])*np.cos(k*p['r2']+p['phi2'])
fig,ax=plt.subplots(1,2,figsize=(13,5))
km=k/1000
ax[0].plot(km,base,'grey',lw=1.5,label=r'$P_0k^ne^{-(k/k_D)^2}$');ax[0].plot(km,base*ac,'b',lw=1.2,label=
↪  '+ acoustic
↪  term')
ax[0].plot(km,base*(ac+dm),'r',lw=2,label=r'full $P_\delta$ (+ damped $r_2$ term)')
ax[0].set_xlabel(r'k [Mpc$^{-1}$]');ax[0].set_ylabel(r'$P_\delta$ [code units,
↪  Gpc$^3$]');ax[0].set_yscale('log');ax[0].set_title('Fitted source spectrum ($r_2$ free)')
ch=np.linspace(2.0,5.2,400);ax[1].plot(ch,np.exp(-0.5*((ch-p['chiw'])/p['sw'])**2),'r',lw=2,label=r'$W(\c
↪  hi)=e^{-(\chi-R_c)^2/2w^2}$, $R_c$=3.572, $w$=0.336
↪  Gpc')
ax[1].set_xlabel(r'$\chi$ [Gpc]');ax[1].set_ylabel('W');ax[1].set_title('Radial window')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d3_stoch.png',dpi=150)
# 4 radiation+matter
ae=1/3400;f=lambda a:a/np.sqrt(ae+a);T=quad(f,0,1)[0];sc=13.8e9/T
a=np.geomspace(1e-6,1,300);t=np.array([quad(f,0,x)[0]*sc for x in a])
fig,ax=plt.subplots(1,2,figsize=(13,5))
ax[0].loglog(a,ae/a**4,'orange',lw=2,label=r'radiation $\rho_r/\rho_{m,0}=a_{\rm
↪  eq}a^{-4}$');ax[0].loglog(a,a**-3.,'g',lw=2,label=r'matter $\rho_m/\rho_{m,0}=a^{-3}$')
ax[0].axvline(ae,color='k',ls='--',label=r'equality $a_{\rm eq}=1/3400$ (41
↪  kyr)');ax[0].axvline(1/1090,color='m',ls=':',label='recombination z=1089 (297 kyr)')
```

```
ax[0].set_xlabel('a');ax[0].set_ylabel(r'$\rho/\rho_{m,0}$')
ax[1].loglog(t/1e3,a,'k',lw=2,label=r'$a(t)$ from $H\propto\sqrt{a_{\rm eq}a^{-4}+a^{-3}}$, $t_0$=13.8 Gyr')
ax[1].loglog(t/1e3,a[-1]*(t/t[-1])**(2/3),'g:',label=r'$t^{2/3}$');ax[1].loglog(t/1e3,ae*(t/40.8e3)**0.5, 'orange',ls=':',label=r'$t^{1/2}$')
ax[1].set_ylim(1e-6,1.5);ax[1].set_xlabel('t [kyr]');ax[1].set_ylabel('a')
legbelow(fig,ax,2);fig.tight_layout(rect=(0,0.18,1,1));fig.savefig('fig/d4_radmat.png',dpi=150)
# 5 modulation
fig,ax=plt.subplots(figsize=(13,3.8));xx=np.linspace(0,3,400);ax.plot(xx*42,shp(xx),'b',lw=2,label=r'$m(r)=\exp\sum_k c_k(\min(r/42,R_b)-1)^{k+1}$, $c$=[−0.041, −0.035, 0.1353, 0], $R_b$=2.011')
ax.set_xlabel('distance from CoE axis point [Gpc]');ax.set_ylabel('m (before normalising)');legbelow(fig,[ax],1);fig.tight_layout(rect=(0,0.12,1,1));fig.savefig('fig/d5_mod.png',dpi=150)
print(p)
```

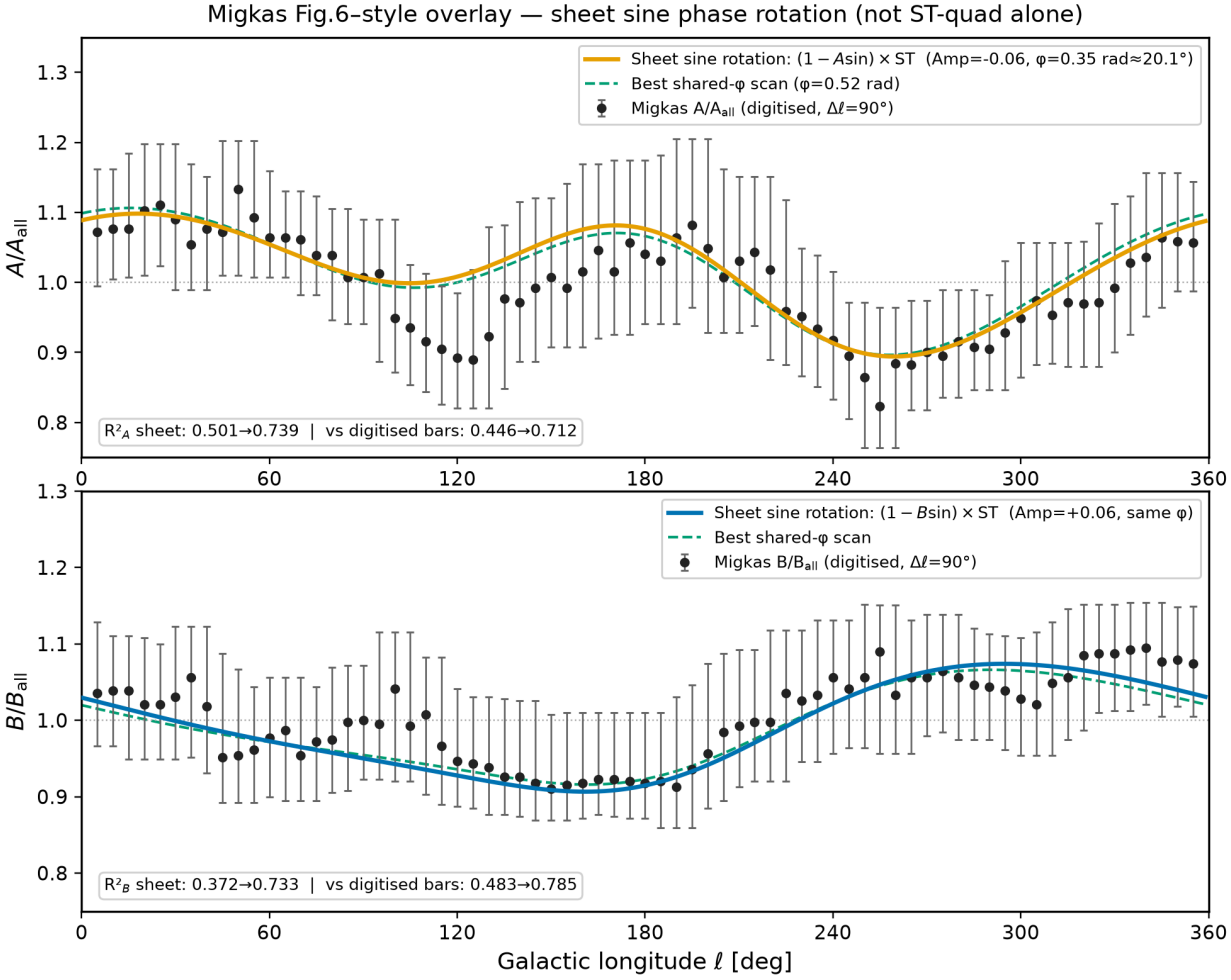
12. Observables: Migkas A/A_all with bars and rotation overlay

Originally made: Fri Oct 23 2026, 8:21 AM CT (*copy time onto the box (true original likely earlier)*)
Paper figure number: this morning Fig. 4; now Fig. 4
Files: `migkas_bars_rotation_overlay.png`
Code status: original (box)

Notes

Migkas A/A_all vs Galactic longitude with bars and rotation overlay. Input: Migkas cluster data (reads a spreadsheet via openpyxl).

Paper caption (caveats as stated): The observables. Top: Migkas $A/A_{\rm all}$ against Galactic longitude (digitised, $\Delta l = 90^\circ$ cones, black with error bars). Bottom: $B/B_{\rm all}$. Solid: spreadsheet shell model with the one-cycle sine rotation (Amp $= \mp 0.06$, $\varphi = 0.35$ rad); dashed: best shared-φ scan. R^2 before and after rotation is given in each panel. Restored from `cosmology/figures`.



Diagnostic: Amp×sin((2π/360)·ℓ+φ) phase alignment from Chris sheet tab "Copy of Copy of Copy of density". ST quadrupole alone is a separate claim.

`migkas_bars_rotation_overlay.png`

Computing chain (run order, full listings)

/workspace/cosmology/figures/migkas_bars_rotation_overlay.py

```
#!/usr/bin/env python3
"""
Migkas Fig.6 overlay with published-style error bars + Chris sheet-style
sine phase-rotation diagnostic.

This is NOT a free A cos φ dipole template. It reproduces the spreadsheet
rotation model:

  BA = Amp * sin((2π/360)·□ + φ)     # Amp≈−0.06, φ≈0.35 rad
  BE = −Amp * same sine
  Model A/A_all = (1 − BA) * AF      # AF = sheet ST H_obs/H series
  Model B/B_all = (1 − BE) * AN      # AN = sheet mass / B-side series

Sheet tab used for cached series: "Copy of Copy of Copy of density"
(R²_A≈0.739, R²_B≈0.733 with rotation; ≈0.50 / ≈0.37 without).

First-principles ST quadrupole shape alone is a separate claim — this panel
is the sheet-style phase-rotation diagnostic vs Migkas bars.
"""
from __future__ import annotations

import csv
import json
from pathlib import Path

import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec

OUT = Path(__file__).resolve().parent
CSV_MIGKAS = Path("/workspace/cosmology/migkas/fig6_digitized.csv")
XLSX = Path(
    "/home/box/agent-data/agents/196ab413-0708-4f81-99fc-58a32c5366c5/"
    "attachments/b3737b0408e9976ecfad92319c9a6bbbf53777e5846e2136703654a0dcf9269a.xlsx"
)
SHEET_CACHE = Path("/tmp/chris_sheet_series.csv")
SHEET_TAB = "Copy of Copy of Copy of density"

# Chris sheet params on that tab (cached)
AMP_A = -0.06
PHI_RAD = 0.35  # ≈20.05°
OMEGA = 2.0 * np.pi / 360.0

C_BLUE = "#0072B2"
C_ORANGE = "#E69F00"
C_BLACK = "#222222"
C_GRAY = "#666666"
C_GREEN = "#009E73"


def style() -> None:
    mpl.rcParams.update(
        {
```

```
            "figure.facecolor": "white",
            "axes.facecolor": "white",
            "savefig.facecolor": "white",
            "savefig.dpi": 220,
            "font.size": 11,
            "axes.labelsize": 12,
            "axes.titlesize": 12,
            "legend.fontsize": 8.5,
            "xtick.labelsize": 10,
            "ytick.labelsize": 10,
            "axes.linewidth": 1.1,
            "lines.linewidth": 2.0,
            "mathtext.fontset": "dejavusans",
        }
    )


def rsq(y: np.ndarray, x: np.ndarray) -> float:
    """Pearson R² (Excel RSQ)."""
    y = np.asarray(y, dtype=float)
    x = np.asarray(x, dtype=float)
    if y.size < 2 or np.std(x) == 0 or np.std(y) == 0:
        return float("nan")
    return float(np.corrcoef(x, y)[0, 1] ** 2)


def load_migkas_csv(path: Path = CSV_MIGKAS) -> dict[str, np.ndarray]:
    rows = list(csv.DictReader(path.open()))
    out = {k: np.array([float(r[k]) for r in rows]) for k in rows[0]}
    return out


def load_sheet_series(cache: Path = SHEET_CACHE) -> dict[str, np.ndarray]:
    """Load AR/AF/AN/BA/BE/BH/BI/BL/BM from pre-extracted CSV or xlsx."""
    if cache.is_file():
        rows = list(csv.DictReader(cache.open()))
        keys = ["AR", "AF", "AN", "BA", "BE", "BH", "BI", "BL", "BM", "BG"]
        return {k: np.array([float(r[k]) for r in rows]) for k in keys}

    # Fallback: extract with openpyxl (slow)
    from openpyxl import load_workbook

    wb = load_workbook(XLSX, data_only=True, read_only=True)
    ws = wb[SHEET_TAB]
    data = {k: [] for k in ["AR", "AF", "AN", "BA", "BE", "BH", "BI", "BL", "BM", "BG"]}
    for r in range(66, 427):
        for k in data:
            data[k].append(float(ws[f"{k}{r}"].value))
    wb.close()
    return {k: np.asarray(v) for k, v in data.items()}


def sine_mod(l_deg: np.ndarray, amp: float, phi: float) -> np.ndarray:
    return amp * np.sin(OMEGA * np.asarray(l_deg, dtype=float) + phi)


def model_rotated(
    af: np.ndarray, an: np.ndarray, l_deg: np.ndarray, amp_a: float, phi: float
) -> tuple[np.ndarray, np.ndarray]:
```

```
    """Chris sheet: Model A = (1−BA)*AF, Model B = (1−BE)*AN, BE=−amp_a."""
    ba = sine_mod(l_deg, amp_a, phi)
    be = sine_mod(l_deg, -amp_a, phi)
    return (1.0 - ba) * af, (1.0 - be) * an


def interp_sheet_to(l_target: np.ndarray, l_src: np.ndarray, y_src: np.ndarray) -> np.ndarray:
    """Periodic-ish interp on 0..360; sheet has 0..360 inclusive."""
    # Drop duplicate 360 if present
    mask = l_src < 360.0 - 1e-9
    ls, ys = l_src[mask], y_src[mask]
    order = np.argsort(ls)
    ls, ys = ls[order], ys[order]
    # wrap pad
    ls_p = np.concatenate([ls - 360.0, ls, ls + 360.0])
    ys_p = np.concatenate([ys, ys, ys])
    return np.interp(np.asarray(l_target, dtype=float) % 360.0, ls_p, ys_p)


def scan_phi(
    l: np.ndarray,
    af: np.ndarray,
    an: np.ndarray,
    a_obs: np.ndarray,
    b_obs: np.ndarray,
    amp: float = AMP_A,
    n: int = 361,
) -> dict:
    phis = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    best = {"phi": 0.0, "r2_a": -1.0, "r2_b": -1.0, "r2_sum": -1.0}
    curve_a, curve_b = [], []
    for phi in phis:
        ma, mb = model_rotated(af, an, l, amp, phi)
        ra, rb = rsq(a_obs, ma), rsq(b_obs, mb)
        curve_a.append(ra)
        curve_b.append(rb)
        s = ra + rb
        if s > best["r2_sum"]:
            best = {
                "phi": float(phi),
                "phi_deg": float(np.rad2deg(phi)),
                "r2_a": float(ra),
                "r2_b": float(rb),
                "r2_sum": float(s),
            }
    best["phi_scan_r2_a"] = curve_a
    best["phi_scan_r2_b"] = curve_b
    best["phis"] = phis.tolist()
    return best


def main() -> None:
    style()
    mig = load_migkas_csv()
    sheet = load_sheet_series()

    l_s = sheet["AR"]
    af_s, an_s = sheet["AF"], sheet["AN"]
    bh_s, bl_s = sheet["BH"], sheet["BL"]
```

```
    bi_s, bm_s = sheet["BI"], sheet["BM"]

    # Sheet R² (exact Excel RSQ on 1° VLOOKUP Migkas)
    r2_a_with_sheet = rsq(sheet["BI"], sheet["BH"])
    r2_b_with_sheet = rsq(sheet["BM"], sheet["BL"])
    r2_a_wo_sheet = rsq(sheet["BI"], sheet["AF"])
    r2_b_wo_sheet = rsq(sheet["BM"], sheet["AN"])

    # Digitized Migkas + asymmetric bars
    l_m = mig["l_deg"]
    a_m = mig["A_over_Aall"]
    b_m = mig["B_over_Ball"]
    a_elo, a_ehi = mig["A_err_low"], mig["A_err_high"]
    b_elo, b_ehi = mig["B_err_low"], mig["B_err_high"]

    # Sheet model curves at digitized longitudes (Chris φ)
    af_m = interp_sheet_to(l_m, l_s, af_s)
    an_m = interp_sheet_to(l_m, l_s, an_s)
    model_a_chris, model_b_chris = model_rotated(af_m, an_m, l_m, AMP_A, PHI_RAD)
    # Also the cached BH/BL interpolated (should match)
    bh_m = interp_sheet_to(l_m, l_s, bh_s)
    bl_m = interp_sheet_to(l_m, l_s, bl_s)

    r2_a_dig_chris = rsq(a_m, model_a_chris)
    r2_b_dig_chris = rsq(b_m, model_b_chris)
    r2_a_dig_wo = rsq(a_m, af_m)
    r2_b_dig_wo = rsq(b_m, an_m)

    # Scan φ on digitized points (shared φ, opposite Amp)
    scan = scan_phi(l_m, af_m, an_m, a_m, b_m, amp=AMP_A)
    model_a_best, model_b_best = model_rotated(
        af_m, an_m, l_m, AMP_A, scan["phi"]
    )
    # Fine curve for plotting (1°)
    l_fine = np.arange(0.0, 360.0, 1.0)
    af_f = interp_sheet_to(l_fine, l_s, af_s)
    an_f = interp_sheet_to(l_fine, l_s, an_s)
    ma_f, mb_f = model_rotated(af_f, an_f, l_fine, AMP_A, PHI_RAD)
    ma_fb, mb_fb = model_rotated(af_f, an_f, l_fine, AMP_A, scan["phi"])

    # ---------- Fig 1: Migkas-like 2-panel overlay ----------
    fig = plt.figure(figsize=(10.5, 8.2))
    gs = GridSpec(2, 1, height_ratios=[1, 1], hspace=0.08, figure=fig)
    ax_a = fig.add_subplot(gs[0])
    ax_b = fig.add_subplot(gs[1], sharex=ax_a)

    ax_a.errorbar(
        l_m,
        a_m,
        yerr=np.vstack([a_elo, a_ehi]),
        fmt="o",
        ms=4.0,
        color=C_BLACK,
        ecolor=C_GRAY,
        elinewidth=0.9,
        capsize=2.0,
        label="Migkas A/A$_\\mathrm{all}$ (digitised, Δ□=90°)",
        zorder=3,
    )
```

```
    ax_a.plot(
        l_fine,
        ma_f,
        color=C_ORANGE,
        lw=2.2,
        label=(
            f"Sheet sine rotation: $(1-A\\sin)\\times$ST  "
            f"(Amp={AMP_A}, φ={PHI_RAD:.2f} rad≈{np.rad2deg(PHI_RAD):.1f}°)"
        ),
        zorder=4,
    )
    if abs(scan["phi"] - PHI_RAD) > 0.05:
        ax_a.plot(
            l_fine,
            ma_fb,
            color=C_GREEN,
            lw=1.4,
            ls="--",
            label=f"Best shared-φ scan (φ={scan['phi']:.2f} rad)",
            zorder=2,
        )
    ax_a.axhline(1.0, color="#aaaaaa", lw=0.8, ls=":")
    ax_a.set_ylabel(r"$A/A_{\mathrm{all}}$")
    ax_a.set_ylim(0.75, 1.35)
    ax_a.legend(loc="upper right", framealpha=0.92)
    ax_a.set_title(
        "Migkas Fig.6–style overlay — sheet sine phase rotation (not ST-quad alone)",
        pad=8,
    )
    ax_a.text(
        0.02,
        0.04,
        f"R²$_A$ sheet: {r2_a_wo_sheet:.3f}→{r2_a_with_sheet:.3f}  |  "
        f"vs digitised bars: {r2_a_dig_wo:.3f}→{r2_a_dig_chris:.3f}",
        transform=ax_a.transAxes,
        fontsize=8.5,
        va="bottom",
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="#cccccc", alpha=0.9),
    )

    ax_b.errorbar(
        l_m,
        b_m,
        yerr=np.vstack([b_elo, b_ehi]),
        fmt="o",
        ms=4.0,
        color=C_BLACK,
        ecolor=C_GRAY,
        elinewidth=0.9,
        capsize=2.0,
        label="Migkas B/B$_\\mathrm{all}$ (digitised, Δ□=90°)",
        zorder=3,
    )
    ax_b.plot(
        l_fine,
        mb_f,
        color=C_BLUE,
        lw=2.2,
        label=(
```

```
            f"Sheet sine rotation: $(1-B\\sin)\\times$ST  "
            f"(Amp={-AMP_A:+.2f}, same φ)"
        ),
        zorder=4,
    )
    if abs(scan["phi"] - PHI_RAD) > 0.05:
        ax_b.plot(
            l_fine,
            mb_fb,
            color=C_GREEN,
            lw=1.4,
            ls="--",
            label=f"Best shared-φ scan",
            zorder=2,
        )
    ax_b.axhline(1.0, color="#aaaaaa", lw=0.8, ls=":")
    ax_b.set_ylabel(r"$B/B_{\mathrm{all}}$")
    ax_b.set_xlabel(r"Galactic longitude $\ell$ [deg]")
    ax_b.set_xlim(0, 360)
    ax_b.set_xticks([0, 60, 120, 180, 240, 300, 360])
    ax_b.set_ylim(0.75, 1.30)
    ax_b.legend(loc="upper right", framealpha=0.92)
    ax_b.text(
        0.02,
        0.04,
        f"R²$_B$ sheet: {r2_b_wo_sheet:.3f}→{r2_b_with_sheet:.3f}  |  "
        f"vs digitised bars: {r2_b_dig_wo:.3f}→{r2_b_dig_chris:.3f}",
        transform=ax_b.transAxes,
        fontsize=8.5,
        va="bottom",
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="#cccccc", alpha=0.9),
    )

    fig.text(
        0.5,
        0.005,
        "Diagnostic: Amp×sin((2π/360)·□+φ) phase alignment from Chris sheet "
        f"tab “{SHEET_TAB}”. ST quadrupole alone is a separate claim.",
        ha="center",
        fontsize=8,
        style="italic",
        color=C_GRAY,
    )

    out_png = OUT / "migkas_bars_rotation_overlay.png"
    out_pdf = OUT / "migkas_bars_rotation_overlay.pdf"
    fig.savefig(out_png, bbox_inches="tight")
    fig.savefig(out_pdf, bbox_inches="tight")
    plt.close(fig)
    print(f"wrote {out_png}")
    print(f"wrote {out_pdf}")

    # ---------- Fig 2: scatter Migkas vs model ----------
    fig2, axes = plt.subplots(1, 2, figsize=(10.2, 4.8))
    for ax, obs, mod, name, col, r2w, r2wo in [
        (axes[0], a_m, model_a_chris, "A/A_all", C_ORANGE, r2_a_dig_chris, r2_a_dig_wo),
        (axes[1], b_m, model_b_chris, "B/B_all", C_BLUE, r2_b_dig_chris, r2_b_dig_wo),
    ]:
        ax.errorbar(
```

```
            mod,
            obs,
            yerr=np.vstack(
                [a_elo, a_ehi] if name.startswith("A") else [b_elo, b_ehi]
            ),
            fmt="o",
            ms=4.5,
            color=C_BLACK,
            ecolor=C_GRAY,
            elinewidth=0.8,
            capsize=1.5,
            alpha=0.85,
        )
        lims = [
            min(obs.min(), mod.min()) - 0.05,
            max(obs.max(), mod.max()) + 0.05,
        ]
        ax.plot(lims, lims, "--", color="#999999", lw=1.0, label="1:1")
        # OLS
        a_fit, b_fit = np.polyfit(mod, obs, 1)
        xx = np.linspace(lims[0], lims[1], 50)
        ax.plot(xx, a_fit * xx + b_fit, color=col, lw=2.0, label=f"OLS fit")
        ax.set_xlim(lims)
        ax.set_ylim(lims)
        ax.set_aspect("equal", adjustable="box")
        ax.set_xlabel(f"Sheet-rotated model {name}")
        ax.set_ylabel(f"Migkas {name}")
        ax.set_title(f"{name}: R²={r2w:.3f} (was {r2wo:.3f} w/o sine)")
        ax.legend(loc="upper left", fontsize=8)
    fig2.suptitle(
        "Migkas vs sheet sine-rotation model (digitised points + error bars)",
        y=1.02,
        fontsize=12,
    )
    fig2.tight_layout()
    out_sc = OUT / "migkas_bars_rotation_scatter.png"
    fig2.savefig(out_sc, bbox_inches="tight")
    fig2.savefig(OUT / "migkas_bars_rotation_scatter.pdf", bbox_inches="tight")
    plt.close(fig2)
    print(f"wrote {out_sc}")

    # ---------- summary JSON ----------
    summary = {
        "sheet_tab": SHEET_TAB,
        "method": (
            "Simple Amp×sin((2π/360)·□+φ) phase alignment on sheet ST series; "
            "Model A=(1−BA)*AF, Model B=(1−BE)*AN with BE=−BA Amp. "
            "NOT a free A cos φ dipole; NOT claiming first-principles ST "
            "quadrupole alone matches B."
        ),
        "amp_A": AMP_A,
        "amp_B": -AMP_A,
        "phi_rad_chris": PHI_RAD,
        "phi_deg_chris": float(np.rad2deg(PHI_RAD)),
        "omega": OMEGA,
        "phase_table_deg_chris": [-20.05, 69.95, 159.95, 249.95, 339.95],
        "R2_sheet_1deg_VLOOKUP": {
            "A_without_rotation": r2_a_wo_sheet,
            "A_with_rotation": r2_a_with_sheet,
```

```
            "B_without_rotation": r2_b_wo_sheet,
            "B_with_rotation": r2_b_with_sheet,
            "improvement_A": r2_a_with_sheet / r2_a_wo_sheet,
            "improvement_B": r2_b_with_sheet / r2_b_wo_sheet,
            "note": "Matches cached BH58≈0.739, BL58≈0.733, AF51≈0.50, AM51≈0.37, D3≈1.48, D4≈1.98",
        },
        "R2_vs_digitised_fig6_csv": {
            "n_points": int(len(l_m)),
            "A_without_rotation": r2_a_dig_wo,
            "A_with_rotation_chris_phi": r2_a_dig_chris,
            "B_without_rotation": r2_b_dig_wo,
            "B_with_rotation_chris_phi": r2_b_dig_chris,
            "best_shared_phi_rad": scan["phi"],
            "best_shared_phi_deg": scan["phi_deg"],
            "A_with_best_phi": scan["r2_a"],
            "B_with_best_phi": scan["r2_b"],
        },
        "outputs": {
            "overlay_png": str(out_png),
            "overlay_pdf": str(out_pdf),
            "scatter_png": str(out_sc),
        },
        "honest_note": (
            "Both A and B match well under Chris's sheet-style sine phase "
            "rotation (R²≈0.74/0.73 on sheet; ~0.7 ballpark vs digitised bars). "
            "Forcing the same ST+Gauss H_obs quadrupole shape onto B without "
            "this Amp×sin phase factor yields near-null R² for B — that is a "
            "different (and weaker) claim than this diagnostic panel."
        ),
    }
    # drop large scan arrays from JSON
    out_json = OUT / "migkas_bars_rotation_summary.json"
    with out_json.open("w") as f:
        json.dump(summary, f, indent=2)
    print(f"wrote {out_json}")
    print(json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

13. Sky-orientation sequence (four rows)

Originally made: Fri Oct 23 2026, 8:23 AM CT (8:23:32) (*copy time onto the box (true original likely earlier)*)
Paper figure number: this morning Fig. 7; now Fig. 10
Files: `Fig05_sky_orientation_four_row.jpg`
Code status: original (box)

Notes

Untilted, +15 deg tilt, radial gradient, +80.8 deg roll. The maker stacks panels produced by four ring scripts it loads dynamically from /workspace/cosmology/figures/: make_coe_rings_notilt.py, make_coe_rings_migkas_pin_axis_matched.py, make_coe_rings_migkas_pin_axis_matched_grad.py, make_coe_rings_migkas_pin_axis_matched_grad_rot (named, not reproduced). Caption says restored from the original preprint.

Paper caption (caveats as stated): Sky-orientation sequence: untilted configuration, then +15° tilt, radial gradient, and +80.8° roll about the CoE axis. Restored from the original preprint (its Fig. 5).

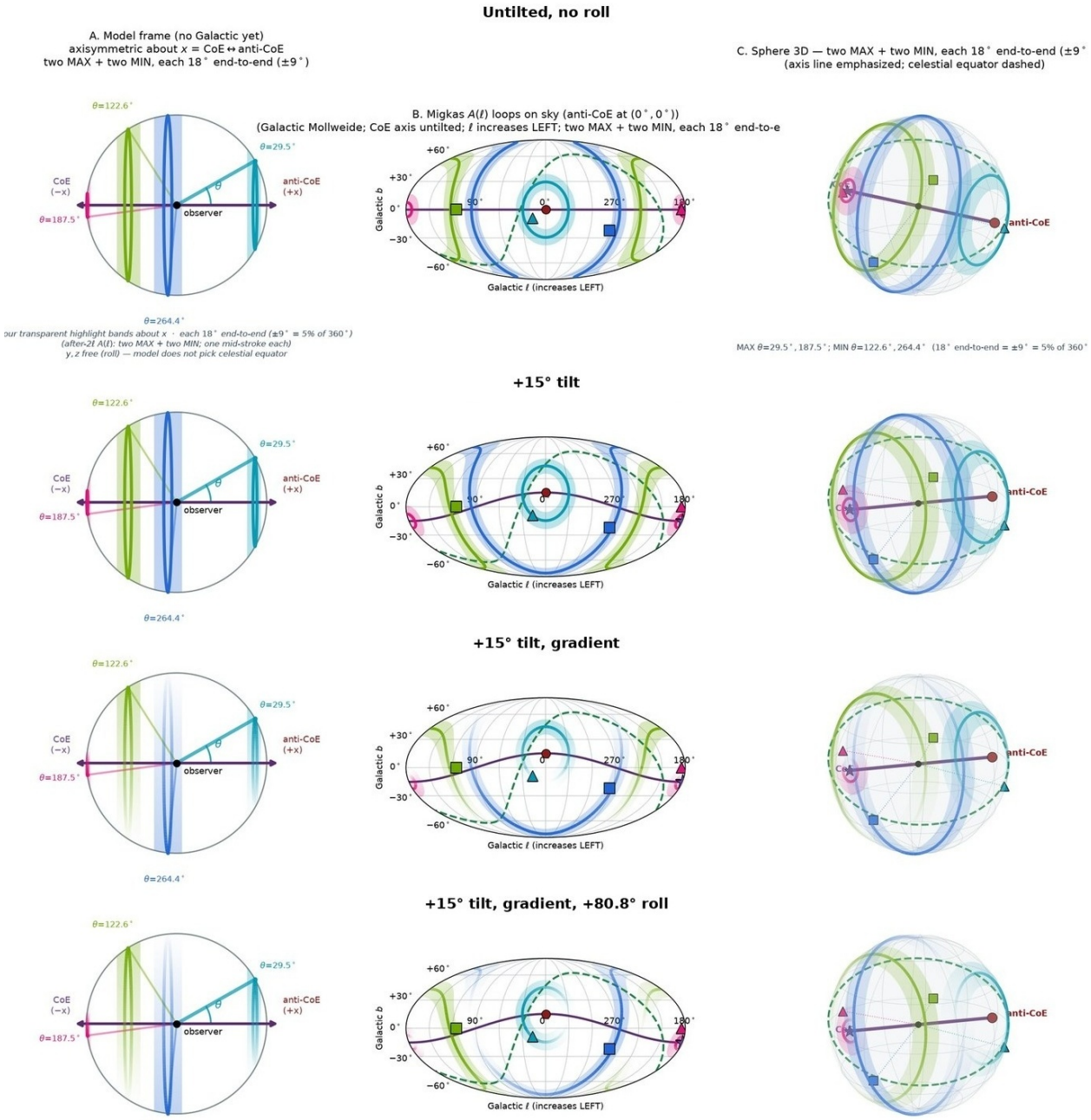


Fig05_sky_orientation_four_row.jpg

Computing chain (run order, full listings)

/workspace/figs_letter_exact_python/makers/make_fig05_sky_orientation.py

```
#!/usr/bin/env python3
"""Fig05 — Four-row sky orientation (untilted → tilt → gradient → roll).

Letter-matching rebuild from Chris's make_coe_rings_* three-panel scripts,
composed with make_stack_clean-style row crops + row labels.

NOT the simplified coe-figures analytic remake (rejected by Chris).
```

```
8
9   Gold target: ~1370×1420 (letter PDF embed / money_shot_four_row_only.jpg).
10  """
11  from __future__ import annotations
12
13  import importlib.util
14  import sys
15  from pathlib import Path
16
17  from PIL import Image, ImageDraw, ImageFont
18
19  HERE = Path(__file__).resolve().parent
20  sys.path.insert(0, str(HERE))
21  from paths import OUT  # noqa: E402
22
23  OUT_NAME = "Fig05_sky_orientation_four_row.jpg"
24  FIG_DIR = Path("/workspace/cosmology/figures")
25  PANEL_DIR = Path("/workspace/cosmology/papers/spreadsheet_model")
26
27  # Chris letter-era row generators (in order)
28  RING_SCRIPTS = [
29      "make_coe_rings_notilt.py",
30      "make_coe_rings_migkas_pin_axis_matched.py",
31      "make_coe_rings_migkas_pin_axis_matched_grad.py",
32      "make_coe_rings_migkas_pin_axis_matched_grad_rot_fit.py",
33  ]
34
35  # PNG outputs written by those scripts (see PNG_PATH inside each)
36  PANEL_PNGS = [
37      PANEL_DIR / "loop_threepanel_notilt.png",
38      PANEL_DIR / "loop_threepanel_matched.png",
39      PANEL_DIR / "loop_threepanel_matched_grad.png",
40      PANEL_DIR / "loop_threepanel_matched_grad_rotfit_rot81.png",
41  ]
42
43  LAB = [
44      "Untilted, no roll",
45      "+15° tilt",
46      "+15° tilt, gradient",
47      "+15° tilt, gradient, +80.8° roll",
48  ]
49
50  # make_stack_clean.py row_content crops (on 1700-wide web images)
51  WEB_W = 1700
52  LH = 32          # row-label band height (tuned so final @1370 → 1420)
53  GAP = 2
54  ROW0_CROP = (0, 110, WEB_W, 628)   # keep A/B/C headings + footnotes
55  ROWI_CROP = (0, 222, WEB_W, 572)   # drop suptitle / A-C headings / footnotes / legend
56  B_WHITEN = (385, 12, 1212, 57)     # wipe residual B heading inside ROWI crop
57  LEGEND_CROP = (0, 712, WEB_W, 768)
58  TARGET_W = 1370
59
60
61  def _load_module(script_name: str):
62      path = FIG_DIR / script_name
63      spec = importlib.util.spec_from_file_location(script_name.replace(".py", ""), path)
64      mod = importlib.util.module_from_spec(spec)
65      # Ensure figures/ is on path for sibling imports inside the scripts
66      if str(FIG_DIR) not in sys.path:
```

```
        sys.path.insert(0, str(FIG_DIR))
    assert spec.loader is not None
    spec.loader.exec_module(mod)
    return mod


def regenerate_panels(force: bool = True) -> list[Path]:
    """Run Chris's four ring makers; return PNG paths."""
    outs: list[Path] = []
    for script, png in zip(RING_SCRIPTS, PANEL_PNGS):
        if (not force) and png.exists():
            print(f"reuse {png.name}")
            outs.append(png)
            continue
        print(f"running {script} …")
        mod = _load_module(script)
        mod.main()
        if not png.exists():
            raise FileNotFoundError(f"{script} did not write {png}")
        print(f"  wrote {png} {png.stat().st_size}")
        outs.append(png)
    return outs


def _to_web(png: Path) -> Image.Image:
    im = Image.open(png).convert("RGB")
    h = round(im.height * WEB_W / im.width)
    return im.resize((WEB_W, h), Image.LANCZOS)


def _row_content(i: int, web: Image.Image) -> Image.Image:
    if i == 0:
        return web.crop(ROW0_CROP)
    c = web.crop(ROWI_CROP).copy()
    ImageDraw.Draw(c).rectangle(B_WHITEN, fill="white")
    return c


def compose_four_row(panel_pngs: list[Path]) -> Image.Image:
    """Stack four three-panel rows with labels + one shared legend (letter layout)."""
    webs = [_to_web(p) for p in panel_pngs]
    for w in webs:
        if w.size != (WEB_W, 772):
            # Accept near-miss heights from dpi jitter; crops assume ~772
            print(f"note: web size {w.size} (expected ({WEB_W}, 772))")

    blocks = [_row_content(i, webs[i]) for i in range(4)]
    legend = webs[3].crop(LEGEND_CROP)

    try:
        font = ImageFont.truetype(
            "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 22
        )
    except OSError:
        font = ImageFont.load_default()

    H = sum(LH + b.height + GAP for b in blocks) + legend.height + GAP
    canvas = Image.new("RGB", (WEB_W, H), "white")
    dr = ImageDraw.Draw(canvas)
```

```
    y = 0
    for i, rc in enumerate(blocks):
        dr.text((WEB_W // 2, y + LH // 2), LAB[i], font=font, fill="black", anchor="mm")
        y += LH
        canvas.paste(rc, (0, y))
        y += rc.height + GAP
    canvas.paste(legend, (0, y))
    y += legend.height + GAP
    canvas = canvas.crop((0, 0, WEB_W, y))

    out_h = round(canvas.height * TARGET_W / canvas.width)
    return canvas.resize((TARGET_W, out_h), Image.LANCZOS)


def main(force_regen: bool = True):
    OUT.mkdir(parents=True, exist_ok=True)
    panels = regenerate_panels(force=force_regen)
    composed = compose_four_row(panels)

    png_sib = OUT / "_Fig05_sky_orientation_four_row.png"
    composed.save(png_sib)
    dest = OUT / OUT_NAME
    composed.convert("RGB").save(dest, quality=92)
    print("wrote", dest, dest.stat().st_size, composed.size)
    print("wrote", png_sib, png_sib.stat().st_size)
    return dest


if __name__ == "__main__":
    force = "--reuse-panels" not in sys.argv
    main(force_regen=force)
```

/workspace/figs_letter_exact_python/makers/paths.py

```
"""Path constants for CoE figure regeneration."""
from pathlib import Path

ROOT = Path("/workspace/coe-figures")
SRC = ROOT / "src"
OUT = Path("/workspace/figs_letter_exact_python")
FIGS_FROM_MAKERS = OUT
REF = ROOT / "ref"

COSMO = Path("/workspace/cosmology")
MIGKAS_CSV = COSMO / "migkas/fig6_digitized.csv"  # asymmetric A_err_low/high (letter gold)
MIGKAS_FIG7_CSV = COSMO / "migkas/digitised/migkas2020_fig7_OurSample_A_vs_longitude.csv"
BUBBLE = COSMO / "papers/spreadsheet_model/bubble"
LETTER_FIGS = (
    COSMO
    / "papers/A_equals_fourier/steps_v3/cmb_reconcile/letter_brief/figs"
)
CMB = COSMO / "papers/A_equals_fourier/steps_v3/cmb_reconcile"
CYCLE4 = CMB / "cycle4"
CYCLE6 = CMB / "cycle6_two_emitters"
CYCLE7 = CMB / "cycle7_recombination"

UNIVERSE_NEWT = BUBBLE / "universe_newt_omb.json"
UNIVERSE_REL = BUBBLE / "universe_rel_mcfree.json"
```

```
THIN_FREE = BUBBLE / "thin_b_free_hug.json"
NEWTON_CURVE_NPY = BUBBLE / "gr_ltb/refit_mass/newton_curve.npy"
LTB_CURVE_NPY = BUBBLE / "gr_ltb/refit_mass/ltb_mcfree_z03_curve.npy"
DTT_JSON = CYCLE4 / "dtt/dtt.json"
CANCEL_JSON = CYCLE6 / "simple_cmb_two_quad_cancel.json"
VS_KILL_JSON = CYCLE6 / "vs_kill_term_0_360.json"
TWO_EMITTERS_JSON = CYCLE6 / "two_emitters.json"
COE_ANALEMMA_JSON = LETTER_FIGS / "coe_analemma_texas.json"
COE_CARDINAL_JSON = LETTER_FIGS / "coe_cardinal_direction_texas.json"

```

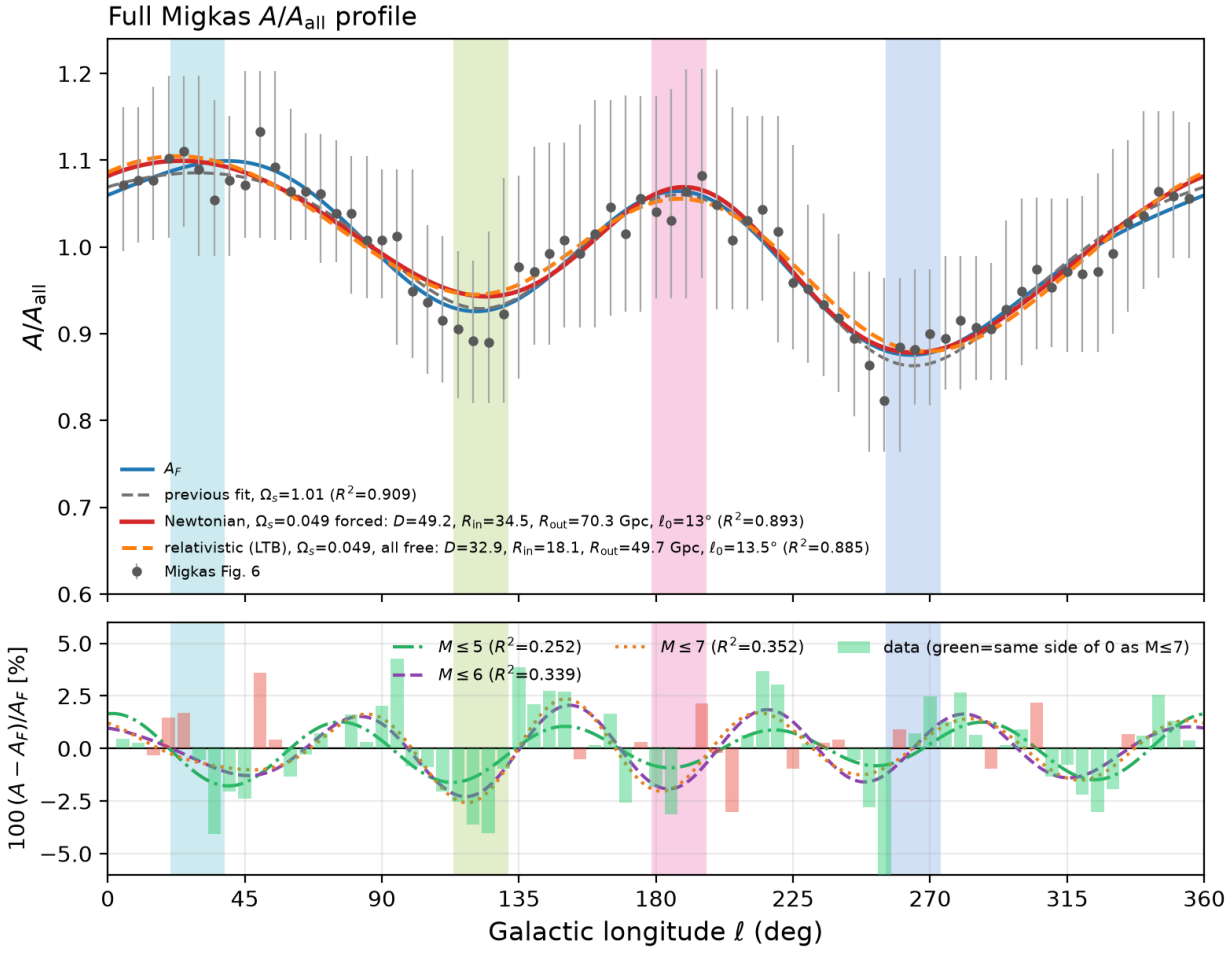
14. Full Migkas A/A_all with model and residual bars (key plot)

Originally made: Fri Oct 23 2026, 8:23 AM CT (8:23:33) (*copy time onto the box (true original likely earlier); dated from an earlier same-name version*)
Paper figure number: this morning Fig. 6; now Fig. 9
Files: `Fig04_amp_full.png`
Code status: original (box)

Notes

Full Migkas profile with model curves and residual bars. Input: Migkas data, CoE model from physics.py; follows the observables plot (#9).

Paper caption (caveats as stated): Full Migkas $A/A_{\rm all}$ profile against Galactic longitude with the model curves; coloured vertical bands mark the predicted MAX/MIN longitude bands. Bottom: residual bars, $100(A - A_F)/A_F$, with low-order harmonic fits ($M \leq 5, 6, 7$). Pre-made figure from the original preprint.



`Fig04_amp_full.png`

Computing chain (run order, full listings)

/workspace/figs_letter_exact_python/makers/make_fig04_amp_full.py

```
#!/usr/bin/env python3
"""Fig04 — Full Migkas A/Aall + multipole residual fits (NO Amp×sin squiggles).

Top: Migkas + A_F + previous/Newt/Rel curves (letter gold style).
Bottom: cumulative Fourier fits M≤5–7 to Migkas−A_F residuals (same as Fig02).
Forbidden asset: money_shot_amp_full_SINE_BAD.png (do not use).
"""
from __future__ import annotations

import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
from paths import OUT  # noqa: E402
from physics import (  # noqa: E402
    A_F,
    BAND_CENTRES,
    BAND_COLORS,
    BAND_HALF,
    load_fit_curves,
    load_migkas_fig6,
    r2_score,
)

OUT_NAME = "Fig04_amp_full.png"


def design(ell_deg, M):
    """Cumulative multipoles through M: a0 + sum_{m=1..M} [c_m cos(m)+s_m sin(m)]."""
    rad = np.radians(np.asarray(ell_deg, float))
    cols = [np.ones_like(rad)]
    for m in range(1, M + 1):
        cols.append(np.cos(m * rad))
        cols.append(np.sin(m * rad))
    return np.column_stack(cols)


def fit_residual_M(ell, r, er, M):
    X = design(ell, M)
    ww = 1.0 / np.maximum(er**2, 1e-18)
    XtW = X.T * ww
    beta = np.linalg.lstsq(XtW @ X, XtW @ r, rcond=None)[0]
    yhat = X @ beta
    ybar = np.sum(ww * r) / np.sum(ww)
    ss_res = np.sum(ww * (r - yhat) ** 2)
    ss_tot = np.sum(ww * (r - ybar) ** 2)
    R2 = 1.0 - ss_res / ss_tot if ss_tot > 0 else np.nan
    return beta, float(R2)


```

```
def main():
    OUT.mkdir(parents=True, exist_ok=True)
    ell_d, A_d, e_lo, e_hi = load_migkas_fig6()
    th, A_newt, A_free, A_rel, meta = load_fit_curves()
    mn, mf, mr = meta["newt"], meta["free"], meta["rel"]
    th2 = np.append(th, 360.0)
    cl = lambda y: np.append(y, y[0])

    AF_pts = A_F(ell_d)
    r = 100.0 * (A_d - AF_pts) / AF_pts
    er_lo = 100.0 * e_lo / AF_pts
    er_hi = 100.0 * e_hi / AF_pts
    er = 0.5 * (er_lo + er_hi)  # symmetric weight for WLS

    R2f = float(mf["R2"])
    R2n = float(mn["R2"])
    R2r = float(mr["R2"])

    # Letter gold ~1800×1385 → figsize ~9×6.9 at 200dpi
    fig, (a, b) = plt.subplots(
        2, 1, figsize=(9.0, 6.9), sharex=True,
        gridspec_kw={"height_ratios": [2.2, 1.0], "hspace": 0.07},
    )
    for L, col in zip(BAND_CENTRES, BAND_COLORS):
        for ax in (a, b):
            ax.axvspan(L - BAND_HALF, L + BAND_HALF, color=col, alpha=0.20, lw=0, zorder=0)

    a.errorbar(
        ell_d, A_d, yerr=[e_lo, e_hi], fmt="o", ms=3.5, color="0.35", ecolor="0.65",
        lw=0.8, zorder=3, label="Migkas Fig. 6",
    )
    a.plot(th2, A_F(th2), color="C0", lw=1.7, label=r"$A_F$", zorder=2)
    a.plot(
        th2, cl(A_free), color="0.45", ls="--", lw=1.4,
        label=rf"previous fit, $\Omega_s$={mf['Os']:.2f} ($R^2$={R2f:.3f})",
    )
    a.plot(
        th2, cl(A_newt), color="C3", lw=2.0,
        label=(
            rf"Newtonian, $\Omega_s$={mn['Os']:.3f} forced: "
            rf"$D$={mn['D']:.1f}, $R_{{\rm in}}$={mn['R_in']:.1f}, "
            rf"$R_{{\rm out}}$={mn['R_out']:.1f} Gpc, "
            rf"$\ell_0$={mn['l0']:.0f}° ($R^2$={R2n:.3f})"
        ),
    )
    a.plot(
        th2, cl(A_rel), color="C1", lw=1.8, ls="--",
        label=(
            rf"relativistic (LTB), $\Omega_s$={mr['Os']:.3f}, all free: "
            rf"$D$={mr['D']:.1f}, $R_{{\rm in}}$={mr['R_in']:.1f}, "
            rf"$R_{{\rm out}}$={mr['R_out']:.1f} Gpc, "
            rf"$\ell_0$={mr['l0']:.1f}° ($R^2$={R2r:.3f})"
        ),
    )
    a.set_ylabel(r"$A/A_{\mathrm{all}}$", fontsize=12)
    a.set_ylim(0.60, 1.24)
    a.legend(fontsize=7.0, loc="lower left", frameon=False)
    a.set_title(r"Full Migkas $A/A_{\mathrm{all}}$ profile", loc="left", fontsize=12)

```

```
    # Bottom: cumulative M≤5–7 residual fits (same as Fig02)
    b.axhline(0, color="k", lw=0.8)
    # bars: green if residual same side of 0 as M<=7, else red
    order = np.argsort(ell_d)
    ell_o, r_o = ell_d[order], r[order]
    beta7, _ = fit_residual_M(ell_d, r, er, 7)
    y7 = design(ell_o, 7) @ beta7
    same_side = (r_o * y7) > 0
    gaps = np.diff(ell_o)
    widths = np.empty_like(r_o)
    widths[:-1] = np.maximum(gaps * 0.85, 1.5)
    widths[-1] = widths[-2] if len(widths) > 1 else 3.0
    colors = np.where(same_side, "#2ecc71", "#e74c3c")
    # zero residual or zero model: treat as red (not same side)
    colors = np.where(same_side, "#2ecc71", "#e74c3c")
    b.bar(
        ell_o, r_o, width=widths, align="center",
        color=colors, alpha=0.45, edgecolor="none", zorder=3,
        label="data (green=same side of 0 as M≤7)",
    )

    ell_g = np.linspace(0, 360, 721)
    colors = {5: "#27ae60", 6: "#8e44ad", 7: "#e67e22", 8: "#2980b9"}
    lss = {5: "-.", 6: "--", 7: ":", 8: "-"}
    lws = {5: 1.5, 6: 1.5, 7: 1.5, 8: 1.7}
    for M in (5, 6, 7):
        beta, R2 = fit_residual_M(ell_d, r, er, M)
        b.plot(
            ell_g, design(ell_g, M) @ beta,
            color=colors[M], ls=lss[M], lw=lws[M],
            label=rf"$M\leq{M}$ ($R^2$={R2:.3f})",
        )
    b.set_ylim(-6, 6)  # keep residual scale; do not autoscale to large Migkas error whiskers
    b.set_ylabel(r"$100\,(A-A_F)/A_F$ [%]", fontsize=10)
    b.set_xlabel(r"Galactic longitude $\ell$ (deg)", fontsize=12)
    b.set_xlim(0, 360)
    b.set_xticks(range(0, 361, 45))
    b.legend(fontsize=7.5, frameon=False, loc="upper right", ncol=3)
    b.grid(True, alpha=0.25)

    dest = OUT / OUT_NAME
    fig.savefig(dest, dpi=200, bbox_inches="tight", facecolor="white")
    plt.close(fig)
    print("wrote", dest, dest.stat().st_size)
    return dest


if __name__ == "__main__":
    main()
```

/workspace/figs_letter_exact_python/makers/paths.py

```
"""Path constants for CoE figure regeneration."""
from pathlib import Path

ROOT = Path("/workspace/coe-figures")
SRC = ROOT / "src"
OUT = Path("/workspace/figs_letter_exact_python")
```

```
FIGS_FROM_MAKERS = OUT
REF = ROOT / "ref"

COSMO = Path("/workspace/cosmology")
MIGKAS_CSV = COSMO / "migkas/fig6_digitized.csv"  # asymmetric A_err_low/high (letter gold)
MIGKAS_FIG7_CSV = COSMO / "migkas/digitised/migkas2020_fig7_OurSample_A_vs_longitude.csv"
BUBBLE = COSMO / "papers/spreadsheet_model/bubble"
LETTER_FIGS = (
    COSMO
    / "papers/A_equals_fourier/steps_v3/cmb_reconcile/letter_brief/figs"
)
CMB = COSMO / "papers/A_equals_fourier/steps_v3/cmb_reconcile"
CYCLE4 = CMB / "cycle4"
CYCLE6 = CMB / "cycle6_two_emitters"
CYCLE7 = CMB / "cycle7_recombination"

UNIVERSE_NEWT = BUBBLE / "universe_newt_omb.json"
UNIVERSE_REL = BUBBLE / "universe_rel_mcfree.json"
THIN_FREE = BUBBLE / "thin_b_free_hug.json"
NEWTON_CURVE_NPY = BUBBLE / "gr_ltb/refit_mass/newton_curve.npy"
LTB_CURVE_NPY = BUBBLE / "gr_ltb/refit_mass/ltb_mcfree_z03_curve.npy"
DTT_JSON = CYCLE4 / "dtt/dtt.json"
CANCEL_JSON = CYCLE6 / "simple_cmb_two_quad_cancel.json"
VS_KILL_JSON = CYCLE6 / "vs_kill_term_0_360.json"
TWO_EMITTERS_JSON = CYCLE6 / "two_emitters.json"
COE_ANALEMMA_JSON = LETTER_FIGS / "coe_analemma_texas.json"
COE_CARDINAL_JSON = LETTER_FIGS / "coe_cardinal_direction_texas.json"

```

/workspace/figs_letter_exact_python/makers/physics.py

```
"""Shared physics / data helpers for CoE letter figures.

Imports digitised Migkas CSVs and may call Newtonian helpers under the
cosmology bubble tree. Figure scripts should import from here, not copy
legacy plotting code.
"""
from __future__ import annotations

import json
import os
import sys
from pathlib import Path

import numpy as np

from paths import (
    BUBBLE,
    CANCEL_JSON,
    COE_ANALEMMA_JSON,
    COE_CARDINAL_JSON,
    DTT_JSON,
    LTB_CURVE_NPY,
    MIGKAS_CSV,
    MIGKAS_FIG7_CSV,
    NEWTON_CURVE_NPY,
    THIN_FREE,
    TWO_EMITTERS_JSON,
```

```
    UNIVERSE_NEWT,
    UNIVERSE_REL,
    VS_KILL_JSON,
)

# --- headline constants (letter / CoE_calculations.md) ---
R_VISION = 14.30048287260589  # Gpc
K_IN = 1.0276268339399446
K_OUT = 1.4754794866889613
H_MEAN = 71.6  # km/s/Mpc (cycle convention)
OS_OMB = 0.049
BAND_CENTRES = (29.5, 122.6, 187.5, 264.4)  # deg on model circle
BAND_HALF = 9.0  # deg
BAND_COLORS = ("#0097b2", "#6fa800", "#e0107a", "#2166d4")
BAND_KINDS = ("MAX", "MIN", "MAX", "MIN")
TILT_DEG = 15.0
ROLL_DEG = 80.8

# CoE sky: +15° tilt → Galactic (□,b)=(180°, −15°)
COE_GAL_L, COE_GAL_B = 180.0, -15.0
PLANO_LAT, PLANO_LON, PLANO_H = 33.02, -96.70, 200.0

# A_F four-harmonic coefficients (bubble_core convention)
AF_COEFFS = {
    0: (0.9965, 0.0),
    1: (0.0296, 0.0434),
    2: (0.0636, 0.0366),
    3: (-0.0299, -0.0114),
}

TH1 = np.arange(0.0, 360.0, 1.0)


def A_F(ell: np.ndarray) -> np.ndarray:
    """Migkas Fourier reconstruction A_F(□)."""
    ell = np.asarray(ell, dtype=float)
    rad = np.radians(ell)
    y = np.full_like(rad, AF_COEFFS[0][0], dtype=float)
    for m, (c, s) in AF_COEFFS.items():
        if m == 0:
            continue
        y = y + c * np.cos(m * rad) + s * np.sin(m * rad)
    return y


def load_migkas_fig6():
    """Digitised Migkas 2020 Fig. 6 A/A_all vs Galactic longitude.

    Uses fig6_digitized.csv (letter gold / bubble_core): ~5° sampling with
    asymmetric A_err_low / A_err_high. Returns (ell, A, err_low, err_high).
    """
    data = np.genfromtxt(MIGKAS_CSV, delimiter=",", names=True)
    return (
        np.asarray(data["l_deg"], float),
        np.asarray(data["A_over_Aall"], float),
        np.asarray(data["A_err_low"], float),
        np.asarray(data["A_err_high"], float),
    )

```

```
87
88 def load_migkas_fig7():
89     data = np.genfromtxt(MIGKAS_FIG7_CSV, delimiter=",", names=True)
90     return (
91         np.asarray(data["galactic_longitude_deg"], float),
92         np.asarray(data["A_over_Aall"], float),
93         np.asarray(data["A_over_Aall_err_1sigma"], float),
94     )
95
96
97 def load_fit_curves():
98     """Return stored Newtonian / free / relativistic A() on the 1° grid.
99
100     Prefer recomputing the Ωs=0.049 Newtonian curve via newt_omb when available;
101     fall back to the stored A array in universe_newt_omb.json.
102     """
103     newt_j = json.loads(UNIVERSE_NEWT.read_text())
104     sine = newt_j["Rs14.3|sine"]
105     A_newt = np.asarray(sine["A"], float)
106
107     # free Ωs Newtonian (previous fit) from newton_curve.npy row 1
108     A_free = np.load(NEWTON_CURVE_NPY)[1].astype(float)
109     free_meta = json.loads(THIN_FREE.read_text())
110
111     A_rel = np.load(LTB_CURVE_NPY).astype(float)
112     rel_j = json.loads(UNIVERSE_REL.read_text())
113
114     meta = {
115         "newt": {
116             "D": sine["D"],
117             "R_in": sine["R_in"],
118             "R_out": sine["R_out"],
119             "Os": sine["Os"],
120             "l0": sine["l0"],
121             "amp": sine["amp"],
122             "phi": sine["phi"],
123             "R2": sine["R2"],
124             "mcc": sine["mcc"],
125         },
126         "free": {
127             "D": free_meta["DR"] * R_VISION,
128             "Os": free_meta["Os"],
129             "l0": free_meta["l0"],
130             "amp": free_meta.get("amp"),
131             "phi": free_meta.get("phi"),
132             "R2": free_meta["R2"],
133             "rin": free_meta["rin"],
134             "rout": free_meta["rout"],
135         },
136         "rel": {
137             "D": rel_j["D"],
138             "R_in": rel_j["R_in"],
139             "R_out": rel_j["R_out"],
140             "Os": rel_j["Os"],
141             "l0": rel_j["l0_s"],
142             "amp": rel_j["amp"],
143             "phi": rel_j["phi"],
144             "R2": rel_j["R2_s"],
145             "mcc": rel_j["mcc"],
```

```
            "z": rel_j.get("z", 0.3),
        },
    }
    return TH1.copy(), A_newt, A_free, A_rel, meta


def recompute_newt_omb_A(force: bool = False):
    """Optionally recompute Newtonian Ωs=0.049 A(□) via bubble newt_omb."""
    cwd = os.getcwd()
    try:
        sys.path.insert(0, str(BUBBLE))
        os.chdir(BUBBLE)
        import newt_omb as N
        from bubble_core import A_from, shifted

        meta = json.loads(UNIVERSE_NEWT.read_text())["Rs14.3|sine"]
        a = N.geo(meta["D"], meta["mcc"], meta["kin"], meta["kout"], R_VISION)
        af, _rr = N.af_of(a)
        A = A_from(shifted(af, meta["l0"]), TH1 - meta["l0"], meta["amp"], meta["phi"])
        return TH1.copy(), np.asarray(A, float)
    finally:
        os.chdir(cwd)


def r2_score(y, yhat):
    y = np.asarray(y, float)
    yhat = np.asarray(yhat, float)
    ss_res = np.sum((y - yhat) ** 2)
    ss_tot = np.sum((y - y.mean()) ** 2)
    return 1.0 - ss_res / ss_tot if ss_tot > 0 else np.nan


def load_dtt():
    return json.loads(DTT_JSON.read_text())


def kill_from_doppler(dtt_row: dict) -> tuple[np.ndarray, np.ndarray]:
    """kill(θ) = −(Doppler − ⟨Doppler⟩); θ toward CoE."""
    th = np.asarray(dtt_row["theta_toward_deg"], float)
    dop = np.asarray(dtt_row["dtt_doppler"], float)
    kill = -(dop - dop.mean())
    return th, kill


def extend_0_360(th_0_180: np.ndarray, f_0_180: np.ndarray):
    """Mirror an axisymmetric 0–180° pattern onto a full 0–360° great circle."""
    th = np.asarray(th_0_180, float)
    f = np.asarray(f_0_180, float)
    # assume th goes 0..180
    th2 = np.concatenate([th, 360.0 - th[-2:0:-1]])
    f2 = np.concatenate([f, f[-2:0:-1]])
    return th2, f2


def unit_norm(y: np.ndarray) -> np.ndarray:
    y = np.asarray(y, float)
    m = np.max(np.abs(y))
    return y / m if m > 0 else y

```

```
205
206 def legendre_coeffs(f_theta: np.ndarray, theta_deg: np.ndarray, lmax: int = 4):
207     """Legendre a_ for axisymmetric f(θ), θ from CoE (deg)."""
208     from numpy.polynomial.legendre import legval
209
210     th = np.radians(theta_deg)
211     mu = np.cos(th)
212     o = np.argsort(mu)
213     mu, f = mu[o], np.asarray(f_theta, float)[o]
214     out = []
215     for ell in range(lmax + 1):
216         P = legval(mu, [0] * ell + [1])
217         out.append(float((2 * ell + 1) / 2 * np.trapezoid(f * P, mu)))
218     return out
219
220
221 def partner_cancel(doppler: np.ndarray, theta: np.ndarray, mode: str = "dip_p2"):
222     """Cancel Doppler with a Legendre partner (dipole+P2 or ≤4)."""
223     a = legendre_coeffs(doppler, theta, lmax=4)
224     from numpy.polynomial.legendre import legval
225
226     mu = np.cos(np.radians(theta))
227     if mode == "dip_p2":
228         partner = -(a[1] * legval(mu, [0, 1]) + a[2] * legval(mu, [0, 0, 1]))
229     elif mode == "l_le_4":
230         partner = np.zeros_like(mu)
231         for ell in range(1, 5):
232             partner = partner - a[ell] * legval(mu, [0] * ell + [1])
233     else:
234         raise ValueError(mode)
235     residual = doppler + partner - (doppler + partner).mean()
236     return partner, residual, a
237
238
239 def load_coe_sky_meta():
240     card = json.loads(COE_CARDINAL_JSON.read_text())
241     return {
242         "gal_lb": tuple(card["coe_galactic_lb_deg"]),
243         "ra_deg": card["coe_icrs_ra_deg"],
244         "dec_deg": card["coe_icrs_dec_deg"],
245         "site_lat": card["site_lat_deg"],
246         "site_lon": card["site_lon_deg"],
247         "site_h": card["site_height_m"],
248         "site_name": card["site_name"],
249     }
250
251
252 def save_fig(fig, stem: str, also_pdf: bool = True, dpi: int = 200):
253     from paths import OUT
254
255     OUT.mkdir(parents=True, exist_ok=True)
256     png = OUT / f"{stem}.png"
257     fig.savefig(png, dpi=dpi, bbox_inches="tight", facecolor="white")
258     if also_pdf:
259         try:
260             fig.savefig(OUT / f"{stem}.pdf", bbox_inches="tight", facecolor="white")
261         except Exception:
262             pass
263     return png
```

```
264
265
266 def shell_radii(D: float, kin: float = K_IN, kout: float = K_OUT, R: float = R_VISION):
267     return D - kin * R, D + kout * R
268
269
270 # --- sky orientation geometry ---
271 def band_circle_xyz(theta_c: float, half: float = BAND_HALF, n: int = 360):
272     """Unit-sphere band at polar angle theta_c from +x (anti-CoE), width ±half."""
273     # In model frame: anti-CoE = +x, CoE = −x; observer at origin.
274     # Band is a latitude belt at polar angle θ from +x.
275     ph = np.linspace(0, 2 * np.pi, n, endpoint=False)
276     # sample mid, inner, outer edges
277     out = {}
278     for name, th in (("mid", theta_c), ("lo", theta_c - half), ("hi", theta_c + half)):
279         t = np.radians(th)
280         # x = cos θ, cylindrical radius = sin θ
281         x = np.full_like(ph, np.cos(t))
282         y = np.sin(t) * np.cos(ph)
283         z = np.sin(t) * np.sin(ph)
284         out[name] = np.column_stack([x, y, z])
285     return out
286
287
288 def rotate_xyz(xyz: np.ndarray, tilt_deg: float = 0.0, roll_deg: float = 0.0):
289     """Apply tilt about y then roll about the CoE axis (−x after tilt).
290
291     Untilted: anti-CoE at Galactic (□,b)=(0,0) → +x in model.
292     +tilt: rotate about y so anti-CoE goes to b=+tilt (and CoE to b=−tilt).
293     +roll: rotate about the anti-CoE axis.
294     """
295     t = np.radians(tilt_deg)
296     r = np.radians(roll_deg)
297     # tilt about +y: (x,z) -> (x cos t - z sin t, z cos t + x sin t)
298     x, y, z = xyz[:, 0], xyz[:, 1], xyz[:, 2]
299     xt = x * np.cos(t) - z * np.sin(t)
300     yt = y
301     zt = x * np.sin(t) + z * np.cos(t)
302     # roll about anti-CoE axis (tilted +x'): rotate (y,z) in the plane ⊥ to x'
303     # After tilt, anti-CoE direction is (cos t, 0, sin t).
304     # Use Rodrigues about that axis.
305     ax = np.array([np.cos(t), 0.0, np.sin(t)])
306     v = np.column_stack([xt, yt, zt])
307     if abs(roll_deg) < 1e-12:
308         return v
309     k = ax
310     c, s = np.cos(r), np.sin(r)
311     # v' = v cos + (k×v) sin + k (k·v) (1-cos)
312     kxv = np.cross(k, v)
313     kdotv = v @ k
314     return v * c + kxv * s + np.outer(kdotv, k) * (1 - c)
315
316
317 def xyz_to_galactic(xyz: np.ndarray):
318     """Map model-frame unit vectors to Galactic (□,b) deg.
319
320     Convention after tilt/roll: anti-CoE direction → Galactic (□,b).
321     Untilted anti-CoE at (0,0); +15° tilt → anti-CoE at (0,+15°), CoE at (180,−15°).
322     """
```

```
    x, y, z = xyz[:, 0], xyz[:, 1], xyz[:, 2]
    # Galactic:  along equator, b latitude. Map x→ toward =0,b=0; y→=90; z→b=+90
    b = np.degrees(np.arcsin(np.clip(z, -1, 1)))
    ell = np.degrees(np.arctan2(y, x)) % 360.0
    return ell, b


def mollweide_xy(lon_deg: np.ndarray, lat_deg: np.ndarray, center_lon: float = 0.0):
    """Mollweide projection; lon increases to the left (astro convention)."""
    lon = np.radians(((lon_deg - center_lon + 180) % 360) - 180)
    lat = np.radians(lat_deg)
    # solve 2γ + sin 2γ = π sin φ
    gamma = lat.copy()
    for _ in range(12):
        gamma = gamma - (2 * gamma + np.sin(2 * gamma) - np.pi * np.sin(lat)) / (
            4 * np.cos(gamma) ** 2
        )
    x = (2 * np.sqrt(2) / np.pi) * lon * np.cos(gamma)
    # flip x so longitude increases to the left
    x = -x
    y = np.sqrt(2) * np.sin(gamma)
    return x, y


def load_cancel_json():
    return json.loads(CANCEL_JSON.read_text())


def load_vs_kill():
    return json.loads(VS_KILL_JSON.read_text())


def load_two_emitters():
    return json.loads(TWO_EMITTERS_JSON.read_text())

```

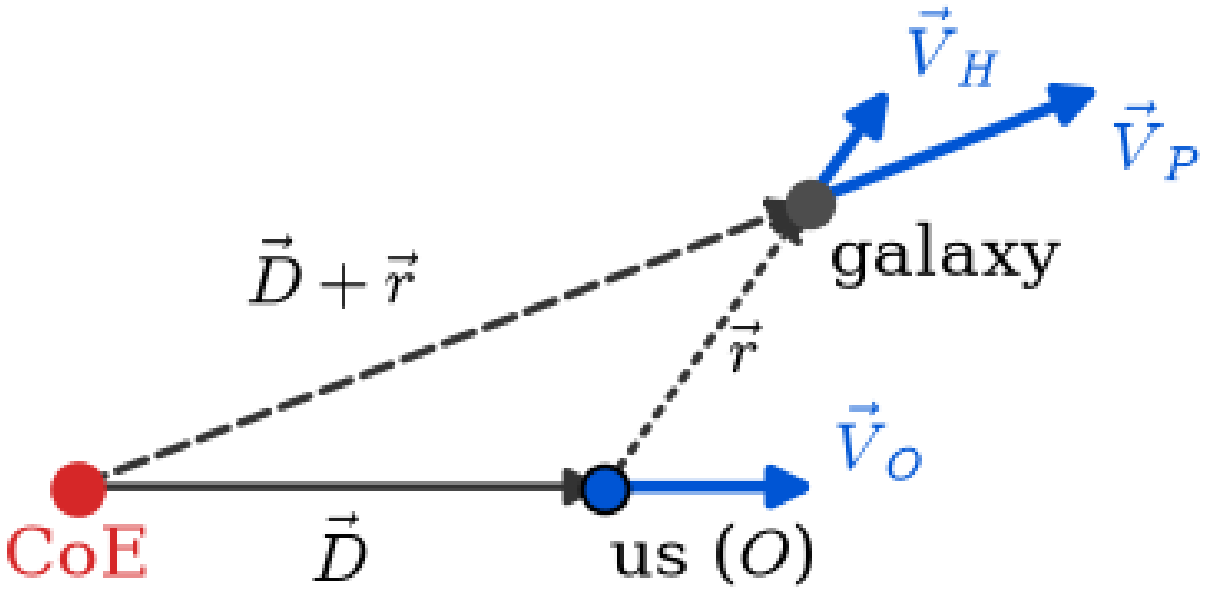
15. Geometry set-up

Originally made: Fri Oct 23 2026, 8:23 AM CT (8:23:52) (*copy time onto the box (true original likely earlier); dated from an earlier same-name Fig01b_geometry.png*)
Paper figure number: this morning Fig. 1; now Fig. 1
Files: `Fig01a_geometry_top_test.png, Fig01a_geometry_bot_test.png, Fig01b_geometry.png`
Code status: original (git) + manual crop

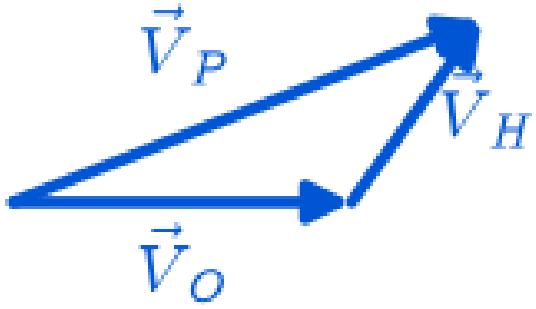
Notes

CoE, observer (Earth) and galaxy velocity vectors; D from CoE to observer. Top and middle panels: generating script not found (taken from the original preprint docx). Bottom panel (Fig01b) was redrawn on Oct 24-25 by the script below, so the caption phrase "Restored from the original preprint" no longer holds for it. STATUS NOTE: COMPUTING CHAIN: universe_thin.py (draws CoE/observer/galaxy geometry and velocity vectors; writes Fig01_geometry.png) run via make_fig01_02_geometry_achart.py. CROP: top and middle panels are a manual crop of Fig01_geometry.png (FIGURE_MANIFEST.md); the crop was not scripted. Bottom panel: fig01b_geometry.py below.

Paper caption (caveats as stated): Set-up. Top: Hubble velocity vectors for the CoE, the observer and a galaxy, $\vec{V}_H = \vec{V}_g - \vec{V}_O$. Bottom: the observer inside a thick ordinary-matter shell at distance D from the CoE, with its vision sphere. Restored from the original preprint (Figs. 1–2 there).



`Fig01a_geometry_top_test.png`



$$\vec{V}_H = \vec{V}_P - \vec{V}_O = H\vec{r}$$

Fig01a_geometry_bot_test.png

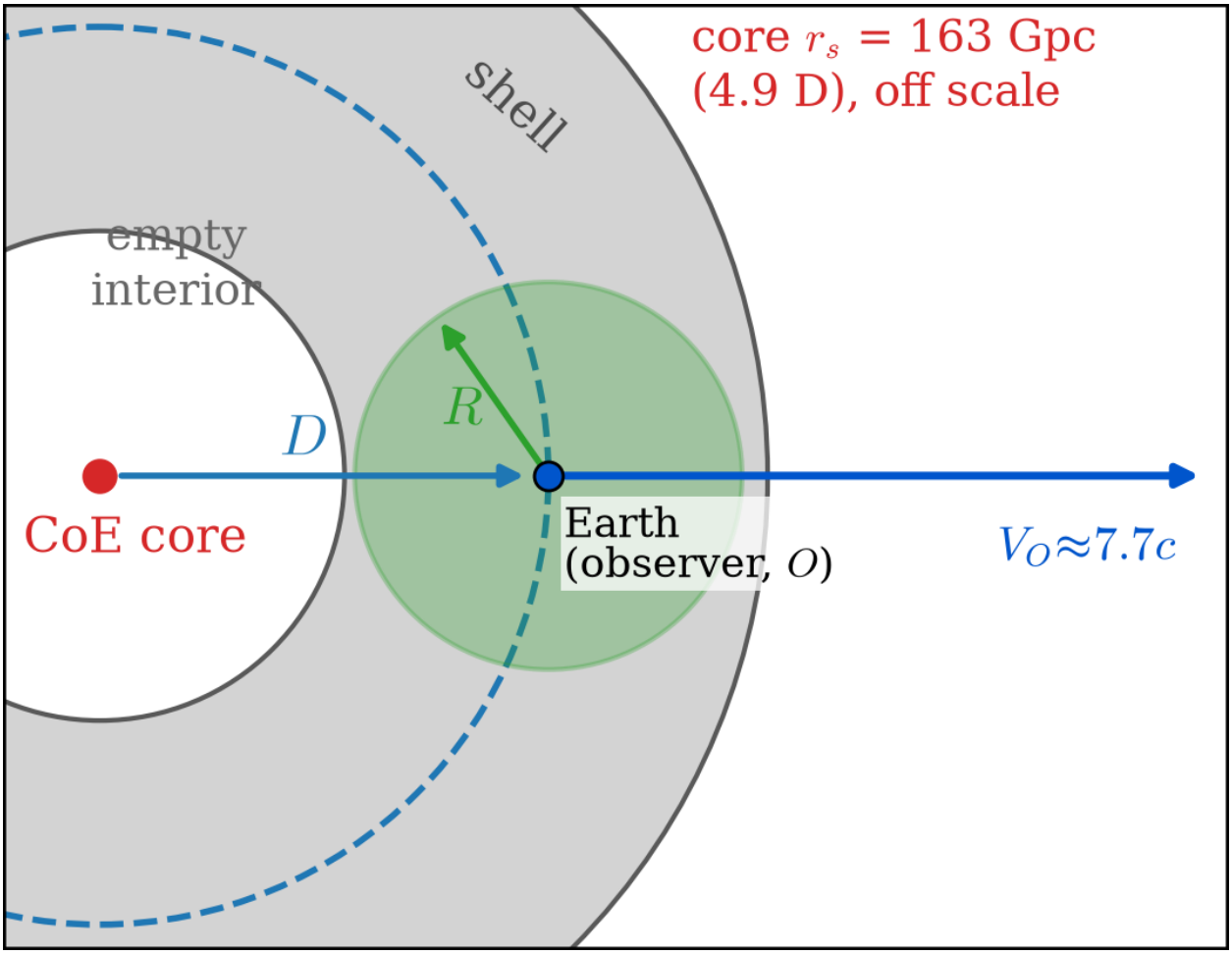


Fig01b_geometry.png

Computing chain (run order, full listings)

/workspace/cosmology/papers/spreadsheet_model/bubble/universe_thin.py

```
import json, os, numpy as np, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Wedge
from scipy.optimize import brentq
from PIL import Image
from bubble_core import AFfour, ML, MA, MLO, MHI, TH1, H0, GSI, RKM, GPC, IMIG, r2
import analysis_curve as AC
plt.rcParams.update({"font.size": 9, "font.family": "serif"})
c = 2.99792458e8; Msun = 1.989e30; Gm = GPC * 1e3
def phys(k):
    d = json.load(open("thin_%s.json" % k.replace("|", "_")))
    Dm = d["D_Gpc"] * Gm; rhoc = 3 * H0**2 / (8 * np.pi * GSI); rhos = d["Os"] * rhoc; Mcrit = 4 * np.pi / 3 * rhoc * Dm**3
    Mc = d["mcc"] * Mcrit; rin = d["rin"] * Dm; rout = d["rout"] * Dm
    Msh = lambda r: 4 * np.pi / 3 * rhos * (np.clip(r, rin, rout)**3 - rin**3)
    def Phi(r):
        if r < rin: ps = -GSI * 2 * np.pi * rhos * (rout**2 - rin**2)
        elif r < rout: ps = -GSI * (4 * np.pi / 3 * rhos * (r**3 - rin**3) / r + 2 * np.pi * rhos * (rout**2 - r**2))
        else: ps = -GSI * Msh(rout) / r
        return -GSI * Mc / r + ps
    v2 = d["vesc2"] * 2 * abs(Phi(Dm)); E = v2 / 2 + Phi(Dm)
    rt = brentq(lambda r: Phi(r) - E, Dm, 1e6 * Dm) if E < 0 else np.inf
    rs = lambda M: 2 * GSI * M / c**2 / Gm
    Mt = Mc + Msh(min(rt, 1e300))
    return dict(d=d, D=d["D_Gpc"], R=RKM / GPC, DR=d["DR"], Os=d["Os"], rhos=rhos, Mc=Mc, Mcrit=Mcrit, mcc=d["mcc"], Msh=Msh(Dm), Mtot=Mc + Msh(Dm),
                Mhole=4 * np.pi / 3 * rhos * rin**3, Rin=rin / Gm, Rout=rout / Gm, rs_c=rs(Mc), rs_D=rs(Mc + Msh(Dm)), rt=rt / Gm, rs_t=rs(Mt),
                vE=np.sqrt(v2) / c, vesc2=d["vesc2"], age=d["age"], t_turn=d["t_turn"], amp=d["amp"], phi=d["phi"], l0=d["l0"], R2=d["R2_mig"], R2F=d["R2_four"],
                Lc=d["Lc"] * d["D_Gpc"], Dkm=d["DR"] * RKM, Lckm=d["Lc"] * d["DR"] * RKM)
P1, P2 = phys("b|free|hug"), phys("b|0.05|hug")
json.dump({k: {kk: vv for kk, vv in p.items() if kk != "d"} for k, p in (("free", P1), ("baryons", P2))}, open("universe_thin.json", "w"), indent=1, default=float)
for p in (P1, P2): print({k: (round(v, 4) if isinstance(v, float) and abs(v) < 1e6 else v) for k, v in p.items() if k != "d"})
# ---- relativistic (exact LTB) thick ordinary-matter shell, z = 0.3 case (gr_ltb/refit_mass) ----
def rel():
    o = json.load(open("universe_rel_mcfree.json"))   # all free: D, Mc, R_in, R_out, l0, Amp, phi (old Mc-constrained fit: best_rows_thick.json)
    D = o["D"]; Dm = D * Gm; rhoc = 3 * H0**2 / (8 * np.pi * GSI); rhos = o["Os"] * rhoc
    rs = lambda M: 2 * GSI * M / c**2 / Gm; Rv = RKM / GPC; Rin, Rout = o["R_in"], o["R_out"]
    cur = np.array(o["A"])
    return dict(D=D, R=Rv, DR=D / Rv, Os=o["Os"], rhos=rhos, Mc=o["Mc_kg"], Mcrit=4 * np.pi / 3 * rhoc * Dm**3, mcc=o["Mc_kg"] / (4 * np.pi / 3 * rhoc * Dm**3),
                Msh=o["M_D_kg"] - o["Mc_kg"], Mtot=o["M_D_kg"], Mshtot=4 * np.pi / 3 * rhos * ((Rout * Gm)**3 - (Rin * Gm)**3), Mhole=4 * np.pi / 3 * rhos * (Rin * Gm)**3,
                Mvis=4 * np.pi / 3 * rhos * (Rv * Gm)**3, Rin=Rin, Rout=Rout, rs_c=rs(o["Mc_kg"]), rs_D=rs(o["M_D_kg"]), vE=H0 * Dm / c, twoE=o["twoE"],
                age=o["age_Gyr"], amp=o["amp"], phi=o["phi"], l0=o["l0_s"], R2=o["R2_s"], R2ns=o["R2_ns"], R2F=r2(AFfour(TH1), cur), H_r=o["H_r"], H_perp=o["H_perp"],
                z=o["z"], Lc=0.005 * D, Dkm=D * GPC, Lckm=0.005 * D * GPC, cur=cur)
P3 = rel()
```

```
def newt_omb():   # Newtonian, Omega_s = 0.049 forced (newt_omb.py)
    o = json.load(open("universe_newt_omb.json"))["Rs14.3|sine"]; D = o["D"]; Dm = D * Gm; rhoc = 3 * H0**2 / (8 * np.pi * GSI); rhos = o["Os"] * rhoc
    rs = lambda M: 2 * GSI * M / c**2 / Gm; Rv = RKM / GPC; cur = np.array(o["A"])
    return dict(D=D, R=Rv, DR=D / Rv, Os=o["Os"], rhos=rhos, Mc=o["Mc_kg"], Mcrit=o["Mc_kg"] / o["mcc"], mcc=o["mcc"], Msh=o["shell_inside_D_kg"], Mtot=o["M_inside_D_kg"],
                Mshtot=o["shell_total_kg"], Mhole=4 * np.pi / 3 * rhos * (o["R_in"] * Gm)**3, Mvis=4 * np.pi / 3 * rhos * (Rv * Gm)**3, Rin=o["R_in"], Rout=o["R_out"],
                rs_c=rs(o["Mc_kg"]), rs_D=rs(o["M_inside_D_kg"]), vE=o["VO_over_c"],
                vesc2=o["vesc2_full_potential"], age=o["age_Gyr"], amp=o["amp"], phi=o["phi"],
                l0=o["l0"],
                R2=o["R2"], R2geo=json.load(open("universe_newt_omb.json"))["Rs14.3|geom"]["R2"], R2F=r2(AFfour(TH1), cur), Lc=0.005 * D, Dkm=D * GPC, Lckm=0.005 * D * GPC, cur=cur)
P4 = newt_omb()
for q in (P1, P2): q["Mshtot"] = 4 * np.pi / 3 * q["rhos"] * ((q["Rout"] * Gm)**3 - (q["Rin"] * Gm)**3); q["Mvis"] = 4 * np.pi / 3 * q["rhos"] * (q["R"] * Gm)**3
json.dump({k: {kk: vv for kk, vv in q.items() if kk not in ("d", "cur")} for k, q in (("free", P1), ("baryons", P2), ("rel", P3), ("newt_omb", P4))}, open("universe_thin.json", "w"), indent=1, default=float)
print("REL", {k: (round(v, 4) if isinstance(v, float) and abs(v) < 1e6 else v) for k, v in P3.items() if k != "cur"})
p = P3; Dg, Rg, l0 = p["D"], p["R"], p["l0"]
lpk = (90 - np.degrees(p["phi"]) + l0) % 360
Ev = np.array([Dg, 0.0])
def sky(l, r): t = np.radians(l - l0); return Ev + r * np.array([np.cos(t), np.sin(t)])
# ---------------- Figure 1: one figure, three axes, unified style ----------------
from matplotlib.gridspec import GridSpec
from itertools import combinations
FS_A, FS_L, FS_T = 7.5, 8.5, 9            # annotations, axis labels/ticks, titles
plt.rcParams.update({"mathtext.fontset": "cm", "axes.linewidth": 0.8, "xtick.labelsize": 8, "ytick.labelsize": 8, "axes.labelsize": FS_L, "axes.titlesize": FS_T,
                     "xtick.major.width": 0.8, "ytick.major.width": 0.8})
CRED, CBLU, CDC, CGRN, CPUR, CSH = "#d62728", "#0055d4", "#1f77b4", "#2ca02c", "#9467bd", "0.82"
VEL = dict(arrowstyle="-|>", color=CBLU, lw=1.3, mutation_scale=9, shrinkA=0, shrinkB=0)
POS = lambda ls="-": dict(arrowstyle="-|>", color="0.2", lw=0.9, ls=ls, mutation_scale=8, shrinkA=0, shrinkB=0)
exA = (-0.6, 5.6, -2.6, 3.4); exB = (-7, 82.7, -34.5, 34.5); E2 = 230
ka = (exA[1] - exA[0]) / (exA[3] - exA[2]); kb = (exB[1] - exB[0]) / (exB[3] - exB[2])
FW = 7.4; L0, G1, G2, R0 = 0.04, 0.58, 0.54, 0.10
H = (FW - L0 - G1 - G2 - R0) / (ka + kb + 1); BOT, TOP = 0.42, 0.24; FH = H + BOT + TOP
fig = plt.figure(figsize=(FW, FH))
gs = GridSpec(1, 5, figure=fig, width_ratios=[ka, G1 / H, kb, G2 / H, 1], wspace=0, left=L0 / FW, right=1 - R0 / FW, bottom=BOT / FH, top=1 - TOP / FH)
vx, ax, bx = fig.add_subplot(gs[0]), fig.add_subplot(gs[2]), fig.add_subplot(gs[4])
def T(axx, *a, **k): k.setdefault("fontsize", FS_A); return axx.text(*a, **k)
# (a) Hubble velocity vectors (schematic)
C0_, O_, P_ = np.array([0, 0.9]), np.array([2.6, 0.9]), np.array([3.6, 2.3]); h_ = 0.4
VO, VP = h_ * (O_ - C0_), h_ * (P_ - C0_); VH = VP - VO
vx.annotate("", xy=O_, xytext=C0_, arrowprops=POS()); vx.annotate("", xy=P_, xytext=C0_, arrowprops=POS("--")); vx.annotate("", xy=P_, xytext=O_, arrowprops=POS(":"))
for base, vec in ((O_, VO), (P_, VP), (P_, VH)): vx.annotate("", xy=base + vec, xytext=base, arrowprops=VEL)
vx.plot(*C0_, "o", color=CRED, ms=5, zorder=6); vx.plot(*O_, "o", color=CBLU, mec="k", mew=0.6, ms=5, zorder=6); vx.plot(*P_, "o", color="0.3", ms=4.5, zorder=6)
T(vx, *(C0_ + [0, -0.18]), "CoE", color=CRED, ha="center", va="top")
T(vx, *(O_ + [0.02, -0.18]), "us ($O$)", ha="left", va="top")
T(vx, *(P_ + [0.1, -0.1]), "galaxy", ha="left", va="top")
T(vx, *((C0_ + O_) / 2 + [0, -0.12]), "$\\vec D$", ha="center", va="top")
T(vx, *((C0_ + P_) / 2 + [-0.12, 0.12]), "$\\vec D+\\vec r$", ha="right", va="bottom")
```

```
T(vx, *((O_ + P_) / 2 + [0.1, -0.05]), "$\\vec r$", ha="left", va="center")
T(vx, *(O_ + VO + [0.08, 0]), "$\\vec V_O$", color=CBLU, ha="left", va="bottom")
T(vx, *(P_ + VP + [0.04, -0.06]), "$\\vec V_P$", color=CBLU, ha="left", va="top")
T(vx, *(P_ + VH + [0.07, -0.02]), "$\\vec V_H$", color=CBLU, ha="left", va="bottom")
A_ = np.array([1.1, -1.45]); s_ = 1.6
for base, vec in ((A_, s_ * VO), (A_, s_ * VP), (A_ + s_ * VO, s_ * VH)): vx.annotate("", xy=base + vec,
↪  xytext=base, arrowprops=VEL)
T(vx, *(A_ + s_ * VO / 2 + [0, -0.1]), "$\\vec V_O$", color=CBLU, ha="center", va="top")
T(vx, *(A_ + s_ * VP / 2 + [-0.1, 0.1]), "$\\vec V_P$", color=CBLU, ha="right", va="bottom")
T(vx, *(A_ + s_ * VO + s_ * VH / 2 + [0.1, 0]), "$\\vec V_H$", color=CBLU, ha="left", va="center")
T(vx, 2.5, -2.55, "$\\vec V_H=\\vec V_P-\\vec V_O=H\\vec r$", ha="center", va="bottom")
vx.set_xlim(exA[0], exA[1]); vx.set_ylim(exA[2], exA[3]); vx.set_aspect("equal"); vx.set_xticks([]);
↪  vx.set_yticks([])
vx.set_title("(a) Hubble velocity vectors")
# (b) Hubble bubble scale
ax.add_patch(Circle((0, 0), p["Rout"], fc=CSH, ec="0.35", lw=0.8, zorder=0)); ax.add_patch(Circle((0, 0),
↪  p["Rin"], fc="white", ec="0.35", lw=0.8, zorder=0.5))
ax.add_patch(Circle((0, 0), Dg, fill=False, color=CDC, lw=1.0, ls="--", zorder=2))
ax.add_patch(Circle(Ev, Rg, fc=CGRN, alpha=0.18, ec=CGRN, lw=1.2, zorder=2))
ax.add_patch(Circle((0, 0), p["rs_c"], fill=False, color=CRED, lw=1.4, zorder=3))
ax.plot(0, 0, "o", color=CRED, ms=5, zorder=6); T(ax, -5.5, 2.4, "CoE core", color=CRED, ha="left",
↪  va="bottom")
ax.plot(*Ev, "o", color=CBLU, mec="k", mew=0.6, ms=5, zorder=7); T(ax, p["Rin"] - 1.4, -1.0, "Earth",
↪  ha="right", va="top", zorder=7)
T(ax, 5.5, 11.7, "empty\ninterior", color="0.4", ha="center", va="center", linespacing=1.1)
T(ax, 0.5 * (Dg + p["Rout"]) * np.cos(np.radians(42)), 0.5 * (Dg + p["Rout"]) * np.sin(np.radians(42)),
↪  "shell", color="0.25", ha="center", va="center", rotation=-48, rotation_mode="anchor")
tq = np.radians(125); q = Ev + Rg * np.array([np.cos(tq), np.sin(tq)])
ax.annotate("", xy=q, xytext=Ev, arrowprops=dict(arrowstyle="-|>", color=CGRN, lw=1.0, mutation_scale=8,
↪  shrinkA=0, shrinkB=0)); T(ax, (Ev[0] + q[0]) / 2 - 1.0, (Ev[1] + q[1]) / 2 - 1.0, "$R$", color=CGRN,
↪  ha="right", va="center")
td = np.radians(-38); qd = Dg * np.array([np.cos(td), np.sin(td)])
ax.annotate("", xy=qd, xytext=(0, 0), arrowprops=dict(arrowstyle="<|-|>", color=CDC, lw=0.9,
↪  mutation_scale=7, shrinkA=0, shrinkB=0)); T(ax, *(qd * 0.58 + [1.4, -1.4]), "$D$", color=CDC,
↪  ha="left", va="top")
T(ax, 43.5, 33, "core $r_s$ = %.0f Gpc\n(%.1f D), off scale" % (p["rs_c"], p["rs_c"] / Dg), color=CRED,
↪  ha="left", va="top", linespacing=1.1)
VX1 = 81; ax.annotate("", xy=(VX1, 0), xytext=Ev, arrowprops=VEL, zorder=8)
vlab = T(ax, VX1, -2.8, "$V_O\\!\\approx\\!%.1fc$" % p["vE"], color=CBLU, ha="right", va="top")
ax.set_xlim(exB[0], exB[1]); ax.set_ylim(exB[2], exB[3]); ax.set_aspect("equal")
ax.set_xlabel("$x$ [Gpc]"); ax.set_ylabel("$y$ [Gpc]"); ax.set_title("(b) Hubble bubble scale")
# (c) Cosmic shell scale
bx.add_patch(Circle((0, 0), p["Rout"], fc=CSH, ec="0.35", lw=0.6, zorder=1)); bx.add_patch(Circle((0, 0),
↪  p["Rin"], fc="white", ec="0.35", lw=0.5, zorder=1.5))
bx.add_patch(Circle((0, 0), p["rs_c"], fill=False, color=CRED, lw=1.4, zorder=3))
bx.add_patch(Circle(Ev, Rg, fc=CGRN, alpha=0.18, ec=CGRN, lw=1.2, zorder=2))   # vision sphere R, to
↪  scale (as in b)
T(bx, 0, 0.87 * p["rs_c"], "core $r_s$\n= %.0f Gpc" % p["rs_c"], color=CRED, ha="center", va="top",
↪  linespacing=1.1); T(bx, p["Rout"] + 6, 25, "shell", color="0.25", ha="left", va="center")
T(bx, 0, -0.996 * E2, "$r_s(<D)$ = %s Gpc\n(%.1f D), off scale" % ("{:,.0f}".format(p["rs_D"]), p["rs_D"]
↪  / Dg), color=CRED, ha="center", va="bottom", linespacing=1.0)
for ang in (45, 135, 225, 315):
    u = np.array([np.cos(np.radians(ang)), np.sin(np.radians(ang))]); bx.annotate("", xy=u * 0.87 *
    ↪  p["rs_c"] * 0.75, xytext=u * p["Rout"] * 1.05, arrowprops=VEL, zorder=4)
bx.set_xlim(-E2, E2); bx.set_ylim(-E2, E2); bx.set_aspect("equal"); bx.set_xticks([-200, 0, 200]);
↪  bx.set_yticks([-200, 0, 200])
bx.set_xlabel("$x$ [Gpc]"); bx.set_title("(c) Cosmic shell scale")
# checks
fig.canvas.draw(); rr = fig.canvas.get_renderer()
```

```
for A_x in (vx, ax, bx):
    bbs = [(t.get_text()[:22], t.get_window_extent(rr)) for t in A_x.texts if t.get_text()]
    for (n1, b1), (n2, b2) in combinations(bbs, 2):
        if b1.overlaps(b2): print("OVERLAP", repr(n1), "|", repr(n2))
    W_ = A_x.get_window_extent(rr)
    for n1, b1 in bbs:
        if b1.x0 < W_.x0 or b1.x1 > W_.x1 or b1.y0 < W_.y0 or b1.y1 > W_.y1: print("OUTSIDE", repr(n1))
for A_x, circ, pad in ((ax, (p["Rin"], p["Rout"], p["rs_c"], Dg), 1.2), (bx, (p["Rin"], p["Rout"],
↪  p["rs_c"]), 17)):
    inv_ = A_x.transData.inverted()
    for t in A_x.texts:
        (x0, y0_), (x1, y1) = inv_.transform(t.get_window_extent(rr)); xs, ys =
        ↪  np.meshgrid(np.linspace(x0, x1, 60), np.linspace(y0_, y1, 20)); rr_ = np.hypot(xs, ys)
        for R_ in circ:
            if rr_.min() - pad < R_ < rr_.max() + pad: print("EDGE", repr(t.get_text()[:20]), round(R_,
            ↪  1), np.round([x0, y0_, x1, y1], 1))
print("axes heights (in)", [round(a_.get_window_extent(rr).height / fig.dpi, 3) for a_ in (vx, ax, bx)],
↪  "H", round(H, 3), "tight W", round(fig.get_tightbbox(rr).width, 3))
fig.savefig("../universe_thin_drawing.png", dpi=200, bbox_inches="tight", pad_inches=0.02); plt.close(fig)
plt.rcParams.update({"mathtext.fontset": "dejavusans", "axes.linewidth": 0.8, "xtick.labelsize": 9,
↪  "ytick.labelsize": 9, "axes.labelsize": 9, "axes.titlesize": 12})
AH = float(os.environ.get("AH", "7.58"))
Af = AFfour(TH1); th2 = np.append(TH1, 360); Af2 = AFfour(th2)
fig, (a, b) = plt.subplots(2, 1, figsize=(6.6, AH), sharex=True, gridspec_kw={"height_ratios": [2.0, 1],
↪  "hspace": 0.06})
a.errorbar(ML, MA, yerr=[MLO, MHI], fmt="o", ms=3, color="0.45", ecolor="0.75", lw=0.8, label="Migkas
↪  Fig. 6")
a.plot(th2, Af2, color="C0", lw=1.5, label="$A_F$")
for pp, col, ls, lab in [(P1, "C3", "-", "Newtonian thin shell + core, $\\Omega_s$=%.2f"), (P4, "C2",
↪  ":", "Newtonian shell + core, $\\Omega_s$=%.3f forced"), (P3, "C1", "--", "relativistic (LTB) shell +
↪  core, $\\Omega_s$=%.3f, $z$=0.3")]:
    A = AC.curve("ptshell", pp["d"]) if "d" in pp else pp["cur"]; assert abs(r2(MA, A[IMIG]) - pp["R2"])
    ↪  < 2e-3, (r2(MA, A[IMIG]), pp["R2"])
    A2 = np.append(A, A[0])
    a.plot(th2, A2, color=col, ls=ls, lw=1.6 if ls != "--" else 1.3, label=(lab % pp["Os"]) + ",
    ↪  $\\ell_0$=%.1f° (fit $R^2$=%.3f; vs $A_F$ %.3f)" % (pp["l0"], pp["R2"], pp["R2F"]))
    b.plot(th2, 100 * (A2 - Af2) / Af2, color=col, ls=ls, lw=1.4 if ls == "-" else 1.2, label={"-":
    ↪  "Newtonian", ":": "Newt. $\\Omega_s$=0.049", "--": "relativistic"}[ls])
a.set_ylabel("$A/A_{\\rm all}$"); a.set_ylim(0.62, 1.22); a.legend(fontsize=7.2, loc="lower left",
↪  frameon=False)
b.axhline(0, color="k", lw=0.6); b.plot(ML, 100 * (MA - AFfour(ML)) / AFfour(ML), "o", ms=2.5,
↪  color="0.45", label="data")
b.set_ylabel("$100\\,(\\cdot-A_F)/A_F$ [%]", fontsize=8); b.set_xlabel("Galactic longitude $\\ell$ (deg)")
b.set_xlim(0, 360); b.set_xticks(range(0, 361, 45)); b.legend(fontsize=7, frameon=False, loc="upper
↪  right", ncol=4)
fig.savefig("../universe_thin_Achart.png", dpi=200, bbox_inches="tight")
for f in ["universe_thin_drawing", "universe_thin_Achart"]:
    im = Image.open("../" + f + ".png").convert("RGB"); im.thumbnail((1700, 1700)); im.save("../" + f +
    ↪  "_web.jpg", quality=88)
```

/workspace/figs_letter_exact_python/makers/make_fig01_02_geometry_achart.py

```
#!/usr/bin/env python3
"""Fig01 geometry + Fig02 Achart — rerun spreadsheet_model/bubble/universe_thin.py.

Pixel-matches letter gold embeds (PNG encoding may differ; RGBA max abs == 0).
"""
from __future__ import annotations
```

```

import runpy
import shutil
from pathlib import Path

OUT = Path("/workspace/figs_letter_exact_python")
BUBBLE = Path("/workspace/cosmology/papers/spreadsheet_model/bubble")
SM = BUBBLE.parent
SCRIPT = BUBBLE / "universe_thin.py"


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    # universe_thin.py writes ../universe_thin_drawing.png and ../universe_thin_Achart.png
    # relative to bubble/; run in that directory via runpy with argv.
    import os, sys
    cwd = os.getcwd()
    sys.path.insert(0, str(BUBBLE))
    try:
        os.chdir(BUBBLE)
        runpy.run_path(str(SCRIPT), run_name="__main__")
    finally:
        os.chdir(cwd)
    src1 = SM / "universe_thin_drawing.png"
    src2 = SM / "universe_thin_Achart.png"
    d1 = OUT / "Fig01_geometry.png"
    d2 = OUT / "Fig02_Achart.png"
    shutil.copy2(src1, d1)
    shutil.copy2(src2, d2)
    print("wrote", d1, d1.stat().st_size)
    print("wrote", d2, d2.stat().st_size)
    return d1, d2


if __name__ == "__main__":
    main()

```

/workspace/coepdf/scripts/fig01b_geometry.py

```

# Fig 1 bottom: observer O (Earth) inside thick shell, distance D from CoE core, vision sphere R.
import matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt,numpy as np
from matplotlib.patches import Circle,FancyArrowPatch
plt.rcParams.update({'mathtext.fontset':'cm','font.family':'serif','font.size':15})
fig=plt.figure(figsize=(5.34,4.12),dpi=200,facecolor='white');ax=fig.add_axes([0,0,1,1]);ax.set_xlim(0,48
↪  1);ax.set_ylim(371,0);ax.axis('off')
C=(38,185);O=(214,185);D=O[0]-C[0]
ax.add_patch(Circle(C,262,fc='lightgrey',ec='0.35',lw=1.5))
ax.add_patch(Circle(C,96,fc='white',ec='0.35',lw=1.5))
ax.add_patch(Circle(C,D,fill=False,ec='tab:blue',ls='--',lw=2))
ax.add_patch(Circle(O,76,fc='tab:green',alpha=.3,ec='tab:green',lw=1.5))
ar=lambda a,b,c,lw=1.6:ax.add_patch(FancyArrowPatch(a,b,arrowstyle='-|>',mutation_scale=14,color=c,lw=lw))
# D: CoE -> observer, drawn as chord-free radius at an angle ending on dashed circle (|D| = CoE-to-O
↪  distance)
ar((C[0]+6,C[1]),(O[0]-7,O[1]),'tab:blue',2);ax.text(118,C[1]-8,'$D$',color='tab:blue',fontsize=18,ha='ce
↪  nter')
t2=np.radians(125);ar(O,(O[0]+76*np.cos(t2),O[1]-76*np.sin(t2)),'tab:green',2)
ax.text(O[0]-42,O[1]-20,'$R$',color='tab:green',fontsize=17)
```

```
ar(O,(472,185),'#0055cc',2.5);ax.text(390,218,r'$V_O\!\approx\!7.7c$',color='#0055cc',fontsize=15)
ax.plot(*C,'o',color='tab:red',ms=10);ax.plot(*O,'o',color='#0055cc',mec='k',ms=9)
ax.text(8,C[1]+30,'CoE core',color='tab:red',ha='left',fontsize=15)
ax.text(68,118,'empty\ninterior',color='0.4',ha='center',fontsize=14,linespacing=1.1)
ax.text(O[0]+6,O[1]+40,'Earth\n(observer, $O$)',color='k',fontsize=13,ha='left',linespacing=1.05,bbox=dic
↪  t(fc='white',ec='none',alpha=.75,pad=1))
ax.text(200,40,'shell',rotation=-42,color='0.3',fontsize=15,ha='center',va='center')
ax.text(270,40,'core $r_s$ = 163 Gpc\n(4.9 D), off scale',color='tab:red',fontsize=14)
ax.add_patch(plt.Rectangle((0,0),481,371,fill=False,ec='k',lw=2))
fig.savefig('fig/Fig01b_geometry.png',facecolor='white')
```

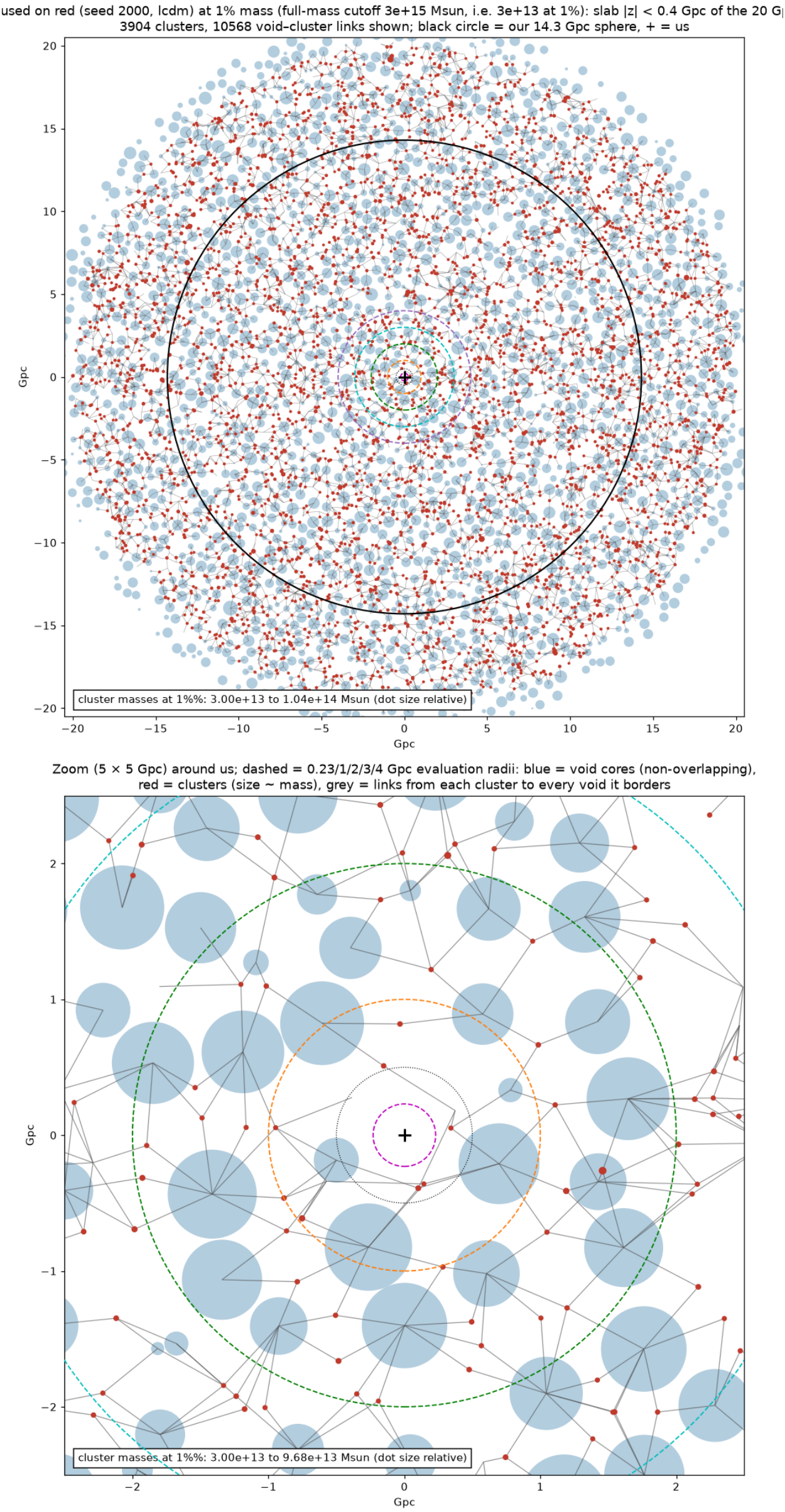
16. Realistic cosmic web slab

Originally made: Fri Oct 23 2026, 8:29 AM CT (*copy time onto the box (true original likely earlier)*)
Paper figure number: this morning Fig. 16; now Fig. 16
Files: `f06.png`
Code status: original (box)

Notes

Slab |z|<0.4 Gpc of a 20 Gpc box, seed 2000, 3904 clusters. Built from web ingredients (#7). Candidates searched: redweb2/stack_s2000.py writes red_web_s2000_stack.png (a spectrum stack, not this slab), so not used. STATUS NOTE: redweb2/bodies3.py output red_web_s2000_bodies_1pct.png is md5-identical to f06.png (SSIM 1.000).

Paper caption (caveats as stated): Realistic web: slab $|z| < 0.4$ Gpc of a 20 Gpc box (seed 2000, 1% mass cut, 3904 clusters, 10568 void–cluster links). Circle: 14.3 Gpc sphere; +: observer. Bottom: zoom (±9 Gpc). [unverified: whether this figure used the same `build.py` as above (whose grids use seeds 11, 12)].



Zoom (5 × 5 Gpc) around us; dashed = 0.23/1/2/3/4 Gpc evaluation radii: blue = void cores (non-overlapping), red = clusters (size ~ mass), grey = links from each cluster to every void it borders

Blue discs = core spheres of voids centred in the slab (each void is the whole cell between its clusters). Red dots = clusters at web nodes. Grey lines = bipartite links (void ↔ cluster only), drawn when both ends are in the slab. Dotted circle = 0.5 Gpc zone kept empty around us.

`f06.png`

Computing chain (run order, full listings)

/workspace/coepdf/book/recovered/f06.py

```
"""Plain side view of one generated network: slab |z| < half-thickness, void core spheres cut by the slab, clusters, and every void-cluster link with both ends in the slab. Full region + zoom."""
import os,sys,numpy as np,matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection,PatchCollection
ME='/workspace/coe-cosmology-gh/draft/current/analyses/void_mass_network'; sys.path.insert(0,ME); import vmn_gen as G
Mmin=float(sys.argv[1]) if len(sys.argv)>1 else 3e15
g=G.generate(Mmin=Mmin,seed=2000,comp_mode="lcdm"); S=g["S"]/1000; XC=g["XC"]/1000; Rv=g["Rv"]/1000; L=g["links"]; R=g["params"]["R_out"]/1000
def draw(ax,lim,hz,cen=(0,0)):
    sv=(np.abs(S[:,2])<hz)&(np.abs(S[:,0]-cen[0])<lim+2)&(np.abs(S[:,1]-cen[1])<lim+2)
    rc=np.sqrt(np.maximum(Rv[sv]**2-np.maximum(np.abs(S[sv,2])-0,0)**2,0))      # radius of the slice through z=0
    pc=PatchCollection([plt.Circle(S[i,:2],r) for i,r in zip(np.where(sv)[0],rc) if r>0],facecolor="#2471a3",alpha=0.35,edgecolor="none",zorder=1)
    ax.add_collection(pc)
    sc=(np.abs(XC[:,2])<hz)&(np.abs(XC[:,0]-cen[0])<lim+2)&(np.abs(XC[:,1]-cen[1])<lim+2)
    lk=L[sc[L[:,0]]&sv[L[:,1]]]
    seg=np.stack([XC[lk[:,0],:2],S[lk[:,1],:2]],1)
    ax.add_collection(LineCollection(seg,colors="0.3",linewidths=0.35 if lim>8 else 0.8,alpha=0.6,zorder=2))
    m=0.01*g["MC"][sc]; ax.scatter(XC[sc,0],XC[sc,1],s=(2 if lim>8 else 8)+(30 if lim>8 else 80)*(m/m.max())**1.5,c="#c0392b",edgecolors="none",zorder=3); ax.text(0.02,0.02,f"cluster masses at 1%%: {m.min():.2e} to {m.max():.2e} Msun (dot size relative)",transform=ax.transAxes,fontsize=10,bbox=dict(fc="w"))
    ax.add_patch(plt.Circle((0,0),14.3,fill=False,color="k",lw=1.4,zorder=4)); ax.plot(0,0,"k+",ms=12,mew=2,zorder=5)
    ax.add_patch(plt.Circle((0,0),0.5,fill=False,color="k",lw=0.8,ls=":",zorder=4))
    for rr,cc in ((0.23,"m"),(1,"tab:orange"),(2,"g"),(3,"c"),(4,"tab:purple")): ax.add_patch(plt.Circle((0,0),rr,fill=False,color=cc,lw=1.2,ls="--",zorder=4))
    ax.set_xlim(cen[0]-lim,cen[0]+lim); ax.set_ylim(cen[1]-lim,cen[1]+lim); ax.set_aspect("equal"); ax.set_xlabel("Gpc"); ax.set_ylabel("Gpc")
    return sv.sum(),sc.sum(),len(lk)
fig,ax=plt.subplots(2,1,figsize=(10,20.5))
a,b,c=draw(ax[0],R+0.5,0.4); ax[0].set_title(f"Web used on red (seed 2000, lcdm) at 1% mass (full-mass cutoff {Mmin:.0e} Msun, i.e. {0.01*Mmin:.0e} at 1%): slab |z| < 0.4 Gpc of the {R:.0f} Gpc region\n{b} clusters, {c} void–cluster links shown; black circle = our 14.3 Gpc sphere, + = us",fontsize=12)
a,b,c=draw(ax[1],2.5,0.4,cen=(0,0)); ax[1].set_title(f"Zoom (5 × 5 Gpc) around us; dashed = 0.23/1/2/3/4 Gpc evaluation radii: blue = void cores (non-overlapping),\nred = clusters (size ~ mass), grey = links from each cluster to every void it borders",fontsize=12)
fig.text(0.5,0.008,"Blue discs = core spheres of voids centred in the slab (each void is the whole cell between its clusters).\nRed dots = clusters at web nodes. Grey lines = bipartite links (void ↔ cluster only),\ndrawn when both ends are in the slab. Dotted circle = 0.5 Gpc zone kept empty around us.",ha="center",fontsize=10.5)
fig.tight_layout(rect=(0,0.045,1,1)); fig.savefig("/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/clusters_voids/quad_balance/mass_family_scan/constrained/red_web_s2000_bodies_1pct.png",dpi=110)
```

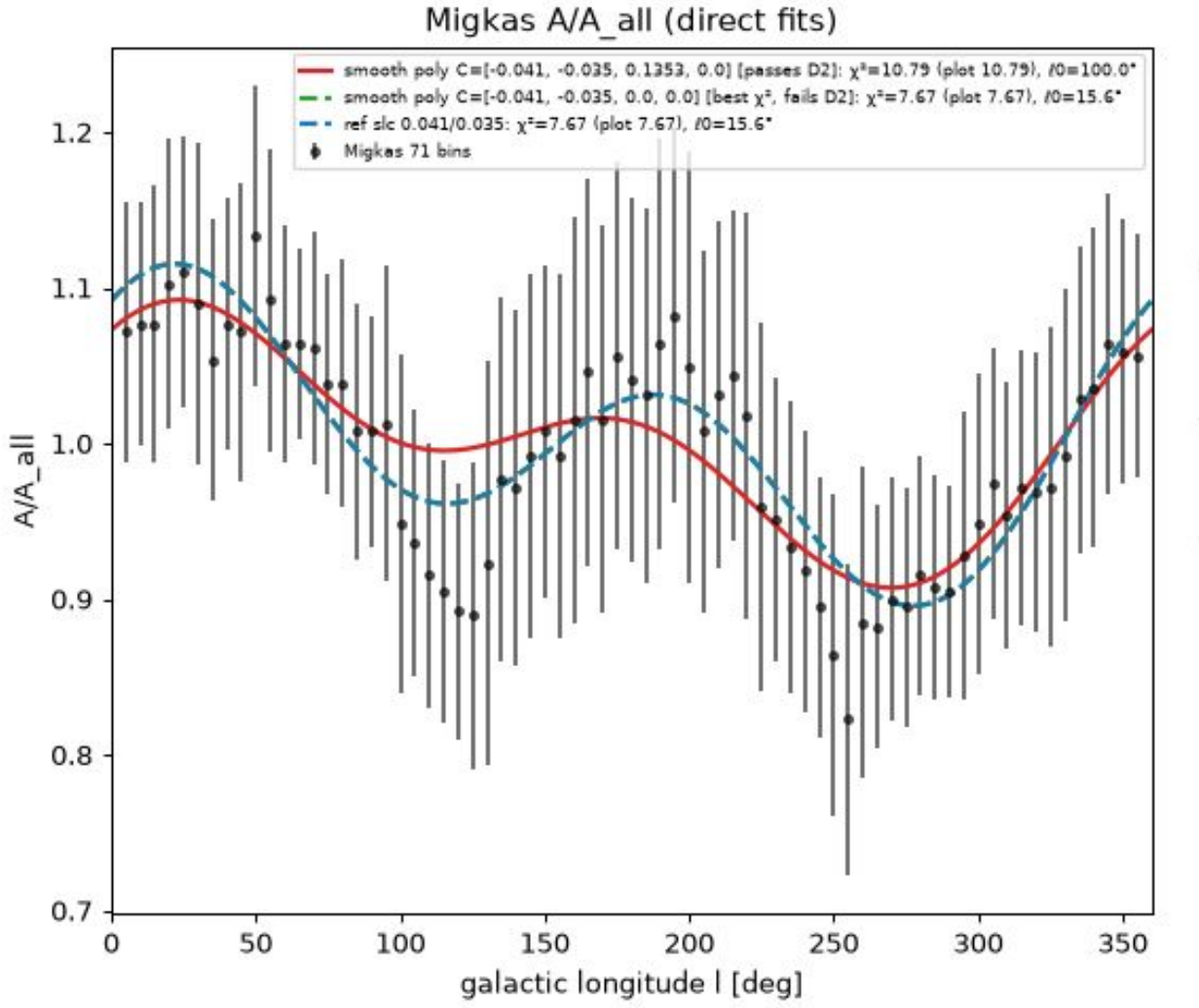
17. Migkas 71 bins with CoE direct fits

Originally made: Fri Oct 23 2026, 8:30 AM CT (8:30:28.584) (*copy time onto the box (true original likely earlier)*)
Paper figure number: this morning Fig. 15; now Fig. 12
Files: `f07.jpg`
Code status: original (box)

Notes

Red polynomial uses the same c_k as m(r) (#27). Caveat: poly_plots.py saves poly_plots.png / poly_spectrum.png, not f07.jpg directly; f07.jpg is taken to be its output per the inventory.

Paper caption (caveats as stated): Migkas $A/A_{\rm all}$ (71 bins, black) with CoE direct fits. Red: smooth polynomial in $\cos\psi$, $\chi^2 = 10.79$, $l_0 = 100°$. Green dashed: best variant, $\chi^2 = 7.67$, $l_0 = 15.6°$. Blue dashed: reference sinusoid (amplitudes 0.041/0.035), $\chi^2 = 7.67$. The two humps come from $\cos\psi(l)$ at fixed galactic latitude. Red uses the c_k of $m(r)$, Fig. fig:mod.



f07.jpg

Computing chain (run order, full listings)

/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/clusters_voids/quad_balance/mass_family_scan/c

```
# usage: poly_plots.py PASS.npz BESTCHI.npz   (both smooth-polynomial runs)
import os,sys,json,numpy as np,warnings;warnings.filterwarnings('ignore')
import matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
H=os.getcwd();FP,FB=[os.path.abspath(a) for a in sys.argv[1:3]];REF=os.path.abspath('../runs_sl2/slc_0.041037_0.035.npz')
os.chdir('/workspace/cosmology-course/notes/accel/gr/three_vel/density')
src=open('/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/clusters_voids/quad_balance/mass_family_scan/migkas_prof.py').read()
exec(src.split('rng=np.random')[0]);os.chdir(H)
sys.argv=[sys.argv[0]];import anP
def mfit(rat,n=40):
    rng=np.random.default_rng(12345);best=None
    for k in range(n):
        r=minimize(chi,LO+(HI-LO)*rng.random(6),args=(rat,),method='Nelder-Mead',options=dict(maxiter=500,xatol=1e-7,fatol=1e-7))
        if best is None or r.fun<best.fun:best=r
    th=best.x
    for j in range(3):th=minimize(chi,th,args=(rat,),method='Nelder-Mead',options=dict(maxiter=5000,xatol=1e-9,fatol=1e-9)).x
    m=model(th,rat);return th,chi(th,rat),1-np.sum((Y-m)**2)/np.sum((Y-Y.mean())**2)
Ls=np.arange(0,361,1.0);Lsave=L.copy()
def curve(t,ra):
    global L;L=np.mod(Ls,360);L[-1]=360.0;v=model(t,ra);L=Lsave;return v
from scipy.interpolate import interp1d
out={}
for tag,fn in [('pass',FP),('bestchi',FB),('ref',REF)]:
    A=anP.an(fn);th,c2,R2=mfit(A['ratio']);cc=float(np.sum(((interp1d(Ls,curve(th,A['ratio']))(np.mod(L,360))-Y)/S)**2))
    d=np.load(fn);A.update(chi2=float(c2),chi2_from_plotted=cc,R2=float(R2),l0=float(th[1]),theta=th.tolist(),frac=float(d['frac']));out[tag]=A
    print(tag,A['C'],'ratio %.5f chi2 %.3f (plot %.3f) R2 %.3f l0 %.2f    a'%(A['ratio'],c2,cc,R2,th[1]),['%.3e'%x for x in A['a']],'D2-6',['%.3g'%x for x in A['Dl'][2:7]])
PL=np.loadtxt('/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/clusters_voids/reproduce_cmb/planckTT.txt')
out['planck_l2_30']=PL[:29].tolist();json.dump(out,open('poly_results.json','w'),indent=1)
col={'pass':'C3','bestchi':'C2','ref':'C0'};lab={k:('smooth poly C=%s'%out[k]['C'] if out[k]['C'] else 'ref slc 0.041/0.035')+(' [passes D2]' if k=='pass' else (' [best χ², fails D2]' if k=='bestchi' else '')) for k in out if k in col}
fig,ax=plt.subplots(1,4,figsize=(25,5.5))
A=ax[0];A.errorbar(L,Y,yerr=S,fmt='o',ms=3,color='k',alpha=.6,label='Migkas 71 bins')
for k in col:A.plot(Ls,curve(np.array(out[k]['theta']),out[k]['ratio']),col[k],ls='-' if k=='pass' else '--',lw=1.8,label=lab[k]+f": χ²={out[k]['chi2']:.2f} (plot {out[k]['chi2_from_plotted']:.2f}), □0={out[k]['l0']:.1f}°")
A.set_xlim(0,360);A.set_xlabel('galactic longitude l  [deg]');A.set_ylabel('A/A_all');A.legend(fontsize=6);A.set_title('Migkas A/A_all (direct fits)')
A=ax[1]
for k in col:
    o=np.load(out[k]['run'])['o'];o=o[np.argsort(o[:,0])];psi=o[:,0];lz=np.log(o[:,1]);w=np.sin(np.radians(psi));d_=-(lz-np.trapezoid(lz*w,psi)/np.trapezoid(w,psi))
    A.plot(psi,d_*1e3,col[k],marker='o',ms=2,label=lab[k])
A.invert_xaxis();A.set_xlabel('ψ [deg] (180 = toward CoE, left)');A.set_ylabel('ΔT/T [10⁻³] (monopole removed)');A.set_title('CMB ΔT/T, ray-traced')
```

```
p=out['pass'];A.text(.03,.04,'pass: '+', '.join(f'a{l+1}={p["a"][l]:.2e}' for l in range(4)),transform=A.transAxes,fontsize=7);A.legend(fontsize=6)
A=ax[2];x=np.linspace(1e-3,2.6,3000);Rb=1.5*(1+14.30048287260589/42)
for k in col:
    C=out[k]['C']
    s=np.exp(sum(c*(np.minimum(x,Rb)-1)**(j+1) for j,c in enumerate(C))) if C else x**-0.041037*np.exp(-0.035*(x-1)**2)
    A.plot(x,s*(x<=Rb),col[k],label=lab[k])
A.axvline(1,color='k',ls=':');A.text(1.02,0.1,'observer',fontsize=8);A.plot(0,0,'k*',ms=12);A.text(0.03,0.05,'CoE',fontsize=8)
A.set_xlabel('comoving distance from CoE x (observer=1)');A.set_ylabel('initial ρ shape (ρ(1)=1)');A.legend(fontsize=6,loc='upper left');A.set_title('Density profile (smooth, no features)')
A=ax[3];k_=np.arange(2,7)
for i,k in enumerate(col):A.bar(k_-0.27+0.27*i,np.array(out[k]['Dl'][2:7]),0.27,color=col[k],alpha=.8,label=lab[k])
A.errorbar(k_,PL[:5,1],yerr=[PL[:5,2],PL[:5,3]],fmt='k_',ms=20,mew=2,capsize=4,label='Planck 2018 TT')
A.set_yscale('log');A.set_xlabel('□');A.set_ylabel('D_□ [μK²] (dipole removed)');A.legend(fontsize=6);A.set_title('Multipoles vs Planck')
plt.tight_layout();plt.savefig('poly_plots.png',dpi=90)
plt.figure(figsize=(9,5.5));ll=np.arange(2,31)
for k in col:plt.semilogy(ll,np.maximum(np.array(out[k]['Dl'][2:31]),1e-6),col[k],marker='o',ms=3,label=lab[k])
plt.errorbar(PL[:29,0],PL[:29,1],yerr=[PL[:29,2],PL[:29,3]],fmt='ks',ms=3,capsize=2,label='Planck 2018 TT')
plt.xlabel('□');plt.ylabel('D_□ [μK²]');plt.title('Angular power spectrum □=2–30 (37 rays, 5° spacing; □≥20 resolution-limited)');plt.legend(fontsize=7);plt.grid(alpha=.3)
plt.tight_layout();plt.savefig('poly_spectrum.png',dpi=90)
```

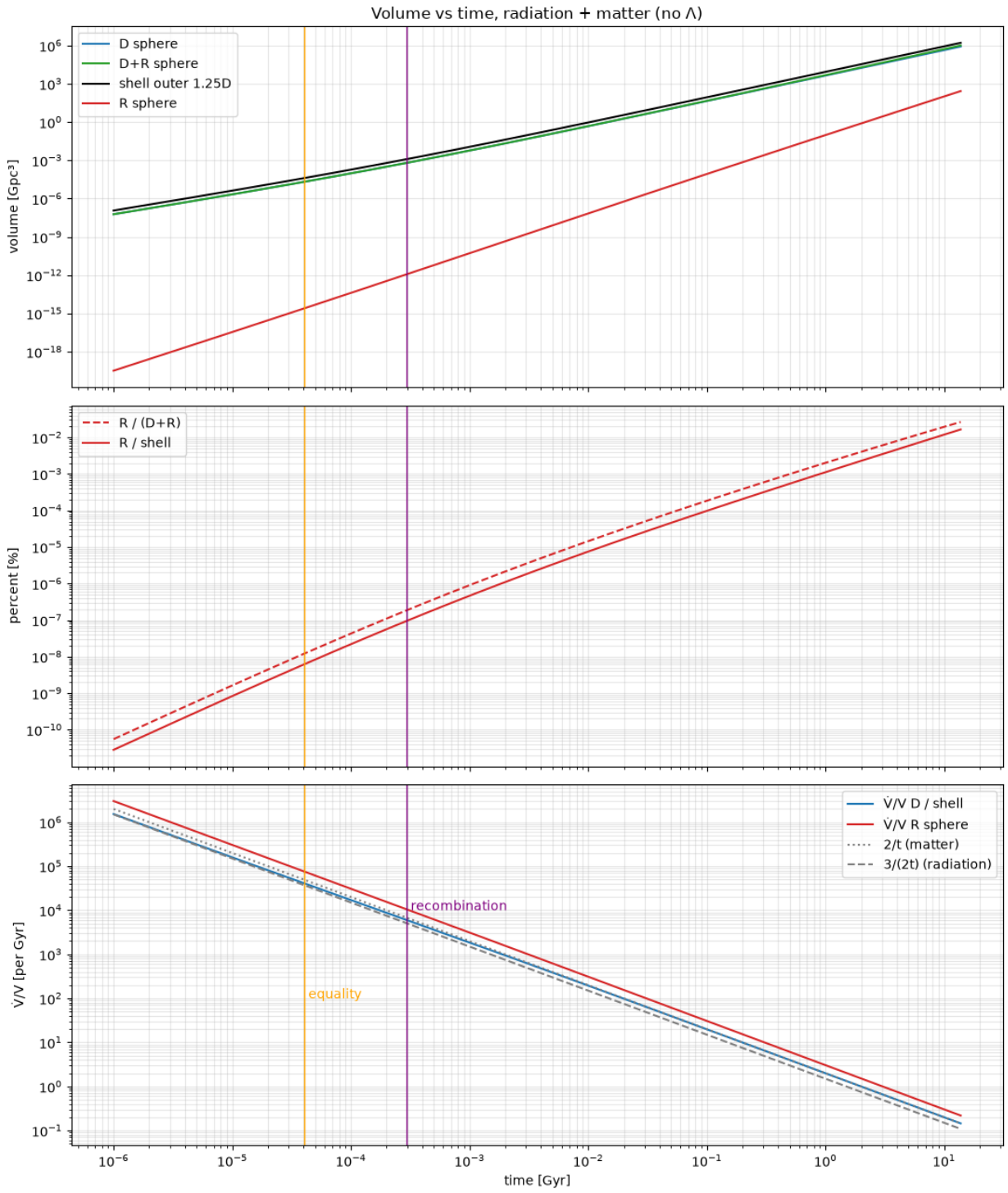
18. Volume vs time (radiation + matter)

Originally made: Fri Oct 23 2026, 8:30 AM CT (8:30:28.612) (*copy time onto the box (true original likely earlier); same second as #16, ordered by paper number*)
Paper figure number: this morning Fig. 2; now Fig. 2
Files: `f02.png`
Code status: no computing code found

Notes

Volumes of the D sphere, D+R sphere and outer sphere from a(t) of Eq. 2; follows from #5. STATUS NOTE: No script that computes these volume curves exists anywhere searched. Searched for computing code: md5 of every image on the box; rg for savefig/imsave targets and figure title strings across /workspace (redweb2, cosmology, cosmology-course, coe-cosmology-gh, coe-cosmology, coe-cosmology-repo, figs_letter_exact_python, coepdf) and /tmp; all 7 git repos (all branches, reflog, stash, dangling objects); 291 background terminal logs; agent-store attachments; sampled pages of the Cosmology, Dumb Bot and Cosmo Writing transcripts. Image-only remake removed from this book.

Paper caption (caveats as stated): Volume versus time with radiation + matter (no Λ). Top: volumes of the D sphere, $D{+}R$ sphere and outer sphere from $a(t)$ of Eq. (2). Middle: V_R as a percentage of the larger volumes. Bottom: $\dot{V}/V = n/t$. Vertical lines: equality (41 kyr), recombination (297 kyr).



f02.png

Computing chain (run order, full listings)

Generating script not found. Searched all `.py` files under /workspace (excluding virtualenvs) for the image filename and its known aliases, and the git repos.

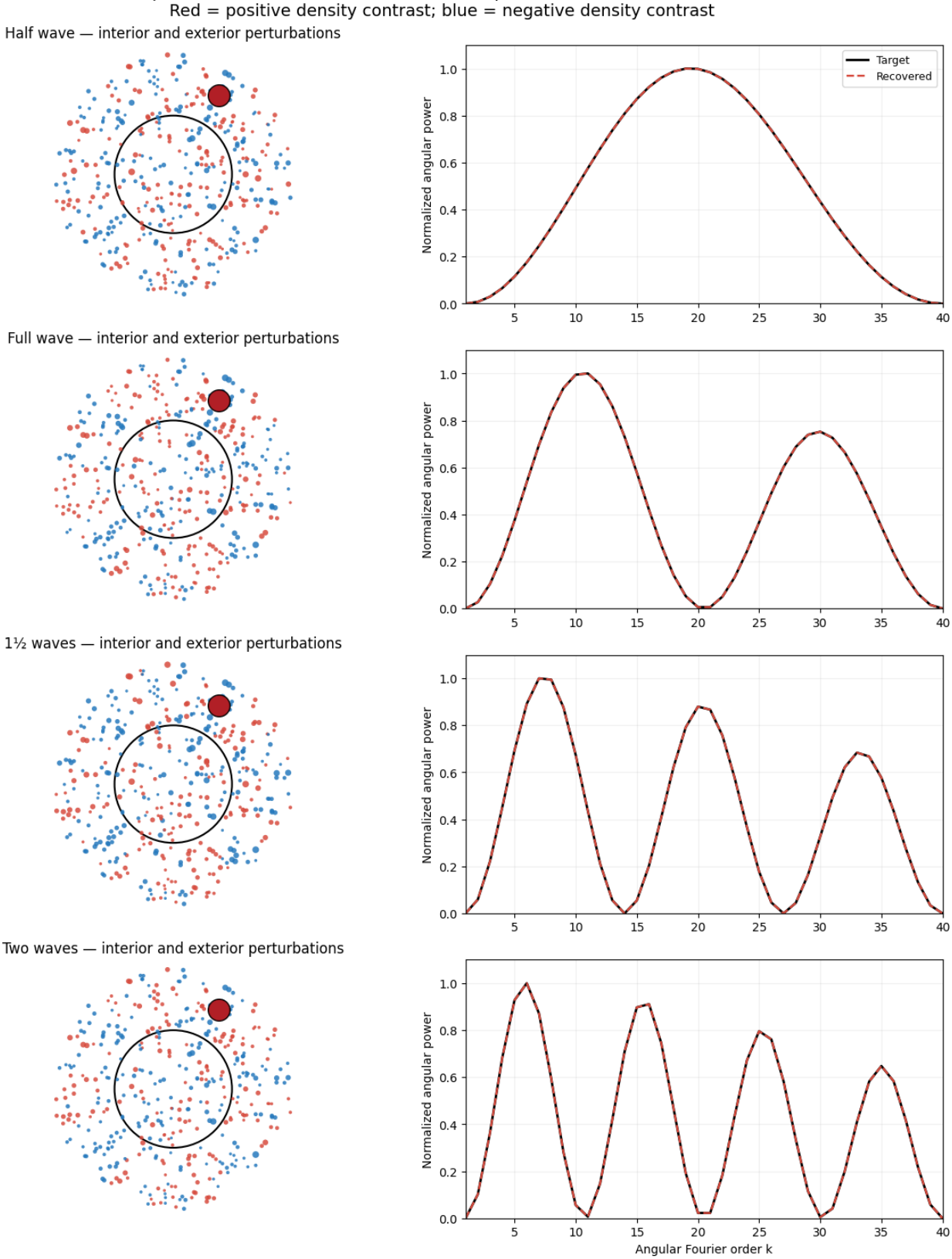
19. Target profiles matched numerically

Originally made: Fri Oct 23 2026, 8:30 AM CT (8:30:28.612) (*copy time onto the box (true original likely earlier)*)
Paper figure number: this morning Fig. 10; now Fig. 19
Files: `f12.png`
Code status: no computing code found

Notes

Fixed mass at 1.55R plus fitted clumps/voids. Per Chris, matched to a different power spectrum from the web/ladder track. STATUS NOTE: Likely made by Chris outside the box (style matches his ring scripts; unconfirmed). Searched for computing code: md5 of every image on the box; rg for savefig/imsave targets and figure title strings across /workspace (redweb2, cosmology, cosmology-course, coe-cosmology-gh, coe-cosmology, coe-cosmology-repo, figs_letter_exact_python, coepdf) and /tmp; all 7 git repos (all branches, reflog, stash, dangling objects); 291 background terminal logs; agent-store attachments; sampled pages of the Cosmology, Dumb Bot and Cosmo Writing transcripts. Image-only remake removed from this book.

Paper caption (caveats as stated): Target profiles matched numerically: a fixed mass at $1.55R$ plus fitted clumps/voids reproduce target angular profiles (half, $1\frac{1}{2}$, $2\frac{1}{2}$, two waves).


Fixed prominent mass at 1.55R and fitted clumps/voids both inside and outside R
Red = positive density contrast; blue = negative density contrast
Half wave — interior and exterior perturbations
Full wave — interior and exterior perturbations
1½ waves — interior and exterior perturbations
Two waves — interior and exterior perturbations
Target
Recovered
Normalized angular power
Angular Fourier order k

f12.png

Computing chain (run order, full listings)

Generating script not found. Searched all `.py` files under /workspace (excluding virtualenvs) for the image filename and its known aliases, and the git repos.

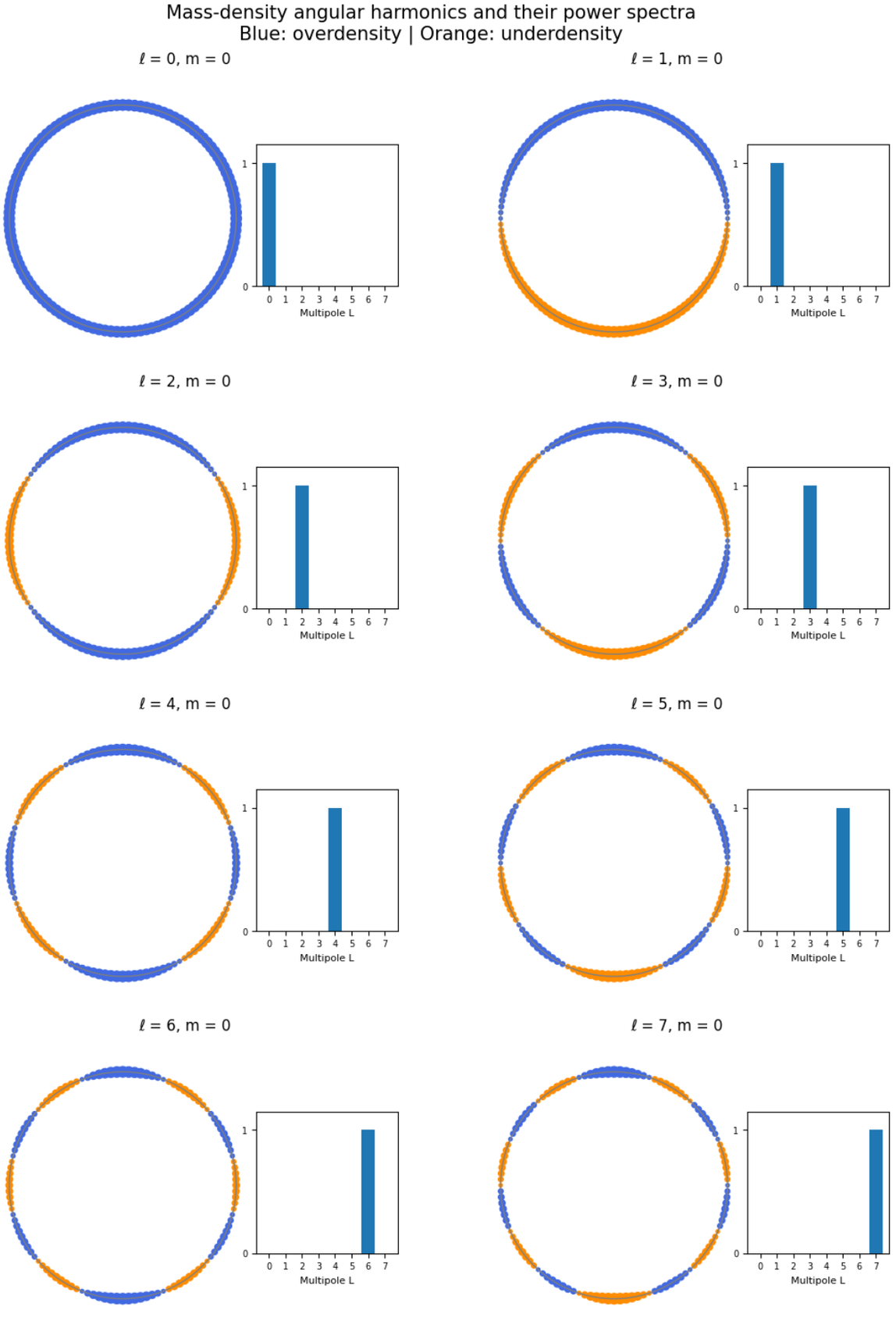
20. Axisymmetric harmonics on a ring

Originally made: Fri Oct 23 2026, 8:30 AM CT (8:30:28.824) (*copy time onto the box (true original likely earlier)*)
Paper figure number: this morning Fig. 9; now Fig. 8
Files: `f11.png`
Code status: no computing code found

Notes

Single harmonics l = 0..7 as density on a ring with the power bar at that l. STATUS NOTE: Searched for computing code: md5 of every image on the box; rg for savefig/imsave targets and figure title strings across /workspace (redweb2, cosmology, cosmology-course, coe-cosmology-gh, coe-cosmology, coe-cosmology-repo, figs_letter_exact_python, coepdf) and /tmp; all 7 git repos (all branches, reflog, stash, dangling objects); 291 background terminal logs; agent-store attachments; sampled pages of the Cosmology, Dumb Bot and Cosmo Writing transcripts. Image-only remake removed from this book.

Paper caption (caveats as stated): Single axisymmetric harmonics $\ell = 0 \ldots 7$ drawn as density on a ring (blue over-, orange under-density) with the power bar at that ℓ.


Mass-density angular harmonics and their power spectra
Blue: overdensity | Orange: underdensity
ℓ = 0, m = 0
ℓ = 1, m = 0
ℓ = 2, m = 0
ℓ = 3, m = 0
ℓ = 4, m = 0
ℓ = 5, m = 0
ℓ = 6, m = 0
ℓ = 7, m = 0
Multipole L

f11.png

Computing chain (run order, full listings)

Generating script not found. Searched all `.py` files under /workspace (excluding virtualenvs) for the image filename and its known aliases, and the git repos.

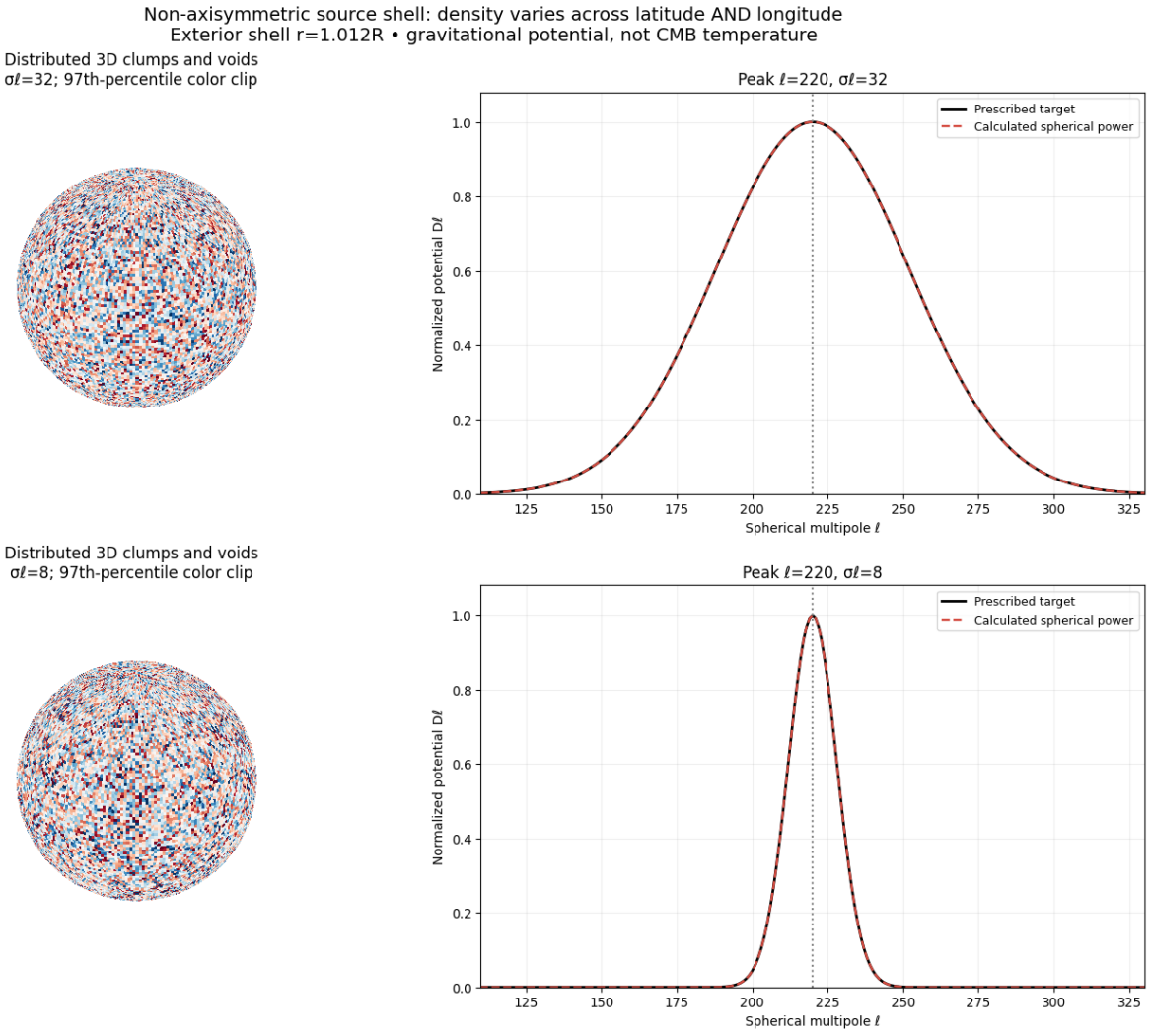
21. Target peak matched numerically

Originally made: Fri Oct 23 2026, 8:30 AM CT (8:30:28.880) (*copy time onto the box (true original likely earlier)*)
Paper figure number: this morning Fig. 11; now Fig. 20
Files: `f13.png`
Code status: original (Chris attachment, Wed Oct 21 6:09 PM CT)

Notes

Non-axisymmetric source shell at r = 1.012R. Per Chris, made because the spectrum from the target-profiles fit (#16) was not there yet. STATUS NOTE: COMPUTING CHAIN (run order): f13.py (computes the non-axisymmetric source shell at r = 1.012R and its C_l / matched peak; writes the plot), with companion ring code. Source: Chris attachment 01M5GBMH5C2D4X1DRCQ4N5TBPC.txt (Wed Oct 21 6:09 PM CT), image attached 6:11 PM CT md5-identical to f13.png. Rerun gives the same curves; only label glyphs and dpi differ (SSIM 0.80).

Paper caption (caveats as stated): Target peak matched numerically: a non-axisymmetric source shell at $r = 1.012R$ (gravitational potential, not temperature) built from 3D clumps/voids reproduces a prescribed peak at $\ell = 220$ with widths $\sigma_\ell = 32$ and 8.



f13.png

Computing chain (run order, full listings)

/workspace/coepdf/book/recovered/f13.py

```
# Distributed spherical clumps and voids: wide/narrow ell=220 spectra
# Original model code used to generate distributed_spherical_clumps_ell220.png
# Requires numpy, scipy, matplotlib
# Note: inverse-designed gravitational-potential spectrum, NOT CMB temperature.

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import eval_legendre
from matplotlib.colors import TwoSlopeNorm
from pathlib import Path

rng = np.random.default_rng(20261021)
ell = np.arange(1, 351)
shell_radius = 1.012
n_axes = 5
```

```
n_theta, n_phi = 130, 260
theta = np.linspace(.003, np.pi-.003, n_theta)
phi = np.linspace(0, 2*np.pi, n_phi, endpoint=False)
tt, pp = np.meshgrid(theta, phi, indexing='ij')
directions = np.stack([np.sin(tt)*np.cos(pp), np.sin(tt)*np.sin(pp), np.cos(tt)], axis=-1)
xyz = directions.reshape(-1, 3)

def unit(a):
    return a / np.linalg.norm(a, axis=-1, keepdims=True)

def build_case(width, seed):
    rg = np.random.default_rng(seed)
    target = np.exp(-.5*((ell-220)/width)**2)
    # Draw distinct orientations for each degree; multiple m components.
    density = np.zeros(len(xyz))
    recovered = np.zeros(len(ell))
    for i, l in enumerate(ell):
        if target[i] < 1e-9:
            continue
        axes = unit(rg.normal(size=(n_axes, 3)))
        weights = rg.choice([-1., 1.], size=n_axes)
        gram = eval_legendre(int(l), np.clip(axes @ axes.T, -1, 1))
        S = max(float(weights @ gram @ weights), 1e-12)
        # Choose each harmonic amplitude to enforce the prescribed target.
        A = shell_radius**(l+1) * np.sqrt(target[i]*(2*l+1)/(l*(l+1)*S))
        basis = sum(weights[j]*eval_legendre(int(l), xyz @ axes[j])
                    for j in range(n_axes))
        density += A*basis
        recovered[i] = l*(l+1)/(2*l+1)*(A/shell_radius**(l+1))**2*S
    density = density.reshape(n_theta, n_phi)
    return target, recovered/recovered.max(), density

cases = [(32, build_case(32, 32)), (8, build_case(8, 8))]
fig = plt.figure(figsize=(14, 11), facecolor='white', layout='constrained')
gs = fig.add_gridspec(2, 2, width_ratios=[1, 1.15])
for row, (width, (target, power, density)) in enumerate(cases):
    ax = fig.add_subplot(gs[row, 0], projection='3d')
    # Display clipping only: actual density values are not changed.
    clip = np.percentile(np.abs(density), 97)
    norm = TwoSlopeNorm(vmin=-clip, vcenter=0, vmax=clip)
    colors = plt.get_cmap('RdBu_r')(norm(density))
    X = np.sin(tt)*np.cos(pp)
    Y = np.sin(tt)*np.sin(pp)
    Z = np.cos(tt)
    ax.plot_surface(X, Y, Z, facecolors=colors, shade=False,
                    linewidth=0, rstride=1, cstride=1, antialiased=False)
    ax.set_box_aspect((1, 1, 1))
    ax.set(xlim=(-1, 1), ylim=(-1, 1), zlim=(-1, 1))
    ax.view_init(elev=24, azim=35)
    ax.set_axis_off()
    ax.set_title(f'Distributed 3D clumps and voids\n'
                 f'sigma_l={width}; 97th-percentile color clip', fontsize=12)

    ax2 = fig.add_subplot(gs[row, 1])
    ax2.plot(ell, target, 'k', lw=2, label='Prescribed target')
    ax2.plot(ell, power, color='#d44b40', ls='--', lw=1.7,
             label='Calculated spherical power')
    ax2.axvline(220, color='.5', ls=':')
    ax2.set(xlim=(110, 330), ylim=(0, 1.08), xlabel='Spherical multipole l',
```

```
            ylabel='Normalized potential D_l', title=f'Peak l=220, sigma_l={width}')
    ax2.legend(fontsize=9)
    ax2.grid(alpha=.2)

fig.suptitle('Non-axisymmetric source shell: density varies across latitude AND longitude\n'
             'Exterior shell r=1.012R | gravitational potential, not CMB temperature',
             fontsize=14)
fig.savefig('distributed_spherical_clumps_ell220.png', dpi=170, bbox_inches='tight')
plt.show()
```

/workspace/coepdf/book/recovered/f13_rings_companion.py

```
#!/usr/bin/env python3
"""Reproducible wide/narrow ell=220 inverse-designed exterior-ring experiment.
Requires numpy, scipy, matplotlib.
Outputs two PNG figures and one comparison PNG in the current directory.

IMPORTANT: This computes gravitational-POTENTIAL angular power, NOT CMB temperature
anisotropy. It prescribes the target power and solves for signed shell contrasts.
Negative contrast means underdensity relative to a positive background.
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.special import eval_legendre
from matplotlib.colors import Normalize
from matplotlib.cm import ScalarMappable

L_MAX=420
N_RINGS=480
R=1.0
SOURCE_RADIUS=1.012*R
PEAK_L=220
WIDTHS=(32,8)
ell=np.arange(1,L_MAX+1)
mu,quad_w=np.polynomial.legendre.leggauss(N_RINGS)
P=np.array([eval_legendre(int(l),mu) for l in ell])

def solve(sigma):
    target=np.exp(-0.5*((ell-PEAK_L)/sigma)**2)
    target/=target.max()
    # Axisymmetric source ring expansion:
    # f(mu)=sum_l (2l+1)/2 * q_l * P_l(mu)
    # q_l=int_-1^1 f(mu)P_l(mu)dmu
    # Potential on R=1 due to exterior shell d>1:
    # Phi_l ∝ q_l/d^(l+1)
    # D_l ∝ l(l+1)(2l+1) |Phi_l|^2, up to common constants.
    q=np.sqrt(target/(ell*(ell+1)*(2*ell+1)))*SOURCE_RADIUS**(ell+1)
    contrast=((2*ell+1)/2*q)@P
    # Normalize density contrast only; normalization cancels in power shape.
    contrast/=np.max(np.abs(contrast))
    q_recovered=P@(quad_w*contrast)
    phi=q_recovered/SOURCE_RADIUS**(ell+1)
    power=ell*(ell+1)*(2*ell+1)*np.abs(phi)**2
    power/=power.max()
    return target,contrast,power

```

```
def plot_case(sigma, filename):
    target,contrast,power=solve(sigma)
    fig=plt.figure(figsize=(15,7.8),facecolor="white",layout="constrained")
    gs=fig.add_gridspec(1,3,width_ratios=[1.25,1.1,1.25])
    ax=fig.add_subplot(gs[0,0],projection="3d")
    az=np.linspace(0,2*np.pi,180)
    # All 480 physical latitude rings; slight line transparency to resolve overlaps.
    for j in range(N_RINGS):
        z=SOURCE_RADIUS*mu[j]
        radius=SOURCE_RADIUS*np.sqrt(1-mu[j]**2)
        color=plt.get_cmap("RdBu_r")((contrast[j]+1)/2)
        ax.plot(radius*np.cos(az),radius*np.sin(az),
                np.full_like(az,z),color=color,lw=.65,alpha=.75)
    ax.set(xlim=(-1.1,1.1),ylim=(-1.1,1.1),zlim=(-1.1,1.1))
    ax.set_box_aspect((1,1,1));ax.view_init(elev=24,azim=35)
    ax.set_title(f"480 latitude rings\nExterior shell r={SOURCE_RADIUS:.3f} R")
    ax.set_axis_off()

    ax2=fig.add_subplot(gs[0,1])
    # Meridian slice: each line is the cross section of an entire ring.
    for j in range(N_RINGS):
        z=SOURCE_RADIUS*mu[j]
        x=SOURCE_RADIUS*np.sqrt(1-mu[j]**2)
        color=plt.get_cmap("RdBu_r")((contrast[j]+1)/2)
        ax2.plot([-x,x],[z,z],color=color,lw=.8)
    ax2.add_patch(plt.Circle((0,0),R,fill=False,color="black",lw=1))
    ax2.set_aspect("equal");ax2.set(xlim=(-1.1,1.1),ylim=(-1.1,1.1))
    ax2.set_xlabel("x / R");ax2.set_ylabel("z / R")
    ax2.set_title("Meridional ring cross-sections")

    ax3=fig.add_subplot(gs[0,2])
    ax3.plot(ell,target,color="black",lw=2.2,label="Prescribed target")
    ax3.plot(ell,power,color="#d24a3b",ls="--",lw=1.8,label="Recovered from rings")
    ax3.axvline(PEAK_L,color=".5",ls=":")
    ax3.set(xlim=(100,340),ylim=(0,1.08),xlabel="Spherical multipole □",
            ylabel="Normalized gravitational-potential D□",
            title=f"Peak □={PEAK_L}; σ□={sigma}\nFWHM ≈ {2.355*sigma:.0f}")
    ax3.grid(alpha=.2);ax3.legend(fontsize=9)
    fig.suptitle("INVERSE-DESIGNED GRAVITATIONAL POTENTIAL — NOT CMB TEMPERATURE",fontsize=14)
    sm=ScalarMappable(norm=Normalize(-1,1),cmap="RdBu_r")
    cb=fig.colorbar(sm,ax=[ax,ax2],orientation="horizontal",shrink=.65,pad=.06)
    cb.set_label("Signed density contrast (normalized): blue = void, red = overdensity")
    fig.savefig(filename,dpi=180,bbox_inches="tight")
    plt.close(fig)
    return target,power

def main():
    outputs=[]
    fig,ax=plt.subplots(figsize=(10,5.5),layout="constrained")
    for sigma,color in zip(WIDTHS,("#6d6d6d","#c9473c")):
        path=f"rings_ell220_sigma{sigma}.png"
        target,power=plot_case(sigma,path)
        outputs.append(path)
        ax.plot(ell,power,color=color,lw=2,label=f"Recovered σ□={sigma}")
    ax.axvline(PEAK_L,color="black",ls=":",lw=1)
    ax.set(xlim=(100,340),ylim=(0,1.08),xlabel="Spherical multipole □",
           ylabel="Normalized gravitational-potential D□",
           title="Wide versus narrow □=220 potential-power peaks")
    ax.grid(alpha=.2);ax.legend()
```

```
    comparison="rings_ell220_spectrum_comparison.png"
    fig.savefig(comparison,dpi=180,bbox_inches="tight")
    plt.close(fig)
    outputs.append(comparison)
    print("Created:",*outputs,sep="\n  ")

if __name__=="__main__":
    main()
```

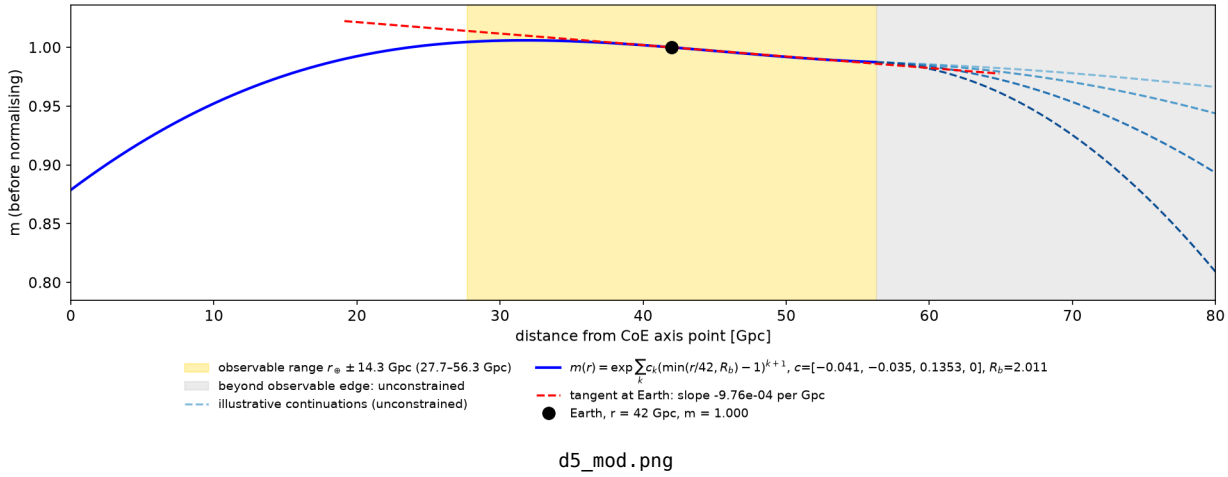
22. Density modulation m(r)

Originally made: Fri Oct 23 2026, 8:30 AM CT (8:30:28.880) (*copy time onto the box (true original likely earlier); same instant as #21, ordered by paper number; redrawn since (last Oct 25 12:27 AM CT)*)
Paper figure number: this morning Fig. 14; now Fig. 6
Files: `d5_mod.png`
Code status: original (box)

Notes

m(r) with Earth at 42 Gpc, band 27.7-56.3 Gpc; illustrative unconstrained falloffs beyond 56.3 Gpc, cut at 80 Gpc. Original version came from coepdf/dens_plots.py (shown under #5). c_k shared with #14.

Paper caption (caveats as stated): CoE modulation $m(r) = \exp\sum_k c_k(\min(r/42, R_b) - 1)^{k+1}$ of the web number density. r is measured from the axis point $42\hat{a}$ Gpc, $\hat{a} \propto (1, 0, \tan 15°)$; $c = [-0.041, -0.035, 0.1353, 0]$, $R_b = 1.5(1 + 1/2.937)$. Normalised to mean 1 in each grid shell. Black dot: Earth at $r = 42$ Gpc ($m = 1$); red dashed: tangent there, slope -9.8×10^{-4} Gpc^{-1} (cf. CatWISE-locked $\rho'/\rho \approx -0.00106$); gold band: how far we see in either direction, $r = 42 \pm 14.3$ Gpc ($d = 14.3$ Gpc, the $D \pm d$ reach of §sec:laws). Grey ($r > 56.3$ Gpc): assumed uniform at the band-edge value $m = 0.987$; dotted: formula continued, not used.



d5_mod.png

Computing chain (run order, full listings)

/workspace/coepdf/d5_mod_earth.py

```
import numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
plt.rcParams.update({'font.size':11})
exec(open('/workspace/coepdf/scripts/shp.py').read())
r0,dv=42.0,14.3005
m=lambda r:shp(r/42)
s=(m(r0+1e-3)-m(r0-1e-3))/2e-3
fig,ax=plt.subplots(figsize=(13,4.6));r=np.linspace(0,80,600)
```

```
ax.axvspan(r0-dv,r0+dv,color='gold',alpha=0.3,label=f'observable range $r_\\oplus\\pm${dv:.1f} Gpc ({r0-dv:.1f}–{r0+dv:.1f} Gpc)')
re=r0+dv;mf=lambda r:np.where(r>re,m(re),m(r))
ax.axvspan(re,80,color='0.85',alpha=0.5,label='beyond observable edge: unconstrained')
se=(m(re)-m(re-1e-3))/1e-3;x=np.linspace(0,80-re,200);ys=[]
for i,k in enumerate([2e-5,6e-5,1.5e-4,3e-4]):
  y=m(re)+se*x-k*x**2;ys+=list(y);ax.plot(re+x,y,ls='--',lw=1.5,color=plt.cm.Blues(0.45+0.15*i),label='illustrative continuations (unconstrained)' if i==0 else None)
ri=r[r<=re];ax.plot(ri,m(ri),'b',lw=2,label=r'$m(r)=\exp\sum_k c_k(\min(r/42,R_b)-1)^{k+1}$, $c$=[−0.041, −0.035, 0.1353, 0], $R_b$=2.011')
rt=np.linspace(r0-1.6*dv,r0+1.6*dv,10);ax.plot(rt,m(r0)+s*(rt-r0),'r--',lw=1.6,label=f'tangent at Earth: slope {s:.2e} per Gpc')
ax.plot(r0,m(r0),'ko',ms=9,label=f'Earth, r = {r0:.0f} Gpc, m = {m(r0):.3f}')
ax.set_xlim(0,80);ax.set_ylim(0.97*min(min(m(ri)),min(ys)),1.03*max(max(m(ri)),max(ys)));ax.set_xlabel('distance from CoE axis point [Gpc]');ax.set_ylabel('m (before normalising)');ax.set_facecolor('w')
fig.legend(loc='lower center',ncol=2,frameon=False,fontsize=9);fig.tight_layout(rect=(0,0.16,1,1));fig.savefig('fig/d5_mod.png',dpi=150,facecolor='w')
print(s,m(r0),m(r0-dv),m(r0+dv))
```

/workspace/coepdf/scripts/shp.py

```
CC=[-0.041,-0.035,0.1353,0.0];Rb=1.5*(1+1/2.937)
shp=lambda x:np.exp(sum(c*(np.minimum(np.abs(x),Rb)-1)**(k+1) for k,c in enumerate(CC)))
```

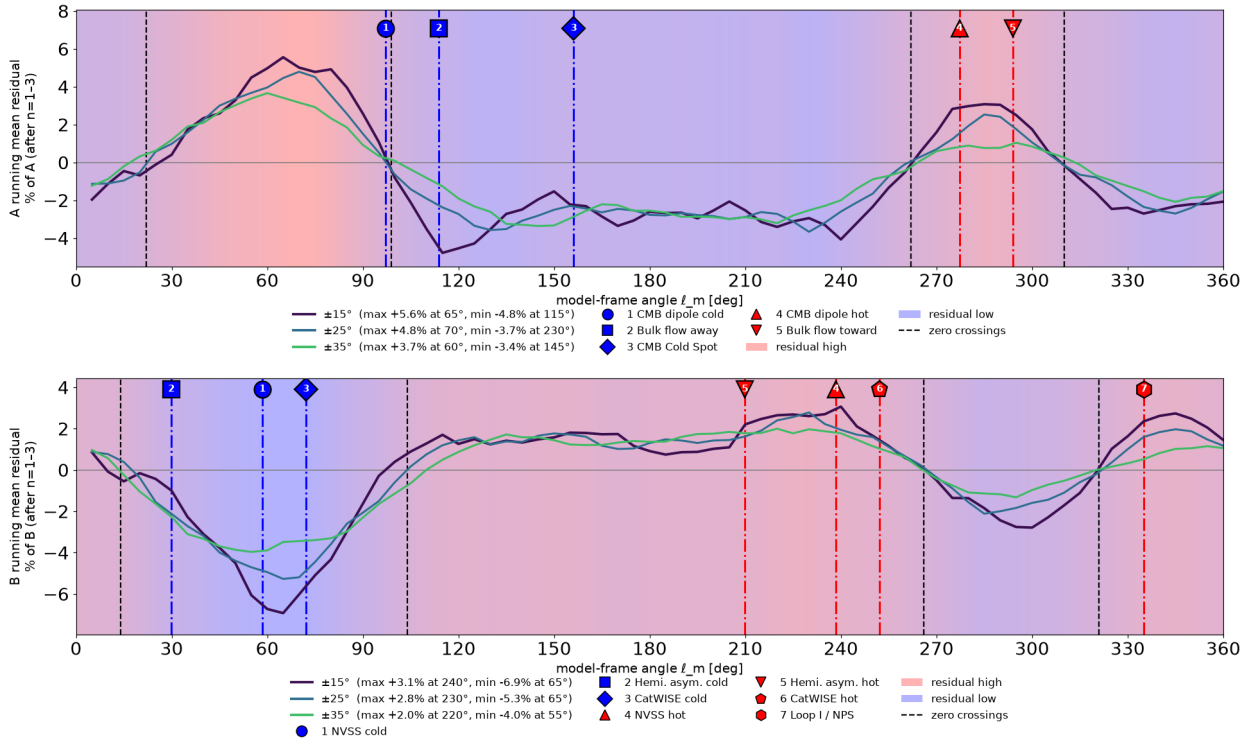
23. A/B running-mean residuals and sky maps

Originally made: Sat Oct 24 2026, 4:40 PM CT (*file creation, coepdf/fig/sky_angle.png*)
Paper figure number: this morning Fig. 27; now Fig. 15
Files: `sky_angle.png, sky_mapA_al.png, sky_mapB_al.png`
Code status: original computing chain (box) + composite

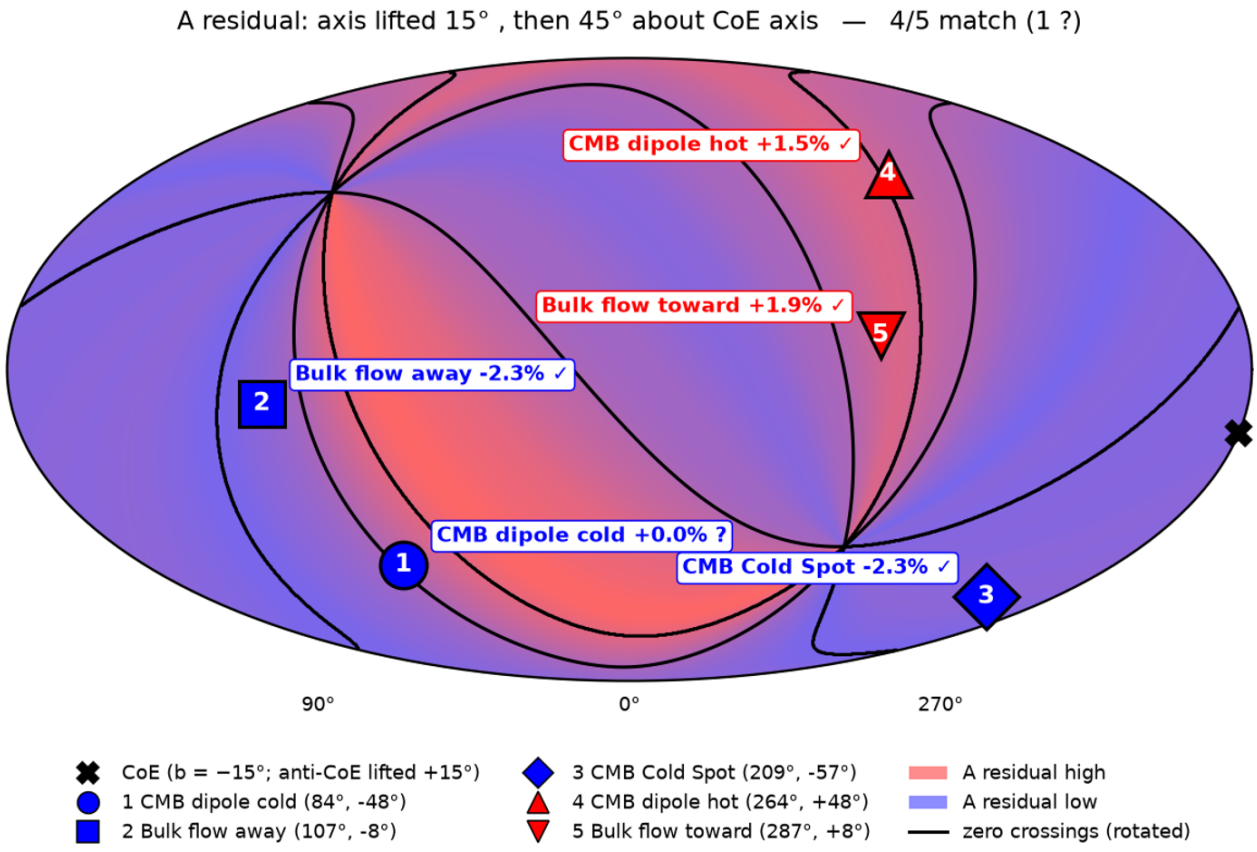
Notes

Residuals vs model-frame angle and residual sky maps for A and B. Depends on the model fits (#11). coepdf/conv.py only references these files in LaTeX; it does not make them. STATUS NOTE: COMPUTING CHAIN (run order): (1) l50/jk2.py (Sat Oct 24 12:03 PM CT) fits the A and B Migkas curves and writes l50/curves.json; (2) fourier_resid/cum2.py reads Migkas digitized data (cosmology/migkas/fig6_digitized.csv) and sets up the angle grid; (3) recon_n3.py removes the n = 1-3 shared harmonics from the A and B residuals; (4) dumpM.py takes the 25-deg running mean of the percent residual and writes /tmp/M25.json (Sat Oct 24 1:50 PM CT); (5) spots.py (named sky features) + /tmp/R.json (hand-entered hi/lo angle windows, 1:46 PM CT, no writer found) feed the residual-vs-angle plot and the sky maps sky_mapA_al.py / sky_mapB_al.py (recovered from /tmp/r45c.py). The plotting heredoc that wrote sky_angle.png itself (4:41 PM CT) was not found; the old image-only remake is removed.

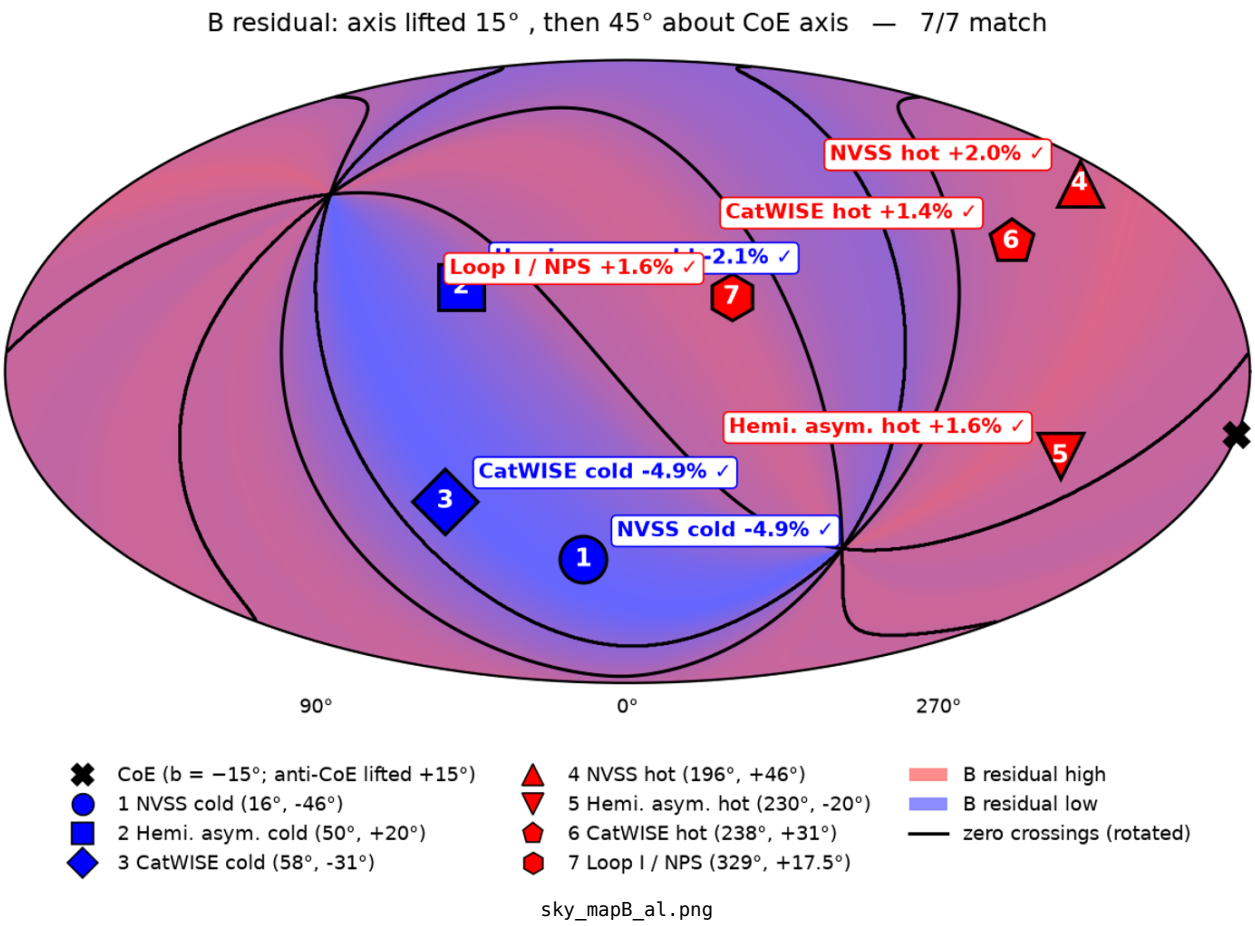
Paper caption (caveats as stated): Top: A and B running-mean residuals vs model-frame angle ($\pm 15/25/35°$). Bottom: residual maps (Galactic Mollweide, $\ell = 0$ at centre, increasing to the left) lifted 15° and rotated 45° about the CoE axis, with spots at published (ℓ, b); left A, right B.



`sky_angle.png`



sky_mapA_al.png



sky_mapB_al.png

Computing chain (run order, full listings)

/workspace/cosmology/figures/l50/jk2.py

```
import numpy as np,json
src=open('jk.py').read().split("allk=np.ones")[0];exec(src)
allk=np.ones(len(z),bool);A0,B0=curves(allk);la,Bg=fit(allk);res=y-(la+Bg*x)
out={'grid':grid.tolist(),'A0':A0.tolist(),'B0':B0.tolist(),'AF':AFm.tolist(),'BL':BLm.tolist()};rep={}
rng=np.random.default_rng(2)
for tag,lo,hi,key in (('Y',15,80,'AB'),('P',260,335,'B')):
  S=(grid>=lo)&(grid<=hi);ref=(AFm if key=='A' else BLm)
  dev=lambda A1,B1:((A1-AFm)[S].mean(),(B1-BLm)[S].mean())
  d0=dev(A0,B0);c=(lo+hi)/2;hw=(hi-lo)/2+45
  idx=np.where(np.abs((l-c+180)%360-180)<=hw)[0];rows=[]
  for i in idx:
    k=allk.copy();k[i]=False;A1,B1=curves(k);d=dev(A1,B1)
    if key=='AB':sc=-(d[0]-d0[0])/0.010+(d[1]-d0[1])/0.009
    else:
      j=0 if key=='A' else 1;sc=-(d[j]-d0[j])*np.sign(d0[j])
    rows.append((sc,int(i)))
  rows.sort(key=lambda r:-r[0]);top=[r[1] for r in rows]
  R={'N':len(idx),'dev0':d0,'list10':[(name[i].strip(),round(l[i],1),round(b[i],1),round(z[i],3),T[i],rou
  ↪  nd(res[i],2)) for i in
  ↪  top[:10]]}
  for kk in (3,5,10):
```

```
    k=allk.copy();k[top[:kk]]=False;A1,B1=curves(k);R[f'top{kk}']=dev(A1,B1);out[f'{tag}A{kk}']=A1.tolist();out[f'{tag}B{kk}']=B1.tolist()
  v=[]
  for _ in range(100):
    k=allk.copy();k[rng.choice(idx,10,replace=False)]=False;v.append(dev(*curves(k)))
  v=np.array(v);R['rand10_mean']=v.mean(0).tolist();R['rand10_sd']=v.std(0).tolist();rep[tag]=R
json.dump(out,open('curves.json','w'));json.dump(rep,open('jk2.json','w'),default=float,indent=1)
print(json.dumps(rep,default=float,indent=1))
```

/workspace/cosmology/figures/fourier_resid/cum2.py

```
import numpy as np,csv
import matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt;plt.rcParams.update({'font.size':18})
from matplotlib.lines import Line2D
from make_A_vs_angle_axisymmetric_before_rotation import periodic_interp
rows=list(csv.DictReader(open('/workspace/cosmology/migkas/fig6_digitized.csv')))
l=np.array([float(r['l_deg']) for r in rows]);th=np.radians(l)
d=np.genfromtxt('residuals.csv',delimiter=',',names=True);Araw,AF,sA=d['A_migkas'],d['A_axisym_AF_smooth'],d['sigma']
Bv=np.array([float(r['B_over_Ball']) for r in rows]);sB=np.array([(float(r['B_err_low'])+float(r['B_err_high']))/2 for r in rows])
gm=np.genfromtxt('sheet_BL_copy2_density_w_rotat.csv',delimiter=',',names=True)
g=np.linspace(0,360,721);gt=np.radians(g)
BG=periodic_interp(gm['ell_deg'],gm['BL_modelB_sheet'],l);BGg=periodic_interp(gm['ell_deg'],gm['BL_modelB_sheet'],g)
AFg=periodic_interp(l,AF,g)
SG=1.;chi=lambda r,s:np.sum(r**2/s**2);o=np.ones_like(th);Z=0*o
rA=Araw-AF;rB=Bv-BG;steps=[]
for n in range(1,5):
 h=np.column_stack([np.cos(n*th),np.sin(n*th)])
 X=np.vstack([np.column_stack([o,Z,h]),np.column_stack([Z,o,SG*h])]);s=np.r_[sA,sB]
 p=np.linalg.lstsq(X/s[:,None],np.r_[rA,rB]/s,rcond=None)[0]
 steps.append(dict(n=n,p=p,C=np.hypot(p[2],p[3]),pk=np.degrees(np.arctan2(p[3],p[2]))%360/n))
 rA=rA-X[:71]@p;rB=rB-X[71:]@p
def term(st,x,k,sg):n=st['n'];p=st['p'];return p[k]+sg*(p[2]*np.cos(n*x)+p[3]*np.sin(n*x))
cols=['purple','tab:blue','tab:red','tab:green','tab:orange']
for nm,Y,sY,base,baseg,k,sg in (('A',Araw,sA,AF,AFg,0,1.),('B',Bv,sB,BG,BGg,1,SG)):
 lab='A/A_all' if nm=='A' else 'B/B_all'
 cum=[(base.copy(),baseg.copy())]
 for st in steps:cum.append((cum[-1][0]+term(st,th,k,sg),cum[-1][1]+term(st,gt,k,sg)))
 fig,axs=plt.subplots(5,2,figsize=(22,30),facecolor='white');print(nm)
plt.rcParams.update({'font.size':20})
for nm,Y,sY,base,baseg,k,sg in (('A',Araw,sA,AF,AFg,0,1.),('B',Bv,sB,BG,BGg,1,SG)):
 lab='A/A_all' if nm=='A' else 'B/B_all'
 cum=[(base.copy(),baseg.copy())]
 for st in steps:cum.append((cum[-1][0]+term(st,th,k,sg),cum[-1][1]+term(st,gt,k,sg)))
 fig,axs=plt.subplots(2,5,figsize=(26,10.8),facecolor='white',sharex=True,sharey='row');print(nm)
 for i in range(5):
  m,mg=cum[i];res=Y-m;c2=chi(res,sY);r2=np.corrcoef(Y,m)[0,1]**2;print(f' {i} chi2={c2:.2f} r2={r2:.3f}')
  a=axs[0,i];a.axvspan(15,80,color='gold',alpha=.18,zorder=0);a.axvspan(260,335,color='lightsteelblue',alpha=.35,zorder=0)
  a.errorbar(l,Y,yerr=sY,fmt='o',color='k',ms=4,alpha=.45,capsize=0,lw=1)
  if i>0:a.plot(g,cum[i-1][1],color='grey',lw=2,ls='--',alpha=.6)
  a.plot(g,mg,color=cols[i],lw=3.5)
  a.set_title('base' if i==0 else f'+ n={i}',fontsize=22,fontweight='bold')
  a.text(.03,.04,f'χ²={c2:.2f}  r²={r2:.3f}',transform=a.transAxes,fontsize=17,color='0.25')
```

```
  b=axs[1,i];b.axvspan(15,80,color='gold',alpha=.18,zorder=0);b.axvspan(260,335,color='lightsteelblue',al
  ↪ pha=.35,zorder=0)
  b.errorbar(l,res,yerr=sY,fmt='o',color='k',ms=4,alpha=.45,capsize=0,lw=1);b.axhline(0,color='grey',lw=1
  ↪ .2)
  if i<4:
   st=steps[i];b.plot(g,term(st,gt,k,sg)-st['p'][k],color=cols[i+1],lw=3.5)
   b.text(.03,.04,f'next n={st["n"]}: C={st["C"]:.3f}',transform=b.transAxes,fontsize=17,color='0.25')
  else:b.text(.03,.04,'final residual',transform=b.transAxes,fontsize=17,color='0.25')
  b.set_xlim(0,360);b.set_xticks([0,90,180,270,360]);b.set_xlabel(' [deg]',fontsize=18)
 axs[0,0].set_ylabel(lab,fontsize=20);axs[1,0].set_ylabel('residual',fontsize=20)
 axs[0,0].set_ylim(0.8,1.2);axs[1,0].set_ylim(-0.2,0.2)
 bn='AF_smooth (axisymmetric)' if nm=='A' else 'CoE sheet (1−BE)·AN'
 h=[Line2D([],[],color='k',marker='o',ls='',alpha=.5,label='Migkas Fig. 6
 ↪ ±σ'),Line2D([],[],color='purple',lw=3,label=f'base:
 ↪ {bn}')]+[Line2D([],[],color=cols[j],lw=3,label=f'shared n={j}') for j in
 ↪ range(1,5)]+[Line2D([],[],color='grey',lw=2,ls='--',label='previous
 ↪ model'),plt.Rectangle((0,0),1,1,color='gold',alpha=.3,label='≈50° deviation
 ↪ (=15–80°)'),plt.Rectangle((0,0),1,1,color='lightsteelblue',alpha=.5,label='≈305° deviation
 ↪ (=260–335°)')]
 fig.legend(handles=h,loc='lower center',ncol=5,fontsize=19,frameon=False)
 fig.tight_layout(rect=(0,0.1,1,1),w_pad=0.3);fig.savefig(f'{nm}_cumulative_harmonics.png',dpi=90,facecol
 ↪ or='white')
```

/workspace/cosmology/figures/fourier_resid/recon_n3.py

```
import numpy as np,json,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
exec(open('cum2.py').read().split("rA=Araw")[0])
d=json.load(open('../l50/curves.json'));g0=np.array(d['grid'])
P=lambda k,x:periodic_interp(g0,np.array(d[k]),x)
A0,B0,AFl,BLl=P('A0',l),P('B0',l),P('AF',l),P('BL',l);AFg,BLg=P('AF',g),P('BL',g)
rA=A0-AFl;rB=B0-BLl;steps2=[]
for n in range(1,4):
  h=np.column_stack([np.cos(n*th),np.sin(n*th)]);X=np.vstack([np.column_stack([o,Z,h]),np.column_stack([Z
  ↪ ,o,h])]);s=np.r_[sA,sB]
  p=np.linalg.lstsq(X/s[:,None],np.r_[rA,rB]/s,rcond=None)[0];steps2.append((n,p));rA=rA-X[:71]@p;rB=rB-X
  ↪ [71:]@p
T=lambda n,p,x,k:p[k]+p[2]*np.cos(n*x)+p[3]*np.sin(n*x)
cols=['purple','tab:blue','tab:red','tab:green']
plt.rcParams.update({'font.size':16});fig,axs=plt.subplots(2,4,figsize=(26,11),facecolor='white',sharex=T
↪ rue,sharey='row')
for row,(nm,Y,sY,mb,mg,k) in enumerate((('A',A0,sA,AFl,AFg,0),('B',B0,sB,BLl,BLg,1))):
  m,mgg=mb.copy(),mg.copy()
  for i in range(4):
    if i>0:n,p=steps2[i-1];pm=mgg.copy();m=m+T(n,p,th,k);mgg=mgg+T(n,p,gt,k)
    a=axs[row,i];a.axvspan(15,80,color='gold',alpha=.18);a.axvspan(260,335,color='lightsteelblue',alpha=.
    ↪ 35)
    a.errorbar(l,Y,yerr=sY,fmt='o',color='k',ms=4,alpha=.5,lw=1)
    if i>0:a.plot(g,pm,color='grey',ls='--',lw=2,alpha=.6)
    a.plot(g,mgg,color=cols[i],lw=3.5)
    r=Y-m;a.text(.03,.04,f'χ²={np.sum(r**2/sY**2):.2f}  r²={np.corrcoef(Y,m)[0,1]**2:.3f}
    ↪ RMS={np.sqrt(np.mean(r**2)):.3f}',transform=a.transAxes,fontsize=14,color='0.2')
    if row==0:a.set_title('CoE model' if i==0 else f'+ shared n={i}',fontweight='bold')
    a.set_xlim(0,360);a.set_xticks([0,90,180,270,360])
  axs[row,0].set_ylabel('A/A_all' if nm=='A' else 'B/B_all')
for a in axs[1]:a.set_xlabel(' [deg]')
from matplotlib.lines import Line2D
```

```
h=[Line2D([],[],color='k',marker='o',ls='',alpha=.5,label='reconstruction, 313 clusters (±σ Fig. 6)'),Line2D([],[],color='purple',lw=3,label='CoE model (A: AF_smooth, B: sheet BL)')]+[Line2D([],[],color=cols[j],lw=3,label=f'+ shared n={j}') for j in (1,2,3)]+[Line2D([],[],color='grey',ls='--',lw=2,label='previous'),plt.Rectangle((0,0),1,1,color='gold',alpha=.3,label='  15–80°'),plt.Rectangle((0,0),1,1,color='lightsteelblue',alpha=.5,label='  260–335°')]
fig.legend(handles=h,loc='lower center',ncol=4,frameon=False,fontsize=15)
fig.tight_layout(rect=(0,.09,1,1));fig.savefig('recon_n3_build.png',dpi=95,facecolor='white')
for n,p in steps2:
  ph=np.degrees(np.arctan2(p[3],p[2]))%360;print(n,round(np.hypot(p[2],p[3]),4),sorted(round((ph/n+360*j/n)%360,1) for j in range(n)))
```

/workspace/cosmology/figures/fourier_resid/dumpM.py

```
exec(open('recon_n3.py').read().split("cols=['purple'")[0])
import json;out={'l':list(map(float,l))}
for nm,Y,mb,k in (('A',A0,AFl,0),('B',B0,BLl,1)):
  m=mb.copy()
  for n,p in steps2[:3]:m=m+T(n,p,th,k)
  r=100*(Y-m)/Y;out[nm]=[float(np.mean(r[np.abs((l-c+180)%360-180)<=25])) for c in l]
json.dump(out,open('/tmp/M25.json','w'))
```

/workspace/coepdf/book/recovered/spots.py

```
SPOTS={'A':[('CMB dipole cold',84,-48,'cold'),('Bulk flow away',107,-8,'cold'),('CMB Cold Spot',209,-57,'cold'),('CMB dipole hot',264,48,'hot'),('Bulk flow toward',287,8,'hot')],
'B':[('NVSS cold',16,-46,'cold'),('Hemi. asym. cold',50,20,'cold'),('CatWISE cold',58,-31,'cold'),('NVSS hot',196,46,'hot'),('Hemi. asym. hot',230,-20,'hot'),('CatWISE hot',238,31,'hot'),('Loop I / NPS',329,17.5,'hot')]}
CC={'hot':'red','cold':'blue','ref':'0.35'}
```

/workspace/coepdf/book/recovered/sky_mapA_al.py

```
import json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
exec(open('/tmp/spots.py').read());MM=json.load(open('/tmp/M25.json'))
Z={'A':[22,99,262,310],'B':[14,104,266,321]}
TILT,ROLL=-15.0,45.0
def vec(l,b):l,b=np.radians(l),np.radians(b);return np.stack([np.cos(b)*np.cos(l),np.cos(b)*np.sin(l),np.sin(b)],-1)
def rot(ax,a):
  ax=ax/np.linalg.norm(ax);a=np.radians(a);K=np.array([[0,-ax[2],ax[1]],[ax[2],0,-ax[0]],[-ax[1],ax[0],0]])
  return np.eye(3)+np.sin(a)*K+(1-np.cos(a))*K@K
R1=rot(np.array([0,1.,0]),TILT)           # CoE (180,0) -> b=+15
coe=R1@vec(180,0);R=rot(coe,ROLL)@R1      # +80 right-handed about CoE axis
def lm(l,b):m=vec(l,b)@R;return np.degrees(np.arctan2(m[...,1],m[...,0]))%360   # model-frame l
def moll(l,b):
  lam=np.radians(((np.asarray(l,float)+180)%360)-180);b=np.radians(np.asarray(b,float));th=b.copy()
  for _ in range(60):th=th-(2*th+np.sin(2*th)-np.pi*np.sin(b))/(2+2*np.cos(2*th)+1e-12)
  return -2*np.sqrt(2)/np.pi*lam*np.cos(th),np.sqrt(2)*np.sin(th)
# inverse mollweide grid
X,Y=np.meshgrid(np.linspace(-2*np.sqrt(2),2*np.sqrt(2),1400),np.linspace(-np.sqrt(2),np.sqrt(2),700))
ok=(X/(2*np.sqrt(2)))**2+(Y/np.sqrt(2))**2<=1
th=np.arcsin(np.clip(Y/np.sqrt(2),-1,1));B=np.degrees(np.arcsin(np.clip((2*th+np.sin(2*th))/np.pi,-1,1)))
```

```
L=np.degrees(-np.pi*X/(2*np.sqrt(2)*np.cos(th)+1e-12))%360
LM=lm(L,B);rows=[]
fig,axs=plt.subplots(2,1,figsize=(15,15),facecolor='white')
for ax,K in zip(axs,'AB'):
  v=np.interp(LM,MM['l'],MM[K],period=360);gm=np.abs(v[ok]).max()
  img=np.ones(X.shape+(4,));img[...,3]=0
  f=np.clip((v/gm+1)/2,0,1)
  img[...,0]=f;img[...,1]=0;img[...,2]=1-f;img[...,3]=np.where(ok,.6,0)
  ax.imshow(img,extent=(X.min(),X.max(),Y.min(),Y.max()),origin='lower',interpolation='bilinear')
  for z in Z[K]:
    d=(LM-z+180)%360-180;ax.contour(X,Y,np.where(ok,d,np.nan),[0],colors='k',linewidths=2)
  t=np.linspace(0,2*np.pi,400);ax.plot(2*np.sqrt(2)*np.cos(t),np.sqrt(2)*np.sin(t),'k',lw=1.5)
  for lg in (0,90,270):ax.text(moll(lg,0)[0],-1.55,f'{lg}°',ha='center',fontsize=12)
  for nm_,(cl,cb),mk in [('CoE',(180,0),'X')]:
    c=R1@vec(cl,cb);c2=np.degrees([np.arctan2(c[1],c[0])%(2*np.pi),np.arcsin(c[2])]);xs,ys=moll(*c2);ax.p
    ↪ lot(xs,ys,'kX',ms=16)
  MK=['o','s','D','^','v','p','h'];h=[Line2D([],[],ls='',marker='X',color='k',ms=13,label='CoE (b = −15°;
  ↪ anti-CoE lifted +15°)')];sc=0
  for i,(nm,sl,sb,c) in enumerate(SPOTS[K]):
    val=float(np.interp(lm(sl,sb),MM['l'],MM[K],period=360));okk=(val>0)==(c=='hot');sc+=okk
    rows.append((K,nm,sl,sb,round(float(lm(sl,sb)),1),round(val,2),okk))
    xs,ys=moll(sl,sb);xs,ys=float(xs),float(ys)
    ax.plot(xs,ys,MK[i],ms=28,mfc=CC[c],mec='k',mew=2,zorder=7);ax.text(xs,ys,str(i+1),ha='center',va='ce
    ↪ nter',color='white',fontsize=15,fontweight='bold',zorder=8)
    ax.annotate(f"{nm} {val:+.1f}% {'✓' if okk else ('?' if abs(val)<0.1 else
    ↪ '✗')}",(xs,ys),xytext=(-20,14) if xs>0 else (20,14),textcoords='offset points',ha='right' if xs>0
    ↪ else 'left',fontsize=12.5,fontweight='bold',color=CC[c],zorder=9,bbox=dict(fc='white',ec=CC[c],bo
    ↪ xstyle='round,pad=.25'))
    h.append(Line2D([],[],ls='',marker=MK[i],mfc=CC[c],mec='k',ms=13,label=f'{i+1} {nm} ({sl}°,
    ↪ {sb:+g}°)'))
  h+=[plt.Rectangle((0,0),1,1,fc='red',alpha=.45,label=f'{K} residual
  ↪ high'),plt.Rectangle((0,0),1,1,fc='blue',alpha=.45,label=f'{K} residual
  ↪ low'),Line2D([],[],color='k',lw=2,label='zero crossings (rotated)')]
  ax.legend(handles=h,loc='upper center',bbox_to_anchor=(.5,-.02),ncol=3,fontsize=11.5,frameon=False)
  ax.set_title(f'{K} residual: axis lifted 15° , then 45° about CoE axis   −   {sc}/{len(SPOTS[K])}
  ↪ match'+(' (1 ?)' if K=='A' else ''),fontsize=15)
  ax.set_aspect('equal');ax.axis('off');ax.set_xlim(-2.9,2.9);ax.set_ylim(-1.65,1.5)
fig.suptitle('Galactic Mollweide,  = 0 at centre,  increases to the left; spots at published (,
↪ b)',fontsize=13,y=.995)
fig.tight_layout();fig.savefig('overlay_lift15_r45_bluered.png',dpi=110,facecolor='white')
for r in rows:print(r)
```

/workspace/coepdf/book/recovered/sky_mapB_al.py

```
import json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
exec(open('/tmp/spots.py').read());MM=json.load(open('/tmp/M25.json'))
Z={'A':[22,99,262,310],'B':[14,104,266,321]}
TILT,ROLL=-15.0,45.0
def vec(l,b):l,b=np.radians(l),np.radians(b);return
↪ np.stack([np.cos(b)*np.cos(l),np.cos(b)*np.sin(l),np.sin(b)],-1)
def rot(ax,a):
  ax=ax/np.linalg.norm(ax);a=np.radians(a);K=np.array([[0,-ax[2],ax[1]],[ax[2],0,-ax[0]],[-ax[1],ax[0],0]
  ↪ ])
  return np.eye(3)+np.sin(a)*K+(1-np.cos(a))*K@K
R1=rot(np.array([0,1.,0]),TILT)           # CoE (180,0) -> b=+15
coe=R1@vec(180,0);R=rot(coe,ROLL)@R1      # +80 right-handed about CoE axis
def lm(l,b):m=vec(l,b)@R;return np.degrees(np.arctan2(m[...,1],m[...,0]))%360   # model-frame l
```

```
def moll(l,b):
  lam=np.radians(((np.asarray(l,float)+180)%360)-180);b=np.radians(np.asarray(b,float));th=b.copy()
  for _ in range(60):th=th-(2*th+np.sin(2*th)-np.pi*np.sin(b))/(2+2*np.cos(2*th)+1e-12)
  return -2*np.sqrt(2)/np.pi*lam*np.cos(th),np.sqrt(2)*np.sin(th)
# inverse mollweide grid
X,Y=np.meshgrid(np.linspace(-2*np.sqrt(2),2*np.sqrt(2),1400),np.linspace(-np.sqrt(2),np.sqrt(2),700))
ok=(X/(2*np.sqrt(2)))**2+(Y/np.sqrt(2))**2<=1
th=np.arcsin(np.clip(Y/np.sqrt(2),-1,1));B=np.degrees(np.arcsin(np.clip((2*th+np.sin(2*th))/np.pi,-1,1)))
L=np.degrees(-np.pi*X/(2*np.sqrt(2)*np.cos(th)+1e-12))%360
LM=lm(L,B);rows=[]
fig,axs=plt.subplots(2,1,figsize=(15,15),facecolor='white')
for ax,K in zip(axs,'AB'):
  v=np.interp(LM,MM['l'],MM[K],period=360);gm=np.abs(v[ok]).max()
  img=np.ones(X.shape+(4,));img[...,3]=0
  f=np.clip((v/gm+1)/2,0,1)
  img[...,0]=f;img[...,1]=0;img[...,2]=1-f;img[...,3]=np.where(ok,.6,0)
  ax.imshow(img,extent=(X.min(),X.max(),Y.min(),Y.max()),origin='lower',interpolation='bilinear')
  for z in Z[K]:
    d=(LM-z+180)%360-180;ax.contour(X,Y,np.where(ok,d,np.nan),[0],colors='k',linewidths=2)
  t=np.linspace(0,2*np.pi,400);ax.plot(2*np.sqrt(2)*np.cos(t),np.sqrt(2)*np.sin(t),'k',lw=1.5)
  for lg in (0,90,270):ax.text(moll(lg,0)[0],-1.55,f'{lg}°',ha='center',fontsize=12)
  for nm_,(cl,cb),mk in [('CoE',(180,0),'X')]:
    c=R1@vec(cl,cb);c2=np.degrees([np.arctan2(c[1],c[0])%(2*np.pi),np.arcsin(c[2])]);xs,ys=moll(*c2);ax.p
    ↪ lot(xs,ys,'kX',ms=16)
  MK=['o','s','D','^','v','p','h'];h=[Line2D([],[],ls='',marker='X',color='k',ms=13,label='CoE (b = −15°;
  ↪ anti-CoE lifted +15°)')];sc=0
  for i,(nm,sl,sb,c) in enumerate(SPOTS[K]):
    val=float(np.interp(lm(sl,sb),MM['l'],MM[K],period=360));okk=(val>0)==(c=='hot');sc+=okk
    rows.append((K,nm,sl,sb,round(float(lm(sl,sb)),1),round(val,2),okk))
    xs,ys=moll(sl,sb);xs,ys=float(xs),float(ys)
    ax.plot(xs,ys,MK[i],ms=28,mfc=CC[c],mec='k',mew=2,zorder=7);ax.text(xs,ys,str(i+1),ha='center',va='ce
    ↪ nter',color='white',fontsize=15,fontweight='bold',zorder=8)
    ax.annotate(f"{nm} {val:+.1f}% {'✓' if okk else ('?' if abs(val)<0.1 else
    ↪ '✗')}",(xs,ys),xytext=(-20,14) if xs>0 else (20,14),textcoords='offset points',ha='right' if xs>0
    ↪ else 'left',fontsize=12.5,fontweight='bold',color=CC[c],zorder=9,bbox=dict(fc='white',ec=CC[c],bo
    ↪ xstyle='round,pad=.25'))
    h.append(Line2D([],[],ls='',marker=MK[i],mfc=CC[c],mec='k',ms=13,label=f'{i+1} {nm} ({sl}°,
    ↪ {sb:+g}°)'))
  h+=[plt.Rectangle((0,0),1,1,fc='red',alpha=.45,label=f'{K} residual
  ↪ high'),plt.Rectangle((0,0),1,1,fc='blue',alpha=.45,label=f'{K} residual
  ↪ low'),Line2D([],[],color='k',lw=2,label='zero crossings (rotated)')]
  ax.legend(handles=h,loc='upper center',bbox_to_anchor=(.5,-.02),ncol=3,fontsize=11.5,frameon=False)
  ax.set_title(f'{K} residual: axis lifted 15° , then 45° about CoE axis   −   {sc}/{len(SPOTS[K])}
  ↪ match'+(' (1 ?)' if K=='A' else ''),fontsize=15)
  ax.set_aspect('equal');ax.axis('off');ax.set_xlim(-2.9,2.9);ax.set_ylim(-1.65,1.5)
fig.suptitle('Galactic Mollweide,  = 0 at centre,  increases to the left; spots at published (,
↪ b)',fontsize=13,y=.995)
fig.tight_layout();fig.savefig('overlay_lift15_r45_bluered.png',dpi=110,facecolor='white')
for r in rows:print(r)
```

24. Spot-scores table

Originally made: Sat Oct 24 2026, 4:46 PM CT (*file creation, coepdf/fig/sky_tab_only.png*)
Paper figure number: this morning Fig. 26; now Fig. 14
Files: `sky_tab_only.png`
Code status: recovered (box /tmp)

Notes

Spot scores by sign of the tied residual (A vs B spots). Built from #23. STATUS NOTE: Table section of /tmp/comb45c.py; same content, 5 px wider at dpi 120. Inputs as for #23 (M25.json from dumpM.py).

Paper caption (caveats as stated): Spot scores by sign of the tied residual. The qualitative assignment of each spot to A or B is given in the text below.

Axis lifted 15°, then 45° counterclockwise about CoE axis — Score: A 4/5, B 7/7, total 11/12, 1 on the zero line (?)

Tied to	Spot	ℓ, b	Hot/cold	Needs	Residual at spot (%)	Match	Why A or B
A	CMB dipole cold	84, −48	cold	< 0	+0.02	?	Our motion → distance/velocity
A	Bulk flow away	107, −8	cold	< 0	−2.27	✓	Recession velocity → distance
A	CMB Cold Spot	209, −57	cold	< 0	−2.33	✓	Eridanus supervoid → distance
A	CMB dipole hot	264, +48	hot	> 0	+1.54	✓	Our motion → distance/velocity
A	Bulk flow toward	287, +8	hot	> 0	+1.90	✓	Recession velocity → distance
B	NVSS cold	16, −46	cold	< 0	−4.87	✓	Source density → sample
B	Hemi. asym. cold	50, +20	cold	< 0	−2.11	✓	Power direction → sample
B	CatWISE cold	58, −31	cold	< 0	−4.87	✓	Matter dipole → sample
B	NVSS hot	196, +46	hot	> 0	+2.03	✓	Source density → sample
B	Hemi. asym. hot	230, −20	hot	> 0	+1.62	✓	Power direction → sample
B	CatWISE hot	238, +31	hot	> 0	+1.39	✓	Matter dipole → sample
B	Loop I / NPS	329, +17.5	hot	> 0	+1.60	✓	Soft X-ray foreground → slope

sky_tab_only.png

Computing chain (run order, full listings)

/workspace/coepdf/book/recovered/spots.py

```
SPOTS={'A':[('CMB dipole cold',84,-48,'cold'),('Bulk flow away',107,-8,'cold'),('CMB Cold Spot',209,-57,'cold'),('CMB dipole hot',264,48,'hot'),('Bulk flow toward',287,8,'hot')],
'B':[('NVSS cold',16,-46,'cold'),('Hemi. asym. cold',50,20,'cold'),('CatWISE cold',58,-31,'cold'),('NVSS hot',196,46,'hot'),('Hemi. asym. hot',230,-20,'hot'),('CatWISE hot',238,31,'hot'),('Loop I / NPS',329,17.5,'hot')]}
CC={'hot':'red','cold':'blue','ref':'0.35'}
```

/workspace/coepdf/book/recovered/sky_tab_only.py

```
import matplotlib;matplotlib.use('Agg')
import matplotlib.pyplot as plt
from PIL import Image
top=Image.open('combined_v5.png').crop((0,70,2006,1175))
top.save('/tmp/top.png')
rows=[('A','CMB dipole cold','84, −48','cold','< 0','+0.02','?','Our motion → distance/velocity'),
('A','Bulk flow away','107, −8','cold','< 0','−2.27','✓','Recession velocity → distance'),
('A','CMB Cold Spot','209, −57','cold','< 0','−2.33','✓','Eridanus supervoid → distance'),
```

```
('A','CMB dipole hot','264, +48','hot','> 0','+1.54','✓','Our motion → distance/velocity'),
('A','Bulk flow toward','287, +8','hot','> 0','+1.90','✓','Recession velocity → distance'),
('B','NVSS cold','16, −46','cold','< 0','−4.87','✓','Source density → sample'),
('B','Hemi. asym. cold','50, +20','cold','< 0','−2.11','✓','Power direction → sample'),
('B','CatWISE cold','58, −31','cold','< 0','−4.87','✓','Matter dipole → sample'),
('B','NVSS hot','196, +46','hot','> 0','+2.03','✓','Source density → sample'),
('B','Hemi. asym. hot','230, −20','hot','> 0','+1.62','✓','Power direction → sample'),
('B','CatWISE hot','238, +31','hot','> 0','+1.39','✓','Matter dipole → sample'),
('B','Loop I / NPS','329, +17.5','hot','> 0','+1.60','✓','Soft X-ray foreground → slope')]
fig,ax=plt.subplots(figsize=(18,5.2));ax.axis('off')
t=ax.table(cellText=[r[:7]+(r[7],) for r in rows],colLabels=['Tied to','Spot','□,
↪  b','Hot/cold','Needs','Residual at spot (%)','Match','Why A or
↪  B'],loc='center',cellLoc='center',colWidths=[.06,.15,.1,.07,.06,.13,.06,.3])
t.auto_set_font_size(False);t.set_fontsize(12);t.scale(1,1.6)
for i,r in enumerate(rows,1):
  t[i,3].get_text().set_color('red' if r[3]=='hot' else 'blue')
  t[i,6].get_text().set_color({'✓':'green','?':'darkorange'}.get(r[6],'red'))
ax.set_title('Axis lifted 15°, then 45° counterclockwise about CoE axis  −  Score: A 4/5, B 7/7, total
↪  11/12, 1 on the zero line (?)',fontsize=15)
fig.savefig('/tmp/tab.png',dpi=110,facecolor='white',bbox_inches='tight')
ims=[Image.open(p).convert('RGB') for p in
↪  ['/tmp/top_new.png','overlay_lift15_r45_bluered.png','/tmp/tab.png']]
W=max(i.width for i in ims)
ims=[i.resize((W,int(i.height*W/i.width))) for i in ims]
out=Image.new('RGB',(W,sum(i.height for i in ims)),'white');y=0
for i in ims: out.paste(i,(0,y));y+=i.height
out.save('combined_r45_bluered.png')
```

25. Predicted MAX/MIN bands on Migkas Fig 8

Originally made: Sat Oct 24 2026, 8:17 PM CT (*file creation, co-epdf/fig/Fig06_overlay_one.png*)
Paper figure number: this morning Fig. 8; now Fig. 11
Files: `Fig06_overlay_one.png`
Code status: original computing chain (box) + stack

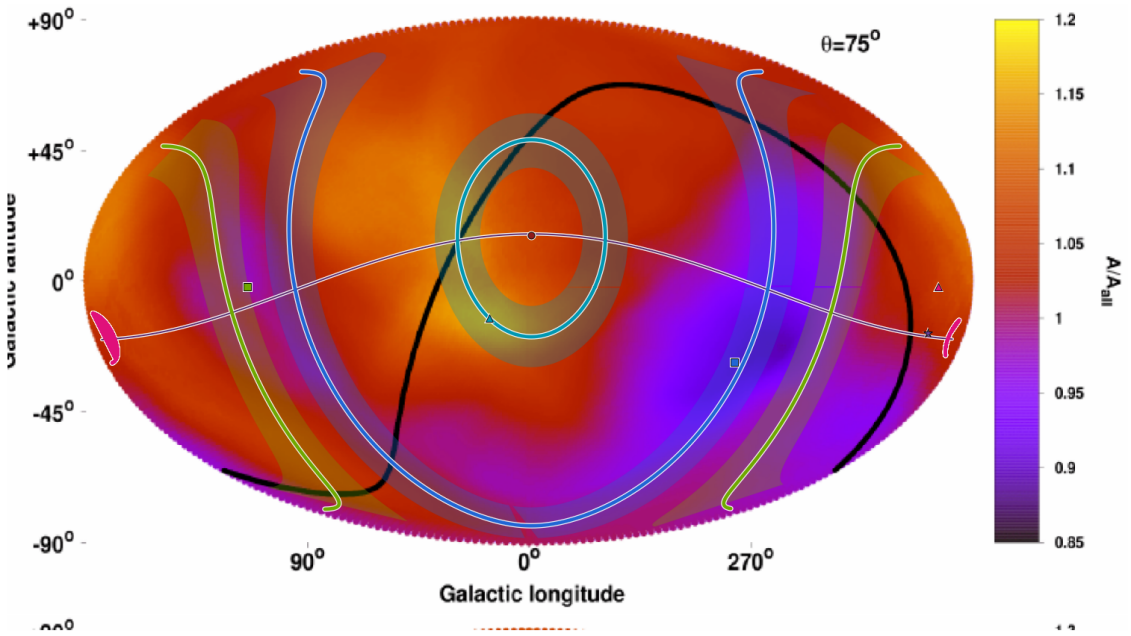
Notes

Bands overlaid on Migkas Fig 8. Related makers exist (figs_letter_exact_python/makers/make_fig06_overlay_stack.py, make_fig06_matplotlib_clean.py) but write Fig06_overlay_stack.png / Fig06_matplotlib_clean.png, not this file. STATUS NOTE: COMPUTING CHAIN: make_migkas_fig8_overlay_matched.py computes the CoE model MAX/MIN bands for the +15 deg tilt, +gradient, and +gradient+80.8 deg roll cases and draws them on Migkas Fig. 8; it writes cosmology/papers/spreadsheet_model/fig8_overlay_matched.png, _grad.png, _grad_rot81.png, which are md5-identical to figs_letter_exact_python/Fig06_overlay_matched*.png. STACK (one line): Fig06_overlay_one.png stacks those three PNGs with row labels; the stacking command (Sat Oct 24 8:17 PM CT) was not found.

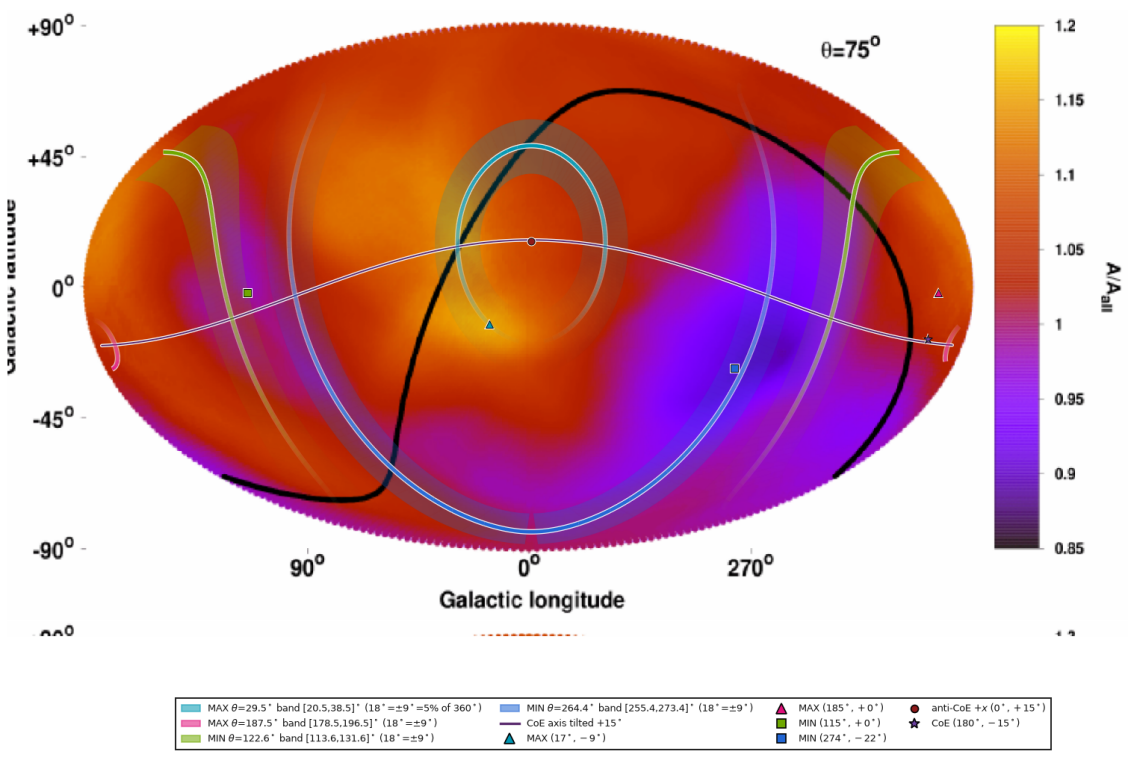
Paper caption (caveats as stated): Predicted MAX/MIN bands on Migkas Fig. 8 (two MAX and two MIN bands, each 18° wide) for the +15° tilt alone (top), tilt with gradient (middle), and tilt, gradient and +80.8° roll (bottom). Regenerated from code (the original preprint's Fig. 6).

Migkas Fig. 8 — $\theta = 75^\circ$ $A/A_{\rm all}$ (their numbered colorbar)

+15° tilt



+15° tilt + gradient



+15° tilt + gradient + 80.8° roll

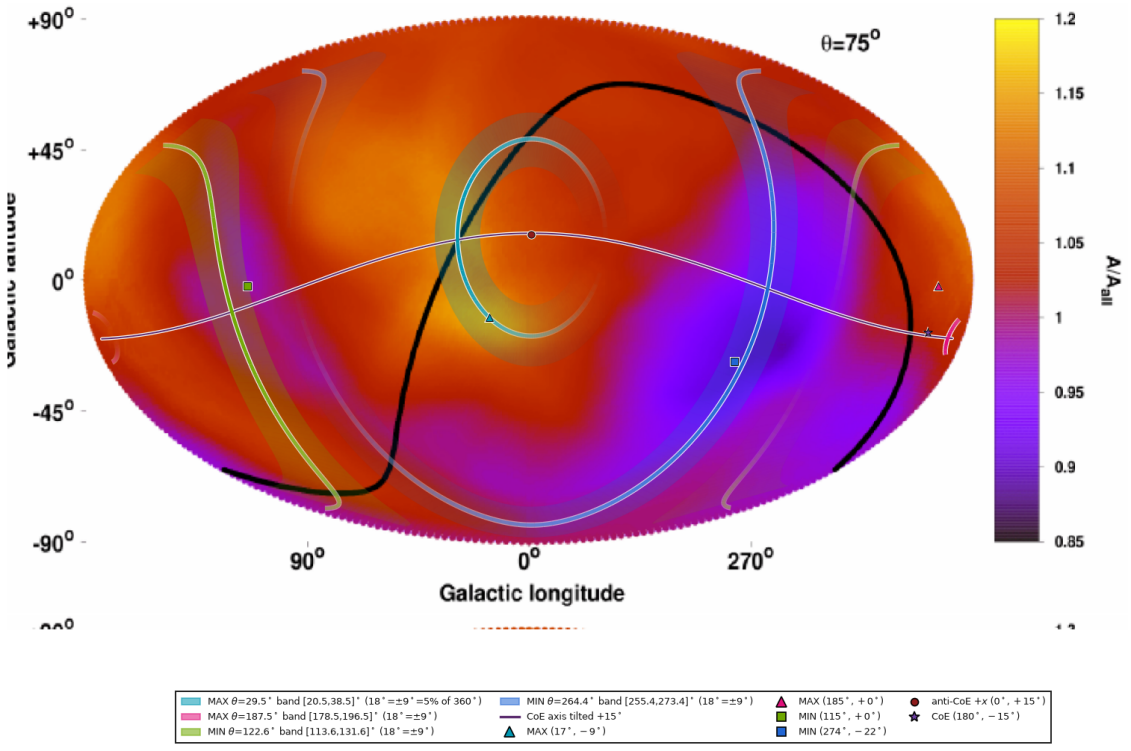


Fig06_overlay_one.png

Computing chain (run order, full listings)

/workspace/cosmology/figures/make_migkas_fig8_overlay_matched.py

```
#!/usr/bin/env python3
"""
Superimpose CoE MAX/MIN highlight bands + Migkas markers onto the REAL Migkas
Fig. 8 theta=75 A/A_all Mollweide map.

Matches make_coe_rings_migkas_pin_axis.py (three-panel Panel B) geometry:
  Exactly FOUR features — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ + ONE solid mid-stroke:
    MAX 30°  → [21°, 39°]   color #00e5ff (bright cyan)
    MAX 185° → [176°, 194°] color #ff2d95 (magenta/hot pink)
    MIN 115° → [106°, 124°] color #b8f200 (lime)
    MIN 265° → [256°, 274°] color #4fc3f7 (sky blue)
  Drawing: clean PolyCollection fill + one mid-stroke (optional thin white
  underlay on mid only). NO black rim, NO thick white halo stacks, NO dual
  boundary-edge strokes. No 283° companion. No Dec=0 overlay.
  MAX triangles: (17°, −9°) and (185°, 0°)
  MIN squares:   (115°, 0°) and (274°, −22°)
  CoE axis tilted +30°: anti-CoE (0°,+30°), CoE (180°,−30°) (purple).

Registration (unchanged from working Fig. 8):
  GOOD_REG = (cx, cy, Rx, Ry) = (863.0, 763.25, 693.9927, 392.542425)
  Source: FULL_migkas_theta75.png
  CoE axis tilted +30°; clip/inset so overlays stay inside colored oval
  lon_disp = -wrap(□)

Outputs:
  migkas_fig8_theta75_with_coe_rings.png
  migkas_fig8_theta75_with_coe_rings.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.collections import PolyCollection
from matplotlib.lines import Line2D
from matplotlib.patches import Ellipse, Patch
from PIL import Image

OUT_DIR = Path(__file__).resolve().parent
SM_DIR = Path("/workspace/cosmology/papers/spreadsheet_model")
PNG_PATH = SM_DIR / "fig8_overlay_matched.png"
JSON_PATH = SM_DIR / "fig8_overlay_matched.json"

# Versions = panel B of each three-panel figure (grad: opacity gradient, roll: deg CCW about CoE axis)
VERSIONS = [
    ("matched", False, 0.0),
    ("matched_grad", True, 0.0),
    ("matched_grad_rot45", True, 45.0),
    ("matched_grad_rot81", True, 80.8),
    ("matched_rot81", False, 80.8),  # roll + all loops, no opacity gradient
]
GRAD = False
ROLL = 0.0
SHOW_LEGEND = False  # letter Fig.6: no legend box/strip

SOURCE_CANDIDATES = [
    Path("/workspace/cosmology/map_compare/FULL_migkas_theta75.png"),
```

```
    "Ry": 392.542425,
}

# Sky markers (, b): map extrema + strip stand-ins (same as 3-panel Panel B).
MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

# Galactic pose (Chris 2026-09-22): anti-CoE (+D/+x) at (0°, +30°); CoE at (180°, −30°)
ANTI_COE_LB = (0.0, 15.0)
COE_LB = (180.0, -15.0)
AXIS_TILT_B_DEG = 15.0


def lb_to_n(l_deg, b_deg):
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


D_HAT = lb_to_n(*ANTI_COE_LB)  # +D = anti-CoE = +x


def great_circle_through_poles(D, n=721):
    """Great circle through ±D (CoE axis meridian on the sky)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs

# High-contrast loop colors that pop on magma (avoid reds/oranges/corals)
C_MAX_30 = "#0097b2"    # cyan — MAX 29.5° (three-panel matched)
C_MAX_185 = "#e0107a"   # magenta — MAX 187.5°
C_MIN_115 = "#6fa800"   # green — MIN 122.6°
C_MIN_265 = "#2166d4"   # blue — MIN 264.4°
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]
C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds (match three-panel)
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"


def wrap_lon_deg(lon):
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg):
    return -wrap_lon_deg(ell_deg)


```

```
def n_to_lb(n):
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n=2500):
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.clip(
        np.sin(dec) * np.sin(dec_ngp)
        + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp),
        -1.0,
        1.0,
    )
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = (
        np.sin(dec) * np.cos(dec_ngp)
        - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    )
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def small_circle_about_axis(axis, mu, n=721):
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (
        np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2)
    )


def mollweide_xy(lon_rad, lat_rad, n_iter=64):
    """Standard Mollweide; stable bracketed solve of 2θ+sin(2θ)=π sin(lat)."""
    lon = np.asarray(lon_rad, dtype=float)
    lat = np.asarray(lat_rad, dtype=float)
    target = np.pi * np.sin(np.clip(lat, -np.pi / 2, np.pi / 2))
    lo = np.full_like(lat, -np.pi / 2, dtype=float)
    hi = np.full_like(lat, np.pi / 2, dtype=float)
    for _ in range(n_iter):
        mid = 0.5 * (lo + hi)
        g = 2.0 * mid + np.sin(2.0 * mid)
        lo = np.where(g < target, mid, lo)
        hi = np.where(g >= target, mid, hi)
    theta = 0.5 * (lo + hi)
    theta = np.where(lat >= np.pi / 2 - 1e-15, np.pi / 2, theta)
    theta = np.where(lat <= -np.pi / 2 + 1e-15, -np.pi / 2, theta)
```

```
    x = (2.0 * np.sqrt(2.0) / np.pi) * lon * np.cos(theta)
    y = np.sqrt(2.0) * np.sin(theta)
    return x, y


def lb_to_px(ell_deg, b_deg, cx, cy, Rx, Ry):
    lon = np.deg2rad(lon_disp_from_ell(np.asarray(ell_deg, dtype=float)))
    lat = np.deg2rad(np.asarray(b_deg, dtype=float))
    x, y = mollweide_xy(lon, lat)
    Xn = x / (2.0 * np.sqrt(2.0))
    Yn = y / np.sqrt(2.0)
    return cx + Xn * Rx, cy - Yn * Ry


def as_xy(px, py):
    return float(np.asarray(px).reshape(-1)[0]), float(np.asarray(py).reshape(-1)[0])


def chamfer_distance_to_dark(dark):
    h, w = dark.shape
    INF = 1e6
    D = np.where(dark, 0.0, INF)
    for y in range(h):
        for x in range(w):
            v = D[y, x]
            if x > 0:
                v = min(v, D[y, x - 1] + 1.0)
            if y > 0:
                v = min(v, D[y - 1, x] + 1.0)
            if x > 0 and y > 0:
                v = min(v, D[y - 1, x - 1] + 1.414)
            if x + 1 < w and y > 0:
                v = min(v, D[y - 1, x + 1] + 1.414)
            D[y, x] = v
    for y in range(h - 1, -1, -1):
        for x in range(w - 1, -1, -1):
            v = D[y, x]
            if x + 1 < w:
                v = min(v, D[y, x + 1] + 1.0)
            if y + 1 < h:
                v = min(v, D[y + 1, x] + 1.0)
            if x + 1 < w and y + 1 < h:
                v = min(v, D[y + 1, x + 1] + 1.414)
            if x > 0 and y + 1 < h:
                v = min(v, D[y + 1, x - 1] + 1.414)
            D[y, x] = v
    return D


def detect_oval_bbox(rgb):
    h, w, _ = rgb.shape
    white = (rgb[:, :, 0] > 250) & (rgb[:, :, 1] > 250) & (rgb[:, :, 2] > 250)
    oval_left = np.full(h, np.nan)
    oval_right = np.full(h, np.nan)
    for y in range(h):
        row = ~white[y]
        d = np.diff(np.r_[0, row.astype(int), 0])
        starts = np.where(d == 1)[0]
        ends = np.where(d == -1)[0] - 1
```

```
        if len(starts) == 0:
            continue
        lengths = ends - starts + 1
        candidates = [
            (int(lengths[i]), int(starts[i]), int(ends[i]))
            for i in range(len(starts))
            if starts[i] < 0.72 * w and lengths[i] > 40
        ]
        if not candidates:
            continue
        candidates.sort(reverse=True)
        _, L, R = candidates[0]
        oval_left[y] = L
        oval_right[y] = R
    widths = np.where(np.isfinite(oval_left), oval_right - oval_left, 0.0)
    yeq = int(np.argmax(widths))
    thresh = 0.08 * widths[yeq]
    y0 = yeq
    while y0 > 0 and widths[y0 - 1] >= thresh:
        y0 -= 1
    y1 = yeq
    while y1 < h - 1 and widths[y1 + 1] >= thresh:
        y1 += 1
    x0 = float(oval_left[yeq])
    x1 = float(oval_right[yeq])
    return {
        "x0": int(x0),
        "x1": int(x1),
        "y0": int(y0),
        "y1": int(y1),
        "yeq": int(yeq),
        "cx": 0.5 * (x0 + x1),
        "cy": 0.5 * (y0 + y1),
        "Rx": 0.5 * (x1 - x0),
        "Ry": 0.5 * (y1 - y0 + 1),
    }


def refine_registration(rgb, seed):
    """Gray/dark Dec=0 lock (pre medial-axis). Prefer GOOD_REG hard-set in main."""
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)

    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)

    def median_dist(cx, cy, Rx, Ry):
        px = cx + Xn * Rx
        py = cy - Yn * Ry
        m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
        xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
        yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
        return float(np.median(dist[yi, xi]))

```

```
    base = {k: float(seed[k]) for k in ("cx", "cy", "Rx", "Ry")}
    best_sc = median_dist(**base)
    best = dict(base)
    for dcx in np.linspace(-10, 10, 21):
        for dcy in np.linspace(-10, 10, 21):
            for sRx in (0.98, 0.99, 1.0, 1.01, 1.02):
                for sRy in (0.98, 0.99, 1.0, 1.01, 1.02):
                    p = dict(
                        cx=base["cx"] + dcx,
                        cy=base["cy"] + dcy,
                        Rx=base["Rx"] * sRx,
                        Ry=base["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
    b = best
    for dcx in np.linspace(-2, 2, 17):
        for dcy in np.linspace(-2, 2, 17):
            for sRx in np.linspace(0.995, 1.005, 11):
                for sRy in np.linspace(0.995, 1.005, 11):
                    p = dict(
                        cx=b["cx"] + dcx,
                        cy=b["cy"] + dcy,
                        Rx=b["Rx"] * sRx,
                        Ry=b["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
    best["ceq_median_px"] = float(best_sc)
    return best


def choose_source():
    for p in SOURCE_CANDIDATES:
        if p.is_file():
            return p
    raise FileNotFoundError("No Migkas Fig.8 theta=75 source image found")


def sample_rgb(rgb, ell, b, cx, cy, Rx, Ry, r=4):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    xi, yi = int(round(as_xy(px, py)[0])), int(round(as_xy(px, py)[1]))
    patch = rgb[max(0, yi - r) : yi + r + 1, max(0, xi - r) : xi + r + 1]
    return patch.mean(axis=(0, 1))


def validate_landmarks(rgb, reg):
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    pts = {}
    landmarks = [
        ("center", 0.0, 0.0),
        ("ell90", 90.0, 0.0),
        ("ell270", 270.0, 0.0),
        ("north", 0.0, 90.0),
        ("south", 0.0, -90.0),
```

```
    ]
    for i, (ell, b) in enumerate(MIGKAS_MAXES):
        landmarks.append((f"migkas_max{i}", ell, b))
    for i, (ell, b) in enumerate(MIGKAS_MINS):
        landmarks.append((f"migkas_min{i}", ell, b))
    for name, ell, b in landmarks:
        x, y = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        pts[name] = {"ell": ell, "b": b, "px": x, "py": y}

    assert pts["ell90"]["px"] < pts["center"]["px"], "l=90 must be LEFT of center"
    assert pts["ell270"]["px"] > pts["center"]["px"], "l=270 must be RIGHT of center"
    assert abs(pts["center"]["px"] - cx) < 1.5
    assert abs(pts["center"]["py"] - cy) < 1.5
    assert pts["north"]["py"] < pts["center"]["py"]
    assert pts["south"]["py"] > pts["center"]["py"]

    # Map-extrema pair: (17,-9) should sit in a warmer patch than (274,-22).
    cmax = sample_rgb(rgb, *MIGKAS_MAXES[0], cx, cy, Rx, Ry)
    cmin = sample_rgb(rgb, *MIGKAS_MINS[1], cx, cy, Rx, Ry)
    assert (cmax[0] + cmax[1]) > (cmin[0] + cmin[1]) + 50
    return {
        "landmarks_px": pts,
        "migkas_max0_rgb_mean": [float(x) for x in cmax],
        "migkas_min1_rgb_mean": [float(x) for x in cmin],
    }


def inside_ellipse(px, py, ecx, ecy, eRx, eRy, r_lim=1.0):
    """Boolean mask: points inside axis-aligned ellipse (image y down)."""
    px = np.asarray(px, dtype=float)
    py = np.asarray(py, dtype=float)
    Xn = (px - ecx) / eRx
    Yn = (py - ecy) / eRy
    return Xn * Xn + Yn * Yn <= float(r_lim) * float(r_lim)


def plot_lb_curve(
    ax, ell, b, cx, cy, Rx, Ry, inset=1.0, jump_px=45.0,
    clip_r2=None, clip_path=None, color_oval=None, **kw
):
    """Draw curve in oval; inset<1 keeps strokes inside the published Mollweide edge."""
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    px = np.asarray(px, dtype=float).reshape(-1)
    py = np.asarray(py, dtype=float).reshape(-1)
    # Pull points slightly toward oval center so thick strokes do not spill past the rim.
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    Xn = (px - cx) / Rx
    Yn = (cy - py) / Ry
    r2_lim = (inset * inset) if clip_r2 is None else float(clip_r2)
    inside = Xn * Xn + Yn * Yn <= r2_lim
    # Also require published colored oval (seed) so registration oversize cannot bleed.
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
        inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
    px = np.where(inside, px, np.nan)
    py = np.where(inside, py, np.nan)

    segs = []
```

```
    cur_x, cur_y = [], []
    for x, y in zip(px, py):
        if not (np.isfinite(x) and np.isfinite(y)):
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [], []
            continue
        if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [float(x)], [float(y)]
        else:
            cur_x.append(float(x))
            cur_y.append(float(y))
    if len(cur_x) >= 2:
        segs.append((cur_x, cur_y))
    if clip_path is not None:
        kw["clip_path"] = clip_path
        kw["clip_on"] = True
    for sx, sy in segs:
        (line,) = ax.plot(sx, sy, **kw)
        if clip_path is not None:
            line.set_clip_path(clip_path)
            line.set_clip_on(True)


def plot_closed_ring(
    ax, mu, cx, cy, Rx, Ry, color, lw, inset=1.0, jump_px=80.0,
    clip_path=None, color_oval=None, ls="-", wash_alpha=None,
):
    """
    Exact projected small-circle about D_HAT=+x: n·x̂ = μ.

    Parameterize on the sphere, convert to Galactic (ℓ,b), apply
    lon_disp = -wrap(ℓ), Mollweide-project, and stroke continuous polylines.
    Split wherever display-longitude jumps across the antimeridian (or pixel
    distance jumps). Never invent interior chords, ellipse fits, or decorative
    oval-rim bridges.

    Notes on θ labels used elsewhere:
      μ=cos(30°)  → r=30° about +x (closed oval near map center)
      μ=cos(185°)=cos(175°) → r=175° about +x ≡ 5° about CoE
        → small arcs on BOTH left and right map edges (not a rim loop)
      μ=cos(115°) → r=115° about +x (large seam-crossing curve)
      μ=cos(265°)=cos(95°) → r=95° about +x (near-meridian, seam-crossing)
    """
    pts = small_circle_about_axis(D_HAT, mu, n=4001)
    # Keep sampling order around the axis (azimuth) — stable for splitting.
    phi = np.arctan2(pts[:, 2], pts[:, 1])
    order = np.argsort(phi)
    pts = pts[order]

    ell = np.empty(len(pts))
    b = np.empty(len(pts))
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)

    lon = np.deg2rad(lon_disp_from_ell(ell))  # λ ∈ (-π, π]
    lat = np.deg2rad(b)
```

```
498
499     # Split on antimeridian jumps in display longitude BEFORE pixel projection.
500     dlon = np.diff(lon)
501     cuts = np.where(np.abs(dlon) > np.pi)[0] + 1
502     # Also close the periodic sample: check wrap between last and first.
503     # Build open segments from azimuth-ordered points.
504     idx = np.concatenate([[0], cuts, [len(lon)]])
505     lon_segs = []
506     for i0, i1 in zip(idx[:-1], idx[1:]):
507         if i1 - i0 >= 2:
508             lon_segs.append((lon[i0:i1].copy(), lat[i0:i1].copy()))
509
510     # Circle sampling is cyclic. Merge the artificial last→first break when
511     # that periodic join is not itself an antimeridian crossing. Otherwise a
512     # seam-crossing circle falsely appears as three pieces with a tiny hole.
513     if len(lon_segs) > 1 and abs(lon[0] - lon[-1]) <= np.pi:
514         first_lo, first_la = lon_segs[0]
515         last_lo, last_la = lon_segs[-1]
516         lon_segs = [(
517             np.concatenate([last_lo, first_lo]),
518             np.concatenate([last_la, first_la]),
519         )] + lon_segs[1:-1]
520
521     # Only now pin TRUE seam endpoints to ±π. Never snap the arbitrary
522     # parameterization start/end to the boundary.
523     snapped = []
524     for lo, la in lon_segs:
525         if abs(abs(lo[0]) - np.pi) < np.deg2rad(10):
526             lo[0] = np.sign(lo[0] if lo[0] != 0 else 1.0) * np.pi
527         if abs(abs(lo[-1]) - np.pi) < np.deg2rad(10):
528             lo[-1] = np.sign(lo[-1] if lo[-1] != 0 else 1.0) * np.pi
529         snapped.append((lo, la))
530     lon_segs = snapped
531
532     def stroke_xy(xs, ys):
533         if len(xs) < 2:
534             return
535         if wash_alpha is not None:
536             # Soft μ-wash stroke: no white underlay; translucent color only.
537             layers = ((color, lw, 2, float(wash_alpha)),)
538         else:
539             # Clean mid-stroke: optional thin white underlay only (lw+1.5), then color.
540             # No black outer rim, no thick white halo stack.
541             layers = (
542                 ("white", lw + 1.5, 3, 0.95),
543                 (color, lw, 4, 1.0),
544             )
545         for col, width, z, a in layers:
546             (line,) = ax.plot(
547                 xs, ys, color=col, lw=width, linestyle=ls,
548                 solid_capstyle="round", solid_joinstyle="round",
549                 zorder=z, alpha=a,
550                 clip_path=clip_path, clip_on=True,
551             )
552             if clip_path is not None:
553                 line.set_clip_path(clip_path)
554                 line.set_clip_on(True)
555
556     r2_lim = inset * inset + 1e-10
```

```
    drawn = 0
    for lo, la in lon_segs:
        # Mollweide in normalized oval coords, then pixels.
        x_m, y_m = mollweide_xy(lo, la)
        Xn = x_m / (2.0 * np.sqrt(2.0))
        Yn = y_m / np.sqrt(2.0)
        px = cx + Xn * Rx
        py = cy - Yn * Ry
        # Mild inset so thick strokes stay inside published oval (no rim-snap).
        px = cx + (px - cx) * inset
        py = cy + (py - cy) * inset
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        inside = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)

        # Pixel-space split on NaN / large jumps / outside oval.
        cur_x, cur_y = [], []
        for x, y, ok in zip(px, py, inside):
            if (not ok) or (not np.isfinite(x)) or (not np.isfinite(y)):
                if len(cur_x) >= 2:
                    stroke_xy(cur_x, cur_y)
                    drawn += 1
                cur_x, cur_y = [], []
                continue
            if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
                if len(cur_x) >= 2:
                    stroke_xy(cur_x, cur_y)
                    drawn += 1
                cur_x, cur_y = [float(x)], [float(y)]
            else:
                cur_x.append(float(x))
                cur_y.append(float(y))
        if len(cur_x) >= 2:
            # Close only if this single segment is already nearly a loop
            # (non-wrapping ring such as MAX 30°).
            if (
                drawn == 0
                and len(lon_segs) == 1
                and np.hypot(cur_x[0] - cur_x[-1], cur_y[0] - cur_y[-1]) < jump_px
            ):
                cur_x.append(cur_x[0])
                cur_y.append(cur_y[0])
            stroke_xy(cur_x, cur_y)
            drawn += 1


def _ring_px_arrays(mu, cx, cy, Rx, Ry, inset, n_pts=721):
    """Small-circle n·D=μ → pixel (px, py) arrays, azimuth-ordered, with inset."""
    pts = small_circle_about_axis(D_HAT, float(mu), n=n_pts)
    phi = np.arctan2(pts[:, 2], pts[:, 1])
    order = np.argsort(phi)
    pts = pts[order]
    ell = np.empty(n_pts)
    b = np.empty(n_pts)
    for i, p in enumerate(pts):
```

```
        ell[i], b[i] = n_to_lb(p)
    lon = np.deg2rad(lon_disp_from_ell(ell))
    lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    Xn = x_m / (2.0 * np.sqrt(2.0))
    Yn = y_m / np.sqrt(2.0)
    px = cx + Xn * Rx
    py = cy - Yn * Ry
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    return lon, px, py


def fill_highlight_band(
    ax, theta_lo, theta_hi, theta_mid, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    """
    One highlight band matching three-panel Panel B:
      continuous transparent polygon strips between ±BAND_HALF edges
      (broken only at Mollweide antimeridian / large pixel jumps — NOT a dense
      multi-stroke wash) + ONE solid mid-stroke at θ_mid.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, px_a, py_a = _ring_px_arrays(mu_a, cx, cy, Rx, Ry, inset, n_pts=n_pts)
    lon_b, px_b, py_b = _ring_px_arrays(mu_b, cx, cy, Rx, Ry, inset, n_pts=n_pts)

    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        ok = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    gap = 0.9 * np.pi
    bad = np.zeros(n_pts - 1, dtype=bool)
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
        elif np.hypot(px_a[i + 1] - px_a[i], py_a[i + 1] - py_a[i]) > jump_px:
            bad[i] = True
        elif np.hypot(px_b[i + 1] - px_b[i], py_b[i + 1] - py_b[i]) > jump_px:
            bad[i] = True
        elif (not _inside(px_a[i], py_a[i])) or (not _inside(px_b[i], py_b[i])):
            bad[i] = True
        elif (not _inside(px_a[i + 1], py_a[i + 1])) or (not _inside(px_b[i + 1], py_b[i + 1])):
            bad[i] = True

```

```
    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        i0, i1 = i, j
        if i1 - i0 >= 1:
            xs = np.concatenate([px_a[i0:i1 + 1], px_b[i0:i1 + 1][::-1]])
            ys = np.concatenate([py_a[i0:i1 + 1], py_b[i0:i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1

    if polys:
        # Soft transparent highlight (clean pre-halo look).
        coll = PolyCollection(
            polys, facecolors=color, edgecolors="none",
            alpha=0.30, zorder=2, linewidths=0,
        )
        ax.add_collection(coll)
        if clip_path is not None:
            coll.set_clip_path(clip_path)
            coll.set_clip_on(True)

    # ONE solid mid-stroke only (thin white underlay lw+1.5). No boundary-edge
    # strokes — dual edges with halos multiply visual lines into "extra loops".
    plot_closed_ring(
        ax, mu_mid, cx, cy, Rx, Ry, color, lw=2.8,
        inset=inset, clip_path=clip_path, color_oval=color_oval,
    )



# --- Opacity gradient (same rule as make_coe_rings_migkas_pin_axis_matched_grad*.py):
# alpha *= (1+cos psi)/2, psi = ring azimuth from the θ-ray point (ring point most
# along sign(sin θ)·Ŷ, Ŷ = Galactic +z projected ⊥ D), rolled by ROLL (Y' = cY − sZ).
from matplotlib.collections import LineCollection
from matplotlib.colors import to_rgba


def _ring_basis_roll(D, roll_deg):
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([0.0, 1.0, 0.0])
    y = tmp - np.dot(tmp, a) * a
    y /= np.linalg.norm(y)
    z = np.cross(a, y)
    z /= np.linalg.norm(z)
    r = np.deg2rad(float(roll_deg))
    c, s_ = np.cos(r), np.sin(r)
    return c * y - s_ * z, s_ * y + c * z


def _grad_ring_px(mu, theta_mid, cx, cy, Rx, Ry, inset, n):
```

```
    a = D_HAT / np.linalg.norm(D_HAT)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    rho = np.sqrt(1.0 - mu * mu)
    y, z = _ring_basis_roll(a, ROLL)
    phi0 = 0.5 * np.pi if np.sin(np.deg2rad(theta_mid)) >= 0 else 1.5 * np.pi
    phi = phi0 + np.linspace(-np.pi, np.pi, n)
    pts = (mu * a)[None, :] + rho * (np.outer(np.sin(phi), y) + np.outer(np.cos(phi), z))
    ell = np.array([n_to_lb(p)[0] for p in pts]); b = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ell)); lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    px = cx + x_m / (2.0 * np.sqrt(2.0)) * Rx
    py = cy - y_m / np.sqrt(2.0) * Ry
    px = cx + (px - cx) * inset; py = cy + (py - cy) * inset
    return lon, px, py, 0.5 * (1.0 + np.cos(phi - phi0))


def _rgba(color, alphas):
    r, g, b_, _ = to_rgba(color)
    al = np.clip(np.asarray(alphas, dtype=float), 0.0, 1.0)
    out = np.empty((al.size, 4)); out[:, 0], out[:, 1], out[:, 2], out[:, 3] = r, g, b_, al
    return out


def _runs(seg_sc, valid, n_levels=64):
    lev = np.round(np.clip(seg_sc, 0.0, 1.0) * n_levels) / n_levels
    ok = (lev > 0) & valid
    runs, i, n = [], 0, lev.size
    while i < n:
        if not ok[i]:
            i += 1; continue
        j = i + 1
        while j < n and ok[j] and lev[j] == lev[i]:
            j += 1
        runs.append((i, j, float(lev[i]))); i = j
    return runs


def fill_highlight_band_grad(
    ax, theta_lo, theta_hi, theta_mid, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    """Same band (alpha 0.30) + mid-stroke (white lw+1.5 @0.95 under colour lw 2.8)
    as fill_highlight_band, but with the (1+cos psi)/2 opacity gradient."""
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        ok = ((px - cx) / Rx) ** 2 + ((cy - py) / Ry) ** 2 <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    def _clip(coll):
        if clip_path is not None:
            coll.set_clip_path(clip_path); coll.set_clip_on(True)
        ax.add_collection(coll)

```

```
    gap = 0.9 * np.pi
    lon_a, pxa, pya, sc = _grad_ring_px(mu_a, theta_mid, cx, cy, Rx, Ry, inset, n_pts)
    lon_b, pxb, pyb, _ = _grad_ring_px(mu_b, theta_mid, cx, cy, Rx, Ry, inset, n_pts)
    qa = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    ins_a, ins_b = _inside(pxa, pya), _inside(pxb, pyb)
    valid = ((qa.max(0) - qa.min(0)) <= gap)
    valid &= np.hypot(np.diff(pxa), np.diff(pya)) <= jump_px
    valid &= np.hypot(np.diff(pxb), np.diff(pyb)) <= jump_px
    valid &= ins_a[:-1] & ins_a[1:] & ins_b[:-1] & ins_b[1:]
    runs = _runs(0.5 * (sc[:-1] + sc[1:]), valid)
    if runs:
        polys = [np.column_stack([np.concatenate([pxa[i0:i1 + 1], pxb[i0:i1 + 1][::-1]]),
                                  np.concatenate([pya[i0:i1 + 1], pyb[i0:i1 + 1][::-1]])])
                 for i0, i1, _ in runs]
        _clip(PolyCollection(polys, facecolors=_rgba(color, [0.30 * r[2] for r in runs]),
                             edgecolors="none", linewidths=0, zorder=2))
    lon_m, pxm, pym, scm = _grad_ring_px(mu_mid, theta_mid, cx, cy, Rx, Ry, inset, 4001)
    ins = _inside(pxm, pym)
    valid = (np.abs(np.diff(lon_m)) <= np.pi) & (np.hypot(np.diff(pxm), np.diff(pym)) <= jump_px)
    valid &= ins[:-1] & ins[1:]
    runs = _runs(0.5 * (scm[:-1] + scm[1:]), valid)
    if runs:
        segs = [np.column_stack([pxm[i0:i1 + 1], pym[i0:i1 + 1]]) for i0, i1, _ in runs]
        _clip(LineCollection(segs, colors=_rgba("white", [0.95 * r[2] for r in runs]),
                             linewidths=2.8 + 1.5, zorder=3, capstyle="butt", joinstyle="round"))
        _clip(LineCollection(segs, colors=_rgba(color, [1.0 * r[2] for r in runs]),
                             linewidths=2.8, zorder=4, capstyle="butt", joinstyle="round"))


def draw_overlay(rgb, reg, seed, source):

    h, w, _ = rgb.shape
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    # Published colored Mollweide (seed detect) — used to keep overlays inside
    # the Migkas map even when registration ellipse is slightly larger.
    scx = float(seed["cx"])
    scy = float(seed["cy"])
    sRx = float(seed["Rx"])
    sRy = float(seed["Ry"])

    dpi = 120
    _extra_h = 1.35 if SHOW_LEGEND else 0.35  # tighten when legend omitted
    fig, ax = plt.subplots(figsize=(w / dpi, (h / dpi) + _extra_h), dpi=dpi)
    ax.imshow(rgb, origin="upper", extent=(0, w, h, 0), interpolation="nearest")
    ax.set_xlim(0, w)
    ax.set_ylim(h, 0)
    ax.set_aspect("equal")
    ax.axis("off")

    # clip_path = registered oval (cx,cy,Rx,Ry). Registration itself is unchanged.
    # Because the refined ellipse can sit a bit outside the published colored
    # map, we ALSO inset strokes and cull/inset markers against the seed oval.
    clip_oval = Ellipse(
        (cx, cy), width=2.0 * Rx, height=2.0 * Ry,
        facecolor="none", edgecolor="none", transform=ax.transData,
    )
    ax.add_patch(clip_oval)

    # Stroke inset so thick white+color underlays do not AA-bleed past the rim.
```

```
    STROKE_INSET = 0.97
    # Cull points outside this fraction of the published colored seed oval.
    COLOR_R = 0.978
    color_oval = (scx, scy, sRx, sRy, COLOR_R)

    def stroke_curve(ell, b, color, lw, ls="-"):
        # White underlay then color — stands out on magma without looking oversized.
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
            color_oval=color_oval,
            color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
            solid_capstyle="round", zorder=3,
        )
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
            color_oval=color_oval,
            color=color, lw=lw, ls=ls, alpha=1.0,
            solid_capstyle="round", zorder=4,
        )

    def mark_point(ell, b, marker, color, ms):
        """Place marker; pull limb symbols inward so halo stays in colored oval."""
        px, py = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        # Approx marker+halo radius in pixels (ms is points).
        halo_ms = ms + 1.8
        ms_px = 0.5 * halo_ms * (dpi / 72.0)
        # Normalized margin inside the published colored oval.
        margin = ms_px / min(sRx, sRy) + 0.012
        r_lim = max(0.55, COLOR_R - margin)

        Xn = (px - scx) / sRx
        Yn = (py - scy) / sRy
        r_col = float(np.hypot(Xn, Yn))

        if r_col > 1e-9:
            # Scale toward colored-oval center so the whole symbol fits.
            if r_col > r_lim:
                scale = r_lim / r_col
                px = scx + (px - scx) * scale
                py = scy + (py - scy) * scale
                Xn = (px - scx) / sRx
                Yn = (py - scy) / sRy
                r_col = float(np.hypot(Xn, Yn))
            # Mild extra inset for everyone (matches stroke inset spirit).
            mild = min(STROKE_INSET, 0.995)
            px = scx + (px - scx) * mild
            py = scy + (py - scy) * mild
            Xn = (px - scx) / sRx
            Yn = (py - scy) / sRy
            r_col = float(np.hypot(Xn, Yn))

        # Skip if still too close to the rim for this marker size.
        if r_col + margin > COLOR_R + 1e-6:
            return

        def _mark(m_size, m_color, m_edge, m_ew, z):
            (line,) = ax.plot(
                px, py, marker=marker, ms=m_size, color=m_color,
                markeredgecolor=m_edge, markeredgewidth=m_ew, linestyle="None",
```

```
                zorder=z, clip_path=clip_oval, clip_on=True,
            )
            line.set_clip_path(clip_oval)
            line.set_clip_on(True)

        _mark(ms + 1.8, "white", "white", 0, 6)
        _mark(ms, color, "k", 0.55, 7)

    # CoE axis at geometric inset=1.0. Do NOT overlay Dec=0 — Migkas already
    # printed the celestial equator on Fig. 8; a second green dashed stroke fights it.
    def stroke_curve_geom(ell, b, color, lw, ls="-"):
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
            color_oval=color_oval,
            color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
            solid_capstyle="round", zorder=3,
        )
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
            color_oval=color_oval,
            color=color, lw=lw, ls=ls, alpha=1.0,
            solid_capstyle="round", zorder=4,
        )

    # CoE↔anti-CoE axis = great circle through ±D (tilted +30°; not Galactic equator)
    _ax_ell, _ax_b = great_circle_through_poles(D_HAT, n=721)
    stroke_curve_geom(_ax_ell, _ax_b, C_AXIS, lw=1.15)

    # Exactly four highlight bands (match three-panel Panel B): each ±9°
    # (= 18° end-to-end = 5% of 360°) + one solid mid-stroke. No companion.
    for (lo, hi), mid, color in zip(MAX_BANDS, THETA_MAX, COLORS_MAX):
        (fill_highlight_band_grad if GRAD else fill_highlight_band)(
            ax, lo, hi, mid, cx, cy, Rx, Ry, color,
            inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
        )
    for (lo, hi), mid, color in zip(MIN_BANDS, THETA_MIN, COLORS_MIN):
        (fill_highlight_band_grad if GRAD else fill_highlight_band)(
            ax, lo, hi, mid, cx, cy, Rx, Ry, color,
            inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
        )

    # Markers (limb ones get stronger inset inside mark_point); per-loop colors
    mark_point(ANTI_COE_LB[0], ANTI_COE_LB[1], "o", C_ANTI, ms=5.5)
    mark_point(COE_LB[0], COE_LB[1], "*", C_COE, ms=7.5)
    for (ell, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        mark_point(ell, b, "^", color, ms=6.5)
    for (ell, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        mark_point(ell, b, "s", color, ms=6.0)

    handles = [
        Patch(
            facecolor=C_MAX_30, alpha=0.55, edgecolor=C_MAX_30,
            label=(
                rf"MAX $\theta\!=\!{THETA_MAX[0]:g}^\circ$ band "
                rf"[{MAX_BANDS[0][0]:g},{MAX_BANDS[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=$\pm9^\circ$=5% of $360^\circ$)"
            ),
        ),
        Patch(
```

```
            facecolor=C_MAX_185, alpha=0.55, edgecolor=C_MAX_185,
            label=(
                rf"MAX $\theta\!=\!{THETA_MAX[1]:g}^\circ$ band "
                rf"[{MAX_BANDS[1][0]:g},{MAX_BANDS[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=$\pm9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.55, edgecolor=C_MIN_115,
            label=(
                rf"MIN $\theta\!=\!{THETA_MIN[0]:g}^\circ$ band "
                rf"[{MIN_BANDS[0][0]:g},{MIN_BANDS[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=$\pm9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.55, edgecolor=C_MIN_265,
            label=(
                rf"MIN $\theta\!=\!{THETA_MIN[1]:g}^\circ$ band "
                rf"[{MIN_BANDS[1][0]:g},{MIN_BANDS[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=$\pm9^\circ$)"
            ),
        ),
        Line2D([0], [0], color=C_AXIS, lw=2.0, label=r"CoE axis tilted $+15^\circ$"),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D([0], [0], marker="o", color=C_ANTI, markeredgecolor="k",
               linestyle="None", ms=6,
               label=r"anti-CoE $+x$ $(0^\circ,+15^\circ)$"),
        Line2D([0], [0], marker="*", color=C_COE, markeredgecolor="k",
               linestyle="None", ms=9,
               label=r"CoE $(180^\circ,-15^\circ)$"),
    ]
    if SHOW_LEGEND:
        leg = ax.legend(
            handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.012),
            ncol=4, fontsize=7.5, frameon=True, fancybox=False,
            edgecolor="#222222", framealpha=1.0, facecolor="white",
            labelcolor="#111111", borderpad=0.6, handlelength=2.0,
        )
        leg.get_frame().set_linewidth(1.2)
```

```
    fig.suptitle(
        r"Migkas Fig. 8 $\theta=75^\circ$ $A/A_{\rm all}$"
        r" + four CoE highlight bands, each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
        "\n"
        r"(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at $(0^\circ,+15^\circ)$; "
        r"CoE axis tilted $+15^\circ$; $\ell$ increases LEFT"
        + (r"; gradient: darkest at $\theta$ ray, transparent opposite" if GRAD else "")
        + (rf"; ${ROLL:+g}^\circ$ CCW about CoE axis" if abs(ROLL) > 1e-9 else "")
        + ")",
        fontsize=10.5, y=0.985,
    )
    fig.subplots_adjust(left=0.01, right=0.99, top=0.92, bottom=(0.13 if SHOW_LEGEND else 0.02))
    fig.savefig(PNG_PATH, dpi=dpi, facecolor="white")
    plt.close(fig)
    print(f"wrote {PNG_PATH}")


def main():
    source = choose_source()
    print(f"source: {source}")
    rgb = np.asarray(Image.open(source).convert("RGB"))
    seed = detect_oval_bbox(rgb)
    print("oval seed:", {k: seed[k] for k in ("cx", "cy", "Rx", "Ry", "x0", "x1", "y0", "y1")})
    # Restore previous visually-good oval (Chris preferred). No medial-axis Rx stretch.
    reg = dict(GOOD_REG)
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)
    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)
    px = reg["cx"] + Xn * reg["Rx"]
    py = reg["cy"] - Yn * reg["Ry"]
    m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
    xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
    yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
    reg["ceq_median_px"] = float(np.median(dist[yi, xi]))
    reg["registration_source"] = "hard-set GOOD_REG (pre medial-axis retune)"
    print("restored reg:", {k: reg[k] for k in ("cx", "cy", "Rx", "Ry", "ceq_median_px")})
    checks = validate_landmarks(rgb, reg)
    print(
        "validation OK — max0 RGB", checks["migkas_max0_rgb_mean"],
        "min1 RGB", checks["migkas_min1_rgb_mean"],
    )
    global PNG_PATH, JSON_PATH, GRAD, ROLL
    for name, g, r in VERSIONS:
        PNG_PATH = SM_DIR / f"fig8_overlay_{name}.png"
        JSON_PATH = SM_DIR / f"fig8_overlay_{name}.json"
        GRAD, ROLL = g, r
        draw_overlay(rgb, reg, seed, source)
        img = Image.open(PNG_PATH).convert("RGB")
        img.resize((1700, round(img.height * 1700 / img.width)), Image.LANCZOS).save(
            SM_DIR / f"fig8_overlay_{name}_web.jpg", quality=88)

    meta = {
        "png": str(PNG_PATH),
```

```
        "script": str(Path(__file__).resolve()),
        "source_image": str(source),
        "convention": (
            "Galactic Mollweide; lon_disp=-wrap(ell); ell increases LEFT "
            "(Migkas Fig.8)"
        ),
        "axis_tilt_b_deg": AXIS_TILT_B_DEG,
        "anti_CoE_lb_deg": [float(ANTI_COE_LB[0]), float(ANTI_COE_LB[1])],
        "CoE_lb_deg": [float(COE_LB[0]), float(COE_LB[1])],
        "D_hat_galactic_xyz": D_HAT.tolist(),
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "migkas_max_lb_deg": [list(p) for p in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(p) for p in MIGKAS_MINS],
        "theta_max_from_plus_x_deg": list(THETA_MAX),
        "theta_min_from_plus_x_deg": list(THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in MAX_BANDS],
        "min_bands_deg": [list(b) for b in MIN_BANDS],
        "band_intervals": {
            "MAX_30": [float(MAX_BANDS[0][0]), float(MAX_BANDS[0][1])],
            "MAX_185": [float(MAX_BANDS[1][0]), float(MAX_BANDS[1][1])],
            "MIN_115": [float(MIN_BANDS[0][0]), float(MIN_BANDS[0][1])],
            "MIN_265": [float(MIN_BANDS[1][0]), float(MIN_BANDS[1][1])],
        },
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "palette": "magma-popping cyan/magenta/lime/sky (no reds/oranges)",
        },
        "mu_max_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MAX_BANDS],
        "mu_min_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MIN_BANDS],
        "scan_stop_source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke; "
            "matches three-panel Panel B; no companion"
        ),
        "matches_three_panel": str(OUT_DIR / "make_coe_rings_migkas_pin_axis.py"),
        "oval_seed_bbox": {
            "x0": seed["x0"], "x1": seed["x1"], "y0": seed["y0"], "y1": seed["y1"],
            "cx": seed["cx"], "cy": seed["cy"], "Rx": seed["Rx"], "Ry": seed["Ry"],
        },
        "registration": {
            "cx": reg["cx"], "cy": reg["cy"], "Rx": reg["Rx"], "Ry": reg["Ry"],
            "ceq_median_px": reg["ceq_median_px"],
            "registration_source": reg.get("registration_source"),
            "ceq_residual_before_registration": dict(GOOD_REG),
            "rejected_medial_axis_reg": {
                "cx": 865.6, "cy": 766.4, "Rx": 706.19, "Ry": 393.70,
                "note": "medial-axis / achromatic DT-ridge retune; blew up Rx; reverted",
            },
            "dofs": ["cx", "cy", "Rx", "Ry"],
            "drawing": {
```

```
                "dec0_drawn": False,
                "dec0_note": "not drawn; use Migkas published celestial equator",
                "coe_axis_inset": 1.0,
                "rings_inset": 0.97,
                "note": "CoE axis geometric inset=1; rings mild inset; no Dec=0 overlay",
            },
            "method": (
                "oval seed detect + hard-set to pre-retune visually-good (cx,cy,Rx,Ry); "
                "medial-axis Rx stretch rejected; Dec=0 NOT overlaid (published CEQ kept); "
                "CoE-axis inset=1.0; rings inset~0.97"
            ),
        },
        "landmark_checks": checks,
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent PolyCollection "
            "band (θ±9°, 18° end-to-end) + ONE solid mid-stroke at center θ. "
            "Fill alpha≈0.30. Mid-stroke: thin white underlay (lw+1.5) then color "
            "(lw=2.8). NO black outer rim; NO thick white halo stacks; NO dual "
            "boundary-edge strokes. Colors: cyan/magenta/lime/sky for magma. "
            "No 283° companion. Dec=0 NOT overlaid."
        ),
        "legend_summary": (
            "four highlight bands, each 18° end-to-end (±9° = 5% of 360%); "
            "MAX cyan+magenta, MIN lime+sky; intervals [21,39]/[176,194]/[106,124]/[256,274]"
        ),
        "notes": [
            "Exactly 4 features matching three-panel Panel B geometry/width",
            "Each = ONE smooth transparent PolyCollection ribbon (±9°) + ONE solid mid-stroke only",
            "Clean draw: fill alpha=0.30; mid-stroke thin white underlay (lw+1.5) + color lw=2.8; no
            ↪  black rim; no boundary-edge halos",
            "No multi-stroke scribble wash; no 283° companion; no Dec=0 overlay",
            "MAX bands [21,39] & [176,194]; MIN bands [106,124] & [256,274]",
            "Colors: MAX_30=#00e5ff, MAX_185=#ff2d95, MIN_115=#b8f200, MIN_265=#4fc3f7",
            "band_half_width_deg=9, band_full_width_deg=18 (5% of 360°)",
            "Exact n·x̂=μ small-circles; seam-split polygon fill; mid via plot_closed_ring",
            "No ellipse-fit, no interior chords, no decorative oval-rim loops",
            "No blocking text box on the map; legend below",
            "Colorbar preserved from source (overlays stay on oval)",
            "Overlays clip_path = registered Ellipse(cx,cy,Rx,Ry); rings stroke inset 0.97",
            "Dec=0 NOT drawn (Migkas Fig.8 already has celestial equator); CoE axis = great circle
            ↪  through ±D (tilted +30°) at inset=1.0",
            "axis_tilt_b_deg=30; anti-CoE (0°,+30°), CoE (180°,−30°); Migkas markers fixed at true sky
            ↪  positions",
            "Registration hard-set to pre medial-axis params (863,763.25,693.9927,392.542425); DT-ridge
            ↪  fit rejected",
            "Limb markers inset toward colored-oval center so halo stays inside",
            "Points culled at COLOR_R=0.978 of seed oval; limb markers inset to fit halo",
        ],
    }
    pass  # per-version JSON not written (PNG only)
    print(f"wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

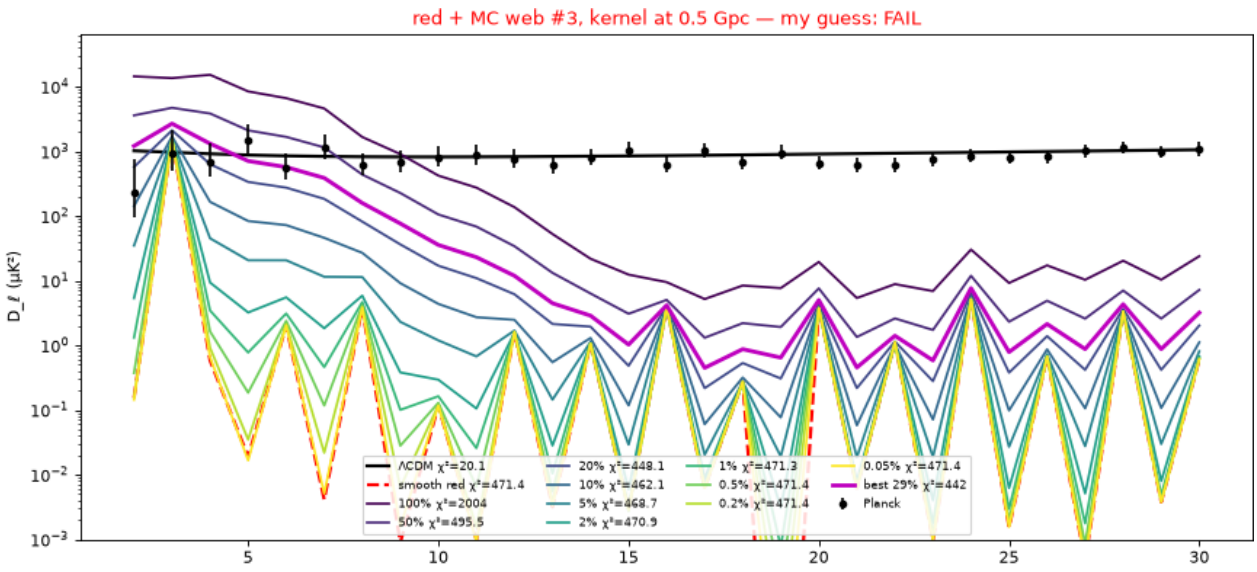
26. Web smoothed ladder (R = 0.5-14 Gpc)

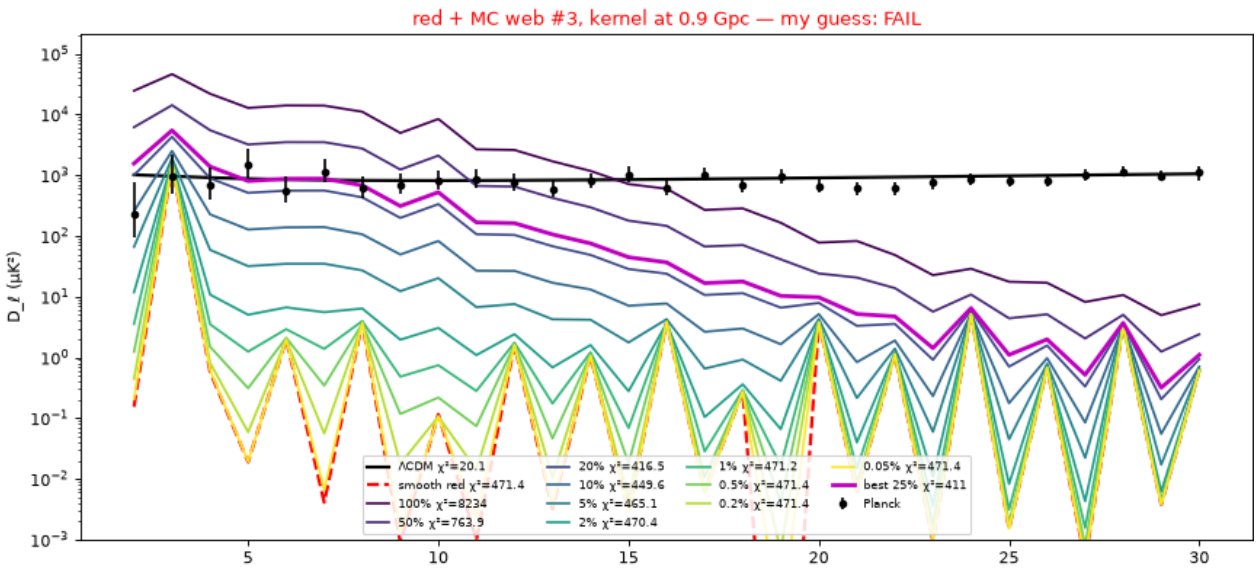
Originally made: Sat Oct 24 2026, 11:29 PM CT (*file creation, coepdf/fig/f19mc3b_stack.png (git commit 6cfb9e6 message mentions an MC web #3 kernel ladder on Oct 23 1:08 AM CT, earlier file not md5-matched)*)
Paper figure number: this morning Fig. 17; now Fig. 17
Files: `f19mc3b_stack.png`
Code status: original computing chain (box) + restack

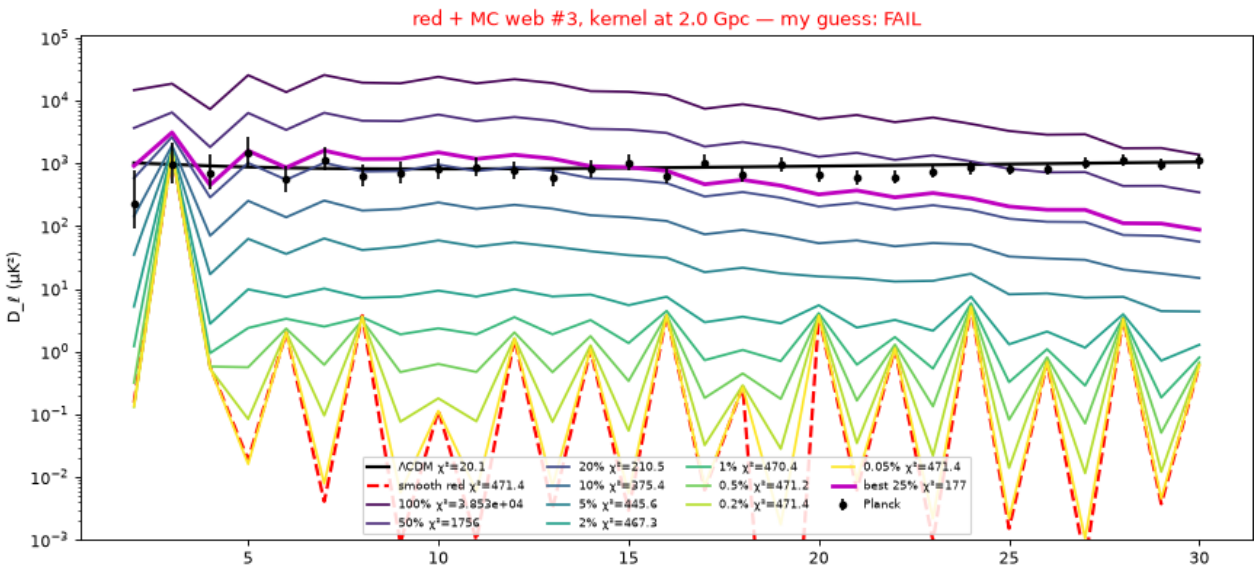
Notes

Red + Monte-Carlo web #3 smoothed with kernels R = 0.5..14 Gpc. Builds on #13 and #1. STATUS NOTE: COMPUTING CHAIN (run order): thin4_run.py writes the red (smooth) model sky sm_red.npy; mc.py (imports build.py, cosmo.py) draws a random Gaussian random field + Poisson halo web with the 42 Gpc density gradient, computes the acceleration and the recombination-light sky smoothed with kernels R = 0.5..14 Gpc, saving mc_sky.npy; mc_bracket.py (uses analyze.py) fits the bracket guesses to samples/bracket_guesses.json; mc_br_plot.py 3 writes samples/mc3_bracket.png. RESTACK (one line): f19mc3b.png (2640x810) is a 3-across rearrangement of the six mc3 bracket panels (880x2400 original; not md5-identical to the current rerun file), and f19mc3b_stack.png restacks f19mc3b.png.

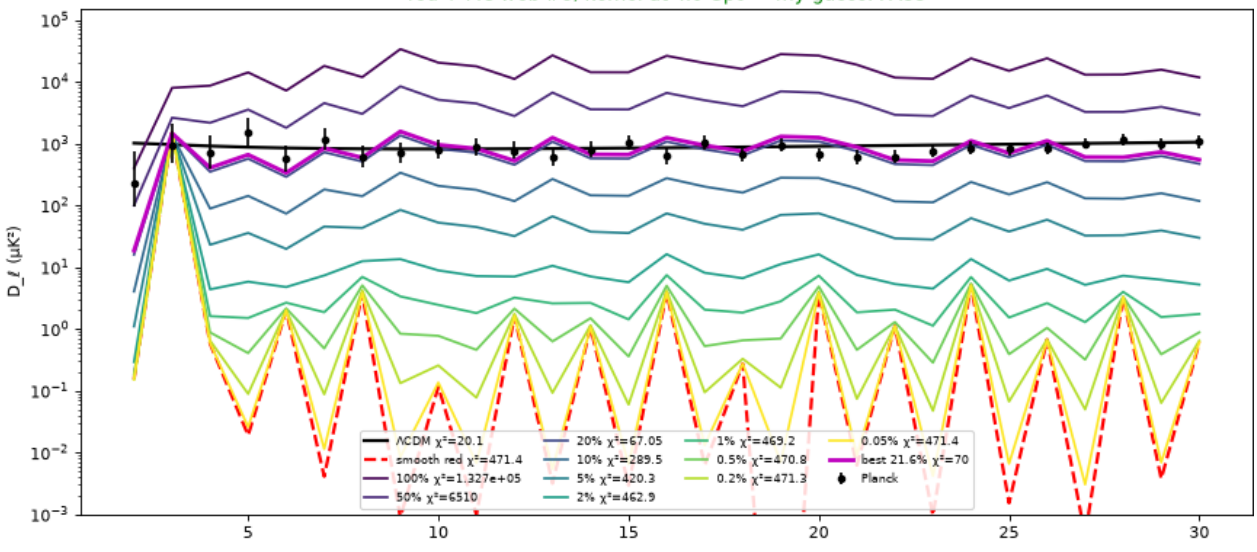
Paper caption (caveats as stated): Red + Monte-Carlo web #3, kernel $R = 0.5$, 0.9, 2.0, 4, 8, 14 Gpc (top to bottom); key in text.



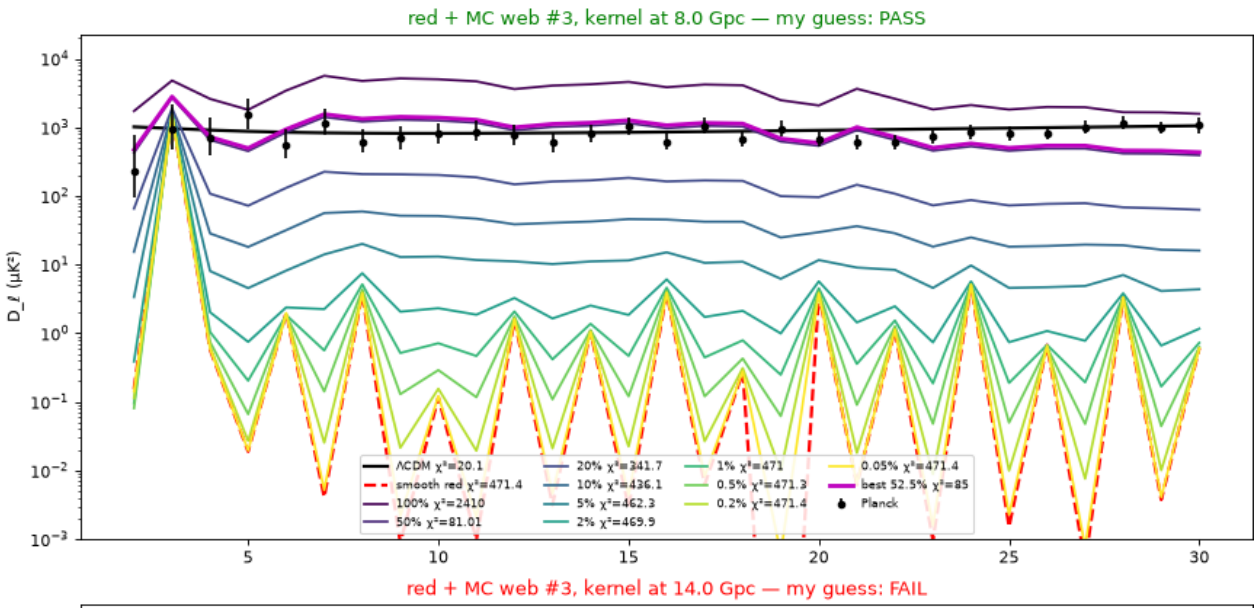




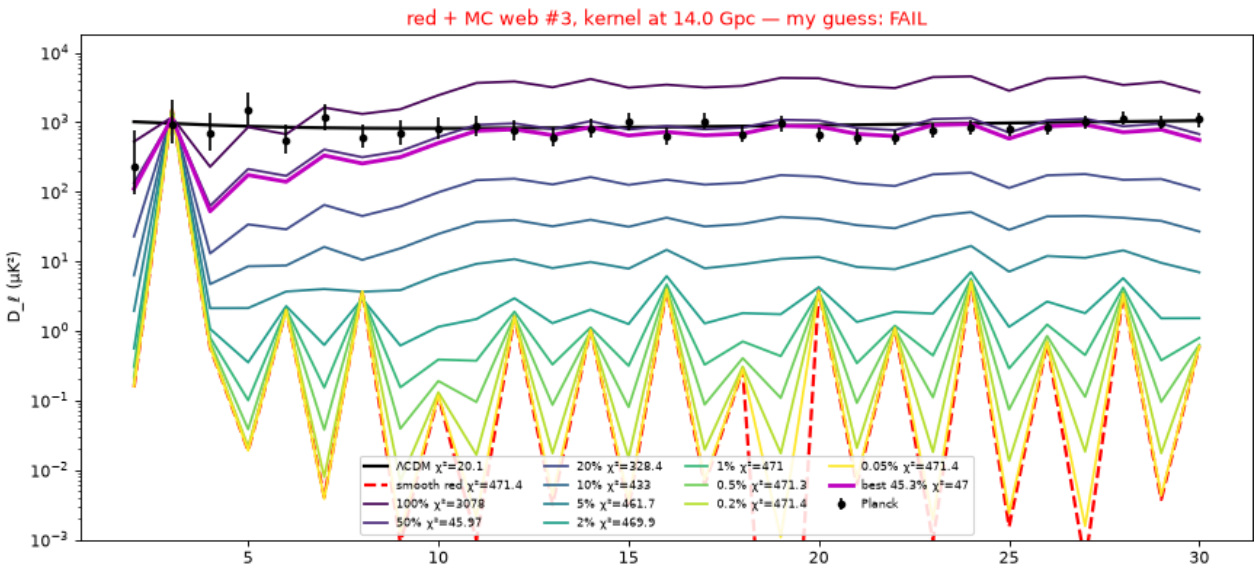
red + MC web #3, kernel at 4.0 Gpc — my guess: PASS



red + MC web #3, kernel at 8.0 Gpc — my guess: PASS



red + MC web #3, kernel at 14.0 Gpc — my guess: FAIL



f19mc3b_stack.png

Computing chain (run order, full listings)

/workspace/redweb2/thin4_run.py

```
import os,sys,json,numpy as np,healpy as hp
from scipy.interpolate import CubicSpline
A='/workspace/coe-cosmology-gh/draft/current/analyses'
g={'__file__':A+'/shell_ensemble/ensemble.py','__name__':'imp'};os.chdir(A+'/shell_ensemble')
src=open(g['__file__']).read();exec(src[:src.index('# ---------- beta=0')],g)
field3,th,KG,RV,FLOOR,phi=[g[k] for k in('field3','th','KG','RV','FLOOR','phi')]
sys.path.insert(0,A+'/void_mass_network');sys.path.insert(0,'/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/rotfit');import vmn_gen as G;MSUN=1.989e30
ELL=np.radians(phi+180);Ch=np.array([np.cos(ELL),np.sin(ELL)]);L_G=922.65
FLOC=-0.9/KG*(RV/L_G)*(np.cos(th)*Ch[0]+np.sin(th)*Ch[1]);FLOC-=FLOC.mean()
web=G.generate(Mmin=3e15,seed=2000,comp_mode='lcdm');X,M=G.kernel_bodies(web)
pp=web['params'];vv=np.linalg.norm(web['S'],axis=1)<pp['R_out'];nv=int(vv.sum());nc=len(web['MC'])
gid=np.concatenate([np.arange(nv),np.arange(nv),nv+np.arange(nc)])  # void id shared by its -D and +W body; clusters own ids
far=np.linalg.norm(np.vstack([web['cen'][vv],web['cen'][vv],web['XC']]),axis=1)>pp['gap_us'];gid=gid[far];assert len(gid)==len(M)
import fit as _f0
CC=[-0.041,-0.035,0.1353,0.0];Rb=1.5*(1+1/2.937);aC=np.array([1,0,np.tan(np.radians(15))]);aC/=np.linalg.norm(aC);PC=42.0*aC
shp=lambda x:np.exp(sum(c*(np.minimum(np.abs(x),Rb)-1)**(k+1) for k,c in enumerate(CC)))
PU=np.vstack([web['cen'][vv],web['XC']])/1000.0;xu=np.linalg.norm(PU-PC,axis=1)/42.0;pu=shp(xu)/shp(xu).max()
ku=np.random.default_rng(777).random(len(pu))<pu;np.savez('/workspace/redweb2/thin4_keep.npz',ku=ku,pu=pu,xu=xu,nv=nv)
dk=ku[gid];X,M=X[dk],M[dk];gid=gid[dk];print('density keep',ku[:nv].mean(),ku[nv:].mean(),pu.min(),pu.max(),flush=True)
XF,MF=X.copy(),M.copy()
P=M@field3(*[np.arctan2((X/RV)[:,1],(X/RV)[:,0]),np.arcsin((X[:,2]/np.linalg.norm(X,axis=1))),np.linalg.norm(X,axis=1)/RV])/KG if False else None
P=np.zeros(360)
for i in range(0,len(M),20000):
  Y=X[i:i+20000]/RV;D=np.linalg.norm(Y,axis=1);P+=M[i:i+20000]@field3(np.arctan2(Y[:,1],Y[:,0]),np.arcsin(Y[:,2]/D),D)/KG
P-=M.sum()*FLOC;ver=dict(n_bodies=len(M),pattern_rms_over_floor=float(np.sqrt(np.mean(P**2))/FLOOR),repo_value=149.87782489337775,repo_n_bodies=272175)
print(ver,flush=True)
# full-sky generalisation of field3: ring point at rho*n (RV units) for every sky direction n
CON='/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/clusters_voids/quad_balance/mass_family_scan/constrained'
B='/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/clusters_voids/reproduce_cmb'
ns=32;V=np.array(hp.pix2vec(ns,np.arange(hp.nside2npix(ns)))).T
def sky(rho):
  Y=X/RV;out=np.zeros(len(V));D3=np.linalg.norm(Y,axis=1)**3
  for i in range(0,len(M),4000):
    y=Y[i:i+4000];d=y[:,None,:]-rho[None,:,None]*V[None];r3=np.linalg.norm(d,axis=2)**3
    ar=np.einsum('bpk,pk->bp',d,V)/r3-(y@V.T)/D3[i:i+4000,None];out+=(M[i:i+4000]/KG)@ar
  return out-out.mean()
# ring check of full-sky kernel in the plane b=0
o=np.load(CON+'/runs_poly/f20000_p_-0.041_-0.035_0.1353_0.npz')['o'];o=o[np.argsort(o[:,0])]
sys.path.insert(0,B+'/../../rotfit');os.chdir(B);import fit
aG=fit.vec(0,15);psi=np.degrees(np.arccos(np.clip(V@aG,-1,1)))
sm=np.exp(-CubicSpline(o[:,0],np.log(o[:,1]))(psi));sm=sm/sm.mean()-1
wf=CubicSpline(o[:,0],o[:,4]);pf=CubicSpline(o[:,0],o[:,5])
t=V-(V@aG)[:,None]*aG;tn=np.linalg.norm(t,axis=1);t=np.where(tn[:,None]>1e-9,t/np.maximum(tn,1e-12)[:,None],0)
w=wf(psi);ph=pf(psi);E=42.0*(-aG[None,:]+w[:,None]*(np.cos(ph)[:,None]*aG[None,:]+np.sin(ph)[:,None]*t));dist=np.linalg.norm(E,axis=1)
T0=2.7255e6;l=np.arange(31)
```

```
    return np.sqrt(2/np.pi)*rho_m/M*(dc/s)*np.abs(ls)*np.exp(-dc**2/(2*s*s)),s
def svdw_dndlnR(Rnl):  # Sheth & van de Weygaert 2004, linear-theory void size function (dv=-2.7), R_nl=1.7 R_L
    RL=Rnl/1.7;s=np.array([sigR(r) for r in RL]);D=abs(dv)/(dc+abs(dv));x=D*s/abs(dv)
    j=np.arange(1,200)[:,None];f=2*np.sum(j*np.pi*x**2*np.sin(j*np.pi*D)*np.exp(-(j*np.pi*x)**2/2),0)
    dls=np.gradient(np.log(1/s),np.log(RL));return f/(4/3*np.pi*RL**3)*dls   # per lnR_L = per lnR_nl
if __name__=='__main__':
    print('A',A,'sig8',sigR(8/h),'sig(100/h)',sigR(100/h),'sig(140/h)',sigR(140/h))
    M=np.geomspace(1e14,2e15,30);n,s=ps_dndlnM(M);print('n(>1e14) Mpc^-3',np.trapezoid(n,np.log(M)),'frac mass',np.trapezoid(n*M,np.log(M))/rho_m)
    R=np.geomspace(10,50,30);nv=svdw_dndlnR(R);print('nvoid Mpc^-3',np.trapezoid(nv,np.log(R)),'vol frac',np.trapezoid(nv*4/3*np.pi*R**3,np.log(R)))
    kk=np.geomspace(1e-4,1,400);print('Pk peak k',kk[np.argmax(P(kk))],'-> scale 2pi/k Mpc',2*np.pi/kk[np.argmax(P(kk))])
```

/workspace/redweb2/realweb/build.py

```
# realistic web: groups of independent Poisson-thinned bodies (halos PS 1e14-2e15 Msun, SvdW compensated voids 10-50 Mpc)
# painted (CIC + Gaussian smoothing at cell scale) on two grids; FFT gravity; tidal sky signal on spheres R
import os,sys,json,time,numpy as np,healpy as hp,scipy.fft as F
from scipy.ndimage import map_coordinates
from cosmo import *
OUT='/workspace/redweb2/realweb/';NG=100;RS=[0.5,0.9,2.0,4.0,8.0,14.0]
MSUN=1.989e30;KG=7.7e51/0.0039;RV=14.3
ns_=32;V=np.array(hp.pix2vec(ns_,np.arange(hp.nside2npix(ns_)))).T
CC=[-0.041,-0.035,0.1353,0.0];Rb=1.5*(1+1/2.937);aC=np.array([1,0,np.tan(np.radians(15))]);aC/=np.linalg.norm(aC);PC=42.0*aC
shp=lambda x:np.exp(sum(c*(np.minimum(np.abs(x),Rb)-1)**(k+1) for k,c in enumerate(CC)))
GR=[dict(name='fine',L=20.0,N=384,rin=0.0,rout=5.0,seed=11),dict(name='coarse',L=60.0,N=384,rin=5.0,rout=15.0,seed=12)]
Mb=np.geomspace(1e14,2e15,41);nM,sM=ps_dndlnM(Mb);bM=1+(dc**2/sM**2-1)/dc   # PS peak-background bias
Rv=np.geomspace(10,50,41);nV=svdw_dndlnR(Rv);BV=-1.5;DV=-0.85
def setup(g):
    N,L=g['N'],g['L'];dx=L/N;x=(np.arange(N)+0.5)*dx-L/2
    k1=2*np.pi*F.fftfreq(N,dx);k3=2*np.pi*F.rfftfreq(N,dx)
    KX,KY,KZ=np.meshgrid(k1,k1,k3,indexing='ij',sparse=True);K2=KX**2+KY**2+KZ**2
    rng=np.random.default_rng(g['seed']);w=rng.standard_normal((N,N,N),dtype=np.float32)
    kM=np.sqrt(K2)/1000.0   # 1/Mpc
    Pk=np.where(K2>0,P(np.maximum(kM,1e-6)),0)/(dx*1000)**3
    d=F.irfftn(F.rfftn(w,workers=8)*np.sqrt(Pk).astype(np.float32)*np.exp(-K2*dx*dx/2),s=w.shape,workers=8).astype(np.float32);del w
    X,Y,Z=np.meshgrid(x,x,x,indexing='ij',sparse=True);r=np.sqrt(X**2+Y**2+Z**2)
    mask=(r>=g['rin'])&(r<g['rout'])
    mod=shp(np.sqrt((X-PC[0])**2+(Y-PC[1])**2+(Z-PC[2])**2)/42).astype(np.float32);mod/=mod[mask].mean()
    return dict(dx=dx,x0=-L/2,KX=KX,KY=KY,KZ=KZ,K2=K2,d=d,mask=mask,mod=mod,N=N,L=L)
def cic(G,P_,m):
    N=G['N'];u=(P_-G['x0'])/G['dx']-0.5;i=np.floor(u).astype(np.int64);f=u-i;out=np.zeros(N**3)
    for a in (0,1):
      for b in (0,1):
        for c in (0,1):
          wgt=m*np.abs(1-a-f[:,0])*np.abs(1-b-f[:,1])*np.abs(1-c-f[:,2])
          ii=((i[:,0]+a)%N)*N*N+((i[:,1]+b)%N)*N+((i[:,2]+c)%N);out+=np.bincount(ii,wgt,N**3)
    return out.reshape(N,N,N)
def bodies(G,rng,frac):
    dV=(G['dx']*1000)**3;mk=G['mask'];d=G['d'][mk];mod=G['mod'][mk];idx=np.flatnonzero(mk)
```

```
    out=[];N=G['N']
    def place(cnt):
        c=np.repeat(idx,cnt);ii=np.stack(np.unravel_index(c,(N,N,N)),1);return G['x0']+(ii+rng.random(ii.shape))*G['dx']
    # halos, by mass bin
    dl=np.diff(np.log(Mb));Hm=[];Hp=[]
    for j in range(len(Mb)-1):
        lam=0.5*(nM[j]+nM[j+1])*dl[j]*dV*np.maximum(0,1+0.5*(bM[j]+bM[j+1])*d)*mod*frac;cnt=rng.poisson(lam)
        Hp.append(place(cnt));Hm.append(np.exp(rng.uniform(np.log(Mb[j]),np.log(Mb[j+1]),cnt.sum())))
    Hp=np.vstack(Hp);Hm=np.concatenate(Hm)
    dl=np.diff(np.log(Rv));Vp=[];Vr=[]
    for j in range(len(Rv)-1):
        lam=0.5*(nV[j]+nV[j+1])*dl[j]*dV*np.maximum(0,1+BV*d)*mod*frac;cnt=rng.poisson(lam)
        Vp.append(place(cnt));Vr.append(np.exp(rng.uniform(np.log(Rv[j]),np.log(Rv[j+1]),cnt.sum())))
    Vp=np.vstack(Vp);Vr=np.concatenate(Vr);Md=-DV*rho_m*4/3*np.pi*Vr**3   # deficit mass (Msun), local mean density
    return Hp,Hm,Vp,Vr,Md
def paint(G,rng,Hp,Hm,Vp,Vr,Md):
    rho=cic(G,Hp,Hm)+cic(G,Vp,-Md)
    for q in range(8):   # compensating wall: 8 random points on shell at 1.25 Rv
        n=rng.normal(size=Vp.shape);n/=np.linalg.norm(n,axis=1)[:,None];rho+=cic(G,Vp+n*(1.25*Vr/1000)[:,None],Md/8)
    rho[~G['mask']]=0;rho[G['mask']]-=rho[G['mask']].mean()   # neutral against uniform background in the region
    return rho/G['dx']**3   # Msun/Gpc^3
def accel(G,rho):
    rk=F.rfftn(rho.astype(np.float32),workers=8)*np.exp(-G['K2']*G['dx']**2/2)   # finite extent: Gaussian at cell scale
    with np.errstate(divide='ignore',invalid='ignore'):ph=np.where(G['K2']>0,-4*np.pi*rk/G['K2'],0)   # G=1, Phi
    return [F.irfftn(-1j*Kc*ph,s=rho.shape,workers=8) for Kc in (G['KX'],G['KY'],G['KZ'])]   # a=-grad Phi, Msun/Gpc^2
def skyfrom(G,a):
    pts=np.vstack([np.zeros((1,3))]+[R*V for R in RS]);u=((pts-G['x0'])/G['dx']-0.5).T
    A=np.stack([map_coordinates(c,u,order=1,mode='wrap') for c in a],1);a0=A[0];out=[]
    for i,R in enumerate(RS):Ai=A[1+i*len(V):1+(i+1)*len(V)];s=((Ai-a0)*V).sum(1);out.append(s-s.mean())
    return np.array(out)*MSUN/KG*RV**2   # same units as sky(): sum (M/KG) n.[d/|d|^3-y/|y|^3], lengths in RV
if __name__=='__main__':
    ck=OUT+'groups.npz';done=dict(np.load(ck)) if os.path.exists(ck) else {}
    Gs=[setup(g) for g in GR];print('setup done',[float(G['d'][G['mask']].std()) for G in Gs],flush=True)
    for G,g in zip(Gs,GR):np.save(OUT+f"delta_{g['name']}_slice.npy",G['d'][:,:,G['N']//2])
    S=done.get('S',np.zeros((0,2,len(RS),len(V))));stats=list(done.get('stats',np.zeros((0,2,4))))
    for k in range(len(S),NG):
        t=time.time();rng=np.random.default_rng(1000+k);row=[];st=[]
        for G,g in zip(Gs,GR):
            B=bodies(G,rng,1/NG);rho=paint(G,rng,*B);row.append(skyfrom(G,accel(G,rho)));st.append([len(B[1]),B[1].sum(),len(B[3]),B[4].sum()])
            if k==0:np.savez(OUT+f"bodies_g0_{g['name']}.npz",Hp=B[0],Hm=B[1],Vp=B[2],Vr=B[3]);np.save(OUT+f"rho_g0_{g['name']}_slice.npy",rho[:,:,G['N']//2])
        S=np.concatenate([S,np.array(row)[None]]);stats.append(st);np.savez(ck,S=S,stats=np.array(stats))
        print(k,'%.0fs'%(time.time()-t),st,flush=True)
```

/workspace/redweb2/realweb/mc/mc.py

```
# Monte Carlo of full realistic universes (natural amplitude, strength 1): new GRF + Poisson halo field per universe.
```

```
# Fast variant of build.py: halos deposited per cell (Poisson counts in 8 mass bins, NGP at cell, Gaussian smoothing at cell scale);
# compensated voids omitted (net ~0 at >=cell scale; checked against the 100-group sweep via web rms). N=256 grids.
import os,sys,time,json,numpy as np,scipy.fft as F
sys.path.insert(0,'/workspace/redweb2/realweb');import build as B
from cosmo import *
OUT='/workspace/redweb2/realweb/mc/';NU=int(sys.argv[1]) if len(sys.argv)>1 else 1000
GR=[dict(name='fine',L=20.0,N=256,rin=0.0,rout=5.0),dict(name='coarse',L=60.0,N=256,rin=5.0,rout=15.0)]
Me=np.geomspace(1e14,2e15,9);Mf=np.geomspace(1e14,2e15,400);nf,sf=ps_dndlnM(Mf);bf=1+(dc**2/sf**2-1)/dc
bins=[];lnM=np.log(Mf)
for j in range(8):
    s=(Mf>=Me[j])&(Mf<=Me[j+1]);n=np.trapezoid(nf[s],lnM[s]);bins.append((n,np.trapezoid(nf[s]*Mf[s],lnM[s])/n,np.trapezoid(nf[s]*bf[s],lnM[s])/n))
def setup(g):
    N,L=g['N'],g['L'];dx=L/N;x=(np.arange(N)+0.5)*dx-L/2;k1=2*np.pi*F.fftfreq(N,dx);k3=2*np.pi*F.rfftfreq(N,dx)
    KX,KY,KZ=np.meshgrid(k1,k1,k3,indexing='ij',sparse=True);K2=KX**2+KY**2+KZ**2
    X,Y,Z=np.meshgrid(x,x,x,indexing='ij',sparse=True);r=np.sqrt(X**2+Y**2+Z**2);mask=(r>=g['rin'])&(r<g['rout'])
    mod=B.shp(np.sqrt((X-B.PC[0])**2+(Y-B.PC[1])**2+(Z-B.PC[2])**2)/42).astype(np.float32);mod/=mod[mask].mean()
    Pk=(np.where(K2>0,P(np.maximum(np.sqrt(K2)/1000,1e-6)),0)/(dx*1000)**3);amp=(np.sqrt(Pk)*np.exp(-K2*dx*dx/2)).astype(np.float32)
    return dict(dx=dx,x0=-L/2,KX=KX,KY=KY,KZ=KZ,K2=K2,mask=mask,mod=mod[mask],amp=amp,N=N)
Gs=[setup(g) for g in GR]
def universe(seed):
    rng=np.random.default_rng(seed);out=[];nh=0
    for G in Gs:
        N=G['N'];w=rng.standard_normal((N,N,N),dtype=np.float32);d=F.irfftn(F.rfftn(w,workers=8)*G['amp'],s=w.shape,workers=8)[G['mask']];del w
        dV=(G['dx']*1000)**3;rho=np.zeros(int(G['mask'].sum()))
        for n,Mm,b in bins:c=rng.poisson(n*dV*np.maximum(0,1+b*d)*G['mod']);rho+=c*Mm;nh+=c.sum()
        rho-=rho.mean();R=np.zeros((N,N,N),np.float32);R[G['mask']]=rho/G['dx']**3
        out.append(B.skyfrom(G,B.accel(G,R)))
    return out[0]+out[1],nh
if __name__=='__main__':
    ck=OUT+'mc_sky.npy';S=list(np.load(ck)) if os.path.exists(ck) else []
    for i in range(len(S),NU):
        t=time.time();s,nh=universe(500000+i);S.append(s.astype(np.float32))
        if i%5==4 or i==NU-1:np.save(ck,np.array(S))
        print(i,'%.1fs'%(time.time()-t),nh,flush=True)
    np.save(ck,np.array(S))
```

/workspace/redweb2/realweb/analyze.py

```
import json,numpy as np,healpy as hp,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
B='/workspace/cosmology-course/notes/accel/gr/rings_cmb/v11/clusters_voids/reproduce_cmb';Pl=np.loadtxt(B+'/planckTT.txt');Pl=Pl[(Pl[:,0]>=2)&(Pl[:,0]<=30)]
T0=2.7255e6;ll=np.arange(31);l=np.arange(2,31)
def Dl(x):return ll*(ll+1)*hp.anafast(np.asarray(hp.remove_dipole(x)),lmax=30)*T0**2/(2*np.pi)
def chi(D):
  c=0
  for r in Pl:
    L=int(r[0]);e=r[3] if D[L]>r[1] else r[2];c+=((D[L]-r[1])/e)**2
  return float(c)
r2=lambda D:float(np.corrcoef(D[2:31],Pl[:,1])[0,1]**2)
```

```
O='/workspace/redweb2/realweb/';z=np.load(O+'groups.npz');S=z['S'].sum(1);NG=len(S);RS=[0.5,0.9,2.0,4.0,8.0,14.0]
sm=np.load('/workspace/redweb2/sm_red.npy');E=[.01,.02,.05,.1,.2,.5,1.0];rng=np.random.default_rng(42)
res=dict(n_groups=NG,smooth=dict(chi2=chi(Dl(sm)),r2=r2(Dl(sm)),Dl=Dl(sm).tolist()),R={})
def ent(w):D=Dl(sm+w);return dict(chi2=chi(D),r2=r2(D),Dl=D.tolist())
for iR,R in enumerate(RS):
  Sg=S[:,iR];tot=Sg.sum(0);d=dict(web_rms=float(tot.std()),smooth_rms=float(sm.std()),a={},b={},c={})
  for e in E:
    k=max(1,int(round(e*NG)));d['b'][str(e)]=ent(e*tot)
    d['a'][str(e)]=[dict(ent(Sg[rng.choice(NG,k,replace=False)].sum(0)*NG/NG),n_groups=k) for _ in range(5)] if e<1 else [ent(tot)]
    cl=[]
    for _ in range(5):
      keep=rng.uniform(e,1);kk=max(1,int(round(keep*NG)));sc=e/(kk/NG);cl.append(dict(ent(sc*Sg[rng.choice(NG,kk,replace=False)].sum(0)),keep=kk/NG,scale=sc))
    d['c'][str(e)]=cl
  res['R'][str(R)]=d;print(R,'rms web/smooth %.3g'%(tot.std()/sm.std()),{m:{e:round(min(x['chi2'] for x in (d[m][e] if isinstance(d[m][e],list) else [d[m][e]])),1) for e in d[m]} for m in 'abc'},flush=True)
json.dump(res,open(O+'realweb_results.json','w'))
# plots
cols=plt.cm.tab10.colors
for R in map(str,RS):
  d=res['R'][R];cur=[('smooth red (no web)',np.array(res['smooth']['Dl']),res['smooth']['chi2'],'k','--')]
  for i,e in enumerate(E):
    cur.append((f'(b) scale {e*100:g}%',np.array(d['b'][str(e)]['Dl']),d['b'][str(e)]['chi2'],cols[i],'-'))
    for m,ls in (('a',':'),('c','-.')):
      L=d[m][str(e)];bb=min(L,key=lambda x:x['chi2']);cur.append((f'({m}) {e*100:g}% best of {len(L)}',np.array(bb['Dl']),bb['chi2'],cols[i],ls))
  for kind in ('log','lin'):
    plt.figure(figsize=(13,8))
    for lab,D,c2,c,ls in cur:
      vis=kind=='log' or np.any((D[2:31]>0)&(D[2:31]<3000))
      plt.plot(l,D[2:31],color=c,ls=ls,lw=1.6 if ls=='-' else 1.2,marker='.' ,label=(f'{lab}: χ²={c2:.4g}, r²={r2(D):.2f}' if vis else None))
    plt.errorbar(Pl[:,0],Pl[:,1],yerr=[Pl[:,2],Pl[:,3]],fmt='o',color='r',ms=4,capsize=2,label='Planck TT')
    if kind=='log':plt.yscale('log');plt.ylim(bottom=1e-3)
    else:plt.ylim(0,3000)
    plt.title(f'Realistic web, sampling R={R} Gpc: (a) random removal, (b) force scale, (c) random mix; ΛCDM χ²=20.1');plt.xlabel('□');plt.ylabel('D_□ (μK²)')
    plt.legend(fontsize=6,ncol=3,loc='lower center',framealpha=0.85);plt.tight_layout();plt.savefig(O+f'realweb_R{R}_{kind}.png',dpi=100);plt.close()
fig,ax=plt.subplots(1,3,figsize=(18,5))
for a,m in zip(ax,'abc'):
  H=np.array([[min(x['chi2'] for x in (res['R'][str(R)][m][str(e)] if isinstance(res['R'][str(R)][m][str(e)],list) else [res['R'][str(R)][m][str(e)]])) for e in E] for R in RS])
  im=a.imshow(np.log10(H),cmap='viridis_r',aspect='auto');a.set_xticks(range(len(E)),[f'{e*100:g}%' for e in E]);a.set_yticks(range(len(RS)),[f'{R}' for R in RS])
  for i in range(len(RS)):
    for j in range(len(E)):a.text(j,i,f'{H[i,j]:.3g}',ha='center',va='center',fontsize=7,color='w' if np.log10(H[i,j])>np.log10(H).mean() else 'k')
  a.set_xlabel('effect size e');a.set_ylabel('sampling R (Gpc)');a.set_title({'a':'(a) random removal, best of 5','b':'(b) force scale','c':'(c) random mix, best of 5'}[m]+' — χ²');plt.colorbar(im,ax=a,label='log10 χ²')
plt.tight_layout();plt.savefig(O+'realweb_chi2_heatmap.png',dpi=110)
```

/workspace/redweb2/realweb/mc/mc_bracket.py

```
import numpy as np,json
exec(open('/workspace/redweb2/realweb/analyze.py').read().split('O=')[0])
S=np.load('mc_sky.npy',mmap_mode='r');sm=np.load('/workspace/redweb2/sm_red.npy');RS=[0.5,0.9,2.0,4.0,8.0,14.0]
l=np.arange(2,31);E=[1,.5,.2,.1,.05,.02,.01,.005,.002,.0005];pm=np.median(Pl[2:,1]);out=[]
for k in range(len(S)):
  r={}
  for iR,R in enumerate(RS):
    sk=np.array(S[k][iR]);st=[]
    for e in E:
      y=Dl(sm+e*sk)[4:31];st.append((np.median(y),abs(np.polyfit(l[2:],np.log(y),1)[0])<0.05))
    ok=any(st[i][1] and st[i+1][1] and st[i][0]>=pm>=st[i+1][0] for i in range(len(E)-1))
    r[R]=bool(ok)
  out.append(r)
  if k%100==0:print(k,flush=True)
json.dump(out,open('samples/bracket_guesses.json','w'))
for R in RS:print('R',R,sum(o[R] for o in out))
print('all 4/8/14',sum(o[4.0] and o[8.0] and o[14.0] for o in out))
print('ex_pass',[k for k,o in enumerate(out) if o[4.0] and o[8.0] and o[14.0]][:5],'ex_fail',[k for k,o in enumerate(out) if not(o[4.0] or o[8.0] or o[14.0])][:5])
```

/workspace/redweb2/realweb/mc/mc_br_plot.py

```
import numpy as np,json,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
exec(open('/workspace/redweb2/realweb/analyze.py').read().split('O=')[0])
S=np.load('mc_sky.npy',mmap_mode='r');sm=np.load('/workspace/redweb2/sm_red.npy');RS=[0.5,0.9,2.0,4.0,8.0,14.0]
t=np.loadtxt('/workspace/joint/lcdm.txt');l=np.arange(2,31);Lt=np.interp(l,t[:,0],t[:,1])
E=[1,.5,.2,.1,.05,.02,.01,.005,.002,.0005];G=np.logspace(-3.5,0.3,60)
ks=[0,int(__import__('sys').argv[1])];out={};B=json.load(open('samples/bracket_guesses.json'))
for k in ks:
  k=int(k);fig,ax=plt.subplots(6,1,figsize=(11,30));out[k]={}
  for a,iR,R in zip(ax,range(6),RS):
    sk=np.array(S[k][iR]);cs=[chi(Dl(sm+e*sk)) for e in G];eb=G[int(np.argmin(cs))];cb=min(cs)
    Db=Dl(sm+eb*sk)[2:31];y=Db[2:];slope=np.polyfit(l[2:],np.log(y),1)[0];over=Db[:3].max()/Pl[:3,1].max()
    ok=B[k][str(R)]
    out[k][R]=dict(best_e=float(eb),chi2=float(cb),slope=float(slope),lowl_over=float(over),guess='PASS' if ok else 'FAIL')
    a.plot(l,Lt,'k-',lw=2,label='ΛCDM χ²=20.1');D0=Dl(sm);a.plot(l,D0[2:31],'r--',lw=2,label=f'smooth red χ²={chi(D0):.1f}')
    for j,e in enumerate(E):
      D=Dl(sm+e*sk);a.plot(l,D[2:31],color=plt.cm.viridis(j/9),label=f'{e*100:g}% χ²={chi(D):.4g}')
    a.plot(l,Db,'m-',lw=2.5,label=f'best {eb*100:.3g}% χ²={cb:.0f}')
    a.errorbar(Pl[:,0],Pl[:,1],yerr=[Pl[:,2],Pl[:,3]],fmt='o',color='k',ms=4,label='Planck')
    a.set_yscale('log');a.set_ylim(1e-3,None);a.set_ylabel('D_□ (μK²)')
    a.set_title(f'red + MC web #{k}, kernel at {R} Gpc — my guess: {out[k][R]["guess"]}',color='g' if ok else 'r')
    a.legend(loc='lower center',ncol=4,fontsize=7,framealpha=.85)
  ax[-1].set_xlabel('□');plt.tight_layout();plt.savefig(f'samples/mc{k}_bracket.png',dpi=80);plt.close()
json.dump(out,open('samples/guesses.json','w'),indent=1)
for k in out:print(k,{R:(v['guess'],round(v['chi2']),f"{v['best_e']*100:.3g}%") for R,v in out[k].items()})
```

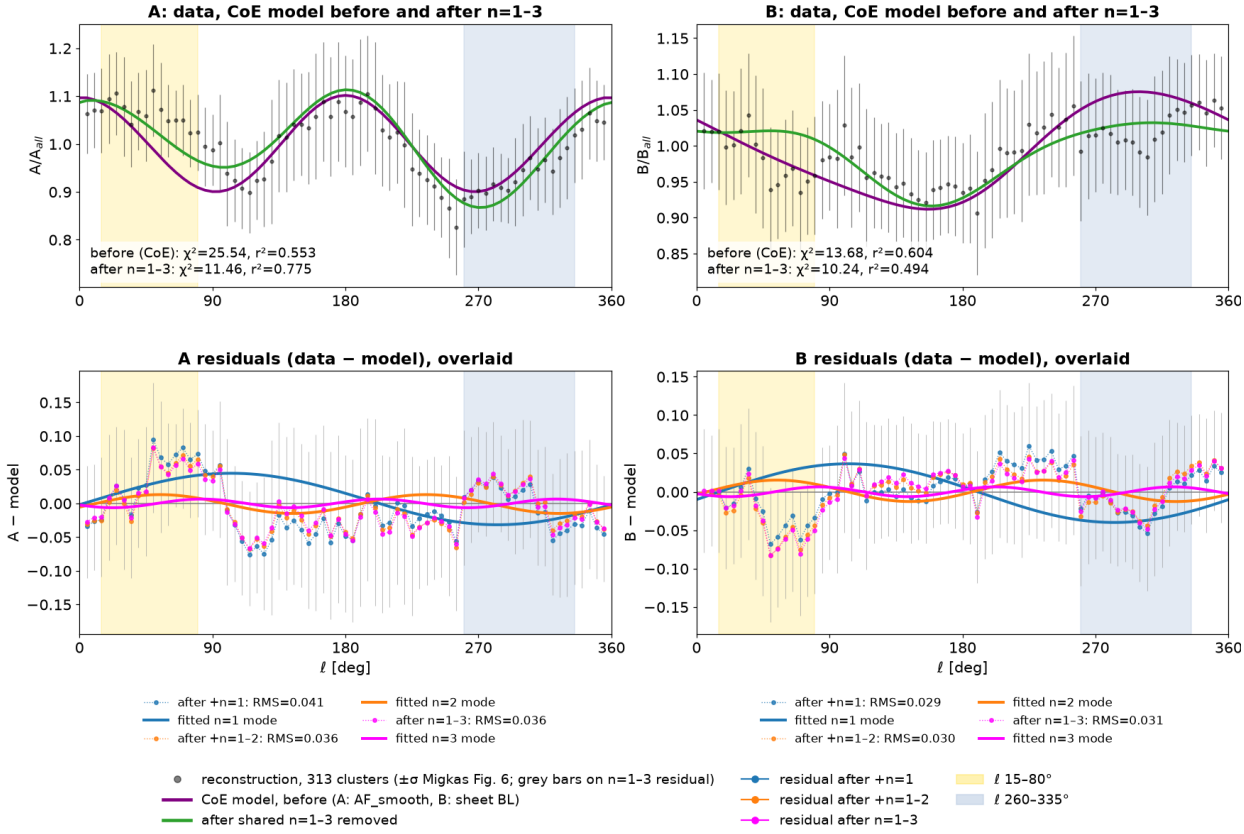
27. Harmonic decomposition 2x2

Originally made: Sun Oct 25 2026, 12:15 AM CT (*file creation (modified again 8:23 AM CT)*)
Paper figure number: this morning Fig. 25; now Fig. 13
Files: `recon_n3_2x2.png`
Code status: original (box)

Notes

A and B data with model before/after removing n = 1-3 harmonics; residuals with fitted modes. Depends on #11. A chi2 25.54 to 11.46, B chi2 13.68 to 10.24.

Paper caption (caveats as stated): Harmonic decomposition, A (left) and B (right). Top: data with the CoE model before harmonics and after removing the shared $n = 1$–3 harmonics (χ^2, Pearson r^2 for both). Bottom: residuals (data − model) after $+n = 1$, $+n = 1$–2 and $n = 1$–3 overlaid, RMS in the legend.



`recon_n3_2x2.png`

Computing chain (run order, full listings)

`/workspace/cosmology/figures/fourier_resid/recon_n3_2x2.py`

```
# Fig 25 2x2: top = data + CoE before + after n=1-3; bottom = residuals after +n=1, +n=1-2, n=1-3 overlaid.
import sys;sys.path.insert(0,'..')
exec(open('recon_n3.py').read().split("cols=['purple'")[0])
from matplotlib.lines import Line2D
plt.rcParams.update({'font.size':14})
RC=['tab:blue','tab:orange','magenta'];lab=['after +n=1','after +n=1–2','after n=1–3']
fig=plt.figure(figsize=(16,11),facecolor='white')
gs=fig.add_gridspec(2,2,height_ratios=[1,1],hspace=0.32,wspace=0.16,left=.07,right=.99,top=.95,bottom=.24)
box=dict(fc='white',ec='none',alpha=.8)
def bands(x):x.axvspan(15,80,color='gold',alpha=.18,zorder=0);x.axvspan(260,335,color='lightsteelblue',alpha=.35,zorder=0)
for k,(nm,Y,sY,mb,mg) in enumerate((('A',A0,sA,AFl,AFg),('B',B0,sB,BLl,BLg))):
  ms=[(mb.copy(),mg.copy())]
  for n,p in steps2:ms.append((ms[-1][0]+T(n,p,th,k),ms[-1][1]+T(n,p,gt,k)))
  st=lambda m:(np.sum((Y-m)**2/sY**2),np.corrcoef(Y,m)[0,1]**2)
  a=fig.add_subplot(gs[0,k]);bands(a)
  a.errorbar(l,Y,yerr=sY,fmt='o',color='k',ms=3,alpha=.5,lw=.8)
  a.plot(g,ms[0][1],color='purple',lw=2.5);a.plot(g,ms[3][1],color='tab:green',lw=2.5)
  (c0,r0),(c3,r3)=st(ms[0][0]),st(ms[3][0])
  a.text(.02,.05,f'before (CoE): χ²={c0:.2f}, r²={r0:.3f}\nafter n=1–3: χ²={c3:.2f}, r²={r3:.3f}',transform=a.transAxes,fontsize=12.5,bbox=box)
  a.set_title(f'{nm}: data, CoE model before and after n=1–3',fontweight='bold',fontsize=15)
  a.set_ylabel('A/A$_{all}$' if nm=='A' else 'B/B$_{all}$');a.set_xlim(0,360);a.set_xticks([0,90,180,270,360])
  b=fig.add_subplot(gs[1,k]);bands(b);b.axhline(0,color='grey',lw=1)
  hh=[]
  for j,i in enumerate((1,2,3)):
    r=Y-ms[i][0];rms=np.sqrt(np.mean(r**2))
    if i==3:b.errorbar(l,r,yerr=sY,fmt='none',ecolor='0.6',lw=.7,alpha=.6,zorder=1)
    hh+=b.plot(l,r,':o',color=RC[j],ms=3.5,lw=0.9,alpha=.8,label=f'{lab[j]}: RMS={rms:.3f}')
    hh+=b.plot(g,T(*steps2[j],gt,k),'-',color=RC[j],lw=2.6,label=f'fitted n={i} mode')
  b.legend(handles=hh,loc='upper center',bbox_to_anchor=(.5,-.2),ncol=2,fontsize=11,frameon=False)
  b.set_title(f'{nm} residuals (data − model), overlaid',fontweight='bold',fontsize=15)
  b.set_ylabel(f'{nm} − model');b.set_xlim(0,360);b.set_xticks([0,90,180,270,360]);b.set_xlabel(' [deg]')
h=[Line2D([],[],color='k',marker='o',ls='',alpha=.5,label='reconstruction, 313 clusters (±σ Migkas Fig. 6; grey bars on n=1–3 residual)'),Line2D([],[],color='purple',lw=3,label='CoE model, before (A: AF_smooth, B: sheet BL)'),Line2D([],[],color='tab:green',lw=3,label='after shared n=1–3 removed')]+[Line2D([],[],color=RC[j],marker='o',lw=1.4,label='residual '+lab[j]) for j in range(3)]+[plt.Rectangle((0,0),1,1,color='gold',alpha=.3,label=' 15–80°'),plt.Rectangle((0,0),1,1,color='lightsteelblue',alpha=.5,label=' 260–335°')]
fig.legend(handles=h,loc='lower center',ncol=3,frameon=False,fontsize=12.5)
fig.savefig('recon_n3_2x2.png',dpi=110,facecolor='white')
```